

To all our customers

Regarding the change of names mentioned in the document, such as Mitsubishi Electric and Mitsubishi XX, to Renesas Technology Corp.

The semiconductor operations of Hitachi and Mitsubishi Electric were transferred to Renesas Technology Corporation on April 1st 2003. These operations include microcomputer, logic, analog and discrete devices, and memory chips other than DRAMs (flash memory, SRAMs etc.)

Accordingly, although Mitsubishi Electric, Mitsubishi Electric Corporation, Mitsubishi Semiconductors, and other Mitsubishi brand names are mentioned in the document, these names have in fact all been changed to Renesas Technology Corp. Thank you for your understanding. Except for our corporate trademark, logo and corporate statement, no changes whatsoever have been made to the contents of the document, and these changes do not constitute any alteration to the contents of the document itself.

Note : Mitsubishi Electric will continue the business operations of high frequency & optical devices and power devices.

Renesas Technology Corp.
Customer Support Dept.
April 1, 2003

M16C/80 Series

Software Manual

MITSUBISHI 16-BIT SINGLE-CHIP
MICROCOMPUTER
M16C FAMILY / M16C/60 SERIES

Using This Manual

This manual is written for the M16C/80 series software. This manual can be used for all types of microcomputers having the M16C/80 series CPU core.

The reader of this manual is expected to have the basic knowledge of electric and logic circuits and microcomputers.

This manual consists of five chapters. The following lists the chapters and sections to be referred to when you want to know details on some specific subject.

- To understand the outline of the M16C/80 series and its features **Chapter 1, "Overview"**
- To understand the operation of each addressing mode **Chapter 2, "Addressing Modes"**
- To understand instruction functions
(Syntax, operation, function, selectable src/dest (label), flag changes, description example, related instructions) **Chapter 3, "Functions"**
- To understand instruction code and cycles **Chapter 4, "Instruction Code/Number of Cycles"**

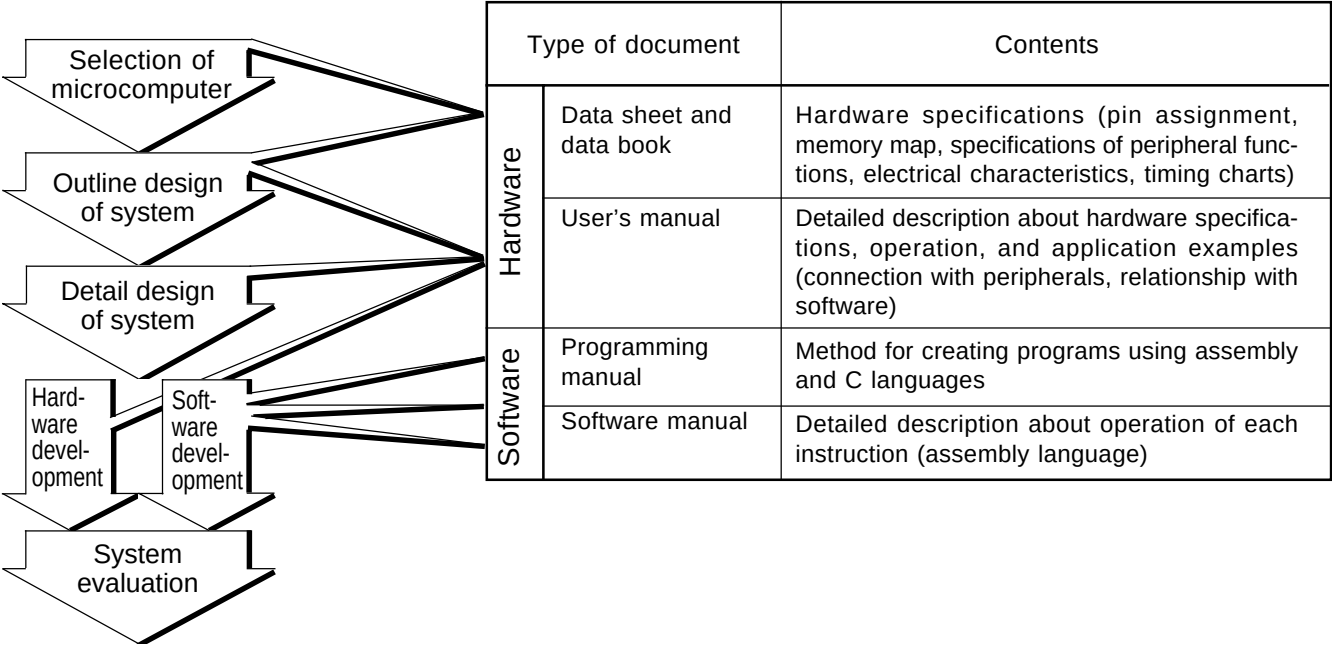
This manual also contains quick references immediately after the Table of Contents. These quick references will help you quickly find the pages for the functions or instruction code/number of cycles you want to know.

- To find pages from mnemonic **Quick Reference in Alphabetic Order**
- To find pages from function and mnemonic **Quick Reference by Function**
- To find pages from mnemonic and addressing **Quick Reference by Addressing**

A table of symbols, a glossary, and an index are appended at the end of this manual.

M16C Family-related document list

Usages
(Microcomputer development flow)



M16C Family Line-up



Table of Contents

Chapter 1	Overview	
1.1	Features of M16C/80 series	2
1.2	Address Space	3
1.3	Register Configuration	4
1.4	Flag Register(FLG)	7
1.5	Register Bank	9
1.6	Internal State after Reset is Cleared	10
1.7	Data Types	11
1.8	Data Arrangement	16
1.9	Instruction Format	18
1.10	Vector Table	19
Chapter 2	Addressing Modes	
2.1	Addressing Modes	22
2.2	Guide to This Chapter	23
2.3	General Instruction Addressing	24
2.4	Specific Instruction Addressing	27
2.5	Bit Instruction Addressing	30
2.6	Bit Instruction Addressing	32
2.7	Read and write operations with 24-bit registers	35
Chapter 3	Functions	
3.1	Guide to This Chapter	38
3.2	Functions	43
3.3	Index Instruction	158
Chapter 4	Instruction Code/Number of Cycles	
4.1	Guide to This Chapter	172
4.2	Instruction Code/Number of Cycles	174

Chapter 5	Interrupt	
5.1	Outline of Interrupt	302
5.2	Interrupt Control	305
5.3	Interrupt Sequence	307
5.4	Return from Interrupt Routine	311
5.5	Interrupt Priority	311
5.6	Multiple Interrupts	312
5.7	Precautions for Interrupts	314
5.8	Exit from Stop Mode and Wait Mode	314
Chapter 6	Calculation Number of Cycles	
6.1	Instruction queue buffer	316

Quick Reference in Alphabetic Order

Mnemonic	See page for function	See page for instruction code/ number of cycles	Mnemonic	See page for function	See page for instruction code/ number of cycles
ABS	43	174	CMPX	72	206
ADC	44	174	DADC	73	206
ADCF	45	176	DADD	74	208
ADD	46	176	DEC	75	210
ADDX	48	183	DIV	76	210
ADJNZ	49	185	DIVU	77	211
AND	50	186	DIVX	78	212
BAND	52	188	DSBB	79	213
BCLR	53	188	DSUB	80	215
BITINDEX	54	189	ENTER	81	217
BM <i>Cnd</i>	55	190	EXITD	82	217
BMEQ/Z	55	190	EXTS	83	218
BMGE	55	190	EXTZ	84	220
BMGEU/C	55	190	FCLR	85	221
BMGT	55	190	FREIT	86	221
BMGTU	55	190	FSET	87	222
BMLE	55	190	INC	88	223
BMLEU	55	190	INDEXB	89	223
BMLT	55	190	INDEXBD	89	224
BMLTU/NC	55	190	INDEXBS	89	224
BMN	55	190	INDEXL	89	225
BMNE/NZ	55	190	INDEXLD	89	225
BMNO	55	190	INDEXLS	89	226
BMO	55	190	INDEXW	89	226
BMPZ	55	190	INDEXWD	89	227
BNAND	56	192	INDEXWS	89	227
BNOR	57	192	INT	90	228
BNOT	58	193	INTO	91	228
BNTST	59	193	J <i>Cnd</i>	92	229
BNXOR	60	194	JEQ/Z	92	229
BOR	61	194	JGE	92	229
BRK	62	195	JGEU/C	92	229
BRK2	63	195	JGT	92	229
BSET	64	196	JGTU	92	229
BTST	65	196	JLE	92	229
BTSTC	66	197	JLEU	92	229
BTSTS	67	198	JLT	92	229
BXOR	68	198	JLTU/NC	92	229
CLIP	69	199	JN	92	229
CMP	70	200	JNE/NZ	92	229

Quick Reference in Alphabetic Order

Mnemonic	See page for function	See page for instruction code/ number of cycles	Mnemonic	See page for function	See page for instruction code/ number of cycles
JNO	92	229	ROT	128	271
JPZ	92	229	RTS	129	272
JMP	93	229	SBB	130	273
JMPI	94	231	SBJNZ	131	275
JMPS	95	232	SC _{cnd}	132	276
JSR	96	233	SCEQ/Z	132	276
JSRI	97	234	SCGE	132	276
JSRS	98	235	SCGEU/C	132	276
LDC	99	235	SCGT	132	276
LDCTX	100	238	SCGTU	132	276
LDIPL	101	239	SCLE	132	276
MAX	102	239	SCLEU	132	276
MIN	103	241	SCLT	132	276
MOV	104	243	SCLTU/NC	132	276
MOVA	106	252	SCN	132	276
MOV _{Dir}	107	253	SCNE/NZ	132	276
MOVHH	107	253	SCNO	132	276
MOVHL	107	253	SCPZ	132	276
MOVLH	107	253	SCMPU	133	277
MOVLL	107	253	SHA	134	278
MOVX	108	255	SHL	136	281
MUL	109	255	SIN	138	283
MULEX	110	257	SMOVB	139	284
MULU	111	257	SMOVF	140	284
NEG	112	259	SMOVU	141	285
NOP	113	259	SOUT	142	285
NOT	114	260	SSTR	143	286
OR	115	260	STC	144	286
POP	117	263	STCTX	145	288
POPC	118	263	STNZ	146	288
POPM	119	264	STZ	147	289
PUSH	120	265	STZX	148	289
PUSHA	121	267	SUB	149	290
PUSHC	122	267	SUBX	151	294
PUSHM	123	268	TST	152	296
REIT	124	269	UND	154	298
RMPA	125	269	WAIT	155	298
ROL	126	270	XCHG	156	299
RORC	127	270	XOR	157	299

Quick Reference by Function

Function	Mnemonic	Content	See page for function	See page for instruction code/ number of cycles
Transfer	MOV	Transfer	104	243
	MOVA	Transfer effective address	106	252
	MOVDir	Transfer 4-bit data	107	253
	MOVX	Transfer extend sign	108	255
	POP	Restore register/memory	117	263
	POPM	Restore multiple registers	119	264
	PUSH	Save register/memory/immediate data	120	265
	PUSHA	Save effective address	121	267
	PUSHM	Save multiple registers	123	268
	STNZ	Conditional transfer	146	288
	STZ	Conditional transfer	147	289
	STZX	Conditional transfer	148	289
	XCHG	Exchange	156	299
Bit manipulation	BAND	Logically AND bits	52	188
	BCLR	Clear bit	53	188
	BITINDEX	Bit index	54	189
	BMCnd	Conditional bit transfer	55	190
	BNAND	Logically AND inverted bits	56	192
	BNOR	Logically OR inverted bits	57	192
	BNOT	Invert bit	58	193
	BNTST	Test inverted bit	59	193
	BNXOR	Exclusive OR inverted bits	60	194
	BOR	Logically OR bits	61	194
	BSET	Set bit	64	196
	BTST	Test bit	65	196
	BTSTC	Test bit & clear	66	197
	BTSTS	Test bit & set	67	198
	BXOR	Exclusive OR bits	68	198
Shift	ROL	Rotate left with carry	126	270
	ROR	Rotate right with carry	127	270
	ROT	Rotate	128	271
	SHA	Shift arithmetic	134	278
	SHL	Shift logical	136	281
Arithmetic	ABS	Absolute value	43	174
	ADC	Add with carry	44	174
	ADCF	Add carry flag	45	176
	ADD	Add without carry	46	176
	ADDX	Add extend sign without carry	48	183
	CLIP	Clip	69	199
	CMP	Compare	70	200

Quick Reference by Function

Function	Mnemonic	Content	See page for function	See page for instruction code/ number of cycles
Arithmetic	CPMX	Compare extended sigh	72	206
	DADC	Decimal add with carry	73	206
	DADD	Decimal add without carry	74	208
	DEC	Decrement	75	210
	DIV	Signed divide	76	210
	DIVU	Unsigned divide	77	211
	DIVX	Singed divide	78	212
	DSBB	Decimal subtract with borrow	79	213
	DSUB	Decimal subtract without borrow	80	215
	EXTS	Extend sign	83	218
	EXTZ	Extend zero	84	220
	INC	Increment	88	223
	MAX	Select maximum value	102	239
	MIN	Select minimum value	103	241
	MUL	Signed multiply	109	255
	MULEX	Multiple extend sign	110	257
	MULU	Unsigned multiply	111	257
	NEG	Two's complement	112	259
	RMPA	Calculate sum-of-products	125	269
	SBB	Subtract with borrow	130	273
	SUB	Subtract without borrow	149	290
	SUBX	Subtract extend without borrow	151	294
Logical	AND	Logical AND	50	186
	NOT	Invert all bits	114	260
	OR	Logical OR	115	260
	TST	Test	152	296
	XOR	Exclusive OR	157	299
Jump	ADJNZ	Add & conditional jump	49	185
	SBJNZ	Subtract & conditional jump	131	275
	JCnd	Jump on condition	92	229
	JMP	Unconditional jump	93	229
	JMPI	Jump indirect	94	231
	JMPS	Jump to special page	95	232
	JSR	Subroutine call	96	233
	JSRI	Indirect subroutine call	97	234
	JSRS	Special page subroutine call	98	235
	RTS	Return from subroutine	129	272
String	SCMPU	String compare unequal	133	277
	SIN	String input	138	283
	SMOVB	Transfer string backward	139	284
	SMOVF	Transfer string forward	140	284

Quick Reference by Function

Function	Mnemonic	Content	See page for function	See page for instruction code/ number of cycles
String	SMOVU	Transfer string	141	285
	SOUT	String output	142	285
	SSTR	Store string	143	286
Other	BRK	Debug interrupt	62	195
	BRK2	Debug interrupt 2	63	195
	ENTER	Build stack frame	81	217
	EXITD	Deallocate stack frame	82	217
	FCLR	Clear flag register bit	85	221
	FREIT	Fast return from interrupt	86	221
	FSET	Set flag register bit	87	222
	INDEX Type	Index	89	223
	INT	Interrupt by INT instruction	90	228
	INTO	Interrupt on overflow	91	228
	LDC	Transfer to control register	99	235
	LDCTX	Restore context	100	238
	LDIPL	Set interrupt enable level	101	239
	NOP	No operation	113	259
	POPC	Restore control register	118	263
	PUSHC	Save control register	122	267
	REIT	Return from interrupt	124	269
	STC	Transfer from control register	144	286
	STCTX	Save context	145	288
	SC <i>cnd</i>	Store on condition	132	276
	UND	Interrupt for undefined instruction	154	298
	WAIT	Wait	155	298

Quick Reference by Addressing (general instruction addressing)

Mnemonic	Addressing																									See page for function	See page for instruction code /number of cycles		
	R0L/R0/R2R0	R0H/R2/-	R1L/R1/R3R1	R1H/R3/-	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24	#IMM8	#IMM16	#IMM24	#IMM32	#IMM	[[An]]	[dsp:8[An]]	[dsp:8[SB/FB]]	[dsp:16[An]]	[dsp:16[SB/FB]]	[dsp:24[An]]	[abs16]			[abs24]	
ABS	√ ²	√	√ ³	√	√	√	√	√	√	√	√								√	√	√	√	√	√	√	√	43	174	
ADC	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√													44	174
ADCF	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√						√	√	√	√	√	√	√	√	√	45	176
ADD ^{*1}	√	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√		√	√	√	√	√	√	√	√	√	√	√	46	176
ADDX	√ ²	√ ⁴	√ ³	√ ⁵	√	√	√	√	√	√	√	√	√	√					√	√	√	√	√	√	√	√	√	48	183
ADJNZ ^{*1}	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√					√										49	185
AND	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√				√	√	√	√	√	√	√	√	√	50	186
BITINDEX	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√															54	189
CLIP	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√													69	199
CMP	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√		√	√	√	√	√	√	√	√	√	√	√	70	200
CMPX	√ ⁶	√	√ ⁷	√	√	√	√	√	√	√	√	√	√	√					√	√	√	√	√	√	√	√	√	72	206
DADC	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√													73	206
DADD	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√													74	208
DEC	√	√	√	√	√	√	√	√	√	√	√	√	√						√	√	√	√	√	√	√	√	√	75	210
DIV	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√				√	√	√	√	√	√	√	√	√	76	210
DIVU	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√				√	√	√	√	√	√	√	√	√	77	211
DIVX	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√				√	√	√	√	√	√	√	√	√	78	212
DSBB	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√				√	√	√	√	√	√	√	√	√	79	213
DSUB	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√	√	√				√	√	√	√	√	√	√	√	√	80	215
ENTER																√												81	217
EXTS	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√															83	218
EXTZ	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√															84	220
INC	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√						√	√	√	√	√	√	√	√	√	88	223
INDEXType	√ ²	√	√ ³	√	√	√	√	√	√	√	√	√	√															89	223

*1 Has special instruction addressing.

*2 Only R0L/R0 can be selected.

*3 Only R1L/R1 can be selected.

*4 Only R0L can be selected.

*5 Only R0H can be selected.

*6 Only R1L can be selected.

*7 Only R1H can be selected.

Quick Reference by Addressing (general instruction addressing)

Mnemonic	Addressing																		See page for function	See page for instruction code /number of cycles
	R0L/R0/R2R0	R0H/R2/-	R1L/R1/R3R1	R1H/R3/-	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24	#IMM8	#IMM16	#IMM24	#IMM32	#IMM		
INT																		✓	90	228
JMP* ¹													✓						93	229
JMPI* ¹	✓ ²	✓ ³	✓ ⁴	✓ ⁵	✓	✓	✓	✓	✓	✓	✓	✓	✓						94	231
JMPS																✓			95	232
JSRI	✓ ²	✓ ³	✓ ⁴	✓ ⁵	✓	✓	✓	✓	✓	✓	✓	✓	✓						97	234
JSRS														✓					98	235
LDC* ¹	✓ ²	✓ ³	✓ ⁴	✓ ⁵	✓	✓	✓	✓	✓	✓	✓	✓	✓		✓	✓			99	235
LDIPL																	✓		101	239
MAX	✓ ⁶	✓	✓ ⁷	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓				102	239
MIN	✓ ⁶	✓	✓ ⁷	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓				103	241
MOV* ¹	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	104	243
MOVA	✓ ⁸		✓ ⁹		✓		✓	✓	✓	✓	✓	✓	✓						106	252
MOVDir	✓ ¹⁰	✓ ¹¹	✓ ¹²	✓ ¹³		✓	✓	✓	✓	✓	✓	✓	✓						107	253
MOVX	✓ ⁸		✓ ⁹		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓				✓	108	255
MUL	✓ ⁶	✓	✓ ⁷		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓			✓	109	255
MULEX				✓ ⁵	✓	✓	✓	✓	✓	✓	✓	✓	✓					✓	110	257
MULU	✓ ⁶	✓	✓ ⁷	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓			✓	111	257
NEG	✓ ⁶	✓	✓ ⁷	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓					✓	112	259
NOT	✓ ⁶	✓	✓ ⁷	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓					✓	114	260
OR	✓ ⁶	✓	✓ ⁷	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓			✓	115	260
POP	✓ ⁶	✓	✓ ⁷	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓					✓	117	263
POPM* ¹	✓	✓	✓	✓															119	264
PUSH	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓		✓	✓	120	265
PUSHA						✓	✓	✓	✓	✓	✓	✓	✓						121	267

*1 Has special instruction addressing.

*2 Only R0/R2R0 can be selected.

*3 Only R2 can be selected.

*4 Only R1/R3R1 can be selected.

*5 Only R3 can be selected.

*6 Only R0L/R0 can be selected.

*7 Only R1L/R1 can be selected.

*8 Only R2R0 can be selected.

*9 Only R3R1 can be selected.

*10 Only R0L can be selected.

*11 Only R0H can be selected.

*12 Only R1L can be selected.

*13 Only R1H can be selected.

Quick Reference by Addressing (general instruction addressing)

Mnemonic	Addressing																		See page for function	See page for instruction code /number of cycles										
	R0L/R0/R2R0	R0H/R2/-	R1L/R1/R3R1	R1H/R3/-	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24	#IMM8	#IMM16	#IMM24	#IMM32	#IMM			[[An]]	[dsp:8[An]]	[dsp:8[SB/FB]]	[dsp:16[An]]	[dsp:16[SB/FB]]	[dsp:24[An]]	[abs16]	[abs24]		
PUSHM ^{*1}	√	√	√	√																							123	268		
ROL	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√							√	√	√	√	√	√	√	√	126	270	
ROR	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√							√	√	√	√	√	√	√	√	127	270	
ROT	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√					√	√	√	√	√	√	√	√	√	√	128	271	
SBB	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√	√	√													130	273	
SBJNZ ^{*1}	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√					√										131	275	
SCCnd	√ ^{*4}	√ ^{*5}	√ ^{*6}	√ ^{*7}	√	√	√	√	√	√	√	√	√							√	√	√	√	√	√	√	√	√	132	276
SHA	√	√	√	√	√	√	√	√	√	√	√	√	√	√				√	√	√	√	√	√	√	√	√	√	√	134	278
SHL	√	√	√	√	√	√	√	√	√	√	√	√	√					√	√	√	√	√	√	√	√	√	√	√	136	281
STC ^{*1}	√ ^{*4}	√ ^{*5}	√ ^{*6}	√ ^{*7}	√	√	√	√	√	√	√	√	√																144	286
STCTX ^{*1}	√	√	√	√																									145	288
STNZ	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√	√	√					√	√	√	√	√	√	√	√	√	146	288
STZ	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√	√	√					√	√	√	√	√	√	√	√	√	147	289
STZX	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√	√	√					√	√	√	√	√	√	√	√	√	148	289
SUB	√	√	√	√	√	√	√	√	√	√	√	√	√	√	√		√			√	√	√	√	√	√	√	√	√	149	290
SUBX	√ ^{*8}	√ ^{*9}	√ ^{*10}	√ ^{*11}	√	√	√	√	√	√	√	√	√	√	√					√	√	√	√	√	√	√	√	√	151	294
TST	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√	√	√														152	296
XCHG	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√							√	√	√	√	√	√	√	√	√	156	299
XOR	√ ^{*2}	√	√	√ ^{*3}	√	√	√	√	√	√	√	√	√	√	√					√	√	√	√	√	√	√	√	√	157	299

*1 Has special instruction addressing.

*2 Only R0L/R0 can be selected.

*3 Only R1L/R1 can be selected.

*4 Only R0 can be selected.

*5 Only R2 can be selected.

*6 Only R1 can be selected.

*7 Only R3 can be selected.

*8 Only R0L/R2R0 can be selected.

*9 Only R0H can be selected.

*10 Only R1L/R3R1 can be selected.

*11 Only R1H can be selected.

Quick Reference by Addressing (special instruction addressing)

Mnemonic	Addressing												See page for function	See page for instruction code /number of cycles
	label	SB/FB	ISP/USP	FLG	INTB	SVPNCT	SVF	DMD0/DMD1	DCT0/DCT1	DRC0/DRC1	DMA0/DMA1	DRA0/DRA1	DSA0/DSA1	
ADD ^{*1}			√										46	176
ADJNZ ^{*1}	√												49	185
JCnd	√												92	229
JMP ^{*1}	√												93	229
JSR ^{*1}	√												96	233
LDC ^{*1}		√	√	√	√	√	√	√	√	√	√	√	99	235
POPC		√	√	√	√	√	√	√	√				118	263
POPM ^{*1}		√											119	264
PUSHC		√	√	√	√	√	√	√	√				122	267
PUSHM ^{*1}		√											123	268
SBJNZ ^{*1}	√												131	275
STC ^{*1}		√	√	√	√	√	√	√	√	√	√	√	144	286

*1 Has general instruction addressing.

Quick Reference by Addressing (bit instruction addressing)

Mnemonic	Addressing											See page for function	See page for instruction code /number of cycles
	bit,R0L/R0H	bit,R1L/R1H	bit,An	bit,[An]	bit,base:11[An]	bit,base:11[SB/FB]	bit,base:19[An]	bit,base:19[SB/FB]	bit,base:27[An]	bit,base:27	bit,base:19	U/I/O/B/S/Z/D/C	
BAND	√	√	√	√	√	√	√	√	√	√	√	52	188
BCLR	√	√	√	√	√	√	√	√	√	√	√	53	188
BM <i>Cnd</i>	√	√	√	√	√	√	√	√	√	√	√	√	190
BNAND	√	√	√	√	√	√	√	√	√	√	√	56	192
BNOR	√	√	√	√	√	√	√	√	√	√	√	57	192
BNOT	√	√	√	√	√	√	√	√	√	√	√	58	193
BNTST	√	√	√	√	√	√	√	√	√	√	√	59	193
BNXOR	√	√	√	√	√	√	√	√	√	√	√	60	194
BOR	√	√	√	√	√	√	√	√	√	√	√	61	194
BSET	√	√	√	√	√	√	√	√	√	√	√	64	196
BTST	√	√	√	√	√	√	√	√	√	√	√	65	196
BTSTC	√	√	√	√	√	√	√	√	√	√	√	66	197
BTSTS	√	√	√	√	√	√	√	√	√	√	√	67	198
BXOR	√	√	√	√	√	√	√	√	√	√	√	68	198
FCLR												√	221
FSET												√	222

Chapter 1

Overview

- 1.1 Features of M16C/80 series**
- 1.2 Address Space**
- 1.3 Register Configuration**
- 1.4 Flag Register (FLG)**
- 1.5 Register Bank**
- 1.6 Internal State after Reset is Cleared**
- 1.7 Data Types**
- 1.8 Data Arrangement**
- 1.9 Instruction Format**
- 1.10 Vector Table**

1.1 Features of M16C/80 series

The M16C/80 series is a single-chip microcomputer developed for built-in applications where the microcomputer is built into applications equipment.

The M16C/80 series supports instructions suitable for the C language with frequently used instructions arranged in one-byte op-code. Therefore, it allows you for efficient program development with few memory capacity regardless of whether you are using the assembly language or C language. Furthermore, some instructions can be executed in one clock cycle, making fast arithmetic processing possible.

Its instruction set consists of 106 discrete instructions matched to the M16C's abundant addressing modes. This powerful instruction set allows to perform register-register, register-memory, and memory-memory operations, as well as arithmetic/logic operations on bits and 4-bit data.

M16C/80 series models incorporate a multiplier, allowing for high-speed computation.

■ Features of M16C/80 series

• Register configuration

Data registers : Four 16-bit registers (of which two registers can be used as 8-bit registers, or two registers are combined and can be used as 32-bit registers)

Address registers : Two 24-bit registers

Base registers : Two 24-bit registers

• Versatile instruction set

C language-suited instructions (stack frame manipulation) : ENTER, EXITD, etc.

Register and memory-indiscriminated instructions : MOV, ADD, SUB, etc.

Powerful bit manipulate instructions : BNOT, BTST, BSET, etc.

4-bit transfer instructions : MOVLL, MOVHL, etc.

Frequently used 1-byte instructions : MOV, ADD, SUB, JMP, etc.

High-speed 1-cycle instructions : MOV, ADD, SUB, etc.

• 16M-byte linear address area

Relative jump instructions matched to distance of jump

• Fast instruction execution time

Shortest 1-cycle instructions : 106 instructions include 39 1-cycle instructions.

■ Speed performance (types incorporating a multiplier, operating at 20 MHz)

Register-register transfer	: 50 ns
Register-memory transfer	: 100 ns
Register-register addition/subtraction	: 50 ns
8 bits x 8 bits register-register operation	: 150 ns
16 bits x 16 bits register-register operation	: 150 ns
16 bits / 8 bits register-register operation	: 0.9 μs
32 bits / 16 bits register-register operation	: 1.2 μs

1.2 Address Space

Fig. 1.2.1 shows an address space.

Addresses 000000₁₆ through 0003FF₁₆ make up an SFR (special function register) area. In individual models of the M16C series, the SFR area extends from 0003FF₁₆ toward lower addresses.

Addresses from 000400₁₆ on make up a memory area. In individual models of the M16C series, a RAM area extends from address 000400₁₆ toward higher addresses, and a ROM area extends from FFFFFFF₁₆ toward lower addresses. Addresses FFFE00₁₆ through FFFFFFF₁₆ make up a fixed vector area.

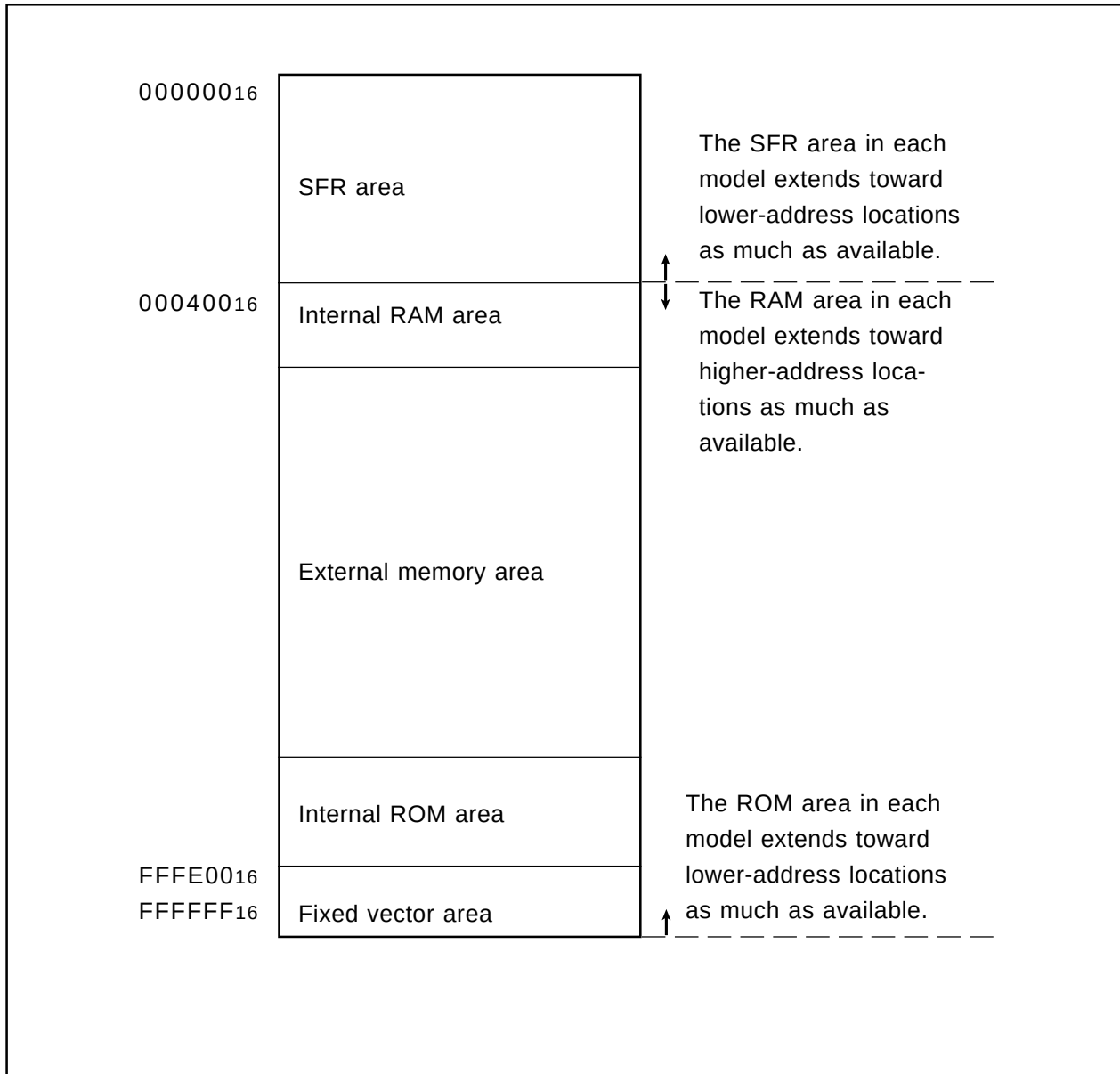


Figure 1.2.1 Address area

1.3 Register Configuration

The central processing unit (CPU) contains the 28 registers shown in Figure 1.3.1. Of these registers, R0, R1, R2, R3, A0, A1, FB, and SB each consist of two sets of registers configuring two register banks.

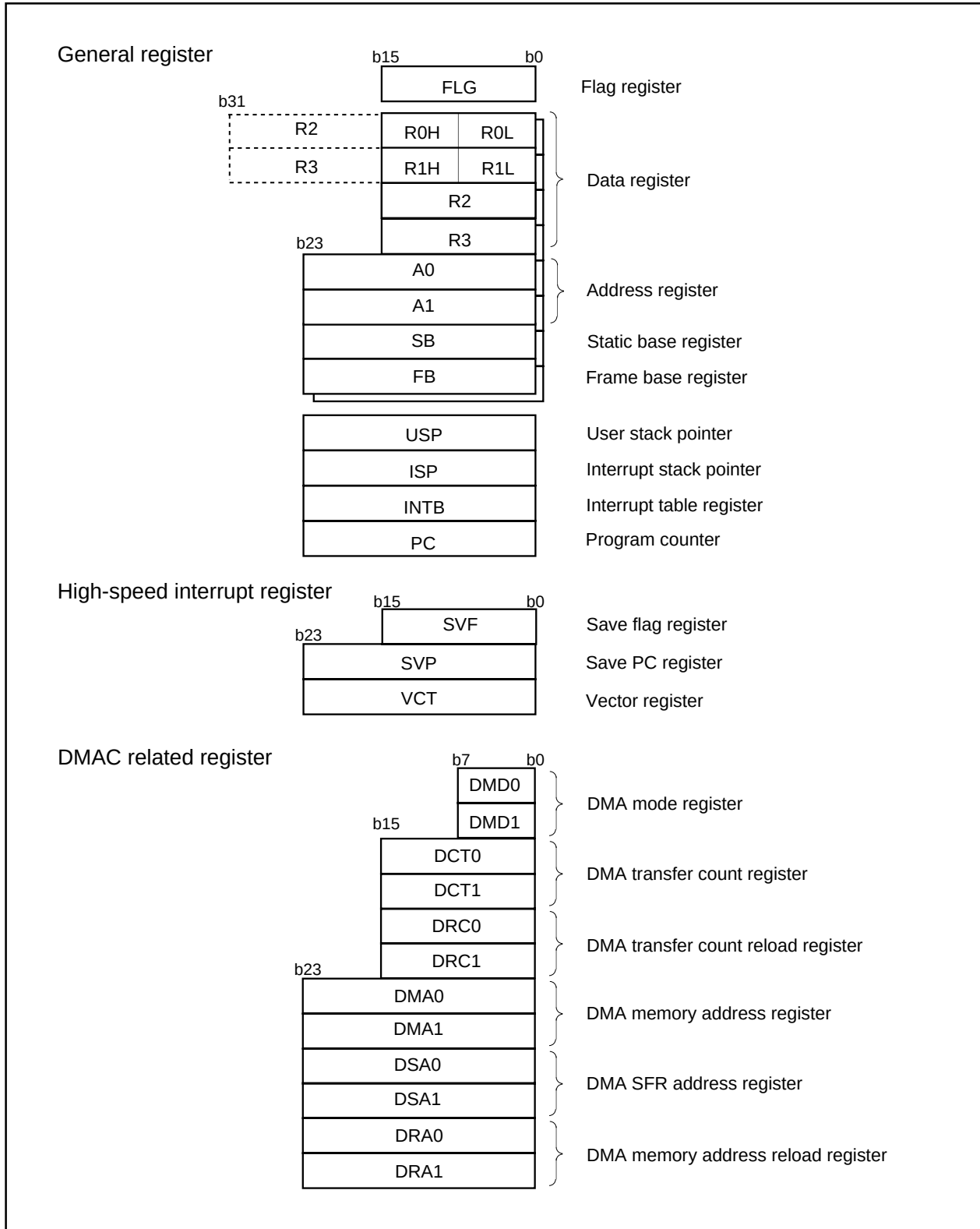


Figure 1.3.1 CPU register configuration

(1) Data registers (R0, R0H, R0L, R1, R1H, R1L, R2, R3, R2R0, and R3R1)

These registers consist of 16 bits, and are used primarily for transfers and arithmetic/logic operations. Registers R0 and R1 can be halved into separate high-order (R0H, R1H) and low-order (R0L, R1L) parts for use as 8-bit data registers. Moreover, you can combine R2 and R0 or R3 and R1 to configure a 32-bit data register (R2R0 or R3R1).

(2) Address registers (A0 and A1)

These registers consist of 24 bits, and have the similar functions as the data registers. These registers are used for address register-based indirect addressing and address register-based relative addressing.

(3) Static base register (SB)

This register consists of 24 bits, and is used for SB-based relative addressing.

(4) Frame base register (FB)

This register consists of 24 bits, and is used for FB-based relative addressing.

(5) Program counter (PC)

This counter consists of 24 bits, indicating the address of an instruction to be executed next.

(6) Interrupt table register (INTB)

This register consists of 24 bits, indicating the initial address of an interrupt vector table.

(7) User stack pointer (USP) and interrupt stack pointer (ISP)

There are two types of stack pointers: user stack pointer (USP) and interrupt stack pointer (ISP), each consisting of 24 bits.

The stack pointer (USP/ISP) you want can be switched by a stack pointer select flag (U flag).

The stack pointer select flag (U flag) is bit 7 of the flag register (FLG).

Set an even number to USP and ISP. When an even number is set, execution becomes efficient.

(8) Flag register (FLG)

This register consists of 11 bits, and is used as a flag, one bit for one flag. For details about the function of each flag, see Section 1.4, "Flag Register (FLG)."

(9) Save flag register (SVF)

This register consists of 16 bits and is used to save the flag register when a high-speed interrupt is generated.

(10) Save PC register (SVP)

This register consists of 16 bits and is used to save the program counter when a high-speed interrupt is generated.

(11) Vector register (VCT)

This register consists of 24 bits and is used to indicate the jump address when a high-speed interrupt is generated.

(12) DMA mode registers (DMD0/DMD1)

These registers consist of 8 bits and are used to set the transfer mode, etc. for DMA.

(13) DMA transfer count registers (DCT0/DCT1)

These registers consist of 16 bits and are used to set the number of DMA transfers performed.

(14) DMA transfer count reload registers (DRC0/DRC1)

These registers consist of 16 bits and are used to reload the DMA transfer count registers.

(15) DMA memory address registers (DMA0/DMA1)

These registers consist of 24 bits and are used to set a memory address at the source or destination of DMA transfer.

(16) DMA SFR address registers (DSA0/DSA1)

These registers consist of 24 bits and are used to set a fixed address at the source or destination of DMA transfer.

(17) DMA memory address reload registers (DRA0/DRA1)

These registers consist of 24 bits and are used to reload the DMA memory address registers.

1.4 Flag Register (FLG)

Figure 1.4.1 shows a configuration of the flag register (FLG). The function of each flag is detailed below.

(1) Bit 0: Carry flag (C flag)

This flag holds a carry, borrow, or shifted-out bit that has occurred in the arithmetic/logic unit.

(2) Bit 1: Debug flag (D flag)

This flag enables a single-step interrupt.

When this flag is set (= 1), a single-step interrupt is generated after an instruction is executed. When an interrupt is acknowledged, this flag is cleared to 0.

(3) Bit 2: Zero flag (Z flag)

This flag is set when an arithmetic operation resulted in 0; otherwise, this flag is 0.

(4) Bit 3: Sign flag (S flag)

This flag is set when an arithmetic operation resulted in a negative value; otherwise, this flag is 0.

(5) Bit 4: Register bank select flag (B flag)

This flag selects a register bank. If this flag is 0, register bank 0 is selected; when the flag is 1, register bank 1 is selected.

(6) Bit 5: Overflow flag (O flag)

This flag is set when an arithmetic operation resulted in overflow.

(7) Bit 6: Interrupt enable flag (I flag)

This flag enables a maskable interrupt.

When this flag is 0, the interrupt is disabled; when the flag is 1, the interrupt is enabled. When the interrupt is acknowledged, this flag is cleared to 0.

(8) Bit 7: Stack pointer select flag (U flag)

When this flag is 0, the interrupt stack pointer (ISP) is selected; when the flag is 1, the user stack pointer (USP) is selected.

This flag is cleared to 0 when a hardware interrupt is acknowledged or an INT instruction of software interrupt numbers 0 to 31 is executed.

(9) Bits 8-11: Reserved area

(10) Bits 12-14: Processor interrupt priority level (IPL)

The processor interrupt priority level (IPL) consists of three bits, allowing you to specify eight processor interrupt priority levels from level 0 to level 7. If a requested interrupt's priority level is higher than the processor interrupt priority level (IPL), this interrupt is enabled.

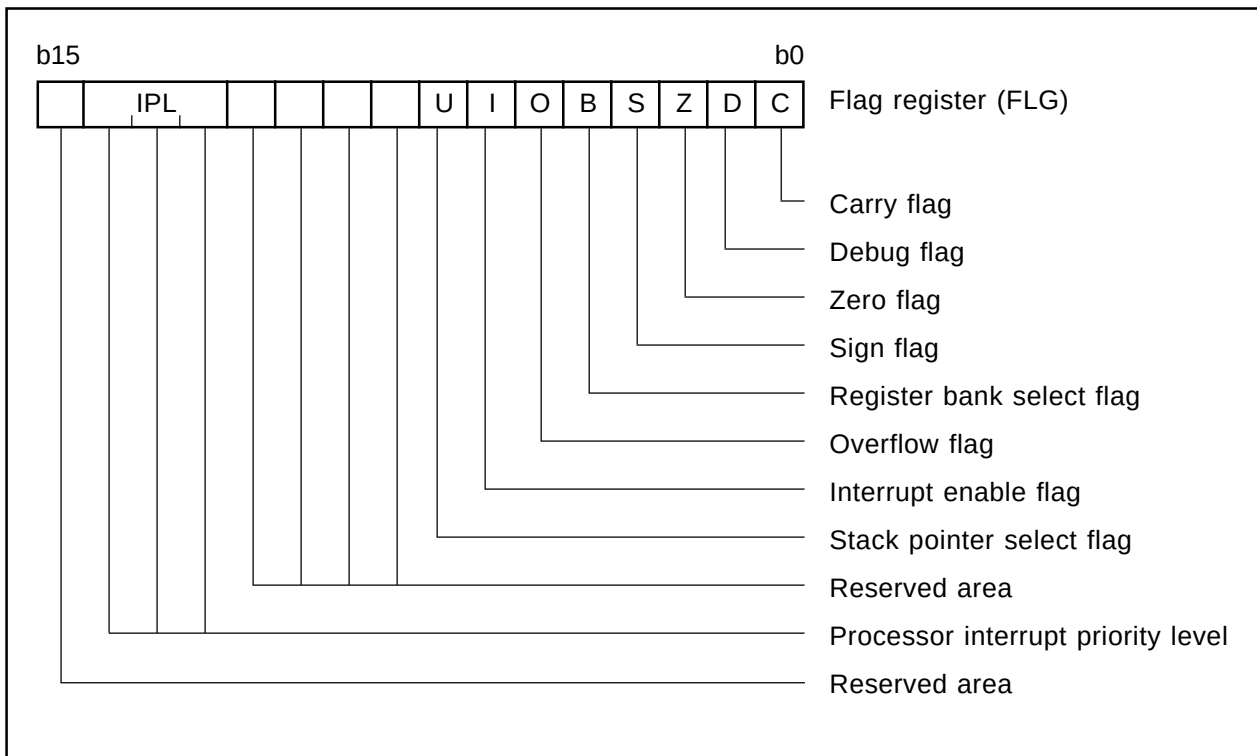
(11) Bit 15: Reserved area

Figure 1.4.1 Configuration of flag register (FLG)

1.5 Register Bank

The M16C has two register banks, each configured with data registers (R0, R1, R2, and R3), address registers (A0 and A1), frame base register (FB), and static base register (SB). These two register banks are switched over by the register bank select flag (B flag) of the flag register (FLG).

Figure 1.5.1 shows a configuration of register banks.

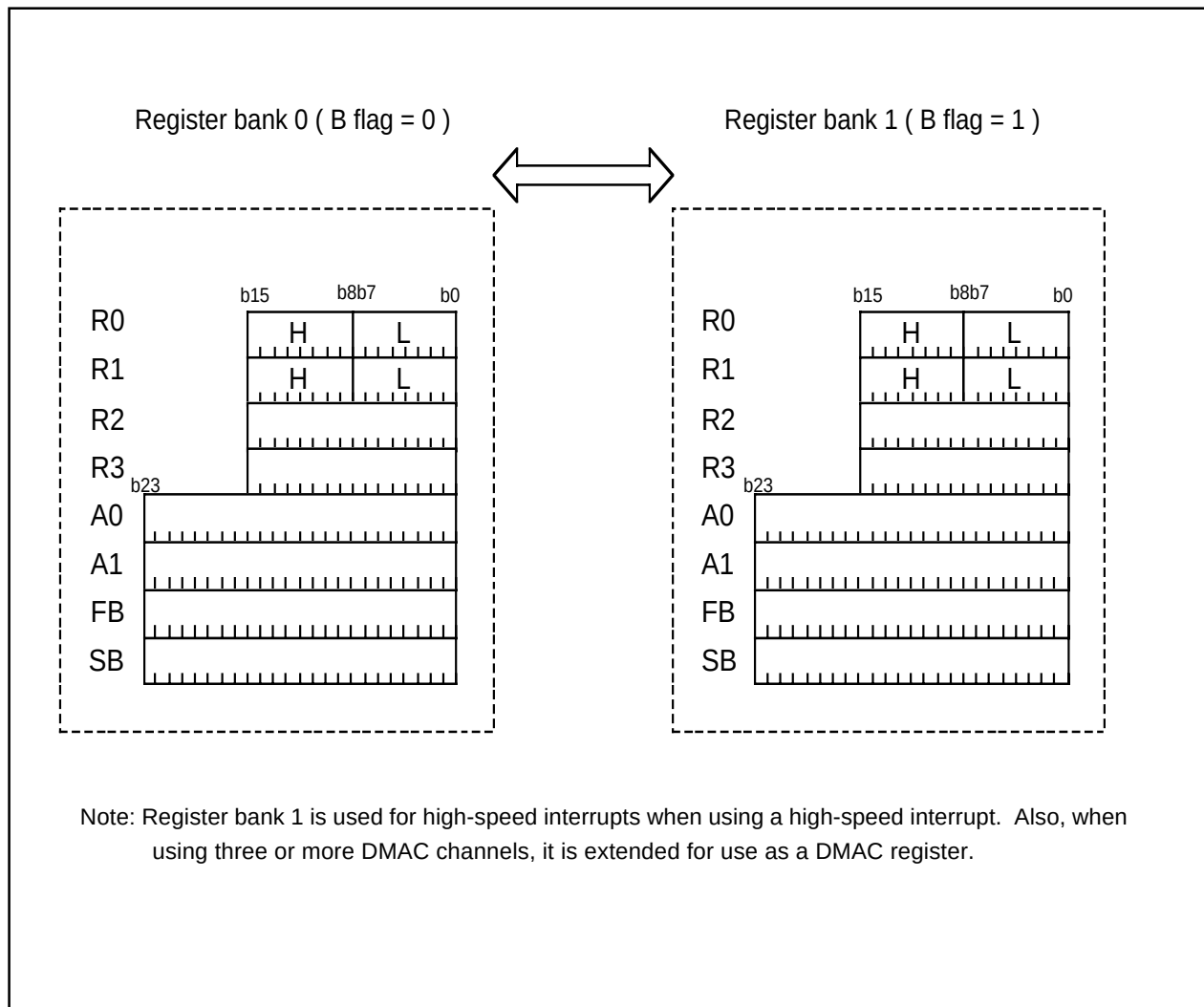


Figure 1.5.1 Configuration of register banks

1.6 Internal State after Reset is Cleared

The following lists the content of each register after a reset is cleared.

• Data registers (R0, R1, R2, and R3)	: 0000 ₁₆
• Address registers (A0 and A1)	: 000000 ₁₆
• Static base register (SB)	: 000000 ₁₆
• Frame base register (FB)	: 000000 ₁₆
• Interrupt table register (INTB)	: 000000 ₁₆
• User stack pointer (USP)	: 000000 ₁₆
• Interrupt stack pointer (ISP)	: 000000 ₁₆
• Flag register (FLG)	: 0000 ₁₆
• DMA mode register (DMD0/DMD1)	: 00 ₁₆
• DMA transfer count register (DCT0/DCT1)	: indeterminate
• DMA transfer count reload register (DRC0/DRC1)	: indeterminate
• DMA memory address register (DMA0/DMA1)	: indeterminate
• DMA SFR address register (DSA0/DSA1)	: indeterminate
• DMA memory address reload register (DRA0/DRA1)	: indeterminate
• Save flag register (SVF)	: indeterminate
• Save PC register (SVP)	: indeterminate
• Vector register (VCT)	: indeterminate

1.7 Data Types

There are four data types: integer, decimal, bit, and string.

1.7.1 Integer

An integer can be a signed or an unsigned integer. A negative value of a signed integer is represented by two's complement.

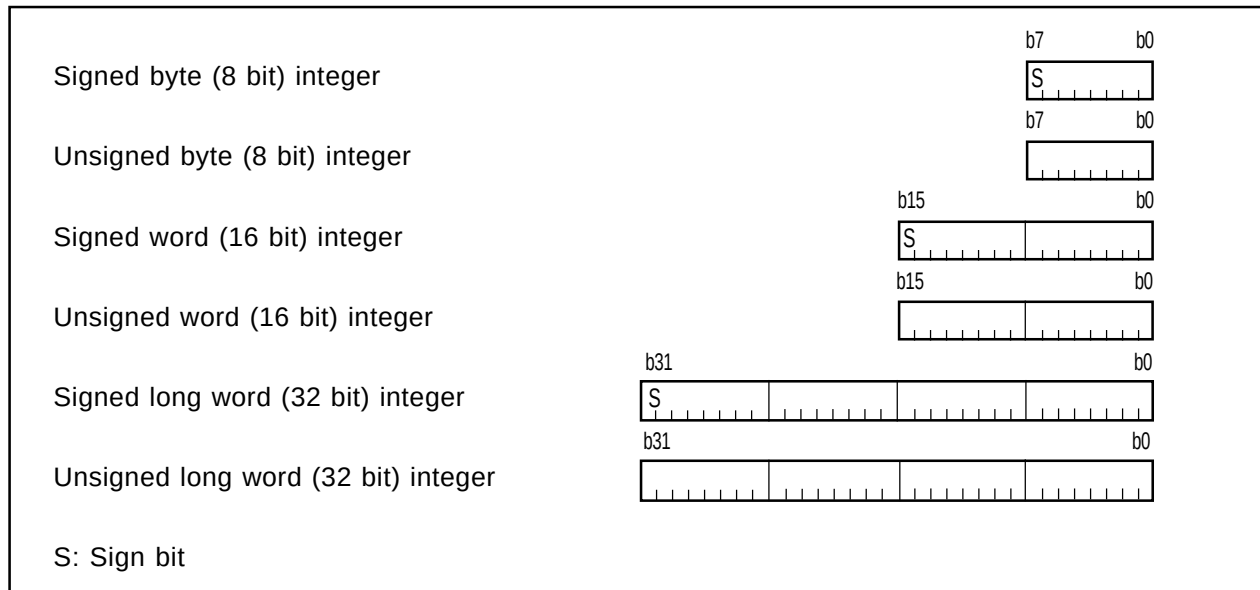


Figure 1.7.1 Integer data

1.7.2 Decimal

This type of data can be used in DADC, DADD, DSBB, and DSUB.

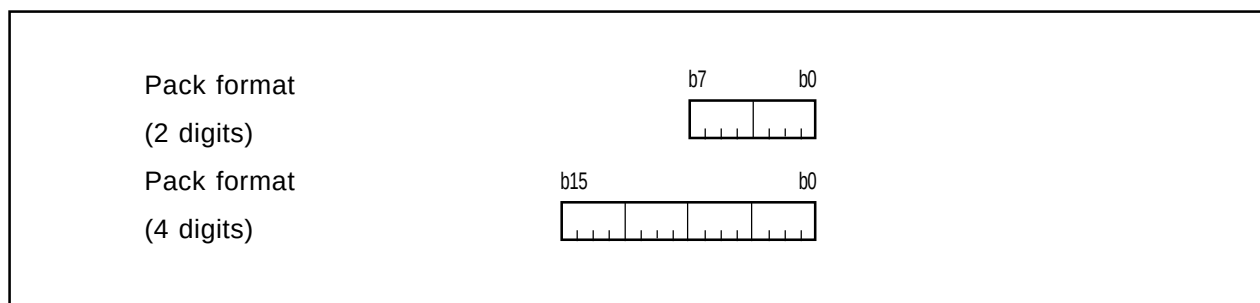


Figure 1.7.2 Decimal data

1.7.3 Bits

(1) Register bits

Figure 1.7.3 shows register bit specification.

Register bits can be specified by register direct (**bit,RnH/RnL** or **bit,An**). Use **bit,RnH/RnL** to specify a bit in data register (**RnH/RnL**); use **bit,An** to specify a bit in address register (**An**).

For bit in **bit,RnH/RnL** and **bit,An**, you can specify a bit number in the range of 0 to 7.

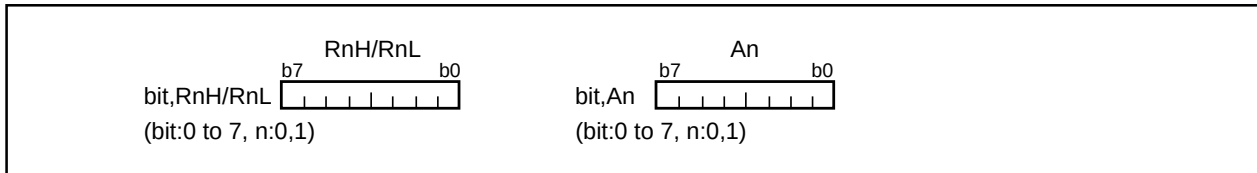


Figure 1.7.3 Register bit specification

(2) Memory bits

Figure 1.7.4 shows addressing modes used for memory bit specification. Table 1.7.1 lists the address range in which you can specify bits in each addressing mode. Be sure to observe the address range in Table 1.7.1 when specifying memory bits.

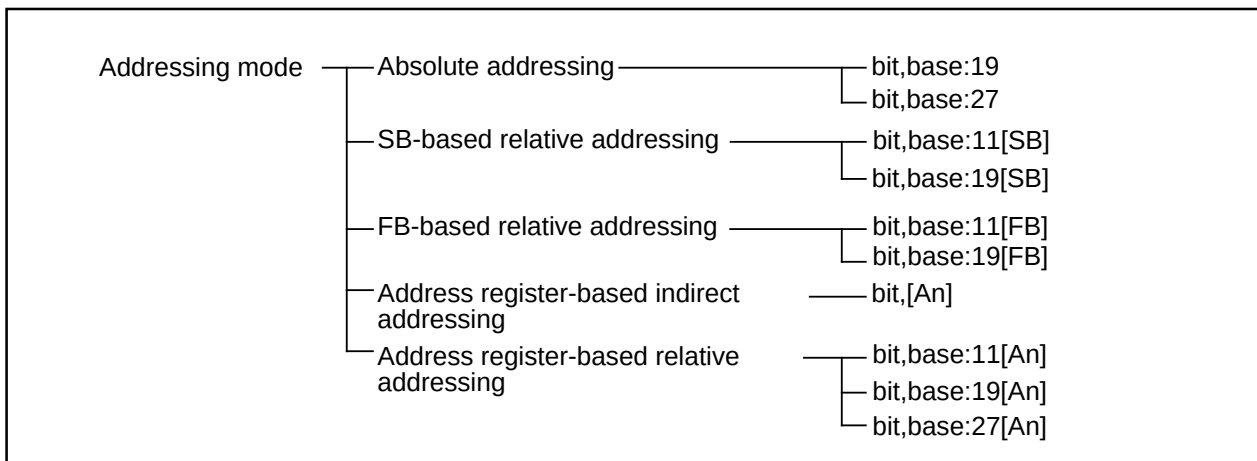


Figure 1.7.4 Addressing modes used for memory bit specification

Table 1.7.1 Bit-Specifying Address Range

Addressing	Specification range		The access range
	Lower limit (address)	Upper limit (address)	
bit,base:19	000000 ₁₆	00FFFF ₁₆	
bit,base:27	000000 ₁₆	FFFFFF ₁₆	
bit,base:11[SB]	[SB]	[SB]+000FF ₁₆	000000 ₁₆ to FFFFFFF ₁₆ .
bit,base:19[SB]	[SB]	[SB]+0FFFF ₁₆	000000 ₁₆ to FFFFFFF ₁₆ .
bit,base:11[FB]	[FB]-000080 ₁₆	[FB]+00007F ₁₆	000000 ₁₆ to FFFFFFF ₁₆ .
bit,base:19[FB]	[FB]-008000 ₁₆	[FB]+007FFF ₁₆	000000 ₁₆ to FFFFFFF ₁₆ .
bit,[An]	000000 ₁₆	FFFFFF ₁₆	
bit,base:11[An]	[An]	[An]+0000FF ₁₆	000000 ₁₆ to FFFFFFF ₁₆ .
bit,base:19[An]	[An]	[An]+00FFFF ₁₆	000000 ₁₆ to FFFFFFF ₁₆ .
bit,base:27[An]	[An]	[An]+FFFFFF ₁₆	000000 ₁₆ to FFFFFFF ₁₆ .

(1) Bit specification by bit, base

Figure 1.7.5 shows the relationship between memory map and bit map.

Memory bits can be handled as an array of consecutive bits. Bits can be specified by a given combination of **bit** and **base**. Using bit 0 of the address that is set to **base** as the reference (= 0), set the desired bit position to **bit**. Figure 1.7.6 shows examples of how to specify bit 2 of address 0000A₁₆.

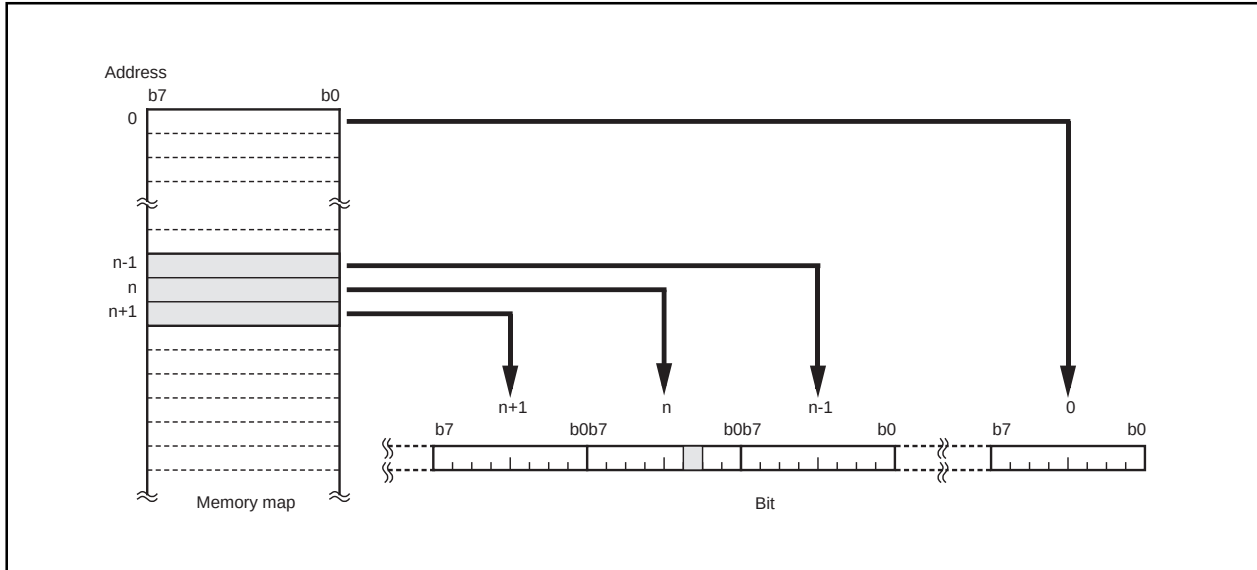


Figure 1.7.5 Relationship between memory map and bit map

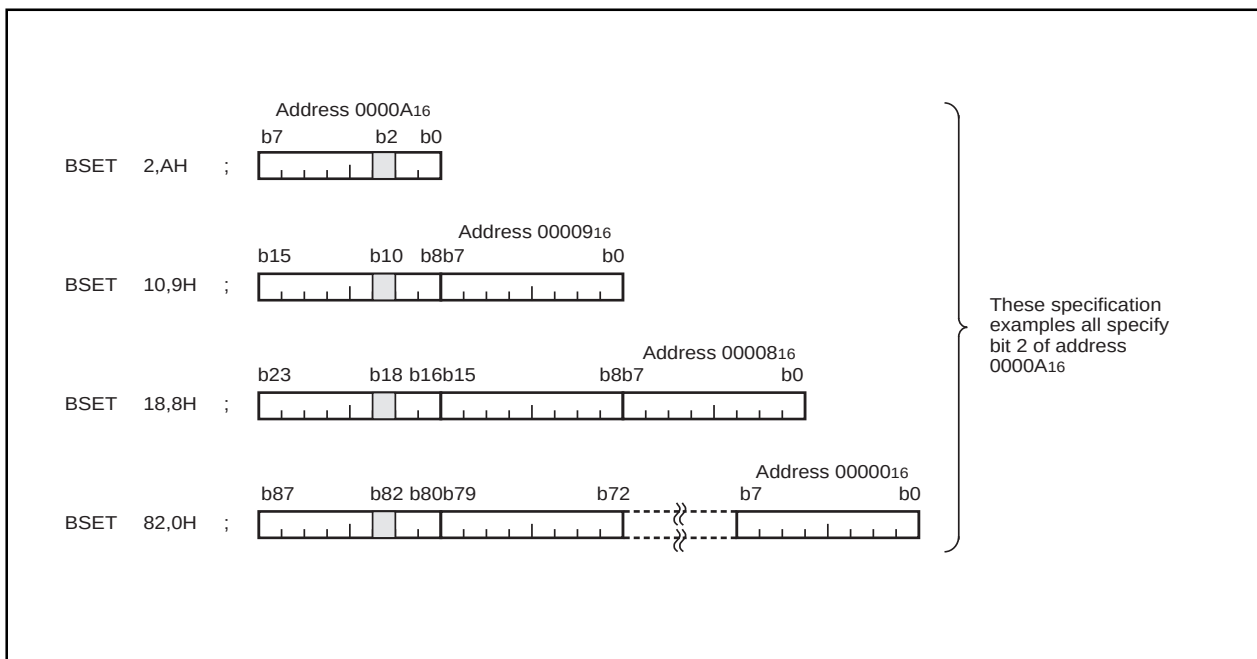


Figure 1.7.6 Examples of how to specify bit 2 of address 0000A₁₆

(2) SB/FB relative bit specification

For SB/FB-based relative addressing, use bit 0 of the address that is the sum of the address set to static base register (**SB**) or frame base register (**FB**) plus the address set to **base** as the reference (= 0), and set the desired bit position to **bit**.

(3) Address register indirect/relative bit specification

For address register indirect addressing, use bit 0 of the address that is set to address register(**An**) as the reference (= 0), and set the desired bit position to **bit**.

For address register indirect addressing, specified bit range is 0 to 7.

For address register relative addressing, use bit 0 of the address that is the sum of the address set to address register (**An**) plus the address set to **base** as the reference (= 0), and set the desired bit position to **bit**.

1.7.4 String

String is a type of data that consists of a given length of consecutive byte (8-bit) or word (16-bit) data. This data type can be used in seven types of string instructions: character string backward transfer (SMOVB instruction), character string forward transfer (SMOVF instruction), specified area initialize (SSTR instruction), character string transfer compare(SCMPU instruction), character string transfer (SMOVU instruction), character string input(SIN instruction) and character string output(SOUT instruction).

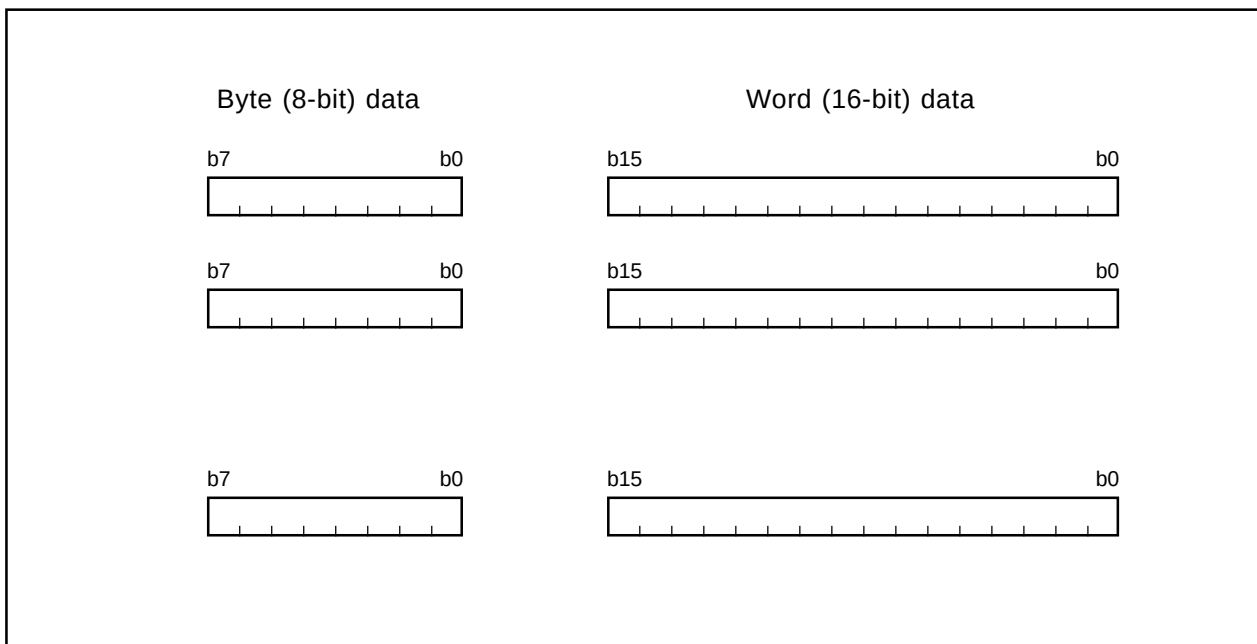


Figure 1.7.7 String data

1.8 Data Arrangement

1.8.1 Data Arrangement in Register

Figure 1.8.1 shows the relationship between a register's data size and bit numbers.

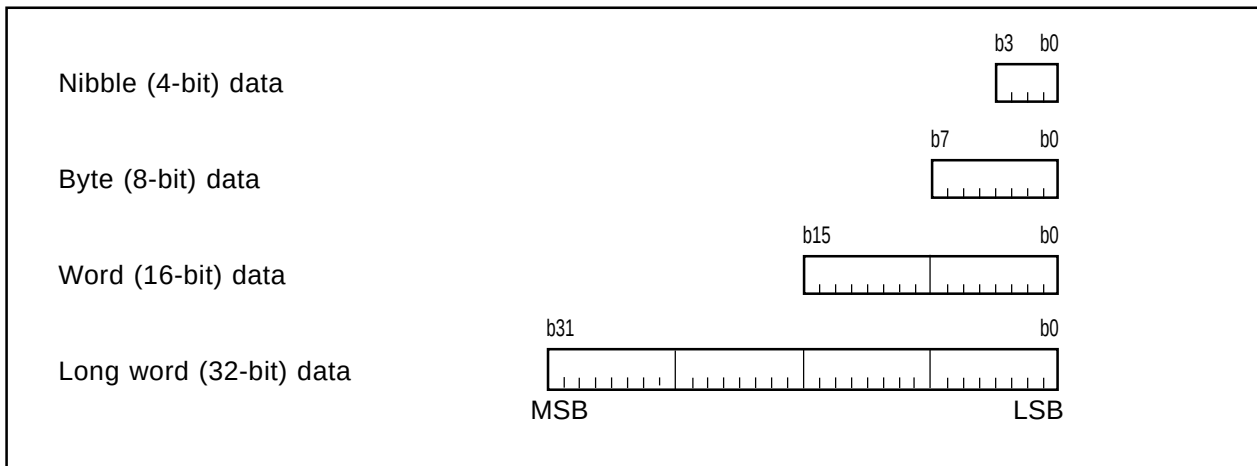


Figure 1.8.1 Data arrangement in register

1.8.2 Data Arrangement in Memory

Figure 1.8.2 shows data arrangement in memory. Figure 1.8.3 shows some examples of operation.

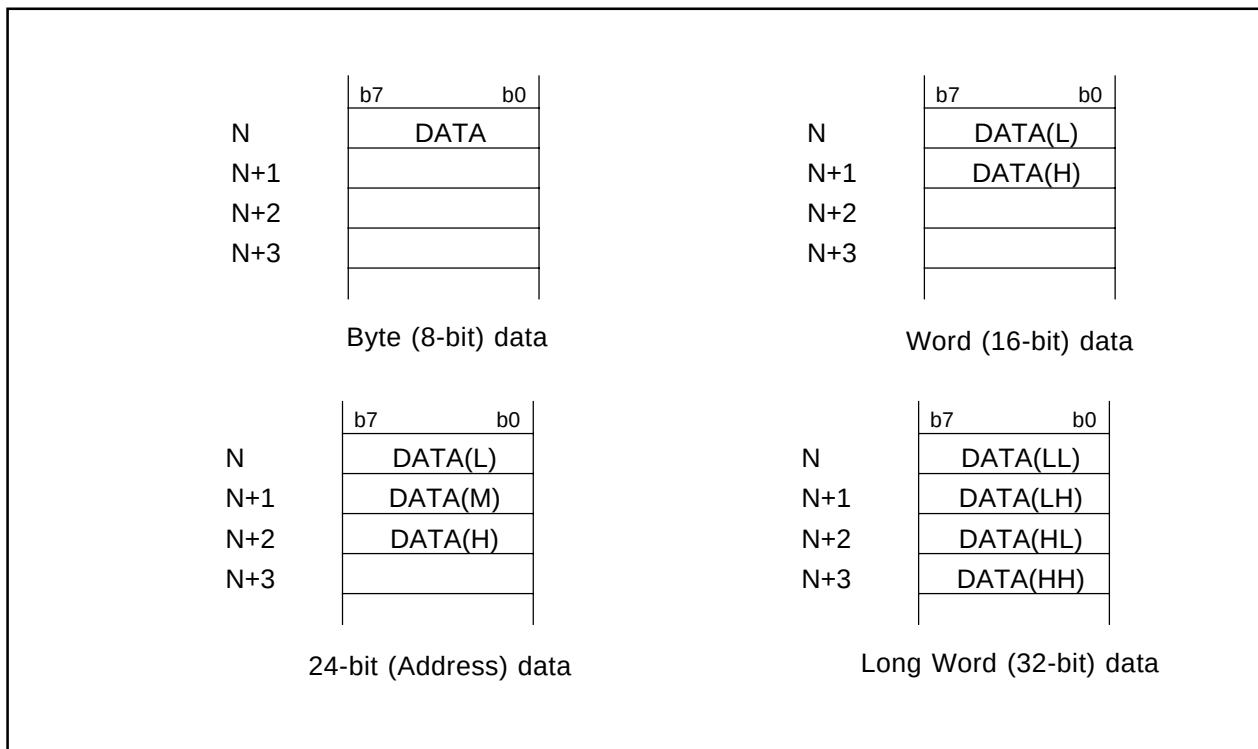


Figure 1.8.2 Data arrangement in memory

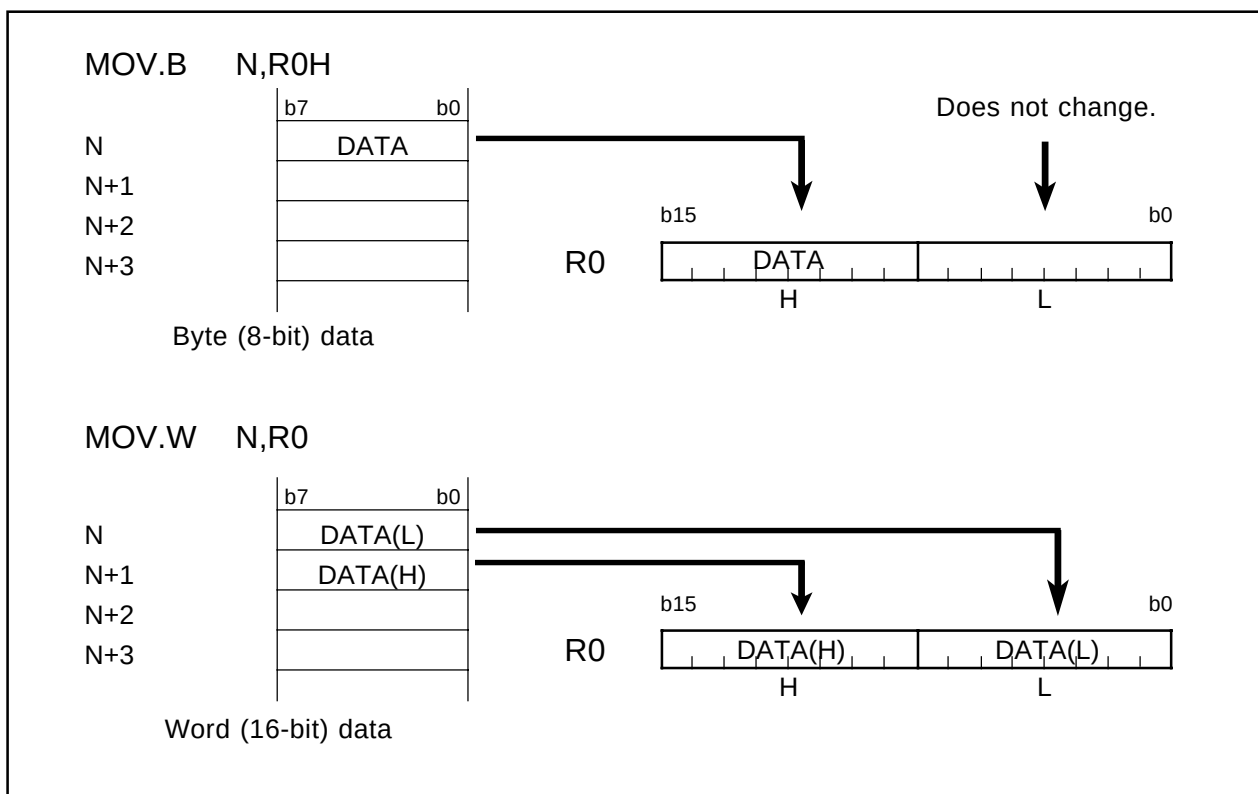


Figure 1.8.3 Examples of operation

1.9 Instruction Format

The instruction format can be classified into four types: generic, quick, short, and zero. The number of instruction bytes that can be chosen by a given format is least for the zero format, and increases successively for the short, quick, and generic formats in that order.

The following describes the features of each format.

(1) Generic format (:G)

Op-code in this format consists of 2 bytes. This op-code contains information on operation and src^{*1} and dest^{*2} addressing modes.

Instruction code here is comprised of op-code (2-3 bytes), src code (0-4 bytes), and dest code (0-3 bytes).

(2) Quick format (:Q)

Op-code in this format consists of two bytes. This op-code contains information on operation and immediate data and dest addressing modes. Note however that the immediate data in this op-code is a numeric value that can be expressed by -7 to +8 or -8 to +7 (varying with instruction).

Instruction code here is comprised of op-code (2 bytes) containing immediate data and dest code (0-3 bytes).

(3) Short format (:S)

Op-code in this format consists of one byte. This op-code contains information on operation and src and dest addressing modes. Note however that the usable addressing modes are limited.

Instruction code here is comprised of op-code (1 byte), src code (0-2 bytes), and dest code (0-2 bytes).

(4) Zero format (:Z)

Op-code in this format consists of one byte. This op-code contains information on operation (plus immediate data) and dest addressing modes. Note however that the immediate data is fixed to 0, and that the usable addressing modes are limited.

Instruction code here is comprised of op-code (1 byte) and dest code (0-2 bytes).

*1 src is the abbreviation of "source."

*2 dest is the abbreviation of "destination."

1.10 Vector Table

The vector table comes in two types: a special page vector table and an interrupt vector table. The special page vector table is a fixed vector table. The interrupt vector table can be a fixed or a variable vector table.

1.10.1 Fixed Vector Table

The fixed vector table is an address-fixed vector table. The special page vector table is allocated to addresses FFFE00₁₆ through FFFFDB₁₆, and part of the interrupt vector table is allocated to addresses FFFFDC₁₆ through FFFFFFF₁₆. Figure 1.10.1 shows a fixed vector table.

The special page vector table is comprised of two bytes per table. Each vector table must contain the 16 low-order bits of the subroutine's entry address. Each vector table has special page numbers (18 to 255) which are used in JSRS and JMPS instructions.

The interrupt vector table is comprised of four bytes per table. Each vector table must contain the interrupt handler routine's entry address.

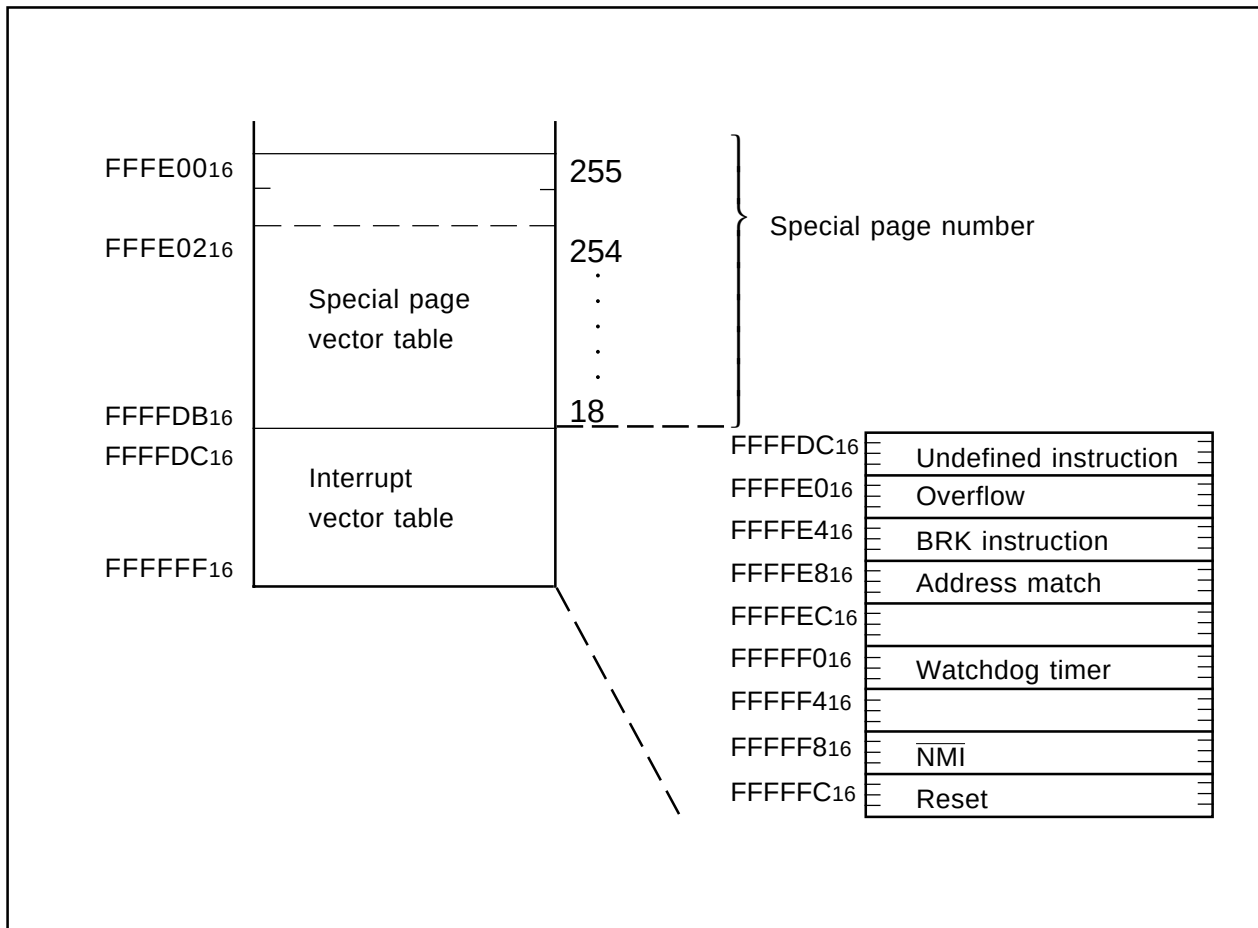


Figure 1.10.1 Fixed vector table

1.10.2 Variable Vector Table

The variable vector table is an address-variable vector table. Specifically, this vector table is a 256-byte interrupt vector table that uses the value indicated by the interrupt table register (INTB) as the entry address (IntBase). Figure 1.10.2 shows a variable vector table.

The variable vector table is comprised of four bytes per table. Each vector table must contain the interrupt handler routine's entry address.

Each vector table has software interrupt numbers (0 to 63). The INT instruction uses these software interrupt numbers.

The built-in peripheral I/O interrupts are assigned to variable vector table by MCU type expansion. Interrupts from the internal peripheral functions are assigned from software interrupt numbers 0. The number of interrupts is different depending on MCU type. To accommodate future increases due to the expansion of product line, Mitsubishi recommend using software interrupt numbers beginning with 63 when you use INT instruction interrupts.

The stack pointer (SP) used for INT instruction interrupts varies with each software interrupt number. For software interrupt numbers 0 through 31, the stack pointer specifying flag (U flag) is saved when an interrupt request is accepted and the interrupt sequence is executed after clearing the U flag to 0 and selecting the interrupt stack pointer (ISP). The U flag that was saved before accepting the interrupt request is restored upon returning from the interrupt handler routine.

For software interrupt numbers 32 through 63, the stack pointer is not switched over.

For peripheral I/O interrupts, the interrupt stack pointer (ISP) is selected irrespective of software interrupt numbers when accepting an interrupt request as for software interrupt numbers 0 through 31.

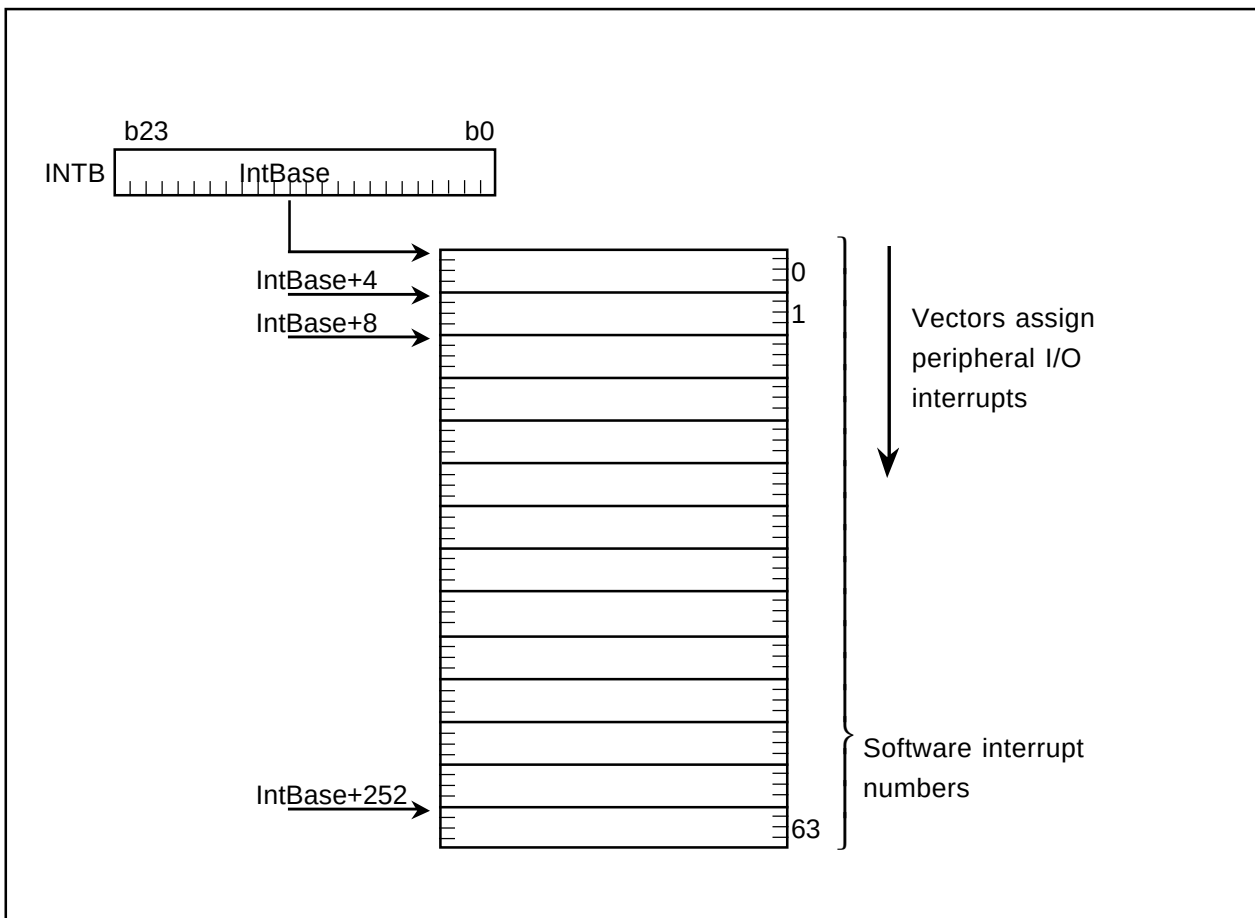


Figure 1.10.2 Variable vector table

Chapter 2

Addressing Modes

- 2.1 Addressing Modes**
- 2.2 Guide to This Chapter**
- 2.3 General Instruction Addressing**
- 2.4 Indirect Instruction Addressing**
- 2.5 Special Instruction Addressing**
- 2.6 Bit Instruction Addressing**
- 2.7 Read and write operations with 24-bit registers**

2.1 Addressing Modes

This section describes addressing mode-representing symbols and operations for each addressing mode. The M16C has four addressing modes outlined below.

(1) General instruction addressing

This addressing accesses an area from address 000000₁₆ through address FFFFFFF₁₆.

The following lists the name of each general instruction addressing:

- Immediate
- Register direct
- Absolute
- Address register indirect
- Address register relative
- SB relative
- FB relative
- Stack pointer relative

(2) Indirect instruction addressing

This addressing accesses an area from address 000000₁₆ through address FFFFFFF₁₆.

The following lists the name of each indirect instruction addressing:

- Absolute indirect
- Two-stage address register indirect
- Address register relative indirect
- SB relative indirect
- FB relative indirect

(3) Special instruction addressing

This addressing accesses an area from address 000000₁₆ through address FFFFFFF₁₆ and control registers.

The following lists the name of each specific instruction addressing:

- Control register direct
- Program counter relative

(4) Bit instruction addressing

This addressing accesses an area from address 000000₁₆ through address FFFFFFF₁₆.

The following lists the name of each bit instruction addressing:

- Register direct
- Absolute
- Address register indirect
- Address register relative
- SB relative
- FB relative
- FLG direct

2.2 Guide to This Chapter

The following shows how to read this chapter using an actual example.

(1)	Address register relative	
(2)	dsp:8[A0] dsp:8[A1] dsp:16[A0] dsp:16[A1]	
(3)	dsp:24[A0] dsp:24[A1]	
(4)		

(1) Name

Indicates the name of addressing.

(2) Symbol

Represents the addressing mode.

(3) Explanation

Describes the addressing operation and the effective address range.

(4) Operation diagram

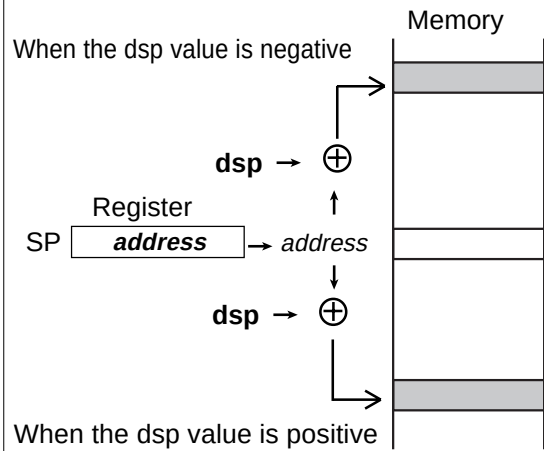
Diagrammatically explains the addressing operation.

2.3 General Instruction Addressing

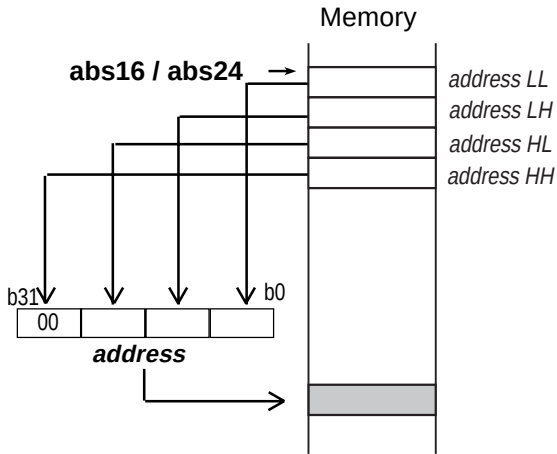
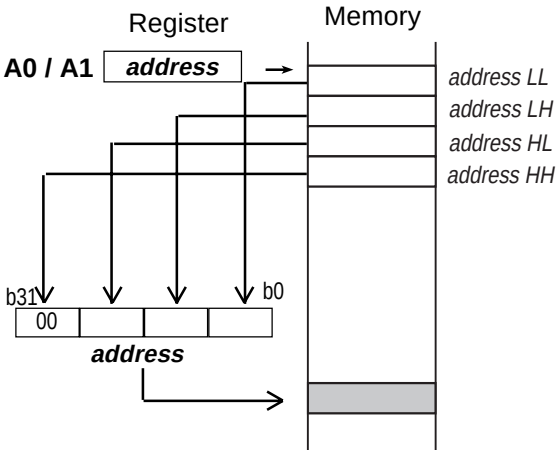
Immediate		
#IMM #IMM8 #IMM16 #IMM32	The immediate data indicated by #IMM is the object to be operated on.	#IMM8 #IMM16 #IMM32
Register direct		
R0L R0H R1L R1H R0 R1 R2 R3 A0 A1 R2R0 R3R1	The specified register is the object to be operated on.	Register R0L / R1L R0H / R1H R0 / R1 / R2 / R3 A0 / A1 R2R0 / R3R1
Absolute		
abs16 abs24	The value indicated by abs constitutes the effective address to be operated on. The effective address range is 0000000₁₆ to 000FFFF₁₆ at abs16, and 0000000₁₆ to 0FFFFFF₁₆ at abs24.	Memory abs16 / abs24
Address register indirect		
[A0] [A1]	The value indicated by the content of address register (A0/A1) constitutes the effective address to be operated on. The effective address range is 0000000₁₆ to 0FFFFFF₁₆.	Register A0 / A1 address Memory

Address register relative		
dsp:8[A0] dsp:8[A1] dsp:16[A0] dsp:16[A1] dsp:24[A0] dsp:24[A1]	The value indicated by displacement (dsp) plus the content of address register (A0/A1)—added not including the sign bits—constitutes the effective address to be operated on. However, if the addition resulted in exceeding $0FFFFFFF_{16}$, the bits above bit 25 are ignored, and the address returns to 00000000_{16}.	
SB relative		
dsp:8[SB] dsp:16[SB]	The address indicated by the content of static base register (SB) plus the value indicated by displacement (dsp)—added not including the sign bits—constitutes the effective address to be operated on. However, if the addition resulted in exceeding $0FFFFFFF_{16}$, the bits above bit 25 are ignored, and the address returns to 00000000_{16}.	
FB relative		
dsp:8[FB] dsp:16[FB]	The address indicated by the content of frame base register (FB) plus the value indicated by displacement (dsp)—added including the sign bits—constitutes the effective address to be operated on. However, if the addition resulted in exceeding 00000000_{16}- $0FFFFFFF_{16}$, the bits above bit 25 are ignored, and the address returns to 00000000_{16} or $0FFFFFFF_{16}$.	

Stack pointer relative	
dsp:8[SP]	The address indicated by the content of stack pointer (SP) plus the value indicated by displacement (dsp) added including the sign bits—constitutes the effective address to be operated on. The stack pointer (SP) here is the one indicated by the U flag. However, if the addition resulted in exceeding 0000000_{16}- $0FFFFFFF_{16}$, the bits above bit 25 are ignored, and the address returns to 0000000_{16} or $0FFFFFFF_{16}$. This addressing can be used in MOV instruction.



2.4 Indirect Instruction Addressing

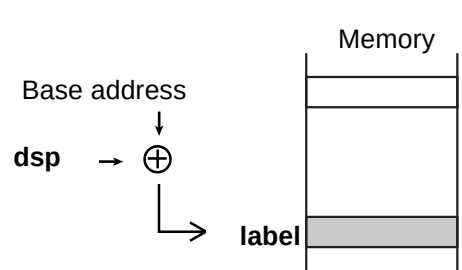
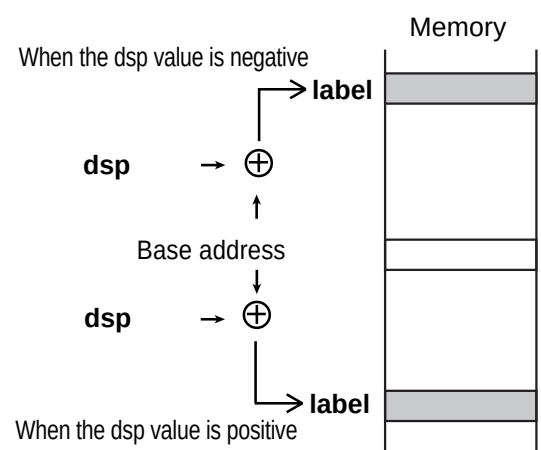
Absolute indirect	 (The upper 8-bit is ignored.)	
Two-stage address register indirect	 (The upper 8-bit is ignored.)	

Address register relative indirect		(The upper 8-bit is ignored.)
[dsp:8[A0]] [dsp:8[A1]] [dsp:16[A0]] [dsp:16[A1]] [dsp:24[A0]] [dsp:24[A1]] The 4-byte value indicated by address register relative constitutes the effective address to be operated on. The effective address range is 00000000₁₆ to 0FFFFFFF₁₆.		
SB relative indirect		(The upper 8-bit is ignored.)
[dsp:8[SB]] [dsp:16[SB]] The 4-byte value indicated by SB relative constitutes the effective address to be operated on. The effective address range is 00000000₁₆ to 0FFFFFFF₁₆.		

FB relative indirect	
[dsp:8[FB]] [dsp:16[FB]]	The 4-byte value indicated by FB relative constitutes the effective address to be operated on. The effective address range is 00000000₁₆ to 0FFFFFFF₁₆. The diagram illustrates the FB relative indirect addressing mode. It shows two identical address calculation sequences. In each sequence, a 32-bit 'address' (represented by a 4-byte field with bits b31 down to b0) is added to the 'dsp' register. The result of this addition is then used to access memory locations address LL, address LH, address HL, and address HH. The upper 8-bit of the address is ignored. (The upper 8-bit is ignored.)

2.5 Special Instruction Addressing

Control register direct		
INTB	The specified control register is the object to be operated on.	INTB
ISP		ISP
SP	This addressing can be used in LDC and STC instructions.	USP
SB		SB
FB	If you specify SP, the stack pointer indicated by the U flag is the object to be operated on.	FB
FLG		FLG
SVP		SVP
VCT		VCT
SVF		SVF
DMD0		DMD0
DMD1		DMD1
DCT0		DCT0
DCT1		DCT1
DRC0		DRC0
DRC1		DRC1
DMA0		DMA0
DMA1		DMA1
DSA0		DSA0
DSA1		DSA1
DRA0		DRA0
DRA1		DRA1

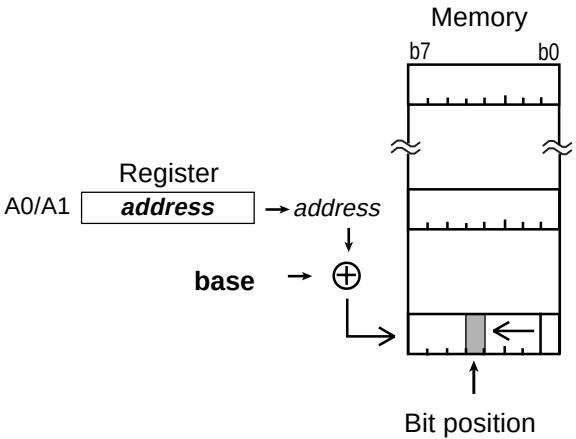
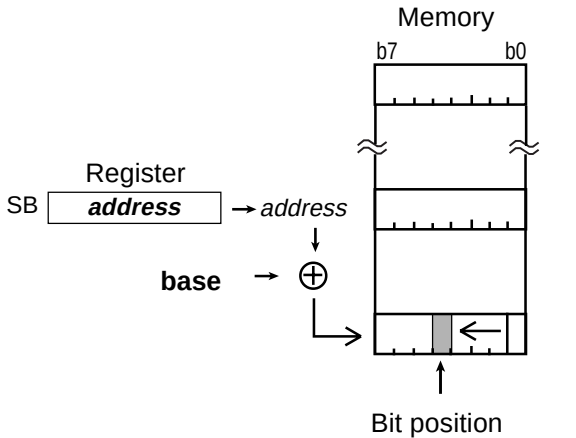
Program counter relative		
label	When the jump length specifier (.length) is (.S)... the base address plus the value indicated by displacement (dsp)—added not including the sign bits—constitutes the effective address. This addressing can be used in JMP instruction.	 $+0 \leq dsp \leq +7$ *1 The base address is the (start address of instruction + 2).
	When the jump length specifier (.length) is (.B) or (.W)... the base address plus the value indicated by displacement (dsp)—added including the sign bits—constitutes the effective address. However, if the addition resulted in exceeding 0000000_{16}-$0FFFFFFF_{16}$, the bits above bit 25 are ignored, and the address returns to 0000000_{16} or $0FFFFFFF_{16}$. This addressing can be used in JMP and JSR instructions.	 When the specifier is (.B), $-128 \leq dsp \leq +127$ When the specifier is (.W), $-32768 \leq dsp \leq +32767$ *2 The base address varies with each instruction.

2.6 Bit Instruction Addressing

This addressing can be used in the following instructions:

BCLR, BSET, BNOT, BTST, BNTST, BAND, BNAND, BOR, BNOR, BXOR, BNXOR, *BM*Cnd**, BTSTS, BTSTC

Register direct		bit , R0L
bit,R0L bit,R0H bit,R1L bit,R1H bit,A0 bit,A1	The specified register bit is the object to be operated on. For the bit position (bit) you can specify 0 to 7. For the address register (A0,A1), you can specify 8 low-order bits.	
Absolute		base
bit,base:19 bit,base:27	The bit that is as much away from bit 0 at the address indicated by base as the number of bits indicated by bit is the object to be operated on. The address range that can be specified by bit,base:19 and bit,base:27 respectively are 0000000₁₆ through 000FFFF₁₆ and 0000000₁₆ through 0FFFFFF₁₆.	
Address register indirect		A0/A1 Register address →
bit,[A0] bit,[A1]	The bit that is as much away from bit 0 at address indicated by address register (A0/A1) as the number of bits is the object to be operated on. Bits at addresses 0000000₁₆ through 0FFFFFF₁₆ can be the object to be operated on. For the bit position (bit) you can specify 0 to 7.	

Address register relative bit,base:11[A0] bit,base:11[A1] bit,base:19[A0] bit,base:19[A1] bit,base:27[A0] bit,base:27[A1] The bit that is as much away from bit 0 at the address indicated by base as the number of bits indicated by address register (A0/A1) is the object to be operated on. However, if the address of the bit to be operated on exceeds 0FFFFFFF₁₆, the bits above bit 25 are ignored and the address returns to 00000000₁₆. The address range that can be specified by bit,base:11, bit,base:19 and bit,base:27 respectively are 256 bytes, 65,536 bytes and 16,777,216 bytes from address register (A0/A1) value.	
SB relative bit,base:11[SB] bit,base:19[SB] The bit that is as much away from bit 0 at the address indicated by static base register (SB) plus the value indicated by base (added not including the sign bits) as the number of bits indicated by bit is the object to be operated on. However, if the address of the bit to be operated on exceeds 0FFFFFFF₁₆, the bits above bit 25 are ignored and the address returns to 00000000₁₆. The address ranges that can be specified by bit,base: 11, and bit,base:19 respectively are 256 bytes, and 65,536 bytes from the static base register (SB) value.	

FB relative		
bit,base:11[FB] bit,base:19[FB]	The bit that is as much away from bit 0 at the address indicated by frame base register (FB) plus the value indicated by base (added including the sign bit) as the number of bits indicated by bit is the object to be operated on. However, if the address of the bit to be operated on exceeds 0000000₁₆-0FFFFFF₁₆, the bits above bit 25 are ignored and the address returns to 0000000₁₆ or 0FFFFFF₁₆. The address range that can be specified by bit,base:11 and bit,base:19 are 128 bytes toward lower addresses or 127 bytes toward higher addresses from the frame base register (FB) value, and 32,768 bytes toward lower addresses or 32,767 bytes toward higher addresses, respectively.	
FLG direct		
U I O B S Z D C	The specified flag is the object to be operated on. This addressing can be used in FCLR and FSET instructions.	

2.7 Read and write operations with 24-bit registers

This section describes operation when 24 bits register(A0, A1) is src or dest for each size specifier (.size/.B .W .L).

When (.B) is specified for the size specifier (.size)

- Read

The 8 low-order bits are read. The flags change states depending on the result of 8-bit operation.

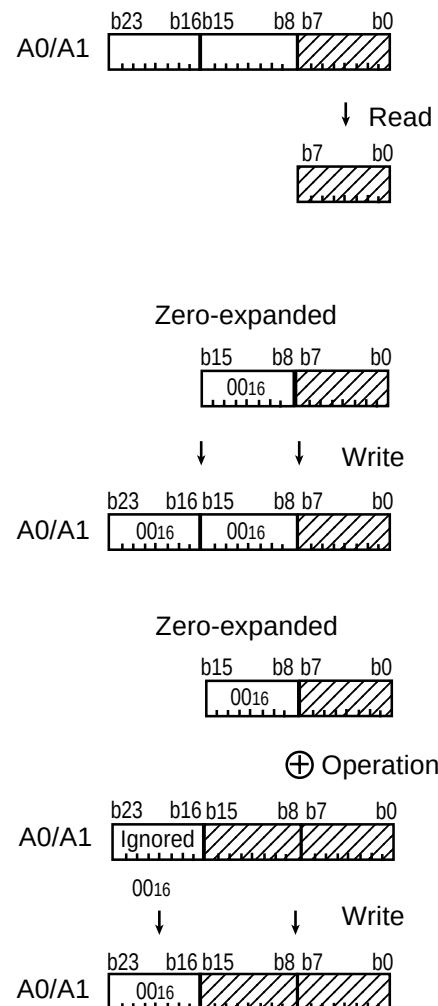
- Write

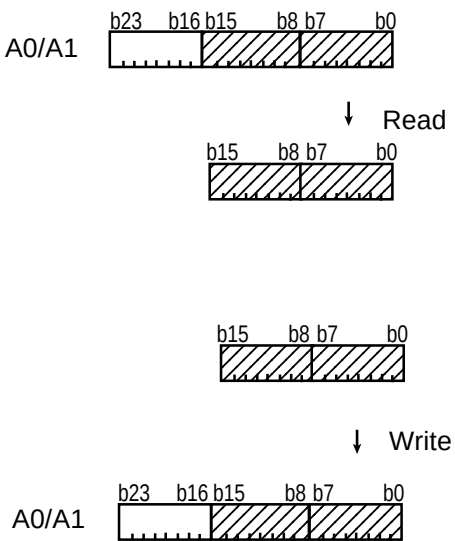
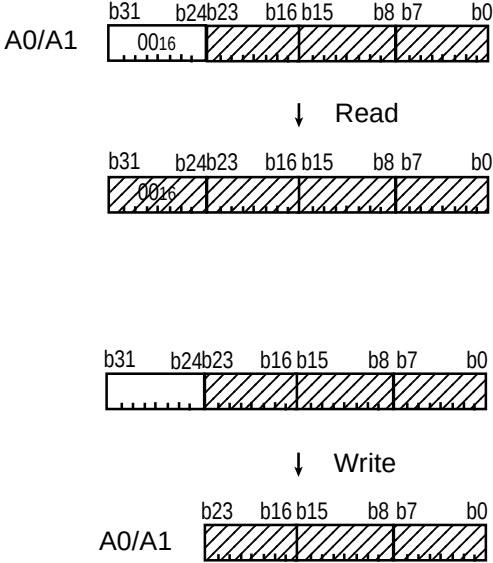
[Transfer instruction]

src is zero-expanded to 16 bits and saved to the low-order 16-bit. In this case, the 8 high-order bits become 0. The flags change states depending on the result of 16-bit transfer data.

[Operating instructions]

src is zero-expanded to perform operation in 16-bit. In this case, the 8 high-order bits become 0. The flags change states depending on the result of 16-bit operation.



When (.W) is specified for the size specifier (.size) • Read The low order 16-bit are read. The flags change states depending on the result of 16-bit operation. • Write Write to the low order 16-bit. In this case, the 8 high-order bits become 0. The flags change states depending on the result of 16-bit transfer data.	 The diagram illustrates the read and write operations for the .W size specifier. It shows a 24-bit register A0/A1 with bits labeled b23, b16, b15, b8, b7, and b0. For the Read operation, the low 16 bits (b15 to b0) are read. For the Write operation, the low 16 bits (b15 to b0) are written, and the high 8 bits (b23 to b16) are zeroed out.
When (.L) is specified for the size specifier (.size) • Read 32 bits are read out after being zero-extended. The flag varies depending on the result of a 32-bit operation. • Write The low-order 24-bit is written, with the 8 high-order bit ignored. The flag varies depending on the result of a 32-bit operation (not the value of the 24-bit register). Example: <code>MOV.L#80000000h,A0</code> Flag status after execution S flag = 1 (The MSB is bit 31.) Z flag = 0 (Set to 1 when all of 32 bits are 0s.) The value of A0 after executing the above instruction becomes 000000₁₆. However, since operation is performed on 32-bit data, the S flag is set to 1 and the Z flag is cleared to 0.	Zero-expanded  The diagram illustrates the zero-expanded read and write operations for the .L size specifier. It shows a 32-bit register A0/A1 with bits labeled b31, b24, b23, b16, b15, b8, b7, and b0. For the Read operation, the low 24 bits (b23 to b0) are read, and the high 8 bits (b31 to b24) are zero-extended. For the Write operation, the low 24 bits (b23 to b0) are written, and the high 8 bits (b31 to b24) are zeroed out.

Chapter 3

Functions

3.1 Guide to This Chapter

3.2 Functions

3.3 Index Instructions

3.1 Guide to This Chapter

This chapter describes the functionality of each instruction by showing syntax, operation, function, selectable src/dest, flag changes, and description examples.

The following shows how to read this chapter by using an actual page as an example.

Chapter 3 Functions

3.2 Functions

(1) **OR**

(2) [Syntax]

(3) OR.size (:format) src,dest

(4) [Operation]

(5) [Function]

(6) [Selectable src/dest]

(7) [Flag Change]

(8) [Description Example]

OR
 [Instruction Code/Number of Cycles]

Logically OR

OR

Page=260

G, S (Can be specified)
B, W

dest ← src ∨ dest
 dest ← [src] ∨ dest

[dest] ← src ∨ [dest]
 [dest] ← [src] ∨ [dest]

- This instruction logically ORs *dest* and *src* together and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

(See the next page for src/dest classified by format.)

src				dest			
R0L/R0/R2R0	R0H/R2/-	R1L/R1/R3R1	R1H/R3/-	R0L/R0/R2R0	R0H/R2/-	R1L/R1/R3R1	R1H/R3/-
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, and #IMM.

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

S : The flag is set when the transfer resulted in MSB of *dest* = 1; otherwise cleared.

Z : The flag is set when the transfer resulted in 0; otherwise cleared.

OR.B Ram:8[SB],R0L ; A0's 8 low-order bits and R0L are ORed.

OR.B:G A0,R0L ; R0L is zero-expanded and ORed with A0.

OR.B:G R0L,A0

OR.B:S #3,R0L

OR.W:G [R1],[A0]

115

(1) Mnemonic

Indicates the mnemonic explained in this page.

(2) Instruction code/number of cycles

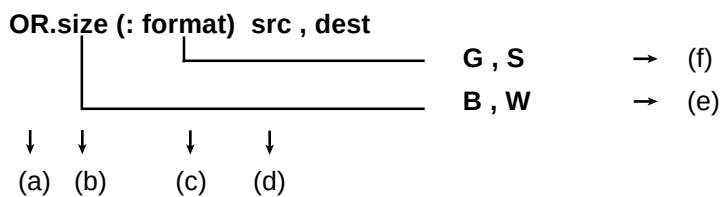
Indicates the page in which instruction code/number of cycles is listed.

Refer to this page for instruction code and number of cycles.

(3) Syntax

Indicates the syntax of the instruction using symbols. If (:format) is omitted, the assembler chooses the optimum specifier.

OR.size (: format) src , dest



G , S → (f)
B , W → (e)

↓ ↓ ↓ ↓
 (a) (b) (c) (d)

(a) Mnemonic **OR**

Describes the mnemonic.

(b) Size specifier **size**

Describes the data size in which data is handled. The following lists the data sizes that can be specified:

- .B Byte (8 bits)
- .W Word (16 bits)
- .L Long word (32 bits)

Some instructions do not have a size specifier.

(c) Instruction format specifier (: format**)**

Describes the instruction format. If (.format) is omitted, the assembler chooses the optimum specifier. If (.format) is entered, its content is given priority. The following lists the instruction formats that can be specified:

- :G Generic format
- :Q Quick format
- :S Short format
- :Z Zero format

Some instructions do not have an instruction format specifier.

(d) Operand **src, dest**

Describes the operand.

(e) Indicates the data size you can specify in (b).**(f) Indicates the instruction format you can specify in (c).**

Chapter 3 Functions

3.2 Functions

(1) **OR**

(2) [Syntax]

(3) **OR.size (:format) src,dest**

(4) [Operation]

(5) [Function]

(6) [Selectable src/dest]

(7) [Flag Change]

(8) [Description Example]

Logically OR

OR [Instruction Code/Number of Cycles]

Page=260

G, S (Can be specified)
W, B

dest ← src ∨ dest [dest] ← src ∨ [dest]
dest ← [src] ∨ dest [dest] ← [src] ∨ [dest]

- This instruction logically ORs *dest* and *src* together and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

src				dest			
R0L/R0/R2R0	R0H/R2/-	R0L/R0/R2R0	R0H/R2/-				
R1L/R1/R3R1	R1H/R3/-	R1L/R1/R3R1	R1H/R3/-				
A0/A0/A0	A1/A1/A1	A0/A0/A0	A1/A1/A1				
dsp:8[A0]	dsp:8[A1]	dsp:8[A0]	dsp:8[A1]				
dsp:16[A0]	dsp:16[A1]	dsp:16[A0]	dsp:16[A1]				
dsp:24[A0]	dsp:24[A1]	dsp:24[A0]	dsp:24[A1]				
abs24	abs16	abs24	abs16				
#IMM8/#IMM16							

*1 Indirect addressing [src]and[dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, and #IMM.

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

S : The flag is set when the transfer resulted in MSB of *dest* = 1; otherwise cleared.

Z : The flag is set when the transfer resulted in 0; otherwise cleared.

OR.B Ram:8[SB],R0L

OR.B:G A0,R0L ; A0' s 8 low-order bits and R0L are ORed.

OR.B:G R0L,A0 ; R0L is zero-expanded and ORed with A0.

OR.B:S #3,R0L

OR.W:G [R1],[A0]

115

(4) Operation

Explains the operation of the instruction using symbols.

(5) Function

Explains the function of the instruction and precautions to be taken when using the instruction.

(6) Selectable *src* / *dest* (label)

If the instruction has an operand, this indicates the format you can choose for the operand.

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

(a) Items that can be selected as *src* (source).

(b) Items that can be selected as *dest* (destination).

(c) Addressing that cannot be selected.

(d) Addressing that can be selected.

(e) Shown on the left side of the slash (R0L) is the addressing when data is handled in bytes (8 bits).
 Shown on the middle side of the slash (R0) is the addressing when data is handled in words (16 bits).
 Shown on the right side of the slash (R2R0) is the addressing when data is handled in words (32 bits).

(7) Flag change

Indicates a flag change that occurs after the instruction is executed. The symbols in the table mean the following:

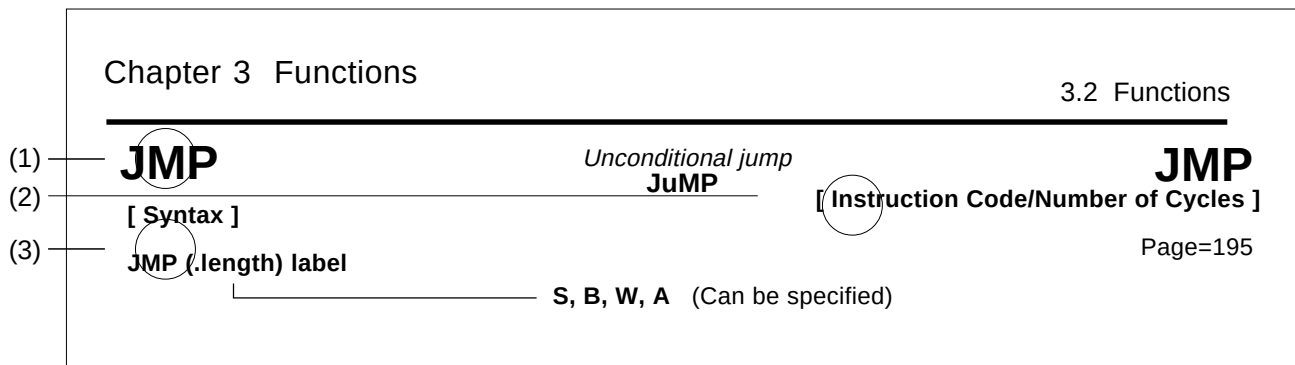
"—" The flag does not change.

"○" The flag changes depending on condition.

(8) Description example

Shows a description example for the instruction.

The following explains the syntax of each jump instruction `JMP`, `JPMI`, `JSR`, and `JSRI` by using an actual example.



(3) Syntax

Indicates the instruction syntax using a symbol.

JMP (.length) label

↓ ↓ ↓

(a) (b) (c)

S, B, W, A → (d)

(a) Mnemonic **JMP**

Describes the mnemonic.

(b) Jump distance specifier **.length**

Describes the distance of jump. If `(.length)` is omitted in `JMP` or `JSR` instruction, the assembler chooses the optimum specifier. If `(.length)` is entered, its content is given priority.

The following lists the jump distances that can be specified:

- .S 3-bit PC forward relative (+2 to +9)
- .B 8-bit PC relative
- .W 16-bit PC relative
- .A 24-bit absolute

(c) Operand **label**

Describes the operand.

(d) Shows the jump distance that can be specified in (b).

ABS

Absolute value
ABSolute

ABS

[Syntax]

ABS.size dest
 └──────────────────┘ B , W

[Instruction Code/Number of Cycles]

Page= 174

[Operation]

dest ← | dest |
 [dest] ← | [dest] |

[Function]

- This instruction takes on an absolute value of *dest* and stores it in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), the 8 high-order bits become 0.

[Selectable dest]

dest*1			
R0L/R0/ R2 R0		R0H/R2/-	
R1L/R1/ R3 R1		R1H/R3/-	
A0 /A0/ A0	A1 /A1/ A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O : The flag is set (= 1) when *dest* before the operation is -128 (.B) or -32768 (.W); otherwise cleared (= 0).
- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is indeterminate.

[Description Example]

ABS.B R0L
 ABS.W [A0]
 ABS.W [[A0]]

ADC

Add with carry

ADC

[Syntax]

[Instruction Code/Number of Cycles]

```
ADC.size  src,dest
└────────────────── B , W
```

Page= 174

[Operation]

$$\text{dest} \leftarrow \text{src} + \text{dest} + C$$

[Function]

- This instruction adds *dest*, *src* and C flag together and stores the result in *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0. Also, when *src* is the address register, the 8 low-order bits of the address register are used as data to be operated on.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest]

src				dest			
R0L/R0/ R2R0		R0H/R2/ -		R0L/R0/ R2R0		R0H/R2/ -	
R1L/R1/ R3R1		R1H/R3/ -		R1L/R1/ R3R1		R1H/R3/ -	
A0/A0/ A0 *1	A1/A1/ A1 *1	[A0]	[A1]	A0/A0/ A0 *1	A1/A1/ A1 *1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O : The flag is set when a signed operation resulted in exceeding +32767 (.W) or - 32768 (.W) or +127 (.B) or -128 (.B); otherwise cleared.
- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when an unsigned operation resulted in exceeding +65535 (.W) or +255 (.B); otherwise cleared.

[Description Example]

ADC.B #2,R0L

ADC.W A0,R0

ADC.B A0.R0L

ADC.B R0L.A0

ADC.W R1,[A1]

; A0's 8 low-order bits and R0L are added.

; R0L is zero-expanded and added with A0.

ADCF

Add carry flag
ADdition Carry Flag

ADCF

[Syntax]

[Instruction Code/Number of Cycles]

```
ADCF.size  dest
|
|_____ B , W
```

Page= 176

[Operation]

$$\text{dest} \leftarrow \text{dest} + C$$
$$[\text{dest}] \leftarrow [\text{dest}] + C$$

[Function]

- This instruction adds *dest* and C flag together and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), the 8 high-order bits become 0.

[Selectable dest]

dest*1			
R0L/R0/ R2R0		R0H/R2/ -	
R1L/R1/ R3R1		R1H/R3/ -	
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O : The flag is set when a signed operation resulted in exceeding +32767 (.W) or -32768 (.W) or +127 (.B) or -128 (.B); otherwise cleared.
- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when an unsigned operation resulted in exceeding +65535 (.W) or +255 (.B); otherwise cleared.

[Description Example]

ADCF.B R0L

ADCF.W Ram:16[A0]

ADD

Add without carry
ADDition

ADD

[Syntax]

ADD.size (:format)

src,dest

[Instruction Code/Number of Cycles]

Page= 176

_____ G , Q , S (Can be specified)
 _____ B , W , L

[Operation]

dest ← dest + src [dest] ← [dest] + src

dest ← dest + [src] [dest] ← [dest] + [src]

[Function]

- This instruction adds *dest* and *src* together and stores the result in *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0. Also, when *src* is the address register, the 8 low-order bits of the address register are used as data to be operated on.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.
- When (.L) is specified for the size specifier (.size) and *dest* is the address register, *dest* is zero-extended to perform operation in 32 bits. The 24 low-order bits of the operation result are stored in *dest*. Also, when *src* is the address register, *src* is zero-extended to perform operation in 32bit. The flags also change states depending on the result of 32-bit operation.
- When (.L) is specified for the size specifier (.size) and *dest* is SP, *dest* is zero-extended to perform operation in 32 bits, and *src* is sign-extended to perform operation in 32 bits. The 24 low-order bits of the operation result are stored in *dest*. The flags also change states depending on the result of 32-bit operation.

[Selectable src/dest]*1(See the next page for *src/dest* classified by format.)

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]	A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM				SP/SP/SP*3			

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

*3 Operation is performed on the stack pointer indicated by the U flag.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O : The flag is set when a signed operation resulted in exceeding +2147483647(.L) or -2147483648(.L), +32767 (.W) or -32768 (.W), or +127 (.B) or -128 (.B); otherwise cleared.
- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when an unsigned operation resulted in exceeding +4294967295(.L) or +65535 (.W) or +255 (.B); otherwise cleared.

[Description Example]

ADD.B [[A0]],abs16

[src/dest Classified by Format]**G format*1**

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]	A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16/#IMM32				SP/SP/SP*3			

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

*3 Operation is performed on the stack pointer indicated by the U flag. You can choose only #IMM16 for *src*. You can choose only (.L) for the size specifier (.size). In this case, you cannot use the indirect addressing mode.

Q format*4

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM3*6/#IMM4*7				SP/SP/SP*5			

*4 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*5 Operation is performed on the stack pointer indicated by the U flag. You can choose only #IMM3 for *src*.

*6 When *dest* is the SP, #IMM3 can be selected. The range of values that can be taken on is $+1 \leq \#IMM3 \leq +8$.

*7 When *dest* is not the SP, #IMM4 can be selected. The range of values that can be taken on is $-8 \leq \#IMM4 \leq +7$.

S format*8

src				dest			
R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16	R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16
#IMM8/#IMM16*9							
#1*10		#2*10		A0*10		A1*10	
#IMM8*10				SP*10			

*8 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*9 You can choose the (.B) and (.W) for the size specifier (.size).

*10 You can choose only (.L) for the size specifier (.size). In this case, you cannot use the indirect addressing mode.

ADDX

Add extend sign without carry
ADDition eXtend sign

ADDX

[Syntax]

ADDX **src,dest**

[Instruction Code/Number of Cycles]

Page=183

[Operation]

dest ← dest + EXTS(src) [dest] ← [dest] + EXTS(src)
 dest ← dest + EXTS([src]) [dest] ← [dest] + EXTS([src])

[Function]

- Sign-extend the 8-bit *src* to 32 bits which are added to the 32-bit *dest*, and the result is stored in *dest*.
- When *dest* is the address register(A0, A1) , *dest* is zero-extended to perform operation in 32 bits. The 24 low-order bits of the operation result are stored in *dest*. The flags also change states depending on the result of 32-bit operation. Also, when *src* is the address register, *src* is zero-extended to perform operation in 8 low-order bits.

[Selectable src/dest]*1

src				dest			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-		R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8							

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O** : The flag is set when a signed operation resulted in exceeding +2147483647(.L) or -2147483648(.L); otherwise cleared.
- S** : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z** : The flag is set when the operation resulted in 0; otherwise cleared.
- C** : The flag is set when an unsigned operation resulted in exceeding +4294967295(.L); otherwise cleared.

[Description Example]

ADDX R0L,A0
 ADDX RAM:8[SB],R2R0
 ADDX [A0],A1

ADJNZ

Add & conditional jump

ADdition then JUMP on Not Zero

ADJNZ

[Syntax]

[Instruction Code/Number of Cycles]

```
ADJNZ.size      src,dest,label
└──────────────────────────────────┘ B , W
```

Page= 185

[Operation]

```
dest ← dest + src
if dest ≠ 0 then jump label
```

[Function]

- This instruction adds *dest* and *src* together and stores the result in *dest*.
- When the addition resulted in any value other than 0, control jumps to **label**. When the addition resulted in 0, the next instruction is executed.
- The op-code of this instruction is the same as that of SBJNZ.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), the 8 high-order bits become 0.

[Selectable src/dest/label]

src	dest				label
#IMM4 ¹	R0L/R0/ R2R0		R0H/R2 ←		PC ² -126 ≤ label ≤ PC ² +129
	R1L/R1/ R3R1		R1H/R3 ←		
	A0/A0/A0	A1/A1/A1	[A0]	[A1]	
	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	
	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	
	dsp:24[A0]	dsp:24[A1]	abs24	abs16	

*1 The range of values that can be taken on is $-8 \leq \#IMM4 \leq +7$.

*2 PC indicates the start address of the instruction.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

ADJNZ.W #-1,R0,label

AND

Logically AND
AND

AND

[Syntax]

AND.size (:format)

src,dest

[Instruction Code/Number of Cycles]

Page=186

[Operation]

$\text{dest} \leftarrow \text{src} \wedge \text{dest}$ $[\text{dest}] \leftarrow \text{src} \wedge [\text{dest}]$
 $\text{dest} \leftarrow [\text{src}] \wedge \text{dest}$ $[\text{dest}] \leftarrow [\text{src}] \wedge [\text{dest}]$

[Function]

- This instruction logically ANDs *dest* and *src* together and stores the result in *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0. Also, when *src* is the address register, the 8 low-order bits of the address register are used as data to be operated on.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest] *1(See the next page for *src/dest* classified by format.)

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R2R0	R1H/R3/-			R1L/R1/R2R0	R1H/R3/-		
A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]	A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP] and #IMM.

*2 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.

Z : The flag is set when the operation resulted in 0; otherwise cleared.

[Description Example]

AND.B Ram:8[SB],R0L

AND.B:G A0,R0L

; A0's 8 low-order bits and R0L are ANDed.

AND.B:G R0L,A0

; R0L is zero-expanded and ANDed with A0.

AND.B:S #3,R0L

AND.W:G [A0],[A1]

[src/dest Classified by Format]**G format^{*1}**

src				dest			
R0L/R0/ R2R0		R0H/R2/ -		R0L/R0/ R2R0		R0H/R2/ -	
R1L/R1/ R2R0		R1H/R3/ -		R1L/R1/ R2R0		R1H/R3/ -	
A0/A0/ A0 ^{*2}	A1/A1/ A1 ^{*2}	[A0]	[A1]	A0/A0/ A0 ^{*2}	A1/A1/ A1 ^{*2}	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

^{*1} Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

^{*2} When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

S format^{*3}

src				dest			
R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16	R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16
#IMM8/#IMM16							

^{*3} Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

BAND

Logically AND bits
Bit AND carry flag

BAND

[Syntax]

BAND src

[Instruction Code/Number of Cycles]

Page=188

[Operation]

$C \leftarrow src \wedge C$

[Function]

- This instruction logically ANDs the C flag and *src* together and stores the result in the C flag.
- When *src* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.

[Selectable src]

src			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	○

Conditions

C : The flag is set when the operation resulted in 1; otherwise cleared.

[Description Example]

- BAND flag
- BAND 4,Ram
- BAND 16,Ram:19[SB]
- BAND 5,[A0]

BCLR

Clear bit
Bit CLear

BCLR

[Syntax]

BCLR **dest**

[Instruction Code/Number of Cycles]

Page= 188

[Operation]

dest ← 0

[Function]

- This instruction stores 0 in *dest*.
- When *dest* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.

[Selectable dest]

dest			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

BCLR flag
 BCLR 4,Ram
 BCLR 16,Ram:19[SB]
 BCLR 5,[A0]

BITINDEX

Bit index
BIT INDEX

BITINDEX

[Syntax]

```
BITINDEX.size      src
                  |
                  +----- B , W
```

[Instruction Code/Number of Cycles]

Page= 189

[Operation]

[Function]

- This instruction modifies addressing of the next bit instruction.
- No interrupt request is accepted immediately after this instruction.
- The operand specified in *src* constitutes the *src* or *dest* index value for the next bit instruction.
- For details, refer to Section 3.3, "Index Instructions."

[Selectable src]

src			
R0L/R0/ R2R0		R0H/R2/ R0	
R1L/R1/ R3R1		R1H/R3/ R1	
A0/A0/ A0	A1/A1/ A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs:24	abs:16

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

BITINDEX R0

BITINDEX [A0]

BM*Cnd**Conditional bit transfer*
Bit Move Condition**BM*Cnd*****[Syntax]****BM*Cnd*** **dest****[Instruction Code/Number of Cycles]**

Page=190

[Operation]

if true then **dest** ← 1
else **dest** ← 0

[Function]

- This instruction transfers the true or false value of the condition indicated by *Cnd* to *dest*. When the condition is true, 1 is transferred; when false, 0 is transferred.
- When *dest* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.
- There are following kinds of *Cnd*.

<i>Cnd</i>	Condition		Expression	<i>Cnd</i>	Condition		Expression
GEU/C	C=1	Equal to or greater than C flag is 1.	$\geq$	LTU/NC	C=0	Smaller than C flag is 0.	$>$
EQ/Z	Z=1	Equal to Z flag is 1.	$=$	NE/NZ	Z=0	Not equal Z flag is 0.	$\neq$
GTU	$C \wedge \bar{Z}=1$	Greater than	$<$	LEU	$C \wedge \bar{Z}=0$	Equal to or smaller than	$\geq$
PZ	S=0	Positive or zero	$0 \leq$	N	S=1	Negative	$0 >$
GE	$S \vee O=0$	Equal to or greater than (signed value)	$\geq$	LE	$(S \vee O) \vee Z=1$	Equal to or smaller than (signed value)	$\geq$
GT	$(S \vee O) \vee Z=0$	Greater than (signed value)	$<$	LT	$S \vee O=1$	Smaller than (signed value)	$>$
O	O=1	O flag is 1.		NO	O=0	O flag is 0.	

[Selectable dest]

dest			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19
C			

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	*1

*1 The flag changes when you specified the C flag for *dest*.**[Description Example]**

BMN 3,Ram:11[SB]
 BMZ C

BNAND

Logically AND inverted bits
Bit Not AND carry flag

BNAND

[Syntax]

BNAND **src**

[Instruction Code/Number of Cycles]

Page=192

[Operation]

$C \leftarrow \overline{\text{src}} \vee C$

[Function]

- This instruction logically ANDs the C flag and inverted *src* together and stores the result in the C flag.
- When *src* is the address register (A0, A1), you can specify the 8 low-order bits for address register.

[Selectable src]

src			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	○

Condition

C : The flag is set when the operation resulted in 1; otherwise cleared.

[Description Example]

BNAND flag
 BNAND 4,Ram
 BNAND 16,Ram:19[SB]
 BNAND 5,[A0]

BNOR

Logically OR inverted bits
Bit Not OR carry flag

BNOR

[Syntax]

BNOR **src**

[Instruction Code/Number of Cycles]

Page= 192

[Operation]

$C \leftarrow \overline{src} \vee C$

[Function]

- This instruction logically ORs the C flag and inverted *src* together and stores the result in the C flag.
- When *src* is the address register (A0, A1), you can specify the 8 low-order bits for address register.

[Selectable src]

src			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	○

Condition

C : The flag is set when the operation resulted in 1; otherwise cleared.

[Description Example]

BNOR flag
 BNOR 4,Ram
 BNOR 16,Ram:19[SB]
 BNOR 5,[A0]

BNOT

Invert bit
Bit NOT

BNOT

[Syntax]

BNOT dest

[Instruction Code/Number of Cycles]

Page=193

[Operation]

dest ← $\overline{\text{dest}}$

[Function]

- This instruction inverts *dest* and stores the result in *dest*.
- When *dest* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.

[Selectable dest]

dest			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

BNOT flag
BNOT 4,Ram
BNOT 16,Ram:19[SB]
BNOT 5,[A0]

BNTST

Test inverted bit
Bit Not TeST

BNTST

[Syntax]

BNTST *src*

[Instruction Code/Number of Cycles]

Page= 193

[Operation]

$Z \leftarrow \overline{\text{src}}$

$C \leftarrow \overline{\text{src}}$

[Function]

- This instruction transfers inverted *src* to the Z flag and inverted *src* to the C flag.
- When *src* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.

[Selectable src]

src			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	○	—	○

Conditions

- Z : The flag is set when *src* is 0; otherwise cleared.
- C : The flag is set when *src* is 0; otherwise cleared.

[Description Example]

BNTST flag

BNTST 4,Ram

BNTST 16,Ram:19[SB]

BNTST 5,[A0]

BNXOR

Exclusive OR inverted bits
Bit Not eXclusive OR carry flag

BNXOR

[Syntax]

BNXOR src

[Instruction Code/Number of Cycles]

Page= 194

[Operation]

$C \leftarrow \overline{src} \vee C$

[Function]

- This instruction exclusive ORs the C flag and inverted *src* and stores the result in the C flag.
- When *src* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.

[Selectable src]

src			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	○

Conditions

C : The flag is set when the operation resulted in 1; otherwise cleared.

[Description Example]

BNXOR flag
BNXOR 4,Ram
BNXOR 16,Ram:19[SB]
BNXOR 5,[A0]

BOR

Logically OR bits
Bit OR carry flag

BOR

[Syntax]

BOR *src*

[Instruction Code/Number of Cycles]

Page= 194

[Operation]

$C \leftarrow src \vee C$

[Function]

- This instruction logically ORs the C flag and *src* together and stores the result in the C flag.
- When *src* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.

[Selectable src]

src			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	○

Conditions

C : The flag is set when the operation resulted in 1; otherwise cleared.

[Description Example]

BOR flag
 BOR 4,Ram
 BOR 16,Ram:19[SB]
 BOR 5,[A0]

BRK

Debug interrupt
BReaK

BRK

[Syntax]
BRK

[Instruction Code/Number of Cycles]
Page= 195

[Operation]

- When anything other than FF₁₆ exists in addresses from FFFFE4₁₆ to FFFFE7₁₆
SP ← SP - 2
M(SP) ← FLG
SP ← SP - 2
M(SP)^{*1} ← (PC + 1)_H
SP ← SP - 2
M(SP) ← (PC + 1)_{ML}
PC ← M(FFFFE4₁₆)

^{*1} The 8 high-order bits become indeterminate.

- When FF₁₆ exists in all addresses from FFFFE4₁₆ to FFFFE7₁₆
SP ← SP - 2
M(SP) ← FLG
SP ← SP - 2
M(SP)^{*2} ← (PC + 1)_H
SP ← SP - 2
M(SP) ← (PC + 1)_{ML}
PC ← M(IntBase)

^{*2} The 8 high-order bits become indeterminate.

[Function]

- This instruction generates a BRK interrupt.
- The BRK interrupt is a nonmaskable interrupt.

[Flag Change]^{*1}

Flag	U	I	O	B	S	Z	D	C
Change	○	○	—	—	—	—	○	—

Conditions

- U : The flag is cleared.
- I : The flag is cleared.
- D : The flag is cleared.

^{*1} The flags are saved to the stack area before the BRK instruction is executed. After the interrupt, the flags change state as shown on the left.

[Description Example]

BRK

BRK2

Debug interrupt2
BReak2

BRK2

[Syntax]
BRK

[Instruction Code/Number of Cycles]
Page=195

[Operation]

SP ← SP - 2
M(SP) ← FLG
SP ← SP - 2
M(SP)*1 ← (PC + 1)H
SP ← SP - 2
M(SP) ← (PC + 1)ML
PC ← M(0020₁₆)

*1 The 8 high-order bits become indeterminate.

- [Function]
- This instruction is provided for exclusive use in debuggers. Do not use it in user programs.
 - A BRK2 interrupt is generated.
 - The BRK2 interrupt is a nonmaskable interrupt.

[Flag Change]*1

Flag	U	I	O	B	S	Z	D	C
Change	○	○	—	—	—	—	○	—

Conditions

- U : The flag is cleared.
I : The flag is cleared.
D : The flag is cleared.

*1 The flags are saved to the stack area before the BRK2 instruction is executed. After the interrupt, the flags change state as shown on the left.

[Description Example]
BRK2

BSET

Set bit
Bit SET

BSET

[Syntax]

BSET dest

[Instruction Code/Number of Cycles]

Page= 196

[Operation]

dest ← 1

[Function]

- This instruction stores 1 in *dest*.
- When *dest* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.

[Selectable dest]

dest			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

BSET flag
BSET 4,Ram
BSET 16,Ram:19[SB]
BSET 5,[A0]

BTST

Test bit
Bit TeST

BTST

[Syntax]

BTST (:format) src

[Instruction Code/Number of Cycles]

Page= 196

G , S (Can be specified)

[Operation]

$Z \leftarrow \overline{\text{src}}$

$C \leftarrow \text{src}$

[Function]

- This instruction transfers inverted *src* to the Z flag and non-inverted *src* to the C flag.
- When *src* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.

[Selectable src]

G format*1

src			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

S format

src
bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	○	—	○

Conditions

- Z : The flag is set when *src* is 0; otherwise cleared.
- C : The flag is set when *src* is 1; otherwise cleared.

[Description Example]

BTST flag

BTST 4,Ram

BTST 16,Ram:19[SB]

BTST 5,[A0]

BTSTC

Test bit & clear
Bit TeST & Clear

BTSTC

[Syntax]

BTSTC **dest**

[Instruction Code/Number of Cycles]

Page= 197

[Operation]

$Z \leftarrow \overline{\text{dest}}$
 $C \leftarrow \text{dest}$
 $\text{dest} \leftarrow 0$

[Function]

- This instruction transfers inverted *dest* to the Z flag and non-inverted *dest* to the C flag. Then it stores 0 in *dest*.
- When *dest* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.
- Do not use this instruction for *dest* in SFR area.

[Selectable dest]

dest			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	○	—	○

Conditions

- Z** : The flag is set when *dest* is 0; otherwise cleared.
C : The flag is set when *dest* is 1; otherwise cleared.

[Description Example]

BTSTC flag
 BTSTC 4,Ram
 BTSTC 16,Ram:19[SB]
 BTSTC 5,[A0]

BTSTS

Test bit & set
Bit TeST & Set

BTSTS

[Syntax]

BTSTS *dest*

[Instruction Code/Number of Cycles]

Page= 198

[Operation]

$Z \leftarrow \overline{\text{dest}}$
 $C \leftarrow \text{dest}$
 $\text{dest} \leftarrow 1$

[Function]

- This instruction transfers inverted *dest* to the Z flag and non-inverted *dest* to the C flag. Then it stores 1 in *dest*.
- When *dest* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.
- Do not use this instruction for *dest* in SFR area.

[Selectable dest]

dest			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	○	—	○

Conditions

- Z** : The flag is set when *dest* is 0; otherwise cleared.
C : The flag is set when *dest* is 1; otherwise cleared.

[Description Example]

BTSTS flag
 BTSTS 4,Ram
 BTSTS 16,Ram:19[SB]
 BTSTS 5,[A0]

BXOR

Exclusive OR bits
Bit eXclusive OR carry flag

BXOR

[Syntax]

BXOR src

[Instruction Code/Number of Cycles]

Page= 198

[Operation]

C ← src ∨ C

[Function]

- This instruction exclusive ORs the C flag and *src* together and stores the result in the C flag.
- When *src* is the address register (A0, A1), you can specify the 8 low-order bits for the address register.

[Selectable src]

src			
bit,R0L	bit,R0H	bit,R1L	bit,R1H
bit,A0	bit,A1	bit,[A0]	bit,[A1]
bit,base:11[A0]	bit,base:11[A1]	bit,base:11[SB]	bit,base:11[FB]
bit,base:19[A0]	bit,base:19[A1]	bit,base:19[SB]	bit,base:19[FB]
bit,base:27[A0]	bit,base:27[A1]	bit,base:27	bit,base:19

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	○

Conditions

C : The flag is set when the operation resulted in 1; otherwise cleared.

[Description Example]

BXOR flag
BXOR 4,Ram
BXOR 16,Ram:19[SB]
BXOR 5,[A0]

CLIP

CLIP
CLIP

CLIP

[Syntax]

CLIP.size src1, src2, dest

B, W

[Instruction Code/Number of Cycles]

Page= 199

[Operation]

```

if      src1 > dest
then    dest ← src1
if      src2 < dest
then    dest ← src2

```

[Function]

- Signed compares src1 and *dest* and stores the content of src1 in *dest* if src1 is greater than *dest*. Next, signed compares src2 and *dest* and stores the content of src2 in *dest* if src2 is smaller than *dest*. When $\text{src1} \leq \text{dest} \leq \text{src2}$, *dest* is not changed.
- When (.W) is specified for the size specifier (.size), *dest* is the address register and writing to *dest*, the 8 high-order bits become 0.
- Src1 and src2 are set "src1<src2".

[Selectable src/dest/label]

src1, src2				dest			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-		R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

```

CLIP.W    #5,#10,R1
CLIP.W    #-5,#5,[A0]

```

CMP

Compare
CoMPare

CMP

[Syntax]

CMP.size (:format) src,dest

_____ **G , Q , S** (Can be specified)
_____ **B , W , L**

[Instruction Code/Number of Cycles]

Page= 200

[Operation]

dest - src [dest] - src
dest - [src] [dest] - [src]

[Function]

- Each flag bit of the flag register varies depending on the result of subtraction of *src* from *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. Also, when *src* is the address register, the 8 low-order bits of the address register are used as data to be operated on.
- When (.L) is specified for the size specifier (.size), and *src* or *dest* is the address register, address register is zero-extended to perform operation in 32 bits. The flags also change states depending on the result of 32-bit operation.

[Selectable src/dest]*1(See the next page for *src/dest* classified by format.)

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]	A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM4/#IMM8/#IMM16/#IMM32							

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 If you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O** : The flag is set when a signed operation resulted in exceeding +2147483647(.L) or -2147483648(.L), +32767 (.W) or -32768 (.W), or +127 (.B) or -128 (.B); otherwise cleared.
- S** : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z** : The flag is set when the operation resulted in 0; otherwise cleared.
- C** : The flag is set when an unsigned operation resulted in any value equal to or greater than 0; otherwise cleared.

[Description Example]

CMP.B:S #10,R0L
 CMP.W:G R0,A0
 CMP.W # -3,R0
 CMP.B #5,Ram:8[FB]
 CMP.B A0,R0L ; A0's 8 low-order bits and R0L are compared.

[src/dest Classified by Format]**G format*¹**

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0* ²	A1/A1/A1* ²	[A0]	[A1]	A0/A0/A0* ²	A1/A1/A1* ²	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM4/#IMM8/#IMM16/#IMM32							

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 If you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

Q format*^{3*4}

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM4* ⁵ /##IMM8/#IMM16/#IMM32							

*3 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*4 You can only specify (.B) or (.W) for the size specifier (.size).

*5 The range of values that can be taken on is $-8 \leq \text{\#IMM4} \leq +7$.

S format*^{6*7}

src				dest			
R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16	R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16
#IMM8/#IMM16							
R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16	R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16
#IMM							

*6 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*7 You can only specify (.B) or (.W) for the size specifier (.size).

CMPX

Compare extended sign
CoMPare eXtend sign

CMPX

[Syntax]

CMPX src,dest

[Instruction Code/Number of Cycles]

Page=206

[Operation]

dest/[dest] - EXTS(src)

[Function]

- Each flag of the flag register changes state according to the result derived by subtracting the sign-extended 32-bit *src* from the 32-bit *dest*.
- When *dest* is the address register (A0, A1), it is zero-extended to perform operation in 32 bits and the flags change their states depending on the result.

[Selectable src/dest]*1

src				dest			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-		R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8							

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O : The flag is set when a signed operation resulted in exceeding +2147483647(.L) or -2147483648(.L), otherwise cleared.
- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when an unsigned operation resulted in any value equal to or greater than 0; otherwise cleared.

[Description Example]

CMPX #10,R2R0
CMPX #5,A0

DADC

Decimal ADDition with Carry

DADC

[Syntax]

[Instruction Code/Number of Cycles]

DADC.size **src,dest**

_____ **B , W**

Page= 206

[Operation]

$$\text{dest} \leftarrow \text{src} + \text{dest} + C$$

[Function]

- This instruction adds *dest*, *src*, and C flag together in decimal and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest]

src				dest			
R0L/R0/ R2R0		R0H/R2/ -		R0L/R0/ R2R0		R0H/R2/ -	
R1L/R1/ R3R1		R1H/R3/ -		R1L/R1/ R3R1		R1H/R3/ -	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	○

Conditions

- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when the operation resulted in exceeding +9999 (.W) or +99 (.B); otherwise cleared.

[Description Example]

DADC.B #3,R0L

DADC.W R1,R0

DADC.W [A0],R2

DADD

Decimal ADDition

DADD

[Syntax]

DADD.size **src,dest**

B , W

[Instruction Code/Number of Cycles]

Page= 208

[Operation]

$$\text{dest} \leftarrow \text{src} + \text{dest}$$

[Function]

- This instruction adds *dest* and *src* together in decimal and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest]

src				dest			
R0L/R0/ R2R0		R0H/R2/ R0		R0L/R0/ R2R0		R0H/R2/ R0	
R1L/R1/ R3R1		R1H/R3/ R1		R1L/R1/ R3R1		R1H/R3/ R1	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	○

Conditions

- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when the operation resulted in exceeding +9999 (.W) or +99 (.B); otherwise cleared.

[Description Example]

```
DADD.B    #3,R0L
DADD.W    R1,R0
DADD.W    [A0],[A1]
```

DEC

Decrement
DECrement

DEC

[Syntax]

DEC.size dest
 └──────────────────┘ B , W

[Instruction Code/Number of Cycles]

Page= 210

[Operation]

dest ← dest - 1 [dest] ← [dest] - 1

[Function]

- This instruction decrements 1 from *dest* and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0.

[Selectable dest]

dest*1			
R0L/R0/ R2 R0		R0H/R2/ -	
R1L/R1/ R3 R1		R1H/R3/ -	
A0 /A0/ A0	A1 /A1/ A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
 Z : The flag is set when the operation resulted in 0; otherwise cleared.

[Description Example]

DEC.W A0
 DEC.B R0L
 DEC.W R0

DIV

Signed divide
DIVide

DIV

[Syntax]

DIV.size **src**

_____ **B , W**

[Instruction Code/Number of Cycles]

Page= 210

[Operation]

- When the size specifier (.size) is (.W)
R0 (quotient), R2 (remainder) $\leftarrow$ R2R0 $\div$ src/[src]
- When the size specifier (.size) is (.B)
R0L (quotient), R0H (remainder) $\leftarrow$ R0 $\div$ src/[src]

[Function]

- This instruction divides R2R0 (R0)*¹ by signed *src* and stores the quotient in R0 (R0L)*¹ and the remainder in R2 (R0H)*¹. The remainder has the same sign as the dividend. Shown in ()¹ are the registers that are operated on when you selected (.B) for the size specifier (.size).
- When (.B) is specified for the size specifier (.size) and *src* is the address register (A0, A1), the 8 low-order bits of the address register are used as data to be operated on. The O flag is set when the operation resulted in the quotient exceeding 8 bits or the divisor is 0. R0L and R0H is undefined.
- When (.W) is specified for the size specifier (.size) and *src* is the address register, the 16 low-order bits of the address register are the data to be operated on. The O flag is set when the operation resulted in the quotient exceeding 16 bits or the divisor is 0. R0 and R2 is undefined.

[Selectable src]

src*2			
R0L/R0/ R2R0		R0H/R2/ -	
R1L/R1/ R3R1		R1H/R3/ -	
A0/A0/ A0	A1/A1/ A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16			

*2 Indirect addressing [src] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	—	—	—	—

Conditions

- O : The flag is set when the operation resulted in the quotient exceeding 16 bits (.W) or 8 bits (.B) or the divisor is 0; otherwise cleared.

[Description Example]

DIV.B A0 ;A0's 8 low-order bits is the divisor.
 DIV.B #4
 DIV.W R0
 DIV.W [[A1]]

DIVU

Unsigned divide
DIVide Unsigned

DIVU

[Syntax]

[Instruction Code/Number of Cycles]

DIVU.size **src** _____ **B, W**

Page= 211

[Operation]

- When the size specifier (.size) is (.W)
 $R0 \text{ (quotient)}, R2 \text{ (remainder)} \leftarrow R2R0 \div \text{src}[\text{src}]$
- When the size specifier (.size) is (.B)
 $R0L \text{ (quotient)}, R0H \text{ (remainder)} \leftarrow R0 \div \text{src}[\text{src}]$

[Function]

- This instruction divides R2R0 (R0)*¹ by unsigned *src* and stores the quotient in R0 (R0L)*¹ and the remainder in R2 (R0H)*¹. Shown in ()¹ are the registers that are operated on when you selected (.B) for the size specifier (.size).
- When (.B) is specified for the size specifier (.size) and *src* is the address register (A0, A1), the 8 low-order bits of the address register are used as data to be operated on. The O flag is set when the operation resulted in the quotient exceeding 8 bits or the divisor is 0. R0L and R0H is undefined.
- When (.W) is specified for the size specifier (.size) and *src* is the address register, the 16 low-order bits of the address register are the data to be operated on. The O flag is set when the operation resulted in the quotient exceeding 16 bits or the divisor is 0. R0 and R2 is undefined.

[Selectable src]

src*2			
R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16			

*2 Indirect addressing [src] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-. SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	—	—	—	—

Conditions

- O : The flag is set when the operation resulted in the quotient exceeding 16 bits (.W) or 8 bits (.B) or the divisor is 0; otherwise cleared.

[Description Example]

DIVU.B	A0	;A0's 8 low-order bits is the divisor.
DIVU.B	#4	
DIVU.W	R0	
DIVU.W	[[A0]]	

DIVX

Singed divide
DIVide eXtension

DIVX

[Syntax]

DIVX.size **src** _____ **B , W**

[Instruction Code/Number of Cycles]

Page= 212

[Operation]

- When the size specifier (.size) is (.W)
 $R0 \text{ (quotient)}, R2 \text{ (remainder)} \leftarrow R2R0 \div \text{src}[\text{src}]$
- When the size specifier (.size) is (.B)
 $R0L \text{ (quotient)}, R0H \text{ (remainder)} \leftarrow R0 \div \text{src}[\text{src}]$

[Function]

- This instruction divides R2R0 (R0)*¹ by signed *src* and stores the quotient in R0 (R0L)*¹ and the remainder in R2 (R0H)*¹. The remainder has the same sign as the divisor. Shown in ()¹ are the registers that are operated on when you selected (.B) for the size specifier (.size).
- When (.B) is specified for the size specifier (.size) and *src* is the address register (A0, A1), the 8 low-order bits of the address register are used as data to be operated on. The O flag is set when the operation resulted in the quotient exceeding 8 bits or the divisor is 0. R0L and R0H is undefined.
- When (.W) is specified for the size specifier (.size) and *src* is the address register, the 16 low-order bits of the address register are the data to be operated on. The O flag is set when the operation resulted in the quotient exceeding 16 bits or the divisor is 0. R0 and R2 is undefined.

[Selectable src]

src*2			
R0L/R0	R2R0	R0H/R2	R2
R1L/R1	R3R1	R1H/R3	R3
A0/A0	A0	A1/A1	A1
[A0]		[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8	#IMM16		

*2 Indirect addressing [src] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	—	—	—	—

Conditions

- O : The flag is set when the operation resulted in the quotient exceeding 16 bits (.W) or 8 bits (.B) or the divisor is 0; otherwise cleared.

[Description Example]

DIVX.B A0 ; A0's 8 low-order bits is the divisor.
 DIVX.B #4
 DIVX.W R0

DSBB

Decimal SuBtract with Borrow

DSBB

[Syntax]

[Instruction Code/Number of Cycles]

DSBB.size **src,dest**

|_____ **B , W**

Page= 213

[Operation]

$$\text{dest} \leftarrow \text{dest} - \text{src} - \overline{C}$$

[Function]

- This instruction subtracts *src* and inverted C flag from *dest* in decimal and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest]

src				dest			
R0L/R0/ R2R0		R0H/R2/ -		R0L/R0/ R2R0		R0H/R2/ -	
R1L/R1/ R3R1		R1H/R3/ -		R1L/R1/ R3R1		R1H/R3/ -	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	○

Conditions

S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.

Z : The flag is set when the operation resulted in 0; otherwise cleared.

C : The flag is set when the operation resulted in any value equal to or greater than 0; otherwise cleared.

[Description Example]

DSBB.B #3,R0L

DSBB.W R1,R0

DSBB.W [A0],[A1]

DSUB

Decimal subtract without borrow
Decimal SUBtract

DSUB

[Syntax]

DSUB.size src,dest
 └──────────────────┘ B , W

[Instruction Code/Number of Cycles]

Page= 215

[Operation]

dest ← dest - src

[Function]

- This instruction subtracts *src* from *dest* in decimal and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest]

src				dest			
R0L/R0/ R2 R0		R0H/R2/ R0		R0L/R0/ R2 R0		R0H/R2/ R0	
R1L/R1/ R3 R1		R1H/R3/ R1		R1L/R1/ R3 R1		R1H/R3/ R1	
A0 /A0/ A0	A1 /A1/ A1	[A0]	[A1]	A0 /A0/ A0	A1 /A1/ A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	○

Conditions

- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when the operation resulted in any value equal to or greater than 0; otherwise cleared.

[Description Example]

DSUB.B #3,R0L
 DSUB.W R1,R0
 DSUB.W [A0],[A1]

ENTER

Build stack frame
ENTER function

ENTER

[Syntax]

ENTER src

[Instruction Code/Number of Cycles]

Page= 217

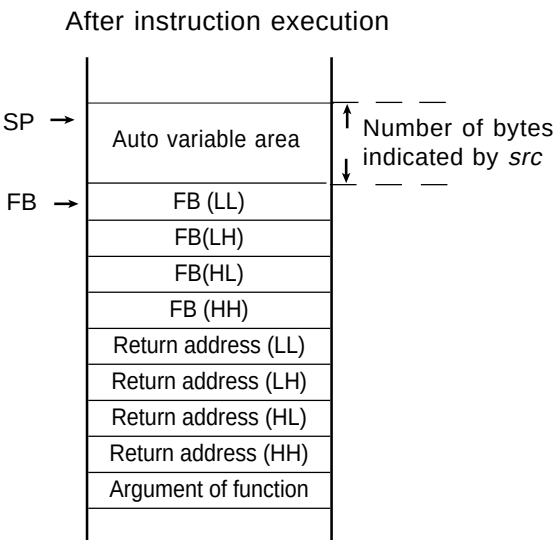
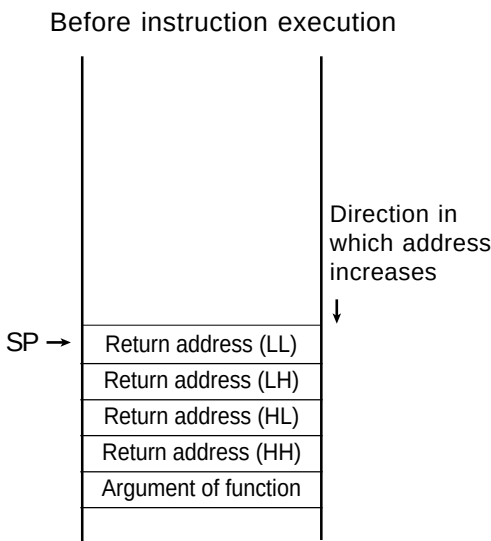
[Operation]

SP ← SP - 2
M(SP)*1 ← FBH
SP ← SP - 2
M(SP) ← FBL
FB ← SP
SP ← SP - src

*1 The 8 high-order bits become indeterminate.

[Function]

- This instruction generates a stack frame. *src* represents the size of the stack frame. Set an even number for *src*. (You can set odd number, but it is more effective to set even number for operation.)
- The diagrams below show the stack area status before and after the ENTER instruction is executed at the beginning of a called subroutine.



[Selectable src]

src
#IMM8

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

ENTER #4

EXITD

Deallocate stack frame EXIT and Deallocate stack frame

EXITD

[Syntax]

EXITD

[Instruction Code/Number of Cycles]

Page=217

[Operation]

```

SP      ← FB
FBL     ← M(SP)
SP      ← SP + 2
FBH     ← M(SP)
SP      ← SP + 2
PCL     ← M(SP)
SP      ← SP + 2
PCH     ← M(SP)*1
SP      ← SP + 2

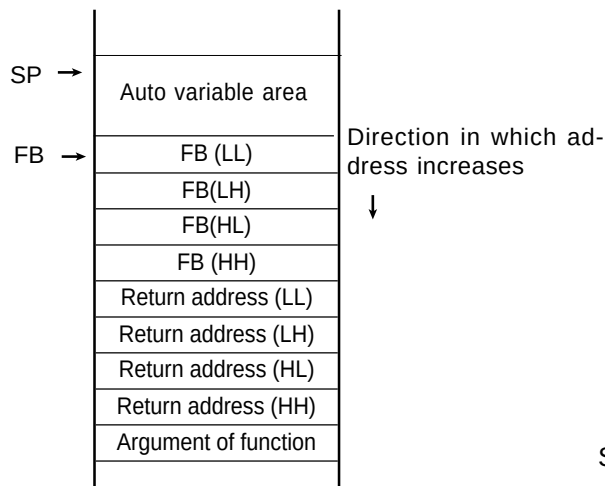
```

*1 The 8 high-order bits become indeterminate.

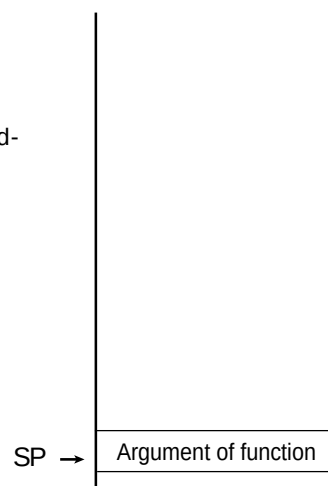
[Function]

- This instruction deallocates the stack frame and exits from the subroutine.
- Use this instruction in combination with the ENTER instruction.
- The diagrams below show the stack area status before and after the EXITD instruction is executed at the end of a subroutine in which an ENTER instruction was executed.

Before instruction execution



After instruction execution



[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

EXITD

EXTS

Extend sign
EXTend Sign

EXTS

[Syntax]

```
EXTS.size  dest
|
|_____ B , W

EXTS.size  src,dest
|
|_____ B
```

[Instruction Code/Number of Cycles]

Page= 218

[Operation]

```
dest ← EXTS(dest)
dest ← EXTS(src)
```

[Function]

- This instruction sign extends *dest* and stores the result in *dest*.
- When you selected (.B) for the size specifier (.size), *src* or *dest* is sign extended to 16 bits. When *dest* is the address register(A0, A1), the 8 high-order bits become 0.
- When you selected (.W) for the size specifier (.size), *dest* is sign extended to 32 bits. When R0 is selected for *dest*, R2 is used for the upper byte; when R1 is selected, R3 is used for the upper byte. When *dest* is the address register, stores the 24 low-order bits of result in *dest*.

[Selectable src/dest]

dest*1			
R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 You can only specify(.B) or (.W) for the size of specifier (.size).

src*2				dest*2			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-		R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16

*2 You can only specify(.B) for the size of specifier (.size).

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.

[Description Example]

EXTS.B	R0L
EXTS.W	R0
EXTS.W	[A0]

EXTZ

Extend zero
EXTend Zero

EXTZ

[Syntax]

EXTZ src,dest

[Instruction Code/Number of Cycles]

Page= 220

[Operation]

dest ← EXTZ(src)

[Function]

- This instruction zero-extends *src* to 16 bits and stores the result in *dest*. When *dest* is the address register(A0, A1), the 8 high-order bits become 0.

[Selectable src/dest]

src				dest			
R0L/ R0/R2 R0		R0H/ R2/-		R0L/R0/R2 R0		R0H/R2/-	
R1L/ R1/R3 R1		R1H/ R3/-		R1L/R1/R3 R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#MM							

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

- S : The flag is always cleared to 0.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.

[Description Example]

EXTZ R0L,R2
EXTZ [A1],[A0]

FCLR

Clear flag register bit
Flag register CLeaR

FCLR

[Syntax]

FCLR dest

[Instruction Code/Number of Cycles]

Page= 221

[Operation]

dest ← 0

[Function]

- This instruction stores 0 in *dest*.

[Selectable dest]

dest							
C	D	Z	S	B	O	I	U

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	*1	*1	*1	*1	*1	*1	*1	*1

*1 The selected flag is cleared to 0.

[Description Example]

FCLR I
FCLR S

FREIT

Fast return from Interrupt
Fast Return from **I**nTerrupt

FREIT

[Syntax]
FREIT

[Instruction Code/Number of Cycles]
Page= 221

[Operation]

FLG ← SVF
PC ← SVP

[Function]

- Restores the contents of PC and FLG from the high-speed interrupt registers that had been saved when accepting a high-speed interrupt request upon returning from the interrupt handler routine.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	*1	*1	*1	*1	*1	*1	*1	*1

*1 Becomes the content of SVF.

[Description Example]

FREIT

FSET

Set flag register bit
Flag register SET

FSET

[Syntax]

FSET dest

[Instruction Code/Number of Cycles]

Page=222

[Operation]

dest ← 1

[Function]

- This instruction stores 1 in *dest*.

[Selectable dest]

dest							
C	D	Z	S	B	O	I	U

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	*1	*1	*1	*1	*1	*1	*1	*1

*1 The selected flag is set (= 1).

[Description Example]

FSET I
FSET S

INC

Increment
INCrement

INC

[Syntax]

[Instruction Code/Number of Cycles]

Page= 223

INC.size **dest**

|_____ **B , W**

[Operation]

$$\text{dest} \leftarrow \text{dest} + 1 \qquad [\text{dest}] \leftarrow [\text{dest}] + 1$$

[Function]

- This instruction adds 1 to *dest* and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0.

[Selectable dest]

dest ^{*1}			
R0L/R0/ R2R0		R0H/R2/ R0	
R1L/R1/ R3R1		R1H/R3/ R1	
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

S : The flag is set when the operation resulted in $MSB = 1$; otherwise cleared.

Z : The flag is set when the operation resulted in 0; otherwise cleared.

[Description Example]

INC.W	A0
INC.B	R0L
INC.B	[[A1]]

INDEXType

Index
INDEX Type

INDEXType

[Syntax]

INDEXType.size src
 _____ B , W

[Instruction Code/Number of Cycles]

Page= 223

[Operation]**[Function]**

- This instruction modifies addressing of the next instruction.
- No interrupts are enabled until after the modifying instruction is executed.
- Use this instruction to access arrays.
- For details, refer to Section 3.3, "Index Instructions."
- There are following types for *Type*:

Type	Function
B BD BS	Modifies the addressing of the next instruction in units of bytes.
W WD WS	Modifies the addressing of the next instruction in units of words.
L LD LS	Modifies the addressing of the next instruction in units of long words.

[Selectable src]

src			
R0L/R0/ R2 R0		R0H/R2/ R0	
R1L/R1/ R3 R1		R1H/R3/ R1	
A0/A0/ A0	A1/A1/ A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

INDEXB.W R0
 INDEXLS.B [A0]

INT

Interrupt by INT instruction
INTerrupt

INT

[Syntax]

INT src

[Instruction Code/Number of Cycles]

Page= 228

[Operation]

SP ← SP - 2
M(SP) ← FLG
SP ← SP - 2
M(SP)*1 ← (PC + 2)H
SP ← SP - 2
M(SP) ← (PC + 2)L
PC ← M(IntBase + src × 4) *1 The 8 high-order bits become indeterminate.

[Function]

- This instruction generates a software interrupt specified by *src*. *src* represents a software interrupt number.
- When *src* is 31 or smaller, the U flag is cleared to 0 and the interrupt stack pointer (ISP) is used.
- When *src* is 32 or larger, the stack pointer indicated by the U flag is used.
- The interrupts generated by the INT instruction are nonmaskable interrupts.
- The interrupt number of *src* must be in the range of 0 to 63, including both ends.

[Selectable src]

src
#IMM6*1*2

*1 #IMM denotes a software interrupt number.
*2 The range of values that can be taken on is $0 \leq \text{\#IMM6} \leq 63$.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	○	○	—	—	—	—	○	—

*3 The flags are saved to the stack area before the INT instruction is executed. After the interrupt, the flags change state as shown on the left.

Conditions

- U : The flag is cleared when the software interrupt number is 31 or smaller. The flag does not change when the software interrupt number is 32 or larger.
- I : The flag is cleared.
- D : The flag is cleared.

[Description Example]

INT #0

INTO

Interrupt on overflow
INTerrupt on Overflow

INTO

[Syntax]
INTO

[Instruction Code/Number of Cycles]
Page= 228

[Operation]
SP ← SP - 2
M(SP) ← FLG
SP ← SP - 2
M(SP)*1 ← (PC + 1)H
SP ← SP - 2
M(SP) ← (PC + 1)L
PC ← M(FFFFE0₁₆)

*1 The 8 high-order bits become indeterminate.

- [Function]
- When the O flag is 1, this instruction generates an overflow interrupt. When the flag is 0, the next instruction is executed.
 - The overflow interrupt is a nonmaskable interrupt.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	○	○	—	—	—	—	○	—

*1 The flags are saved to the stack area before the INTO instruction is executed. After the interrupt, the flags change state as shown on the left.

- Conditions
- U : The flag is cleared.
 - I : The flag is cleared.
 - D : The flag is cleared.

[Description Example]
INTO

JCnd

Jump on condition
Jump on Condition

JCnd

[Syntax]

JCnd **label**

[Instruction Code/Number of Cycles]

Page= 229

[Operation]

if true then jump label

[Function]

- This instruction causes program flow to branch off after checking the execution result of the preceding instruction against the following condition. When the condition indicated by *Cnd* is true, control jumps to **label**. When false, the next instruction is executed.
- The following conditions can be used for *Cnd*:

<i>Cnd</i>	Condition	Expression	<i>Cnd</i>	Condition	Expression
GEU/C	C=1 Equal to or greater than C flag is 1.	$\leq$	LTU/NC	C=0 Smaller than C flag is 0.	$>$
EQ/Z	Z=1 Equal to Z flag is 1.	$=$	NE/NZ	Z=0 Not equal Z flag is 0.	$\neq$
GTU	$C \wedge \bar{Z}=1$ Greater than	$<$	LEU	$C \wedge \bar{Z}=0$ Equal to or smaller than	$\geq$
PZ	S=0 Positive or zero	$0 \leq$	N	S=1 Negative	$0 >$
GE	$S \vee O=0$ Equal to or greater than (signed value)	$\leq$	LE	$(S \vee O) \vee Z=1$ Equal to or smaller than (signed value)	$\geq$
GT	$(S \vee O) \vee Z=0$ Greater than (signed value)	$<$	LT	$S \vee O=1$ Smaller than (signed value)	$>$
O	O=1 O flag is 1.		NO	O=0 O flag is 0.	

[Selectable label]

label	<i>Cnd</i>
$PC^{*1}-127 \leq \text{label} \leq PC^{*1}+128$	GEU/C, GTU, EQ/Z, N, LTU/NC, LEU, NE/NZ, PZ, LE, O, GE, GT, NO, LT

*1 PC indicates the start address of the instruction.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

JEQ label
 JNE label

JMP

*Unconditional jump***JuMP**

JMP

[Syntax]

JMP(.length) label

_____ S , B , W , A

[Instruction Code/Number of Cycles]

Page=229

[Operation]

PC ← label

[Function]

- This instruction causes control to jump to **label**.

[Selectable label]

.length	label
.S	$PC^{*1}+2 \leq \text{label} \leq PC^{*1}+9$
.B	$PC^{*1}-127 \leq \text{label} \leq PC^{*1}+128$
.W	$PC^{*1}-32767 \leq \text{label} \leq PC^{*1}+32768$
.A	abs24

*1 The PC indicates the start address of the instruction.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

JMP label

JMPS

Jump to special page
JuMP Special page

JMPS

[Syntax]

JMPS src

[Instruction Code/Number of Cycles]

Page= 232

[Operation]

PC_H ← FF₁₆
PC_{ML} ← M(FFFE₁₆ - src × 2)

[Function]

- This instruction causes control to jump to the address set in each table of the special page vector table plus FF0000₁₆. The area across which control can jump is from address FF0000₁₆ to address FFFFFFFF₁₆.
- The special page vector table is allocated to an area from address FFFE00₁₆ to address FFFFDB₁₆.
- src represents a special page number. The special page number is 255 for address FFFE00₁₆, and 18 for address FFFFDA₁₆.

[Selectable src]

src
#IMM8*1*2

- *1 #IMM denotes a special page number.
- *2 The range of values that can be taken on is $18 \leq \text{\#IMM8} \leq 255$.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

JMPS #20

JSRI

Indirect subroutine call
Jump SubRoutine Indirect

JSRI

[Syntax]

[Instruction Code/Number of Cycles]

Page=234

JSRI.length **src**

_____ **W , A**

[Operation]

When jump distance specifier (.length) is (.W)

$$SP \leftarrow SP - 2$$
$$M(SP)^{*1} \leftarrow (PC + n^{*2})_H$$
$$SP \leftarrow SP - 2$$
$$M(SP) \leftarrow (PC + n^{*2})_{ML}$$
$$PC \leftarrow PC \pm src$$

*1 The 8 high-order bits become 0.

*2 n denotes the number of instruction bytes.

When jump distance specifier (.length) is (.A)

$$SP \leftarrow SP - 2$$
$$M(SP)^{*1} \leftarrow (PC + n^{*2})H$$
$$SP \leftarrow SP - 2$$
$$M(SP) \leftarrow (PC + n^{*2})H$$

PC ← *src*

100 575

[Function]

This instruction causes control to jump to a subroutine at the address indicated by *src*. When *src* is memory, specify the address at which the low-order address is stored.

- When you selected (.W) for the jump distance specifier (.length), control jumps to a subroutine at the start address of the instruction plus the address indicated by *src* (added including the sign bits). When *src* is memory, the required memory capacity is 2 bytes.
- When *src* is memory and (.A) is selected for the jump distance specifier (.length), the required memory capacity is 3 bytes.

[Selectable src]

When you selected (.W) for the jump distance specifier (.length)

src			
R0L /R0/ R2R0		R0H /R2/ R0	
R1L /R1/ R3R1		R1H /R3/ R1	
A0 / A0 / A0	A1 / A1 / A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

When you selected (.A) for the jump distance specifier (.length)

src			
R0L/R0/R2R0		R0H/R2/	
R1L/R1/R3R1		R1H/R3/	
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

JSRI.A A1

JSRI.W R0

JSRS

Special page subroutine call
Jump SubRoutine Special page

JSRS

[Syntax]

JSRS src

[Instruction Code/Number of Cycles]

Page= 235

[Operation]

SP ← SP - 2
M(SP)*1 ← (PC + 2)_H
SP ← SP - 2
M(SP) ← (PC + 2)_{ML}
PCH ← FF₁₆
PCML ← M (FFFE₁₆ - src × 2)

*1 The 8 high-order bits become 0.

[Function]

This instruction causes control to jump to a subroutine at the address set in each table of the special page vector table plus FF0000₁₆. The area across which program flow can jump to a subroutine is from address FF0000₁₆ to address FFFFFFFF₁₆.

- The special page vector table is allocated to an area from address FFFE00₁₆ to address FFFFDB₁₆.
- src represents a special page number. The special page number is 255 for address FFFE00₁₆, and 18 for address FFFFDA₁₆.

[Selectable src]

src
#IMM8*1*2

- *1 #IMM denotes a special page number.
- *2 The range of values that can be taken on is $18 \leq \text{\#IMM8} \leq 255$.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

JSRS #18

LDC

Transfer to control register
LoaD Control register

LDC**[Syntax]**

LDC **src,dest**

[Instruction Code/Number of Cycles]

Page= 235

[Operation]

dest ← **src**

[Function]

- This instruction transfers *src* to the control register indicated by *dest*.
- When memory is specified for *src*, the following bytes of memory are required.
 2 bytes : DMD0*¹, DMD1*¹, FLG, DCT0, DCT1, DRC0, DRC1, SVF
 4 bytes*² : FB, SB, SP*³, ISP*³, INTB*³, VCT, SVP, DMA0, DMA1, DRA0, DRA1, DSA0, DSA1

*1 The low-order 8 bit of *src* is transferred.

*2 The low-order 24 bit of *src* is transferred.

*3 Set even number for SP, ISP and INTB even though odd number can be set. It is more effective to set even number for operation.

[Selectable src/dest]

src				dest			
R0L/R0/R2R0	R0H/R2/-			DMD0	DMD1	DCT0	DCT1
R1L/R1/R3R1	R1H/R3/-			DRC0	DRC1	FLG	SVF
A0/A0/A0	A1/A1/A1	[A0]	[A1]				
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]				
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]				
dsp:24[A0]	dsp:24[A1]	abs24	abs16				
#IMM16/#IMM24							
R0L/R0/R2R0	R0H/R2/-			FB	SB	SP* ⁴	ISP
R1L/R1/R3R1	R1H/R3/-			INTB	VCT	SVP	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	DMA0	DMA1	DRA0	DRA1
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	DSA0	DSA1		
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]				
dsp:24[A0]	dsp:24[A1]	abs24	abs16				
#IMM16/#IMM24							

*4 Operation is performed on the stack pointer indicated by the U flag.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	*5	*5	*5	*5	*5	*5	*5	*5

*5 The flag changes only when *dest* is FLG.

[Description Example]

LDC A0,FB

LDCTX

Restore context
LoaD ConTeXt

LDCTX

[Syntax]

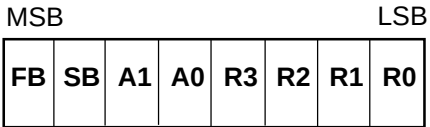
LDCTX abs16,abs24

[Instruction Code/Number of Cycles]

Page=238

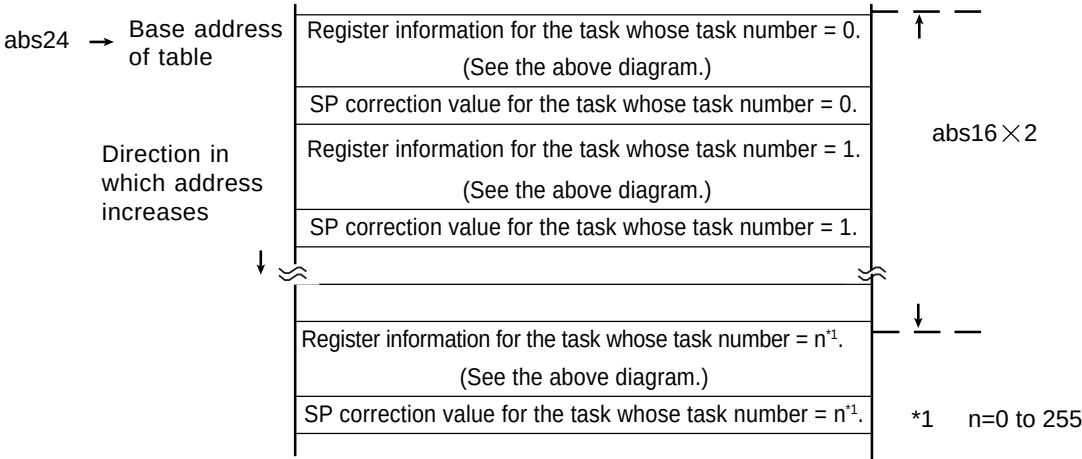
[Function]

- This instruction restores task context from the stack area.
- Set the RAM address that contains the task number in abs16 and the start address of table data in abs24.
- The required register information is specified from table data by the task number and the data in the stack area is transferred to each register according to the specified register information. Then the SP correction value is added to the stack pointer (SP). For this SP correction value, set the number of bytes you want to the transferred. Calculated as 2 bytes when transferring the R0, R1, R2, or R3 registers. A0, A1, SB, and FB are calculated as 4 bytes.
- Information on transferred registers is configured as shown below. Logic 1 indicates a register to be transferred and logic 0 indicates a register that is not transferred.



← Transferred sequentially beginning with R0

- The table data is comprised as shown below. The address indicated by abs24 is the base address of the table. The data stored at an address apart from the base address as much as twice the content of abs16 indicates register information, and the next address contains the stack pointer correction value.



[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

LDCTX Ram,Rom_TBL

LDIPL

Set interrupt enable level
LoaD Interrupt Permission Level

LDIPL

[Syntax]

LDIPL src

[Instruction Code/Number of Cycles]

Page= 239

[Operation]

IPL ← src

[Function]

- This instruction transfers *src* to IPL.

[Selectable src]

src
#IMM3 ^{*1}

^{*1} The range of values that can be taken on is $0 \leq \text{\#IMM3} \leq 7$.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

LDIPL #2

MAX

Select maximum value

MAX select

MAX

[Syntax]

MAX.size src,dest

B , W

[Instruction Code/Number of Cycles]

Page= 239

[Operation]

```
if (src > dest)
then dest ← src
```

[Function]

- Singed compares *src* and *dest* and transfers *src* to *dest* when *src* is greater than *dest*. No change occurs when *src* is smaller than or equal to *dest*.
- When (.W) is specified for the size specifier (.size), *dest* is the address register and writing to *dest*, the 8 high-order bits of the operation result are become 0. Also, when *src* is the address register, transfers the 16 low-order bits of the address register to *dest*.

[Selectable src/dest]

src				dest			
R0L/R0/ R2R0		R0H/R2/ R0		R0L/R0/ R2R0		R0H/R2/ R0	
R1L/R1/ R3R1		R1H/R3/ R1		R1L/R1/ R3R1		R1H/R3/ R1	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

MAX.B #0ABH,ROL

MAX.W # -1,R2

MIN

Select minimum value

MIN select

MIN

[Syntax]

MIN.size	src,dest
----------	----------

B , W

[Instruction Code/Number of Cycles]

Page=241

[Operation]

```
if (src < dest)
then dest ← src
```

[Function]

- Signed compares *src* and *dest* and transfers *src* to *dest* when *src* is smaller than *dest*. No change occurs when *src* is greater than or equal to *dest*.
- When (.W) is specified for the size specifier (.size), *dest* is the address register and writing to *dest*, the 8 high-order bits of the operation result are become 0. Also, when *src* is the address register, transfers the 16 low-order bits of the address register to *dest*.

[Selectable src/dest]

src				dest			
R0L/R0/ R2R0		R0H/R2/ 		R0L/R0/ R2R0		R0H/R2/ 	
R1L/R1/ R3R1		R1H/R3/ 		R1L/R1/ R3R1		R1H/R3/ 	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

MIN.B	#0ABH,R0L
MIN.W	#-1,R2

MOV

Transfer
MOVE

MOV

[Syntax]**MOV.size (:format) src,dest****[Instruction Code/Number of Cycles]**

Page=243

G, Q, Z, S (Can be specified)
B, W, L

[Operation]

dest ← src [dest] ← src
 dest ← [src] [dest] ← [src]

[Function]

- This instruction transfers *src* to *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0. Also, when *src* is the address register, the 8 low-order bits of the address register are used as data to be operated on.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.
- When (.L) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits of *src* is ignored and the 24 low-order bits of *src* is stored to *dest*. Also, when *src* is the address register, *src* is zero-extended to perform operation in 32 bits. The flags also change states depending on the result of 32-bit operation.

[Selectable src/dest]*1

(See the next page for src/dest classified by format.)

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]	A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM	dsp:8[SP]*3			dsp:8[SP]*3			

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

*3 When *src* or *dest* is dsp:8[SP], you cannot choose indirect addressing [src] or [dest] neither.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

S : The flag is set when the transfer resulted in MSB = 1; otherwise cleared.

Z : The flag is set when the transfer resulted in 0; otherwise cleared.

[Description Example]

MOV.B:S #0ABH,R0L
 MOV.W #-1,R2
 MOV.W [A1],[A2]

[src/dest Classified by Format]**G format *1**

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]	A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16/#IMM32	dsp:8[SP]*3*5			dsp:8[SP]*3*4*5			

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

*3 Operation is performed on the stack pointer indicated by the U flag. You cannot choose dsp:8 [SP] for *src* and *dest* simultaneously.

*4 When you specify (.B) or (.W) for the size specifier (.size) and *src* is not #IMM, you can choose dsp:8 [SP] for *dest*.

*5 When *src* or *dest* is dsp:8[SP], you cannot choose indirect addressing [src] or [dest] neither.

Q format *6*7

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM4*8							

*6 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, dsp:8[SP], and #IMM.

*7 You can only specify (.B) or (.W) for the size specifier (.size).

*8 The range of values that can be taken on is $-8 \leq \#IMM4 \leq +7$.

S format *9

src				dest			
R0L/R0*10*11	dsp:8[SB]*11	dsp:8[FB]*11	abs16*11	R0L/R0*10*11	R1L/R1*11*12	dsp:8[SB]*11	dsp:8[FB]*11
#IMM8/#IMM16*11				abs16*11	A0	A1	
R0L/R0	dsp:8[SB]*14	dsp:8[FB]*14	abs16*14	R0L	R0H	dsp:8[SB]	dsp:8[FB]
#IMM16*13/#IMM24*14				abs16	A0/A0*13/A0*14	A1/A1*13/A1*14	

*9 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, dsp:8[SP], and #IMM.

*10 You cannot choose the same registers for *src* and *dest* simultaneously.

*11 You can only specify (.B) or (.W) for the size specifier (.size).

*12 When *src* is not #IMM8/IMM16, you can only choose R1L/R1 for *dest*.

*13 You can specify (.W) for the size specifier (.size). In this case, you cannot use indirect addressing mode for *dest*.

*14 You can specify (.L) for the size specifier (.size). In this case, you cannot use indirect addressing mode for *dest*.

Z format *15

src				dest			
R0L	R0H	dsp:8[SB]	dsp:8[FB]	R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16
abs16	#0			A0	A1		

*15 You can specify (.B) or (.W) for the size specifier (.size).

MOVA

Transfer effective address
MOVE effective Address

MOVA

[Syntax]

MOVA src,dest

[Instruction Code/Number of Cycles]

Page= 252

[Operation]

dest ← EVA(src)

[Function]

- This instruction transfers the affective address of *src* to *dest*.

[Selectable src/dest]

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM							

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

MOVA Ram:16[SB],A0

MOVDir

Transfer 4-bit data
MOVE nibble

MOVDir

[Syntax]**MOVDir** **src,dest****[Instruction Code/Number of Cycles]**

Page= 253

[Operation]

Dir	Operation
HH	H4:dest ← H4:src
HL	L4:dest ← H4:src
LH	H4:dest ← L4:src
LL	L4:dest ← L4:src

[Function]

- Be sure to choose R0L for either *src* or *dest*.

Dir	Function
HH	Transfers <i>src</i> (8 bits)'s 4 high-order bits to <i>dest</i> (8 bits)'s 4 high-order bits.
HL	Transfers <i>src</i> (8 bits)'s 4 high-order bits to <i>dest</i> (8 bits)'s 4 low-order bits.
LH	Transfers <i>src</i> (8 bits)'s 4 low-order bits to <i>dest</i> (8 bits)'s 4 high-order bits.
LL	Transfers <i>src</i> (8 bits)'s 4 low-order bits to <i>dest</i> (8 bits)'s 4 low-order bits.

[Selectable src/dest]

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM							
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM							

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

MOVHH R0L,[A0]
 MOVHL R0L,[A0]

MOVX

Transfer extend sign
MOVE eXtend sign

MOVX

[Syntax]

MOVX **src,dest**

[Instruction Code/Number of Cycles]

Page= 255

[Operation]

dest/[dest] ← EXTS(src)

[Function]

- Sign-extends the 8-bit immediate to 32 bits before transferring it to *dest*.
- When *dest* is the address register (A0, A1), the 24 low-order bits are transferred. The flags also change state for the 32 bits transfers performed.

[Selectable src/dest]

src				dest*1			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8*2							

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 The range of values that can be taken on is $-128 \leq \#IMM8 \leq +127$

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

S : The flag is set when the transfer resulted in MSB of *dest* = 1; otherwise cleared.

Z : The flag is set when the transfer resulted in 0; otherwise cleared.

[Description Example]

MOVX #10,A0
 MOVX #5,[[A1]]

MUL

Signed multiply
MULTiple

MUL

[Syntax]

MUL.size **src,dest**

└──────────────────┘ **B , W**

[Instruction Code/Number of Cycles]

Page= 255

[Operation]

dest ← dest × src [dest] ← [dest] × src

dest ← dest × [src] [dest] ← [dest] × [src]

[Function]

- This instruction multiplies *src* and *dest* together including the sign bits and stores the result in *dest*.
- When you selected (.B) for the size specifier (.size), *src* and *dest* both are operated on in 8 bits and the result is stored in 16 bits. When you specified an address register(A0, A1) for either *src* or *dest*, operation is performed on the address register's 8 low-order bits. When *dest* is the address register, the 8 high-order bits become 0.
- When you selected (.W) for the size specifier (.size), *src* and *dest* both are operated on in 16 bits and the result is stored in 32 bits. When you specified R0 or R1 for *dest*, the result is stored in R2R0 or R3R1 accordingly. When the address register is selected for *dest*, the 24 low-order bits of the 32-bit operation result is stored. When the address register is selected for *src*, operation is performed using the 16 low-order bits of the register.

[Selectable src/dest]*¹

src				dest			
R0L/R0/ R2R0		R0H/R2/-		R0L/R0/ R2R0		R0H/R2/-	
R1L/R1/ R3R1		R1H/R3/-		R1L/R1/ R3R1		R1H/R3/-	
A0/A0/ A0 * ²	A1/A1/ A1 * ²	[A0]	[A1]	A0/A0/ A0 * ²	A1/A1/ A1 * ²	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

MUL.B A0,R0L ; R0L and A0's 8 low-order bits are multiplied.

MUL.W #3,R0

MUL.B R0L,R1L

MUL.W A0,Ram

MUL.W [A0],[[A1]]

MULEX

Multipl extend sign
MULTiple EXtend

MULEX

[Syntax]

MULEX src

[Instruction Code/Number of Cycles]

Page= 257

[Operation]

R1R2R0 ← R2R0 × src/[src]

[Function]

- Multiplies *src* (16-bit data) and R2R0 including the sign and stores the result in R1R2R0.

[Selectable src]

src*1			
R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 Indirect addressing [src] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

MULEX A0
MULEX R3
MULEX Ram
MULEX [[A0]]

MULU

Unsigned multiply
MULTiple Unsigned

MULU

[Syntax]

MULU.size src,dest
└──────────────────┘ B , W

[Instruction Code/Number of Cycles]

Page=257

[Operation]

dest ← dest × src [dest] ← [dest] × src
dest ← dest × [src] [dest] ← [dest] × [src]

[Function]

- This instruction multiplies *src* and *dest* together not including the sign bits and stores the result in *dest*.
- When you selected (.B) for the size specifier (.size), *src* and *dest* both are operated on in 8 bits and the result is stored in 16 bits. When you specified an address register(A0, A1) for either *src* or *dest*, operation is performed on the address register's 8 low-order bits. When *dest* is the address register, the 8 high-order bits become 0.
- When you selected (.W) for the size specifier (.size), *src* and *dest* both are operated on in 16 bits and the result is stored in 32 bits. When you specified R0 or R1 for *dest*, the result is stored in R2R0 or R3R1 accordingly. When the address register is selected for *dest*, the 24 low-order bits of the 32-bit operation result is stored. When the address register is selected for *src*, operation is performed using the 16 low-order bits of the register.

[Selectable src/dest] *1

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]	A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

MULU.B A0,R0L
MULU.W #3,R0
MULU.B R0L,R1L
MULU.W A0,Ram
MULU.W [R1],[A0]

; R0L and A0's 8 low-order bits are multiplied.

NEG

Two's complement
NEGate

NEG

[Syntax]

NEG.size dest
 └──────────────────┘ B , W

[Instruction Code/Number of Cycles]

Page= 259

[Operation] $\text{dest} \leftarrow 0 - \text{dest}$ $[\text{dest}] \leftarrow 0 - [\text{dest}]$ **[Function]**

- This instruction takes the 2's complement of *dest* and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register(A0, A1), the 8 high-order bits become 0.

[Selectable dest]

dest*1			
R0L/R0/ R2R0		R0H/R2/ -	
R1L/R1/ R3R1		R1H/R3/ -	
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O : The flag is set when *dest* before the operation is - 128 (.B) or - 32768 (.W); otherwise cleared.
- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when the operation resulted in 0; otherwise cleared.

[Description Example]

NEG.B R0L
 NEG.W A1
 NEG.W [[A0]]

NOP

No operation
No OPeration

NOP

[Syntax]

NOP

[Instruction Code/Number of Cycles]

Page= 259

[Operation]

$$PC \leftarrow PC + 1$$
[Function]

- This instruction adds 1 to PC.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

NOP

NOT

Invert all bits
NOT

NOT

[Syntax]

NOT.size dest
 └──────────────────┘ B , W

[Instruction Code/Number of Cycles]

Page= 260

[Operation]

dest ← $\overline{\text{dest}}$ [dest] ← $\overline{[\text{dest}]}$

[Function]

- This instruction inverts *dest* and stores the result in *dest*.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register(A0, A1), the 8 high-order bits become 0.

[Selectable dest]

dest*1			
R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
 Z : The flag is set when the operation resulted in 0; otherwise cleared.

[Description Example]

NOT.B R0L
 NOT.W A1

OR

Logically OR
OR

OR

[Syntax]

OR.size (:format) src,dest

[Instruction Code/Number of Cycles]

Page= 260

└──────────────────────────┐ G , S (Can be specified)
└──────────────────────────┐ B , W

[Operation]

dest ← src ∨ dest [dest] ← src ∨ [dest]
dest ← [src] ∨ dest [dest] ← [src] ∨ [dest]

[Function]

- This instruction logically ORs *dest* and *src* together and stores the result in *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0. Also, when *src* is the address register, the 8 low-order bits of the address register are used as data to be operated on.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest]*¹(See the next page for *src/dest* classified by format.)

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0* ²	A1/A1/A1* ²	[A0]	[A1]	A0/A0/A0* ²	A1/A1/A1* ²	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*¹ Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*² If you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.

Z : The flag is set when the operation resulted in 0; otherwise cleared.

[Description Example]

OR.B Ram:8[SB],R0L

OR.B:G A0,R0L

; A0's 8 low-order bits and R0L are ORed.

OR.B:G R0L,A0

; R0L is zero-expanded and ORed with A0.

OR.B:S #3,R0L

OR.W:G [R1],[A0]

[src/dest Classified by Format]**G format^{*1}**

src				dest			
R0L/R0/ R2R0	R0H/R2/ -			R0L/R0/ R2R0	R0H/R2/ -		
R1L/R1/ R3R1	R1H/R3/ -			R1L/R1/ R3R1	R1H/R3/ -		
A0/A0/ A0 ^{*2}	A1/A1/ A1 ^{*2}	[A0]	[A1]	A0/A0/ A0 ^{*2}	A1/A1/ A1 ^{*2}	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

^{*1} Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

^{*2} If you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

S format^{*2}

src				dest			
R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16	R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16
#IMM8/#IMM16							

^{*2} Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

POP

Restore register/memory
POP

POP

[Syntax]

POP.size dest

└──────────────────────────┘ B , W

[Instruction Code/Number of Cycles]

Page= 263

[Operation]

dest/[dest] ← M(SP)
SP ← SP + 2

*1 Even when (.B) is specified for the size specifier (.size), SP is increased by 2.

[Function]

- This instruction restores *dest* from the stack area.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register(A0, A1), the 8 high-order bits become 0.

[Selectable dest]

dest*2			
R0L/R0/ R2 R0	R0H/R2/ -		
R1L/R1/ R3 R1	R1H/R3/ -		
A0 /A0/ A0 A1 /A1/ A1	[A0] [A1]		
dsp:8[A0] dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	
dsp:16[A0] dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	
dsp:24[A0] dsp:24[A1]	abs24	abs16	

*2 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

POP.B R0L
POP.W A0

POPC

Restore control register
POP Control register

POPC

[Syntax]

POPC dest

[Instruction Code/Number of Cycles]

Page= 263

[Operation]

- When dest is DCT0, DCT1, DMD0, DMD1, DRC0, DRC1, SVF or FLG
 $dest^{*1} \leftarrow M(SP)$
 $SP \leftarrow SP + 2$

*1 The 8 low-order bytes are saved when dest is DMD0 or DMD1.

- When dest is FB, SB, SP, ISP or INTB
 $dest^{*2} \leftarrow M(SP)$
 $SP^{*3} \leftarrow SP + 4$

*2 The 3 low-order byte are saved.

*3 4 is not added to SP when dest is SP, or dest is ISP while U flag is "0".

[Function]

- This instruction restores from the stack area to the control register indicated by *dest*.
- Restored stack area is indicated by the U flag.

[Selectable dest]

dest			
FB	SB	SP ^{*1}	ISP
INTB			
DCT0	DCT1	DMD0	DMD1
DRC0	DRC1	SVF	FLG

*1 Operation is performed on the stack pointer indicated by the U flag.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	*2	*2	*2	*2	*2	*2	*2	*2

*2 The flag changes only when *dest* is FLG.

POPC SB

POPM

Restore multiple registers
POP Multiple

POPM

[Syntax]

POPM **dest**

[Instruction Code/Number of Cycles]

Page= 264

[Operation]

$$\begin{aligned} \text{dest}^{*3} &\leftarrow M(\text{SP}) \\ \text{SP} &\leftarrow \text{SP} + n1^{*1} \times 2 \\ \text{SP} &\leftarrow \text{SP} + n2^{*2} \times 4 \end{aligned}$$

*1 n1 denotes the number of R0, R1, R2 and R3 registers to be restored.

*2 n2 denotes the number of A0, A1, SB and FB registers to be restored.

*3 The 3 low-order bytes are saved when dest is A0, A1, SB and FB.

[Function]

- This instruction restores the registers selected by *dest* collectively from the stack area.
- Registers are restored from the stack area in the following order:

FB	SB	A1	A0	R3	R2	R1	R0
-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------

← Restored sequentially beginning with R0

[Selectable dest]

dest^{*3}							
R0	R1	R2	R3	A0	A1	SB	FB

*3 You can choose multiple *dest*.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

POPM R0,R1,A0,SB,FB

PUSH

Save register/memory/immediate data

PUSH

PUSH

[Syntax]

PUSH.size **src**

└──────────────────┘ **B , W , L**

[Instruction Code/Number of Cycles]

Page= 265

[Operation]

- When the size specifier (.size) is (.B)
 - $SP \leftarrow SP - 2$
 - $M(SP)^{*1} \leftarrow src/[src]$
 - When the size specifier (.size) is (.W)
 - $SP \leftarrow SP - 2$
 - $M(SP) \leftarrow src/[src]$
- *1 The 8 high-order bits become indeterminate.
Even when (.B) is specified for the size specifier (.size) , SP is decreased by 2.

- When the size specifier (.size) is (.L)
 - $SP \leftarrow SP - 4$
 - $M(SP)^{*2} \leftarrow src/[src]$
- *2 When *src* is address register(A0, A1), the 8 high-order bits become 0.

[Function]

- This instruction saves *src* to the stack area.
- When (.W) is specified for the size specifier (.size) and *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src]

src^{*3}			
R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-		
A0 /A0/A0	A1 /A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16/#IMM32			

*3 Indirect addressing [src] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

PUSH.B #5
 PUSH.W #100H
 PUSH.L R2R0

PUSHA

Save effective address
PUSH effective Address

PUSHA

[Syntax]

PUSHA src

[Instruction Code/Number of Cycles]

Page= 267

[Operation]

SP ← SP - 4
M(SP)*1 ← EVA(src)
*1 The 8 high-order bits become indeterminate.

[Function]

- This instruction saves the effective address of *src* to the stack area.

[Selectable src]

src			
R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

PUSHA Ram:8[FB]
PUSHA Ram:16[SB]

PUSHC

Save control register
PUSH Control register

PUSHC

[Syntax]

PUSHC src

[Instruction Code/Number of Cycles]

Page= 267

[Operation]

- When *src* is DCT0, DCT1, DMD0, DMD1, DRC0, DRC1, SVF or FLG
$$SP \leftarrow SP - 2$$
$$M(SP)^{*1} \leftarrow src$$

**1* When *src* is DMD0 or DMD1, the 8 high-order bits become indeterminate.
- When *src* is FB, SB, SP, ISP or INTB
$$SP \leftarrow SP - 4$$
$$M(SP)^{*2} \leftarrow src^{*3}$$

**2* The 8 high-order bits become 0.

**3* SP before 4 is subtracted is saved when *src* is SP, or *src* is ISP while U flag is "0".

[Function]

- This instruction saves the control register indicated by *src* to the stack area.

[Selectable src]

src			
FB	SB	SP ^{*3}	ISP
INTB			
DCT0	DCT1	DMD0	DMD1
DRC0	DRC1	SVF	FLG

^{*3} Operation is performed on the stack pointer indicated by the U flag.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

PUSHC SB

PUSHM

Save multiple registers
PUSH Multiple

PUSHM

[Syntax]

PUSHM src

[Instruction Code/Number of Cycles]

Page= 268

[Operation]

SP ← SP - n1*^{*1} × 2
SP ← SP - n2*^{*1} × 4
M(SP)*^{*3} ← src

- *1 n1 denotes the number of R0, R1, R2 and R3 registers to be saved.
- *2 n2 denotes the number of A0, A1, SB and FB registers to be saved.
- *3 When src is A0, A1, SB or FB, the 8 high-order bits become 0.

[Function]

- This instruction saves the registers selected by src collectively to the stack area.
- The registers are saved to the stack area in the following order:

R0	R1	R2	R3	A0	A1	SB	FB
----	----	----	----	----	----	----	----

← Saved sequentially beginning with FB

[Selectable src]

src* ⁴							
R0	R1	R2	R3	A0	A1	SB	FB

*4 You can choose multiple src.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

PUSHM R0,R1,A0,SB,FB

REIT

Return from interrupt
REturn from InTerrupt

REIT

[Syntax]
REIT

[Instruction Code/Number of Cycles]
Page= 269

[Operation]

PCML ← M(SP)
SP ← SP + 2
PCH ← M(SP)*1
SP ← SP + 2
FLG ← M(SP)
SP ← SP + 2
*1 The 8 high-order bits are saved.

[Function]

- This instruction restores the PC and FLG that were saved when an interrupt request was accepted to return from the interrupt handler routine.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	*1	*1	*1	*1	*1	*1	*1	*1

*1 Becomes the value in the stack.

[Description Example]
REIT

RMPA

Calculate sum-of-products
Repeat MultiPle & Addition

RMPA

[Syntax]

RMPA.size

[Instruction Code/Number of Cycles]

Page= 269

B , W

[Operation]*1**Repeat**

$$R1R2R0 \leftarrow R1R2R0 + M(A0) \times M(A1)$$

$$A0 \leftarrow A0 + 2 (1)^{*2}$$

$$A1 \leftarrow A1 + 2 (1)^{*2}$$

$$R3 \leftarrow R3 - 1$$
Until R3 = 0

*1 When you set a value 0 in R3, this instruction is ingored.

*2 Shown in ()^{*2} applies when (.B) is selected for the size specifier (.size).**[Function]**

- This instruction performs sum-of-product calculations, with the multiplicand address indicated by A0, the multiplier address indicated by A1, and the count of operation indicated by R3. Calculations are performed including the sign bits and the result is stored in R1R2R0 .
- The content of the address register when the instruction is completed indicates the next address of the last-read data.
- When an interrupt request is received during instruction execution, the interrupt is acknowledged after a sum-of- product addition is completed (i.e., after the content of R3 is decremented by 1).
- Make sure that R1R2R0 has the initial value set.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	—	—	—	—

ConditionsO : The flag is set when $+2^{31}-1$ or -2^{31} is exceeded during operation; otherwise cleared.**[Description Example]**

RMPA.B

ROLC

Rotate left with carry
ROTate to Left with Carry

ROLC

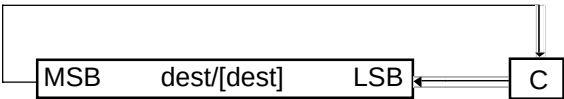
[Syntax]

[Instruction Code/Number of Cycles]

ROLC.size dest
B , W

Page=270

[Operation]



[Function]

This instruction rotates *dest* one bit to the left including the C flag.
When (.W) is specified for the size specifier (.size) and *dest* is the address register(A0, A1), the 8 high-order bits become 0.

[Selectable dest]

dest*1			
R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	○

Conditions

- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in *dest* = 0; otherwise cleared.
- C : The flag is set when the shifted-out bit is 1; otherwise cleared.

[Description Example]

ROLC.B R0L
ROLC.W R0
ROLC.W [[A0]]

RORC

Rotate right with carry
ROtate to Right with Carry

RORC

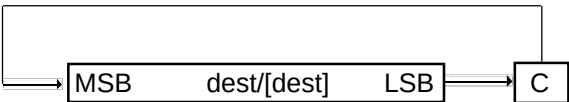
[Syntax]

RORC.size dest B , W

[Instruction Code/Number of Cycles]

Page=270

[Operation]



[Function]

This instruction rotates *dest* one bit to the right including the C flag.
When (.W) is specified for the size specifier (.size) and *dest* is the address register(A0, A1), the 8 high-order bits become 0.

[Selectable dest]

dest*1			
R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	○

Conditions

- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in *dest* = 0; otherwise cleared.
- C : The flag is set when the shifted-out bit is 1; otherwise cleared.

[Description Example]

RORC.B R0L
RORC.W R0
RORC.W [[A0]]

ROT

[Syntax]

ROT.size src,dest

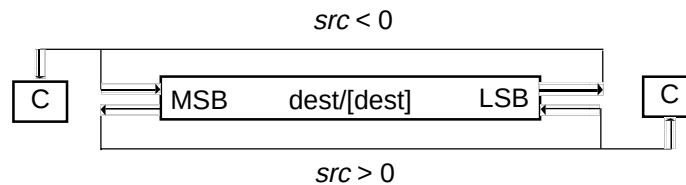
Rotate
ROTate

ROT

[Instruction Code/Number of Cycles]

Page=271

[Operation]



[Function]

- This instruction rotates *dest* left or right the number of bits indicated by *src*. The bit overflowing from LSB (MSB) is transferred to MSB(LSB) and the C flag.
- The direction of rotate is determined by the sign of *src*. When *src* is positive, bits are rotated left; when negative, bits are rotated right.
- When *src* is an immediate, the number of rotates is - 8 to +8(≠0). You cannot set values less than - 8, equal to 0, or greater than +8.
- When *src* is a register, the number of rotates is -16 to +16. Although you can set 0, no bits are rotated and no flags are changed. When you set a value less than -17 or greater than +17, the result of rotation is indeterminate.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register(A0, A1), the 8 high-order bits become 0.

[Selectable src/dest]

src				dest*1			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-		R1L/R1/R3R1*2		R1H/R3/-*2	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM4*3							

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When *src* is R1H, you cannot choose R1 or R1H for *dest*.

*3 The range of values that can be taken on is $-8 \leq \#IMM4 \leq +8$. However, you cannot set 0.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	○

*4 When the number of rotates is 0, no flags are changed.

Conditions

S : The flag is set when the operation resulted in $MSB = 1$; otherwise cleared.

Z : The flag is set when the operation resulted in 0; otherwise cleared.

C : The flag is set when the bit shifted out last is 1; otherwise cleared.

[Description Example]

Description	Example
ROT.B	#1,R0L ; Rotated left

ROT.B #1.R0L : Rotated right

ROT.W R1H,R2

RTS

Return from subroutine
ReTurn from Subroutine

RTS

[Syntax]
RTS

[Instruction Code/Number of Cycles]
Page=272

[Operation]
PCML ← M(SP)
SP ← SP + 2
PCH ← M(SP)*1
SP ← SP + 2
*1 The 8 low-order bits are saved.

[Function]
• This instruction causes control to return from a subroutine.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]
RTS

SBB

Subtract with borrow
SuBtract with Borrow

SBB

[Syntax]

[Instruction Code/Number of Cycles]

SBB.size **src,dest**

_____ **B , W**

Page= 273

[Operation]

$$\text{dest} \leftarrow \text{dest} - \text{src} - \overline{C}$$

[Function]

- This instruction subtracts *src* and inverted C flag from *dest* and stores the result in *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0. Also, when *src* is the address register, the 8 low-order bits of the address register are used as data to be operated on.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest]

src				dest			
R0L/R0/ R2R0		R0H/R2/ -		R0L/R0/ R2R0		R0H/R2/ -	
R1L/R1/ R3R1		R1H/R3/ -		R1L/R1/ R3R1		R1H/R3/ -	
A0/A0/ A0 ^{*1}	A1/A1/ A1 ^{*1}	[A0]	[A1]	A0/A0/ A0 ^{*1}	A1/A1/ A1 ^{*1}	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O : The flag is set when a signed operation resulted in exceeding +32767 (.W) or -32768 (.W), or +127 (.B) or -128 (.B); otherwise cleared.
- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when an unsigned operation resulted in any value equal to or greater than 0; otherwise cleared.

[Description Example]

SBB.B #2,R0L

SBB.W A0.R0

SBB.B A0,R0L

; A0's 8 low-order bits and R0L are operated on.

SBB.B R0L,A0

; R0L is zero-expanded and operated with A0.

SBJNZ

Subtract & conditional jump

SuBtract then Jump on Not Zero

SBJNZ

[Syntax]

[Instruction Code/Number of Cycles]

SBJNZ.size **src,dest,label**

└─────────────────────────── **B , W**

Page= 275

[Operation]

```
dest ← dest - src
if dest ≠ 0 then jump label
```

[Function]

- This instruction subtracts *src* from *dest* and stores the result in *dest*.
- When the operation resulted in any value other than 0, control jumps to **label**. When the operation resulted in 0, the next instruction is executed.
- The op-code of this instruction is the same as that of ADJNZ.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest/label]

src	dest				label
#IMM4 ^{*1}	R0L/R0/ R2R0	R0H/R2/ -			PC ^{*2} -126 ≤ label ≤ PC ^{*2} +129
	R1L/R1/ R3R1	R1H/R3/ -			
	A0/A0/A0	A1/A1/A1	[A0]	[A1]	
	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	
	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	
	dsp:24[A0]	dsp:24[A1]	abs24	abs16	

*1 The range of values that can be taken on is $-7 < \#IMM4 < +8$.

*2 The PC indicates the start address of the instruction.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

```

SBJNZ.W    #1,R0,label
SBJNZ.W    #2,[A1],label

```

SCCnd

Store on condition
Store Condition on Condition

SCCnd

[Syntax]

SCCnd label

[Instruction Code/Number of Cycles]

Page=276

[Operation]

if true then dest ← 1
else dest ← 0

if true then [dest] ← 1
else [dest] ← 0

[Function]

- When the condition specified by *Cnd* is true, this instruction stores a 1 in *dest*; when the condition is false, it stores a 0 in *dest*.
- When *dest* is the address register(A0, A1), the 8 high-order bits of the address register become 0.
- There are following types of *Cnd*:

<i>Cnd</i>	Condition	Expression	<i>Cnd</i>	Condition	Expression
GEU/C	C=1 Equal to or greater than C flag is 1.	$\geq$	LTU/NC	C=0 Smaller than C flag is 0.	$>$
EQ/Z	Z=1 Equal to Z flag is 1.	$=$	NE/NZ	Z=0 Not equal Z flag is 0.	$\neq$
GTU	$C \wedge \overline{Z}=1$ Greater than	$<$	LEU	$C \wedge \overline{Z}=0$ Equal to or smaller than	$\geq$
PZ	S=0 Positive or zero	$0 \leq$	N	S=1 Negative	$0 >$
GE	$S \vee O=0$ Equal to or greater than (signed value)	$\geq$	LE	$(S \vee O) \vee Z=1$ Equal to or smaller than (signed value)	$\geq$
GT	$(S \vee O) \vee Z=0$ Greater than (signed value)	$<$	LT	$S \vee O=1$ Smaller than (signed value)	$>$
O	O=1 O flag is 1.		NO	O=0 O flag is 0.	

[Selectable dest]

dest*1			
R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

SCC R0L

SCC [dsp:8[A0]]

SCMPU

String compare unequal
String CoMPare Unequal

SCMPU

[Syntax]

SCMPU.size

[Instruction Code/Number of Cycles]

Page=277

_____ B , W

[Operation]

- When the size specifier (.size) is (.B)

Repeat

M(A0) – M(A1) (compared by byte)

tmp0 ← M(A0)

tmp2 ← M(A1)

A0 ← A0 + 1

A1 ← A1 + 1

Until (tmp0=0) ∥ (tmp0≠tmp2)

tmp0, tmp2: temporary registers

- When the size specifier (.size) is (.W)

Repeat

M(A0) – M(A1) (compared by byte)

If M(A0)=M(A1) **and** M(A0)≠0 **then** M(A0+1)–M(A1+1)
(compared by byte)

tmp0 ← M(A0)

tmp1 ← M(A0+1)

tmp2 ← M(A1)

tmp3 ← M(A1+1)

A0 ← A0 + 2

A1 ← A1 + 2

Until (tmp0=0) ∥ (tmp1=0) ∥ (tmp0≠tmp2) ∥ (tmp1≠tmp3)

tmp0, tmp1, tmp2, tmp3: temporary registers

[Function]

- Compares strings until contents do not match when compared in the address incrementing direction from the comparison address (A0) to the compared address (A1), until M(A0) = 0 or M(A0+1)=0 (when (.W) is specified for the size specifier (.size)) .
- The contents of the address register (A0, A1) when the instruction is terminated become indeterminate.
- When an interrupt is requested during instruction execution, the interrupt is accepted after comparison of one data is completed.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O : The flag is set when a signed operation of M(A0)–M(A1) resulted in exceeding +127 or -128; otherwise cleared.
- S : The flag is set when the operation of M(A0)–M(A1) resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when fined 0 in M(A0) and terminated, or M(A0)–M(A1)=0 (when compared result is matched); the flag is cleared when M(A0)–M(A1)≠0 (when compared result is not matched).
- C : The flag is set when an unsigned operation of M(A0)–M(A1) resulted in any value equal to or greater then 0; otherwise cleared.

[Description Example]

SCMPU.W

SHA

Shift arithmetic Shift Arithmetic

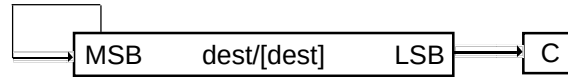
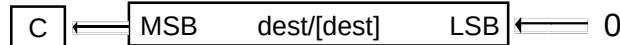
SHA

[Syntax]

SHA.size src,dest

[Instruction Code/Number of Cycles]

Page= 278

[Operation]When $src < 0$ When $src > 0$ **[Function]**

- This instruction arithmetically shifts *dest* left or right the number of bits indicated by *src*. The bit overflowing from LSB(MSB) is transferred to the C flg.
- The direction of shift is determined by the sign of *src*. When *src* is positive, bits are shifted left; when negative, bits are shifted right.
- When *src* is an immediate and you selected (.B) or (.W) for the size specifier (.size), the number of shifts is -8 to +8(≠0). You cannot set values less than -8, equal to 0, or greater than +8. When you selected (.L) for the size specifier (.size), the number of shifts is -16 to +16(≠0). You cannot set values less than -16, equal to 0, or greater than +16.
- When *src* is a register, the number of shifts is -16 to +16. Although you can set 0, no bits are shifted and no flags are changed. When you set a value less than -16 or greater than +16, the result of shift is indeterminate.
- When (.L) is specified for the size specifier (.size) and *dest* is the address register, *dest* is zero-extended to perform operation in 32 bits. The 24 low-order bits of the operation result are stored in *dest*.

[Selectable src/dest]

src				dest*1			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H*2/R3/-		R1L/R1/R3R1*2		R1H/R3/-*2	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM4/#IMM8*3							

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When *src* is R1H, you cannot choose R1, R1H or R3R1 for *dest*.

*3 When (.B) or (.W) is selected for the size specifier (.size), the range of values that can be taken on is $-8 \leq \#IMM4 \leq +8 (\neq 0)$. When (.L) is selected for the size specifier (.size), the range of values that can be taken on is $-16 \leq \#IMM8 \leq +16 (\neq 0)$.

[Flag Change]*¹

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

*¹ When the number of shifts is 0, no flags are changed.

Conditions

O*² : The flag is cleared when all the shift resulted in MSB and shift out bit are the same value; otherwise set.

S*² : The flag is set when the operation resulted in MSB = 1; otherwise cleared.

Z*² : The flag is set when the operation resulted in 0; otherwise cleared.

C*² : The flag is set when the bit at last shifted out is 1; otherwise cleared.

*² When (.L) is specified for the sign specifier (.size) and dest is the address register(A0, A1), the flag become indeterminate.

[Description Example]

SHA.B	#3,R0L	; Arithmetically shifted left
SHA.B	#-3,R0L	; Arithmetically shifted right
SHA.L	R1H,Ram:8[A1]	
SHA.W	R1H,[[A1]]	

SHL

Shift logical
SHift Logical

SHL

[Syntax]

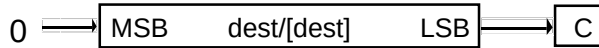
[Instruction Code/Number of Cycles]

SHL.size src,dest
└──────────────────┘ B , W , L

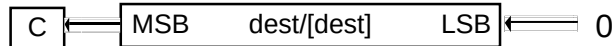
Page=281

[Operation]

When $src < 0$



When $src > 0$



[Function]

- This instruction logically shifts *dest* left or right the number of bits indicated by *src*. The bit overflowing from LSB (MSB) is transferred to the C flag.
- The direction of shift is determined by the sign of *src*. When *src* is positive, bits are shifted left; when negative, bits are shifted right.
- When *src* is an immediate and (.B) or (.W) is specified for the size specifier (.size), the number of shifts is -8 to +8(≠0). You cannot set values less than -8, equal to 0, or greater than +8. When (.L) is specified for the size specifier (.size), the number of shifts is -16 to +16(≠0). You cannot set values less than -16, or greater than +16.
- When *src* is a register, the number of shifts is -16 to +16. Although you can set 0, no bits are shifted and no flags are changed. When you set a value less than -16 or greater than +16, the result of shift is indeterminate.

[Selectable src/dest]

src				dest*1			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H*2/R3/-		R1L/R1/R3R1*2		R1H/R3/-*2	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM4/#IMM8*3							

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When *src* is R1H, you cannot choose R1, R1H or R3R1 for *dest*.

*3 When (.B) or (.W) is selected for the size specifier (.size), the range of values that can be taken on is $-8 \leq \#IMM4 \leq +8 (\neq 0)$. When (.L) is selected for the size specifier (.size), the range of values that can be taken on is $-16 \leq \#IMM8 \leq +16 (\neq 0)$.

[Flag Change]^{*1}

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	○

^{*1} When the number of shifts is 0, no flags are changed.

Conditions

S^{*2}: The flag is set when the operation resulted in MSB = 1; otherwise cleared.

Z^{*2}: The flag is set when the operation resulted in 0; otherwise cleared.

C^{*2}: The flag is set when the bit shifted out last is 1; otherwise cleared.

^{*2} When (.L) is specified for the sign specifier (.size) and dest is the address register(A0, A1), the flag become indeterminate.

[Description Example]

SHL.B	#3,R0L	; Logically shifted left
SHL.B	#-3,R0L	; Logically shifted right
SHL.L	R1H,Ram:8[A1]	
SHL.W	R1H,[[A0]]	

SIN

String input
String INput

SIN

[Syntax]**SIN.size**_____ **B , W****[Instruction Code/Number of Cycles]**

Page=283

[Operation]*1

- When size specifier (.size) is (.B)

While R3≠0 Do

```

M(A1) ← M(A0)
A1 ← A1 + 1
R3 ← R3 - 1

```

End

- When size specifier (.size) is (.W)

While R3≠0 Do

```

M(A1) ← M(A0)
A1 ← A1 + 2
R3 ← R3 - 1

```

End

*1 When you set a value 0 in R3, this instruction is ingored.

[Function]

- Transfers strings from the fixed source address indicated by A0 to the destination address indicated by A1 in the address incrementing direction as many times as specified by R3.
- Set the source of transfer address in A0, the destination address in A1, and the transfer count in R3.
- The content of A1 when the instruction is terminated indicates the next address following the last data transferred.
- When an interrupt is requested during instruction execution, the interrupt is accepted after comparison of one data is completed.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

SIN.W

SMOVB

Transfer string backward
String MOVE Backward

SMOVB

[Syntax]

SMOVB.size

B , W

[Instruction Code/Number of Cycles]

Page=284

[Operation]*1

- When size specifier (.size) is (.B)

While R3≠0 Do

```

M(A1) ← M(A0)
A0 ← A0 - 1
A1 ← A1 - 1
R3 ← R3 - 1

```

End

- When size specifier (.size) is (.W)

While R3≠0 Do

```

M(A1) ← M(A0)
A0 ← A0 - 2
A1 ← A1 - 2
R3 ← R3 - 1

```

End

*1 When you set a value 0 in R3, this instruction is ingored.

[Function]

- This instruction transfers string in successively address decrementing direction from the source address indicated by A0 to the destination address indicated by A1.
- Set the transfer count in R3.
- The address register(A0, A1) when the instruction is completed contains the next address of the last-read data.
- When an interrupt request is received during instruction execution, the interrupt is acknowledged after one data transfer is completed.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

SMOVB.B

SMOVF

Transfer string forward
String MOVE Forward

SMOVF

[Syntax]

SMOVF.size

B, W

[Instruction Code/Number of Cycles]

Page=284

[Operation]^{*1}

- When size specifier (.size) is (.B)

While R3≠0 Do

```

M(A1) ← M(A0)
A0 ← A0 + 1
A1 ← A1 + 1
R3 ← R3 - 1

```

End

- When size specifier (.size) is (.W)

While R3≠0 Do

```

M(A1) ← M(A0)
A0 ← A0 + 2
A1 ← A1 + 2
R3 ← R3 - 1

```

End

*1 When you set a value 0 in R3, this instruction is ingored.

[Function]

- This instruction transfers string in successively address incrementing direction from the source address indicated by A0 to the destination address indicated by A1.
- Set the transfer count in R3.
- The address register (A0, A1) when the instruction is completed contains the next address of the last-read data.
- When an interrupt request is received during instruction execution, the interrupt is acknowledged after one ansfer is completed.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

SMOVF.W

SMOVU

Transfer string
String MOVE Unequal

SMOVU

[Syntax]

SMOVU.size

B, W

[Instruction Code/Number of Cycles]

Page=285

[Operation]

- When size specifier (.size) is (.B)

Repeat

M(A1) ← M(A0) (transferred by byte)

tmp0 ← M(A0)

A0 ← A0 + 1

A1 ← A1 + 1

Until tmp0 = 0

tmp0: temporary register

- When size specifier (.size) is (.W)

Repeat

M(A1) ← M(A0) (transferred by word)

tmp0 ← M(A0)

tmp1 ← M(A0 + 1)

A0 ← A0 + 2

A1 ← A1 + 2

Until (tmp0 = 0) || (tmp1 = 0)

tmp0, tmp1: temporary registers

[Function]

- Transfers strings from the source address indicated by A0 to the destination address indicated by A1 in the address incrementing direction until 0 is detected.
- The contents of the address register (A0, A1) when the instruction is terminated become indeterminate.
- When an interrupt is requested during instruction execution, the interrupt is accepted after comparison of one data is completed.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

SMOVU.B

SOUT

Store string output
String OUTput

SOUT

[Syntax]

SOUT.size

B, W

[Instruction Code/Number of Cycles]

Page= 285

[Operation]*1

- When size specifier (.size) is (.B)

While R3≠0 Do

M(A1) ← M(A0)

A0 ← A0 + 1

R3 ← R3 - 1

End

- When size specifier (.size) is (.W)

While R3≠0 Do

M(A1) ← M(A0)

A0 ← A0 + 2

R3 ← R3 - 1

End

*1 When you set a value 0 in R3, this instruction is ignored.

[Function]

- This instruction transfers strings from the source address indicated by A0 to the fixed destination address indicated by A1 in the address incrementing direction as many times as specified by R3.
- Set the source of transfer address in A0, the destination address in A1, and the transfer count in R3.
- The content of A0 when the instruction is terminated indicates the next address following the last data transferred.
- When an interrupt request is received during instruction execution, the interrupt is acknowledged after one data transfer is completed.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

SOUT.W

SSTR

Store string
String SToRe

SSTR

[Syntax]**SSTR.size**_____ **B , W****[Instruction Code/Number of Cycles]**

Page= 286

[Operation]*1

- When size specifier (.size) is (.B)

While R3≠0 Do $M(A1) \leftarrow R0L$ $A1 \leftarrow A1 + 1$ $R3 \leftarrow R3 - 1$ **End**

- When size specifier (.size) is (.W)

While R3≠0 Do $M(A1) \leftarrow R0$ $A1 \leftarrow A1 + 2$ $R3 \leftarrow R3 - 1$ **End**

*1 When you set a value 0 in R3, this instruction is ingored.

[Function]

- This instruction stores string, with the store data indicated by R0L/R0, the transfer address indicated by A1, and the transfer count indicated by R3.
- The content of A1 when the instruction is terminated indicates the next address following the last data transferred.
- When an interrupt request is received during instruction execution, the interrupt is acknowledged after one data transfer is completed.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

SSTR.B

STC

Transfer from control register
STore from Control register

STC

[Syntax]

STC **src,dest**

[Instruction Code/Number of Cycles]

Page= 286

[Operation]

dest ← **src**

[Function]

- This instruction transfers the control register indicated by *src* to *dest*. When *dest* is memory, specify the address in which to store the low-order address.
- When memory is specified for *dest*, the following bytes of memory are required.
 2 bytes : DMD0*¹, DMD1*¹, FLG, DCT0, DCT1, DRC0, DRC1, SVF
 4 bytes : FB*¹, SB*¹, SP*¹, ISP*¹, INTB*¹, VCT*¹, SVP*¹, DMA0*¹, DMA1*¹, DRA0*¹, DRA1*¹,
 DSA0*¹, DSA1*¹

*1 The 1 high-order byte of dest becomes indeterminate.

[Selectable src/dest]

src				dest			
DMD0	DMD1	DCT0	DCT1	R0L/R0/R2R0	R0H/R2/-		
DRC0	DRC1	FLG	SVF	R1L/R1/R3R1	R1H/R3/-		
				A0/A0/A0	A1/A1/A1	[A0]	[A1]
				dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
				dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
				dsp:24[A0]	dsp:24[A1]	abs24	abs16
FB	SB	SP* ²	ISP	R0L/R0/R2R0	R0H/R2/-		
INTB	VCT	SVP		R1L/R1/R3R1	R1H/R3/-		
DMA0	DMA1	DRA0	DRA1	A0/A0/A0	A1/A1/A1	[A0]	[A1]
DSA0	DSA1			dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
				dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
				dsp:24[A0]	dsp:24[A1]	abs24	abs16

*2 Operation is performed on the stack pointer indicated by the U flag.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

STC FLG,R0

STC FB,A0

STCTX

Save context
STore ConTeXt

STCTX

[Syntax]

STCTX abs16,abs24

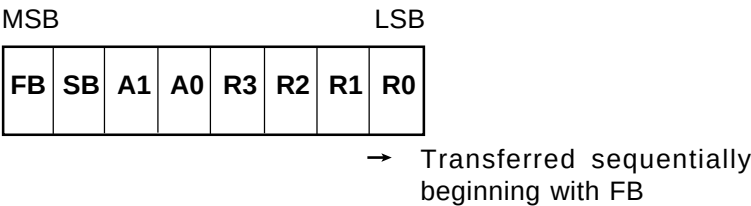
[Instruction Code/Number of Cycles]

Page= 288

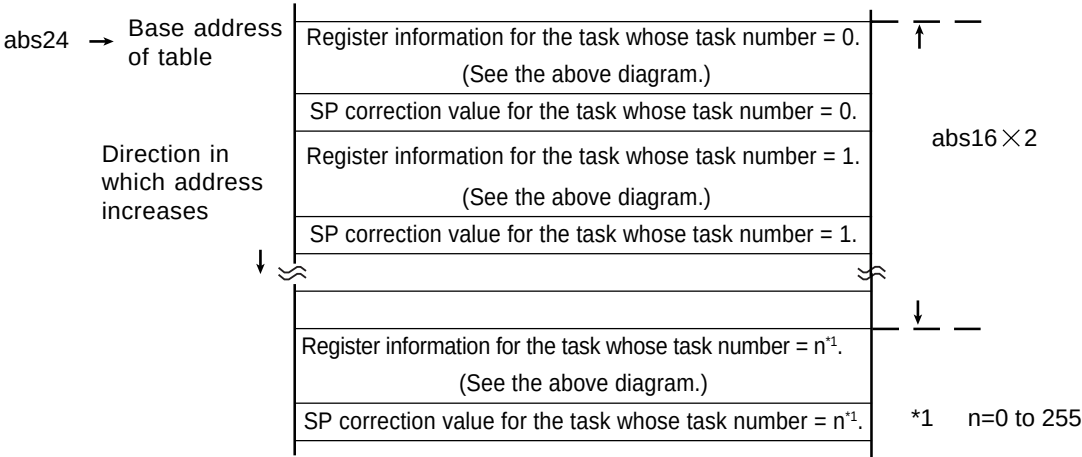
[Operation]

[Function]

- This instruction saves task context to the stack area.
- Set the RAM address that contains the task number in abs16 and the start address of table data in abs24.
- The required register information is specified from table data by the task number and the data in the stack area is transferred to each register according to the specified register information. Then the SP correction value is subtracted to the stack pointer (SP). For this SP correction value, set the number of bytes you want to be transferred. Calculated as 2 bytes when transferring the R0, R1, R2, or R3 registers. A0, A1, SB, and FB are calculated as 4 bytes.
- Information on transferred registers is configured as shown below. Logic 1 indicates a register to be transferred and logic 0 indicates a register that is not transferred.



- The table data is comprised as shown below. The address indicated by abs24 is the base address of the table. The data stored at an address apart from the base address as much as twice the content of abs16 indicates register information, and the next address contains the stack pointer correction value.



[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

STCTX Ram,Rom_TBL

STNZ

Conditional transfer
STore on Not Zero

STNZ

[Syntax]

[Instruction Code/Number of Cycles]

STNZ.size src,dest

└──────────────────────────┘ B , W

Page= 288

[Operation]

if $Z = 0$ **then** $\text{dest}[\text{dest}] \leftarrow \text{src}$

[Function]

- This instruction transfers *src* to *dest* when the Z flag is 0. *dest* is not changed when the Z flag is 1.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0.

[Selectable src/dest]

src				dest*1			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-		R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

STNZ.B #5,Ram:8[SB]

STNZ.W #15,[[A1]]

STZ

Conditional transfer

STZ

[Syntax]

[Instruction Code/Number of Cycles]

STZ.size **src,dest**

Page= 289

B , W

[Operation]

```
if Z = 1 then   dest/[dest] ← src
```

[Function]

- This instruction transfers *src* to *dest* when the Z flag is 1. *dest* is not changed when the Z flag is 1.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0.

[Selectable src/dest]

src				dest*1			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-		R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

STZ.B #5,Ram:8[SB]

STZ.W #10,[[A0]]

STZX

Conditional transfer
STore on Zero eXtention

STZX

[Syntax]

[Instruction Code/Number of Cycles]

STZX.size **src1,src2,dest**

|_____ **B , W**

Page= 289

[Operation]

If $Z = 1$ then $\text{dest} \leftarrow \text{src1}$	If $Z = 1$ then $[\text{dest}] \leftarrow \text{src1}$
else $\text{dest} \leftarrow \text{src2}$	else $[\text{dest}] \leftarrow \text{src2}$

[Function]

- This instruction transfers *src1* to *dest* when the Z flag is 1. When the Z flag is 0, it transfers *src2* to *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0.

[Selectable src/dest]

src				dest ^{*1}			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-		R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

STZX.B #1,#2,Ram:8[SB]

STZX.W #5,#10,[R0]

SUB

Subtract without borrow
SUBtract

SUB

[Syntax]

[Instruction Code/Number of Cycles]

SUB.size (:format) **src,dest**

G , S (Can be specified)

B , W , L

Page=290

[Operation]

```
dest ← dest - src      [dest] ← [dest] - src
dest ← dest - [src]    [dest] ← [dest] - [src]
```

[Function]

- This instruction subtracts *src* from *dest* and stores the result in *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0. Also, when *src* is the address register, the 8 low-order bits of the address register are used as data to be operated on.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.
- When (.L) is specified for the size specifier (.size) and *dest* is the address register, *dest* is zero-extended to perform operation in 32 bits. The 24 low-order bits of the operation result are stored in *dest*. When *src* is the address register, *src* is zero-extended to perform operation in 32 bits. The flags also change states depending on the result of 32-bit operation.

[Selectable src/dest]*¹

(See the next page for *src/dest* classified by format.)

src				dest			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-		R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]	A0/A0/A0*2	A1/A1/A1*2	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16/#IMM32							

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-. SP/SP/SP, dsp:8[SP], and #IMM.

*2 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O : The flag is set when a signed operation resulted in exceeding +2147483647(.L) or -2147483648(.L), +32767 (.W) or -32768 (.W), or +127 (.B) or -128 (.B); otherwise cleared.
- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when an unsigned operation resulted in any value equal to or greater than 0; otherwise cleared.

[Description Example]

SUB.B	A0,R0L	; A0's 8 low-order bits and R0L are operated on.
SUB.B	R0L,A0	; R0L is zero-expanded and operated with A0.
SUB.B	Ram:8[SB],R0L	
SUB.W	#2,[A0]	

[src/dest Classified by Format]**G format*¹**

src				dest			
R0L/R0/R2R0	R0H/R2/-			R0L/R0/R2R0	R0H/R2/-		
R1L/R1/R3R1	R1H/R3/-			R1L/R1/R3R1	R1H/R3/-		
A0/A0/A0* ²	A1/A1/A1* ²	[A0]	[A1]	A0/A0/A0* ²	A1/A1/A1* ²	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16/#IMM32							

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

S format

src				dest* ³			
R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16	R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16
#IMM8/#IMM16* ⁴							

*3 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*4 You can specify only (.B) or (.W) for the size specifier (.size).

SUBX

Subtract extend without borrow
SUBtract eXtend

SUBX

[Syntax]

SUBX *src*,*dest*

[Instruction Code/Number of Cycles]

Page= 294

[Operation]

dest ← *dest* - EXT(*src*)
dest ← *dest* - EXT([*src*])

[*dest*] ← [*dest*] - EXT(*src*)
[*dest*] ← [*dest*] - EXT([*src*])

[Function]

- This instruction subtracts 8-bit *src* from *dest* (32 bits) after sign-extending *src* to 32 bits and stores the result in *dest*.
- When *dest* is the address register (A0, A1), *dest* is zero-extended to perform operation in 32 bits. The 24 low-order bits of the operation result are stored in *dest*. The flags also change states depending on the result of 32-bit operation.

[Selectable src/dest]*1

src				dest			
R0L/R0/R2R0		R0H/R2/-		R0L/R0/R2R0		R0H/R2/-	
R1L/R1/R3R1		R1H/R3/-		R1L/R1/R3R1		R1H/R3/-	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8							

*1 Indirect addressing [*src*] and [*dest*] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	○	—	○	○	—	○

Conditions

- O : The flag is set when a signed operation resulted in exceeding +2147483647(.L) or -2147483648(.L); otherwise cleared.
- S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
- Z : The flag is set when the operation resulted in 0; otherwise cleared.
- C : The flag is set when an unsigned operation resulted in any value equal to or greater than 0; otherwise cleared.

[Description Example]

SUBX R0L,A0
SUBX Ram:8[SB],R2R0
SUBX #2,[A0]

TST

Test
TeST

TST

[Syntax]**[Instruction Code/Number of Cycles]****TST.size(:format)****src,dest**

Page= 296

└──────────────────┬──────────┘ **G , S** (Can be specified)
 └──────────────────┬──────────┘ **B , W**

[Operation]**dest** $\wedge$ **src****[Function]**

- Each flag in the flag register changes state depending on the result of logical AND of *src* and *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. Also, when *src* is the address register, the 8 low-order bits of the address register are used as data to be operated on.
- When (.W) is specified for the size specifier (.size) and *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest](See the next page for *src/dest* classified by format.)

src				dest			
R0L/R0/ R2R0		R0H/R2/ R0		R0L/R0/ R2R0		R0H/R2/ R0	
R1L/R1/ R3R1		R1H/R3/ R1		R1L/R1/ R3R1		R1H/R3/ R1	
A0/A0/ A0 *1	A1/A1/ A1 *1	[A0]	[A1]	A0/A0/ A0 *1	A1/A1/ A1 *1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

- S** : The flag is set when the operation resulted in MSB = 1; otherwise cleared.
Z : The flag is set when the operation resulted in 0; otherwise cleared.

[Description Example]

TST.B #3,R0L

TST.B A0,R0L

; A0's 8 low-order bits and R0L are operated on.

TST.B R0L,A0

; R0L is zero-expanded and operated on with A0.

[src/dest Classified by Format]**G format**

src				dest			
R0L/R0/ R2 R0		R0H/R2/ -		R0L/R0/ R2 R0		R0H/R2/ -	
R1L/R1/ R3 R1		R1H/R3/ -		R1L/R1/ R3 R1		R1H/R3/ -	
A0/A0/ A0 ^{*1}	A1/A1/ A1 ^{*1}	[A0]	[A1]	A0/A0/ A0 ^{*1}	A1/A1/ A1 ^{*1}	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

S format

src				dest			
R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16	R0L/R0	dsp:8[SB]	dsp:8[FB]	abs16
#IMM8/#IMM16							

UND

Interrupt for undefined instruction
UNDEfined instruction

UND

[Syntax]
UND

[Instruction Code/Number of Cycles]
Page= 298

[Operation]

SP ← SP - 2
M(SP) ← FLG
SP ← SP - 2
M(SP)*1 ← (PC + 1)_H
SP ← SP - 2
M(SP) ← (PC + 1)_L
PC ← M(FFFFDC₁₆)
*1 The 8 high-order bits become indeterminate.

[Function]

- This instruction generates an undefined instruction interrupt.
- The undefined instruction interrupt is a nonmaskable interrupt.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	○	○	—	—	—	—	○	—

*1 The flags are saved to the stack area before the UND instruction is executed. After the interrupt, the flag status becomes as shown on the left.

Conditions

- U : The flag is cleared.
- I : The flag is cleared.
- D : The flag is cleared.

[Description Example]

UND

WAIT

[Syntax]
WAIT

Wait
WAIT

WAIT

[Instruction Code/Number of Cycles]
Page= 298

[Operation]

[Function]

- Stops program execution. Program execution is restarted when an interrupt whose priority is higher than that of the stop/wait restoring interrupt priority setup bit is accepted or a reset is generated.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

WAIT

XCHG

Exchange
eXCHAnGe

XCHG

[Syntax]

XCHG.size **src,dest**

B , W

[Instruction Code/Number of Cycles]

Page= 299

[Operation]

$$\text{dest}/[\text{dest}] \quad \longleftrightarrow \quad \text{src}$$

[Function]

- This instruction exchanges contents between *src* and *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is address register(A0, A1), 24 bits of zero-expanded *src* data are placed in the address register and the 8 low-order bits of the address register are placed in *src*.
- When (.W) is specified for the size specifier (.size) and *dest* is address register, 24 bits of zero-expanded *src* data are placed in the address register and the 16 low-order bits of the address register are placed in *src*. When *src* is address register, 24 bits data are placed in the address register and the 16 low-order bits of the address register are placed in *dest*.

[Selectable src/dest]

src				dest*1			
R0L/R0/ R2R0		R0H/R2/ -		R0L/R0/ R2R0		R0H/R2/ -	
R1L/R1/ R3R1		R1H/R3/ -		R1L/R1/ R3R1		R1H/R3/ -	
A0/A0/A0	A1/A1/A1	[A0]	[A1]	A0/A0/A0	A1/A1/A1	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM							

*1 Indirect addressing [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	—	—	—	—

[Description Example]

XCHG.B R0L,A0 ; A0's 8 low-order bits and R0L's zero-expanded value are exchanged.

```
XCHG.W    R0,A1
```

```
XCHG.B    R0L,[A0]
```

XOR

Exclusive OR
eXclusive OR

XOR

[Syntax]

[Instruction Code/Number of Cycles]

XOR.size **src,dest**

Page= 299

B , W

[Operation]

$$\text{dest} \leftarrow \text{dest} \quad \forall \quad \text{src}$$
$$[\text{dest}] \leftarrow [\text{dest}] \vee \text{src}$$
$$\text{dest} \leftarrow \text{dest} \vee [\text{src}]$$
$$[\text{dest}] \leftarrow [\text{dest}] \vee [\text{src}]$$

[Function]

- This instruction exclusive ORs *src* and *dest* together and stores the result in *dest*.
- When (.B) is specified for the size specifier (.size) and *dest* is the address register (A0, A1), *src* is zero-extended to perform operation in 16 bits. In this case, the 8 high-order bits become 0. Also, when *src* is the address register, the 8 low-order bits of the address register are used as data to be operated on.
- When (.W) is specified for the size specifier (.size) and *dest* is the address register, the 8 high-order bits become 0. Also, when *src* is the address register, the 16 low-order bits of the address register are the data to be operated on.

[Selectable src/dest]*¹

src				dest			
R0L/R0/ R2R0		R0H/R2/ R0		R0L/R0/ R2R0		R0H/R2/ R0	
R1L/R1/ R3R1		R1H/R3/ R1		R1L/R1/ R3R1		R1H/R3/ R1	
A0/A0/ A0 *2	A1/A1/ A1 *2	[A0]	[A1]	A0/A0/ A0 *2	A1/A1/ A1 *2	[A0]	[A1]
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16
#IMM8/#IMM16							

*1 Indirect addressing [src] and [dest] can be used in all addressing except R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

*2 When you specify (.B) for the size specifier (.size), you cannot choose A0 and/or A1 for *src* and *dest* simultaneously.

[Flag Change]

Flag	U	I	O	B	S	Z	D	C
Change	—	—	—	—	○	○	—	—

Conditions

S : The flag is set when the operation resulted in MSB = 1; otherwise cleared.

Z : The flag is set when the operation resulted in 0; otherwise cleared.

[Description Example]

XOR.B A0,R0L

: A0's 8 low-order bits and R0L are exclusive ORed.

XOR.B R0L,A0

; R0L is zero-expanded and exclusive ORed with A0.

```
XOR.B    #3,R0L
```

XOR.W A0,A1

XOR.W [A0],[[A1]]

3.3 Index instructions

This section explains each INDEX instruction individually.

The INDEX instructions are provided for use on arrays. The execution addresses are derived by unsigned adding the addresses indicated by src and dest of the next instruction to be executed after the INDEX instruction to the content of src of the INDEX instruction.

The modifiable size is from 0 to 65535(64KB).

No interrupt request is not accepted immediately after the INDEX instruction.

The 10 types of INDEX instructions shown below are supported.

(1) INDEXB.size src

The INDEXB (INDEX Byte) instruction is used for arrays arranged in bytes.

The execution addresses for the INDEXB instruction are derived by unsigned adding the src content of the INDEXB instruction to the addresses indicated by src and dest of the next instruction to be executed.

For the next instruction executed after the INDEXB instruction, be sure to choose memory for both src and dest. Also, specify .B for the size specifier.

Example:

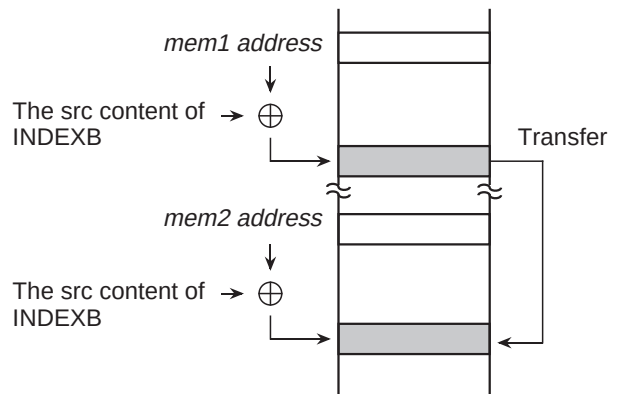
```
INDEXB.B      src
MOV.B:G      mem1,mem2
```

Specify .B Memory

Operation in C language

```
char      src;
char      mem1[],mem2[];
```

```
mem2[src] = mem1[src];
```



Instruction which is modified by INDEXB

The src and dest of

ADC, ADD:G^{*1*2}, AND, CMP:G^{*1}, MAX, MIN, MOV:G^{*1*3}, MUL, MULU, OR, SBB, SUB, TST, XOR.

*1 You can only specify G format.

*2 The SP can not be used in dest of ADD instruction.

*3 The dsp:8[SP] can not be used in src or dest of MOV instruction.

Only above instructions can be used next to INDEXB instruction.

(2) INDEXBD.size src

The INDEXBD (INDEX Byte Dest) instruction is used for arrays arranged in bytes.

The execution addresses for the INDEXBD instruction are derived by unsigned adding the src content of the INDEXBD instruction to the addresses indicated by dest(some instructions are src) of the next instruction to be executed.

For the next instruction executed after the INDEXBD instruction, be sure to choose memory for dest(some instructions are src). Also, specify .B for the size specifier.

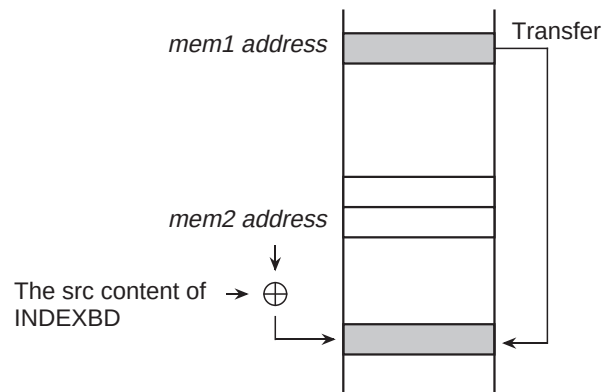
Example:

```
INDEXBD.B      src
MOV.B:G        mem1,mem2
    Specify .B      Memory
```

Operation in C language

```
char      src,mem1;
char      mem2[];
```

```
mem2[src] = mem1;
```

**Instruction which is modified by INDEXBD**

The dest of

ABS, ADC, ADCF, ADD:G^{*1*2}, AND, CLIP, CMP:G^{*1}, DEC, INC, MAX, MIN, MOV:G^{*1*3}, MUL, MULU, NEG, NOT, OR, POP, ROLC, RORC, ROT, SBB, SHA, SHL, STNZ, STZ, STZX, SUB, TST, XCHG, XOR.

The src of

DIV, DIVU, DIVX, PUSH

*1 You can only specify G format.

*2 The SP can not be used in dest of ADD instruction.

*3 The dsp:8[SP] can not be used in src or dest of MOV instruction.

Only above instructions can be used next to INDEXBD instruction.

(3) INDEXBS.size src

The INDEXBS (INDEX Byte Src) instruction is used for arrays arranged in bytes.

The execution addresses for the INDEXBS instruction are derived by unsigned adding the src content of the INDEXBS instruction to the addresses indicated by src of the next instruction to be executed.

For the next instruction executed after the INDEXBS instruction, be sure to choose memory for src. Also, specify .B for the size specifier.

Example:

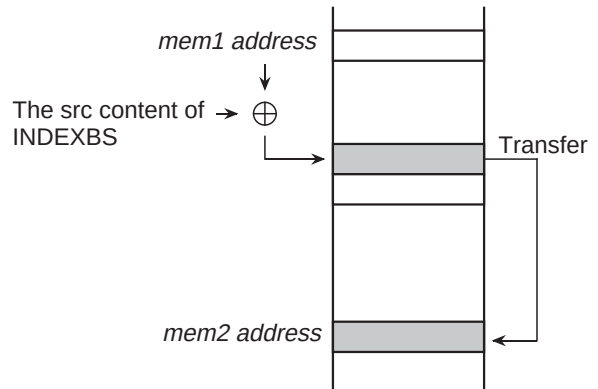
```
INDEXBS.B      src
MOV.B:G      mem1, mem2
```

Specify .B Memory

Operation in C language

```
char      src, mem2;
char      mem1[];
```

```
mem2 = mem1[src];
```

**Instruction which is modified by INDEXBS**

The src of

ADC, ADD:G^{*1*2}, AND, CMP:G^{*1}, MAX, MIN, MOV:G^{*1*3}, MUL, MULU, OR, SBB, SUB, TST, XOR

*1 You can only specify G format.

*2 The SP can not be used in dest of ADD instruction.

*3 The dsp:8[SP] can not be used in src or dest of MOV instruction.

Only above instructions can be used next to INDEXBS instruction.

(4)INDEXW.size src

The INDEXW (INDEX Word) is used for arrays arranged in words.

The execution addresses for the INDEXW instruction are derived by unsigned adding twice the src content of the INDEXW instruction to the addresses indicated by src and dest of the next instruction to be executed. The range of src of INDEXW instruction that can be taken on is from 0 to 32767. You can not set otherwise.

For the next instruction executed after the INDEXW instruction, be sure to choose memory for both src and dest. Also, specify .W for the size specifier.

Example:

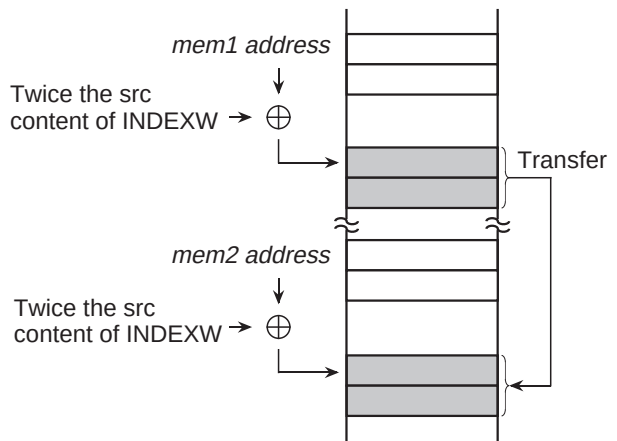
```
INDEXW.B            src
MOV.W:G            mem1,mem2
```

Specify .W Memory

Operation in C language

```
char                src;
char                mem1[],mem2[];
```

```
mem2[src] = mem1[src];
```

**Instruction which is modified by INDEXW**

The src and dest of

ADC, ADD:G^{*1*2}, AND, CMP:G^{*1}, MAX, MIN, MOV:G^{*1*3}, MUL, MULU, OR, SBB, SUB, TST, XOR.

*1 You can only specify G format.

*2 The SP can not be used in dest of ADD instruction.

*3 The dsp:8[SP] can not be used in src or dest of MOV instruction.

Only above instructions can be used next to INDEXW instruction.

(5) INDEXWD.size src

The INDEXWD (INDEX Word Dest) is used for arrays arranged in words.

The execution addresses for the INDEXWD instruction are derived by unsigned adding twice the src content to the addresses indicated by dest (some instructions are src) of the next instruction to be executed.

The range of src of INDEXWD instruction that can be taken on is from 0 to 32767. You cannot set otherwise.

For the next instruction executed after the INDEXWD instruction, be sure to choose memory for dest (some instructions are src). Also, specify .W for the size specifier.

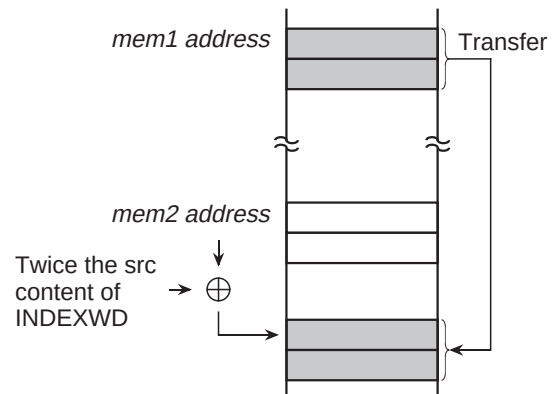
Example:

```
INDEXWD.B      src
MOV.W:G        mem1,mem2
    Specify .W      Memory
```

Operation in C language

```
char      src;
int        mem1;
int        mem2[];
```

```
mem2[src] = mem1;
```

**Instruction which is modified by INDEXWD**

The dest of

ABS, ADC, ADCF, ADD:G^{*1*2}, AND, CLIP, CMP:G^{*1}, DEC, INC, MAX, MIN, MOV:G^{*1*3},
MUL, MULU, NEG, NOT, OR, POP, ROLC, RORC, ROT, SBB, SCcnd, SHA, SHL,
STNZ, STZ, STZX, SUB, TST, XCHG, XOR.

The src of

DIV, DIVU, DIVX, PUSH, JMPI, JSRI.

*1 You can only specify G format.

*2 The SP can not be used in dest of ADD instruction.

*3 The dsp:8[SP] can not be used in src or dest of MOV instruction.

Only above instructions can be used next to INDEXWD instruction.

(6) INDEXWS.size src

The INDEXWS (INDEX Word Src) is used for arrays arranged in words.

The execution addresses for the INDEXWS instruction are derived by unsigned adding twice the src content of the INDEXWS instruction to the addresses indicated by src of the next instruction to be executed. The range of src of INDEXWS instruction that can be taken on is from 0 to 32767. You can not set otherwise.

For the next instruction executed after the INDEXWS instruction, be sure to choose memory for src. Also, specify .W for the size specifier.

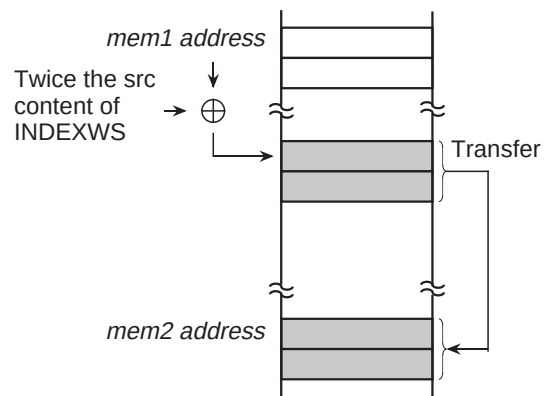
Example:

```
INDEXWS.B      src
MOV.W:G mem1,mem2
    Specify .W      Memory
```

Operation in C language

```
char      src;
int      mem1[];
int      mem2[];
```

```
mem2 = mem1[src];
```

**Instruction which is modified by INDEXWS**

The src of

ADC, ADD:G^{*1*2}, AND, CMP:G^{*1}, MAX, MIN, MOV:G^{*1*3}, MUL, MULU, OR, SBB, SUB, TST, XOR.

*1 You can only specify G format.

*2 The SP can not be used in dest of ADD instruction.

*3 The dsp:8[SP] can not be used in src or dest of MOV instruction.

Only above instructions can be used next to INDEXWS instruction.

(7) INDEXL.size src

The INDEXL (INDEX Long word) is used for arrays arranged in long words.

The execution addresses for the INDEXL instruction are derived by unsigned adding four times the src content of the INDEXL instruction to the addresses indicated by src and dest of the next instruction to be executed. The range of src of INDEXL instruction that can be taken on is from 0 to 16383. You can not set otherwise.

For the next instruction executed after the INDEXL instruction, be sure to choose memory for both src and dest. Also, specify .L for the size specifier.

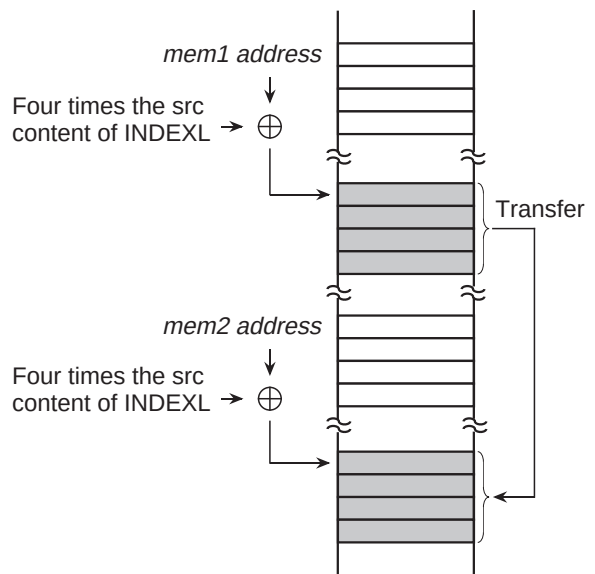
Example:

```
INDEXL.B            src
MOV.L:G            mem1,mem2
   Specify .L       Memory
```

Operation in C language

```
char            src;
long            mem1[],mem2[];

mem2[src] = mem1[src];
```

**Instruction which is modified by INDEXL**

The src and dest of

ADD:G*1*2, CMP:G*1, MOV:G*1*3, SUB.

*1 You can only specify G format.

*2 The SP can not be used in dest of ADD instruction.

*3 The dsp:8[SP] can not be used in src or dest of MOV instruction.

Only above instructions can be used next to INDEXL instruction.

(8) INDEXLD.size src

The INDEXLD (INDEX Long word Dest) is used for arrays arranged in long words.

The execution addresses for the INDEXLD instruction are derived by unsigned adding four times the src content of the INDEXLD instruction to the addresses indicated by dest (some instructions are src) of the next instruction to be executed. The range of src of INDEXLD instruction that can be taken on is from 0 to 16383. You can not set otherwise.

For the next instruction executed after the INDEXLD instruction, be sure to choose memory for dest (some instructions are src). Also, specify .L for the size specifier.

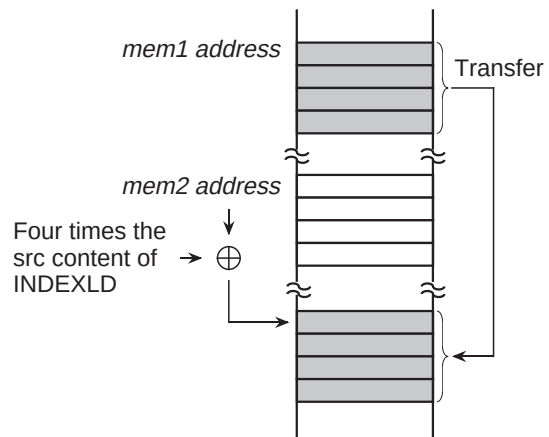
Example:

```
INDEXLD.B      src
MOV.L:G      mem1,mem2
   Specify .L      Memory
```

Operation in C language

```
char      src;
long      mem1;
long      mem2[];
```

```
mem2[src] = mem1;
```

**Instruction which is modified by INDEXLD**

The dest of ADD:G*¹*², CMP:G*¹, MOV:G*¹*³, SUB, SHA, SHL.

The src of JMPL, JSRL.

*1 You can only specify G format.

*2 The SP can not be used in dest of ADD instruction.

*3 The dsp:8[SP] can not be used in src or dest of MOV instruction.

Only above instructions can be used next to INDEXLD instruction.

(9) INDEXLS.size src

The INDEXLS (INDEX Long word Src) is used for arrays arranged in long words.

The execution addresses for the INDEXLS instruction are derived by unsigned adding four times the src content of the INDEXLS instruction to the addresses indicated by src of the next instruction to be executed. The range of src of INDEXLS instruction that can be taken on is from 0 to 16383. You cannot set otherwise.

For the next instruction executed after the INDEXLS instruction, be sure to choose memory for src. Also, specify .L for the size specifier.

Example:

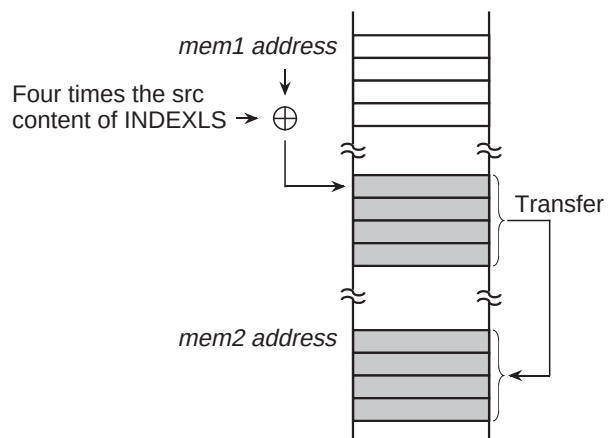
```
INDEXLS.B      src
MOV.L:G      mem1, mem2
```

Specify .L Memory

Operation in C language

```
char      src;
long      mem1[];
long      mem2;
```

```
mem2 = mem1[src];
```

**Instruction which is modified by INDEXLS**

The src of ADD:G*1*2, CMP:G*1, MOV:G*1*3, SUB.

*1 You can only specify G format.

*2 The SP can not be used in dest of ADD instruction.

*3 The dsp:8[SP] can not be used in src or dest of MOV instruction.

Only above instructions can be used next to INDEXLS instruction.

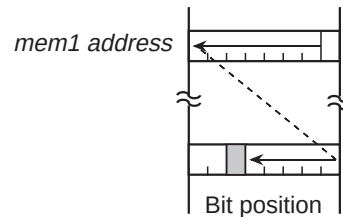
(10) BITINDEX.size src

The BITINDEX instruction is operated on the bit that is apart from bit 0 of the address indicated by dest as many bits as indicated by src of BITINDEX.

Make sure the next instruction to be executed after BITINDEX is a bit instruction. Also, be sure to specify memory for src or dest.

Example:

BITINDEX.B/W src
BSET 3,mem1
 Bit instruction Memory
 Becomes invalid.

**Instruction which is modified by BITINDEX**

The src of BAND, BNAND, BNOR, BNTST, BNXOR, BOR, BTST:G^{*1}, BXOR.

The dest of BCLR, BMcnd, BNOT, BSET, BTSTC, BTSTS.

*1 You can only specify G format.

(11) Next instructions that can be executed after INDEX

The table below lists the next instructions that can be executed after each INDEX instruction.

	Valid instruction	
INDEXB.B/.W*2	ADC, ADD:G*4, AND, CMP:G, MAX, MIN, MOV:G*3, MUL, MULU, OR, SBB, SUB, TST, XOR The src and dest of above instructions.	
INDEXBD.B/.W*2	ABS, ADC, ADCF, ADD:G*4, AND, CLIP, CMP:G, DEC, INC, MAX, MIN, MOV:G*3, MUL, MULU, NEG, NOT, OR, POP, ROLC, RORC, ROT, SBB, SCcnd, SHA, SHL, STNZ, STZ, STZX, SUB, TST, XCHG, XOR The dest of above instructions.	DIV, DIVU, DIVX, PUSH The src of above instructions.
INDEXBS.B/.W*2	ADC, ADD:G*4, AND, CMP:G, MAX, MIN, MOV:G*3, MUL, MULU, OR, SBB, SUB, TST, XOR The src of above instructions.	
INDEXW.B/.W*2	ADC, ADD:G*4, AND, CMP:G, MAX, MIN, MOV:G*3, MUL, MULU, OR, SBB, SUB, TST, XOR The src and dest of above instructions.	
INDEXWD.B/.W*2	ABS, ADC, ADCF, ADD:G*4, AND, CLIP, CMP:G, DEC, INC, MAX, MIN, MOV:G*3, MUL, MULU, NEG, NOT, OR, POP, ROLC, RORC, ROT, SBB, SHA, SHL, STNZ, STZ, STZX, SUB, TST, XCHG, XOR The dest of above instructions.	DIV, DIVU, DIVX, PUSH, JMPI, JSRI The src of above instructions.
INDEXWS.B/.W*2	ADC, ADD:G*4, AND, CMP:G, MAX, MIN, MOV:G*3, MUL, MULU, OR, SBB, SUB, TST, XOR The src of above instructions.	
INDEXL.B/.W*2	ADD:G*4, CMP:G, MOV:G*3, SUB The src and dest of above instructions.	
INDEXLD.B/.W*2	ADD:G*4, CMP:G, MOV:G*3, SHA, SHL, SUB The dest of above instructions.	JMPI*1, JSRI*1 The src of above instructions.
INDEXLS.B/.W*2	ADD:G*4, CMP:G, MOV:G*3, SUB The src of above instructions.	
BITINDEX.B/.W	BAND, BNAND, BNOR, BNTST, BNXOR, BOR, BTST:G, BXOR The src of above instructions.	BCLR, BMcnd, BNOT, BSET, BTSTC, BTSTS The dest of above instructions.

*1 Since the size is specified for .A(3 bytes) by .L(4 bytes), care must be taken when using the data table.

*2 The ADD, CMP, and MOV instructions are valid in only the G format.

*3 The dsp:8[SP] cannot be used in src or dest of MOV instruction.

*4 The SP cannot be used in src or dest of ADD instruction.

(12) Addressing modes

The table below lists the addressing modes that become valid in the next instructions that can be executed after INDEX. Indirect addressing modes can be used in each instruction.

src				dest			
[A0]	[A1]			[A0]	[A1]		
dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]	dsp:8[A0]	dsp:8[A1]	dsp:8[SB]	dsp:8[FB]
dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]	dsp:16[A0]	dsp:16[A1]	dsp:16[SB]	dsp:16[FB]
dsp:24[A0]	dsp:24[A1]	abs24	abs16	dsp:24[A0]	dsp:24[A1]	abs24	abs16

*1 For the MOV instruction you cannot use dsp8:[SP].

*2 The SP in the ADD instruction cannot be used.

*3 You cannot use R0L/R0/R2R0, R0H/R2/-, R1L/R1/R3R1, R1H/R3/-, SP/SP/SP, dsp:8[SP], and #IMM.

Chapter 4

Instruction Code/Number of Cycles

4.1 Guide to This Chapter

4.2 Instruction Code/Number of Cycles

4.1 Guide to This Chapter

This chapter describes instruction code and number of cycles for each op-code.

The following shows how to read this chapter by using an actual page as an example.

Chapter 4 Instruction Code
4.2 Instruction Code/Number of Cycles

LDIPL

(1) **(1) LDIPL #IMM**

(3)

(4)

[Number of Bytes/Number of Cycles]
Bytes/Cycles 2/2

MAX

(1) **(1) MAX.size #IMM,dest**

(3)

.size	SIZE
.B	0
.W	1

dest		d4	d3	d2	d1	d0
Rn	R0L/R0/---	1	0	0	1	0
	R1L/R1/---	1	0	0	1	1
	R0H/R2/-	1	0	0	0	0
	R1H/R3/-	1	0	0	0	1
An	A0	0	0	0	1	0
	A1	0	0	0	1	1
[An]	[A0]	0	0	0	0	0
	[A1]	0	0	0	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0
	dsp:8[A1]	0	0	1	0	1

dest		d4	d3	d2	d1	d0
dsp:8[SB]	dsp:8[SB]	0	0	1	1	0
	dsp:8[FB]	0	0	1	1	1
dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	dsp:16[A1]	0	1	0	0	1
dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	dsp:16[FB]	0	1	0	1	1
dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	dsp:24[A1]	0	1	1	0	1
abs16	abs16	0	1	1	1	1
	abs24	0	1	1	1	0

(4) [Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	4/3	4/3	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5

(1) Mnemonic

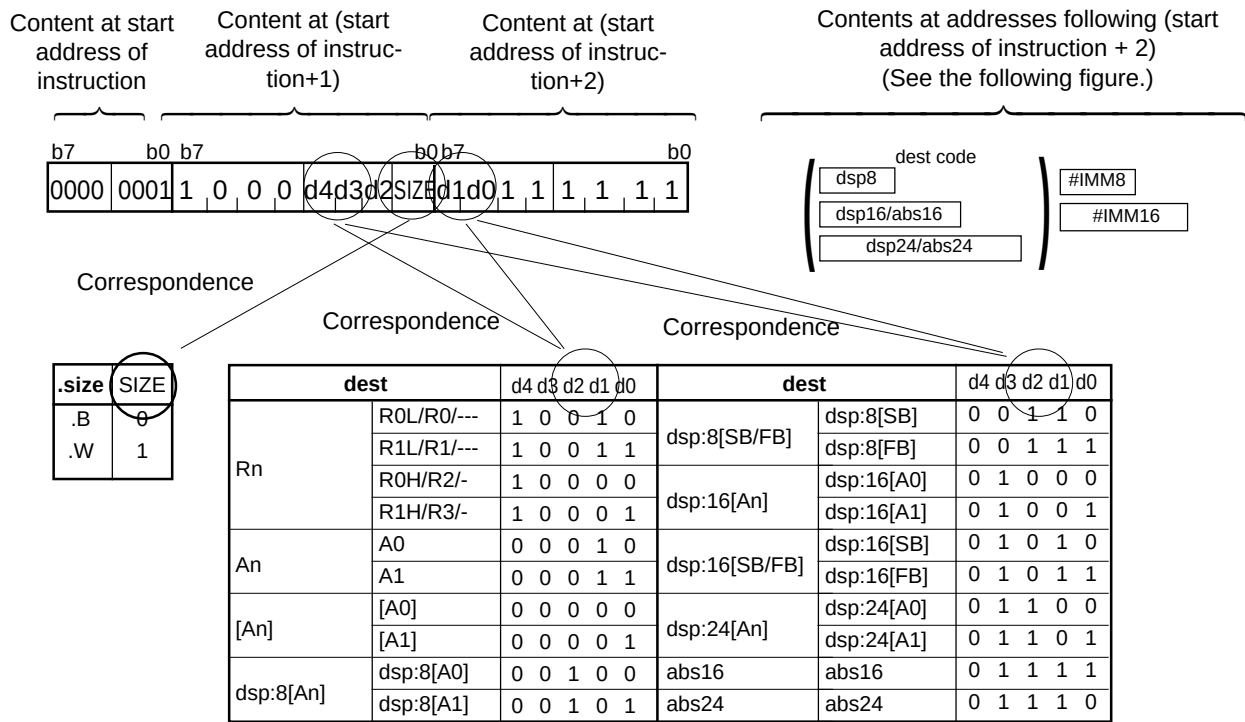
Shows the mnemonic explained in this page.

(2) Syntax

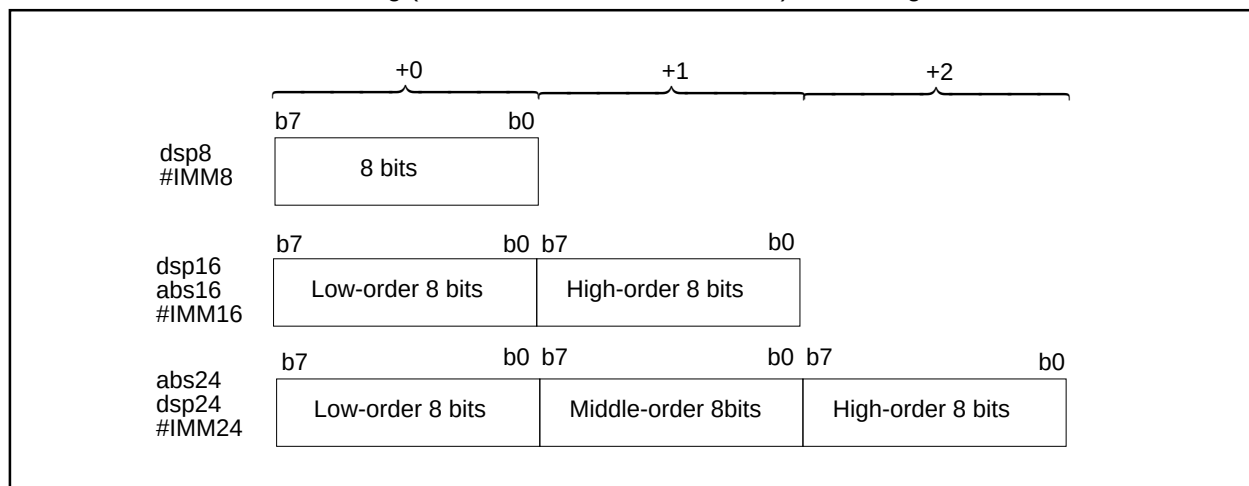
Shows an instruction syntax using symbols.

(3) Instruction code

Shows instruction code. Entered in () are omitted depending on src/dest you selected.



Contents at addresses following (start address of instruction + 2) are arranged as follows:

**(4) Table of cycles**

Shows the number of cycles required to execute this instruction and the number of instruction bytes.

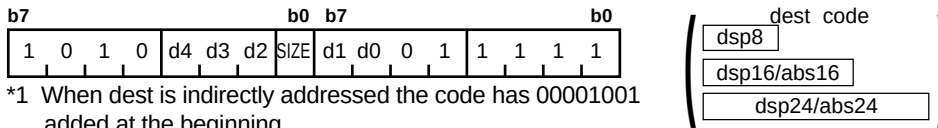
The number of cycles shown are the minimum possible, and they vary depending on the following conditions:

- Number of bytes that have been loaded in the instruction queue buffer
- Accessing of an external memory using 8-bit external bus
- Whether a wait is inserted in the bus cycle

Instruction bytes are indicated on the left side of the slash and execution cycles are indicated on the right side.

ABS

(1) ABS.size dest



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

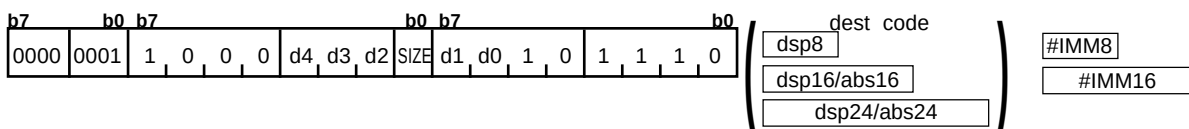
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3, respectively.

ADC

(1) ADC.size #IMM, dest

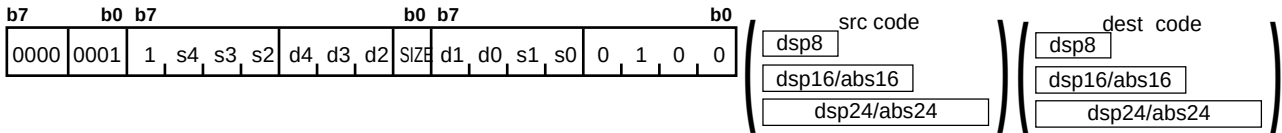


.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	4/1	4/1	4/3	5/3	5/3	6/3	6/3	7/3	6/3	7/3

*1 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

ADC**(2) ADC.size src, dest**

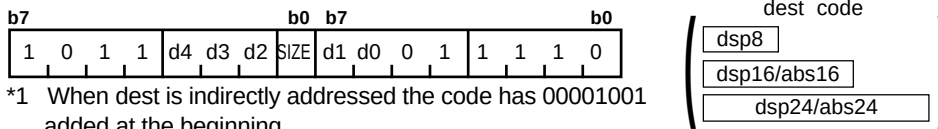
.size	SIZE	src/dest					s4	s3	s2	s1	s0	src/dest					s4	s3	s2	s1	s0
							d4	d3	d2	d1	d0						d4	d3	d2	d1	d0
.B	0	Rn	R0L/R0/---				1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]				0	0	1	1	0
.W	1		R1L/R1/---				1	0	0	1	1		dsp:8[FB]				0	0	1	1	1
			R0H/R2/-				1	0	0	0	0	dsp:16[An]	dsp:16[A0]				0	1	0	0	0
			R1H/R3/-				1	0	0	0	1		dsp:16[A1]				0	1	0	0	1
		An	A0				0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]				0	1	0	1	0
			A1				0	0	0	1	1		dsp:16[FB]				0	1	0	1	1
		[An]	[A0]				0	0	0	0	0	dsp:24[An]	dsp:24[A0]				0	1	1	0	0
			[A1]				0	0	0	0	1		dsp:24[A1]				0	1	1	0	1
		dsp:8[An]	dsp:8[A0]				0	0	1	0	0	abs16	abs16				0	1	1	1	1
			dsp:8[A1]				0	0	1	0	1	abs24	abs24				0	1	1	1	0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3
An	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3
[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:8[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
dsp:16[SB/FB]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
dsp:24[An]	6/3	6/3	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4
abs16	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs24	6/3	6/3	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4

ADCF

(1) ADCF.size dest



.size	SIZE	dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

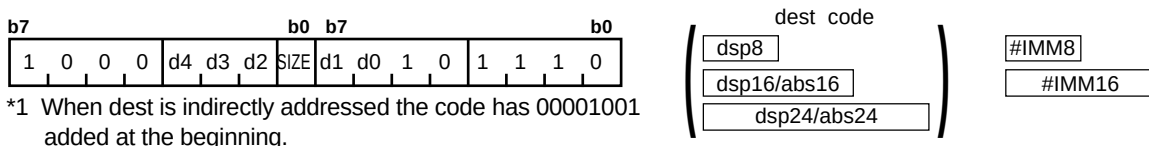
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

ADD

(1) ADD.size:G #IMM,dest



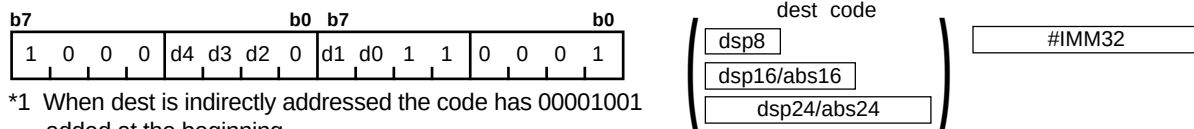
.size	SIZE	dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

ADD**(2) ADD.L:G #IMM,dest**

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

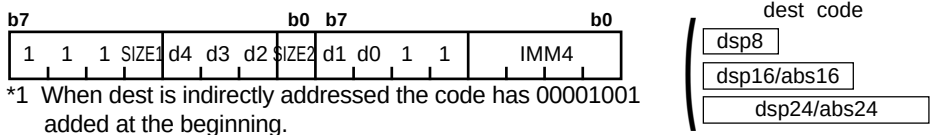
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	6/2	6/2	6/5	7/5	7/5	8/5	8/5	9/5	8/5	9/5

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

ADD

(3) ADD.size:Q #IMM, dest



.size	SIZE1	SIZE2
.B	0	0
.W	0	1
.L	1	0

#IMM	IMM4	#IMM	IMM4
0	0 0 0 0	-8	1 0 0 0
+1	0 0 0 1	-7	1 0 0 1
+2	0 0 1 0	-6	1 0 1 0
+3	0 0 1 1	-5	1 0 1 1
+4	0 1 0 0	-4	1 1 0 0
+5	0 1 0 1	-3	1 1 0 1
+6	0 1 1 0	-2	1 1 1 0
+7	0 1 1 1	-1	1 1 1 1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

When (.B) and (.W) is specified for the size specifier (.size)

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

When (.L) is specified for the size specifier (.size)

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/2	2/2	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4

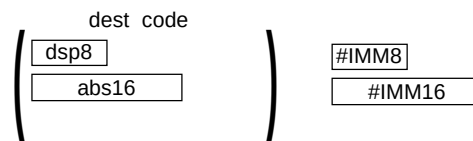
*3 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

ADD**(4) ADD.size:S #IMM, dest**

b7	b6	b5	b4	b3	b2	b1	b0
0	0	d1	d0	0	1	1	SIZE

*1 When dest is indirectly addressed the code has 00001001 added at the beginning.

.size	SIZE	dest	d1	d0
.B	0	Rn	0	0
.W	1	dsp:8[SB]	1	0
		dsp:8[FB]	1	1
		abs16	0	1



dest	Rn	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/1	3/3	4/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier (.size) the number of bytes in the table is increased by 1.

ADD**(5) ADD.L:S #IMM, A0/A1**

b7	b6	b5	b4	b3	b2	b1	b0
1	0	IMM	0	1	1	0	d0

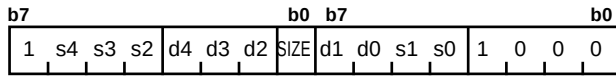
#IMM	IMM	A0/A1	d0
#1	0	A0	0
#2	1	A1	1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	1/2
--------------	-----

ADD

(6) ADD.size:G src, dest

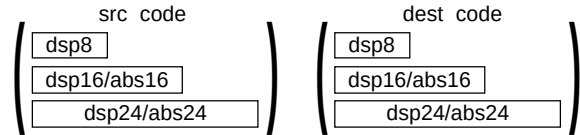


*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed

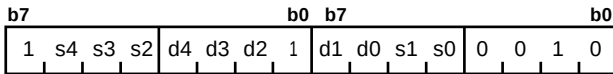


.size	SIZE	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
.B	0						
.W	1						
Rn		R0L/R0/---		1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
		R1L/R1/---		1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
		R0H/R2/-		1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
		R1H/R3/-		1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An		A0		0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
		A1		0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]		[A0]		0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
		[A1]		0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]		dsp:8[A0]		0 0 1 0 0	abs16	abs16	0 1 1 1 1
		dsp:8[A1]		0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
An	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
[An]	2/3	2/3	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4
dsp:8[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[SB/FB]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:16[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:24[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs16	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
abs24	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

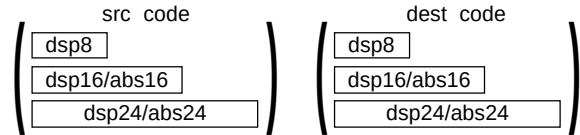
ADD**(7) ADD.L:G src, dest**

*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

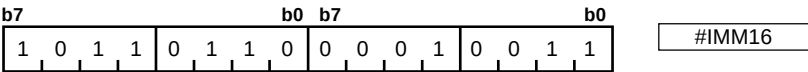
[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
An	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
[An]	2/5	2/5	2/8	3/8	3/8	4/8	4/8	5/8	4/8	5/8
dsp:8[An]	3/5	3/5	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8
dsp:8[SB/FB]	3/5	3/5	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8
dsp:16[An]	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
dsp:16[SB/FB]	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
dsp:24[An]	5/5	5/5	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8
abs16	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
abs24	5/5	5/5	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

ADD

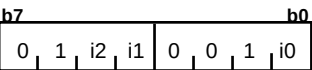
(8) ADD.L:G #IMM16, SP



[Number of Bytes/Number of Cycles]	
Bytes/Cycles	4/2

ADD

(9) ADD.L:Q #IMM3, SP



#IMM3	i2 i1 i0	#IMM3	i2 i1 i0
+1	0 0 0	+5	1 0 0
+2	0 0 1	+6	1 0 1
+3	0 1 0	+7	1 1 0
+4	0 1 1	+8	1 1 1

[Number of Bytes/Number of Cycles]	
Bytes/Cycles	1/1

(10) ADD.L:S #IMM8, SP

b7	b0	b7	b0
1 0 1 1	0 1 1 0	0 0 0 0	0 0 1 1

#IMM8

[Number of Bytes/Number of Cycles]

Bytes/Cycles	3/2
--------------	-----

ADDX**(1) ADDX #IMM, dest**

b7	b0	b7	b0
1 0 0 0	d4 d3 d2 0	d1 d0 0 1	0 0 0 1

dest code

dsp8

dsp16/abs16

dsp24/abs24

#IMM8

*1 When dest is indirectly addressed the code has 00001001 added at the beginning.

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

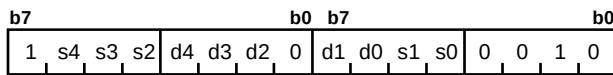
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/2	3/2	3/5	4/5	4/5	5/5	5/5	6/5	5/5	6/5

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

ADDX

(2) ADDX src, dest

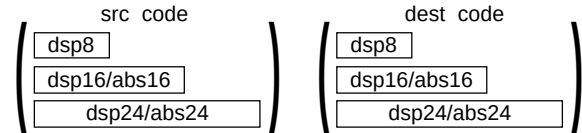


*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
Rn	R0L/---/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	R1L/---/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/---/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/---/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
Rn	---/---/R2R0	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	---/---/R3R1	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	---/---/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	---/---/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
An	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
[An]	2/5	2/5	2/8	3/8	3/8	4/8	4/8	5/8	4/8	5/8
dsp:8[An]	3/5	3/5	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8
dsp:8[SB/FB]	3/5	3/5	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8
dsp:16[An]	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
dsp:16[SB/FB]	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
dsp:24[An]	5/5	5/5	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8
abs16	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
abs24	5/5	5/5	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

ADJNZ

(1) ADJNZ.size #IMM, dest, label



dsp8 (label code) = address indicated by label - (start address of instruction + 2)

.size	SIZE	#IMM	IMM4	#IMM	IMM4
.B	0	0	0 0 0 0	-8	1 0 0 0
.W	1	+1	0 0 0 1	-7	1 0 0 1
		+2	0 0 1 0	-6	1 0 1 0
		+3	0 0 1 1	-5	1 0 1 1
		+4	0 1 0 0	-4	1 1 0 0
		+5	0 1 0 1	-3	1 1 0 1
		+6	0 1 1 0	-2	1 1 1 0
		+7	0 1 1 1	-1	1 1 1 1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

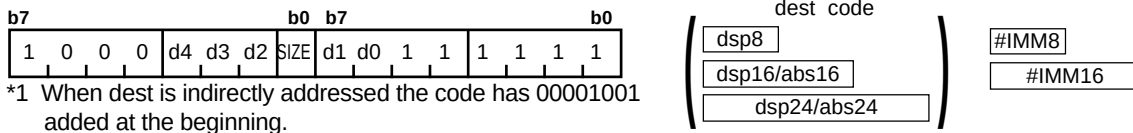
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4

*1 When branched to label, the number of cycles in the table is increased by 2.

AND

(1) AND.size:G #IMM, dest



.size	SIZE	dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

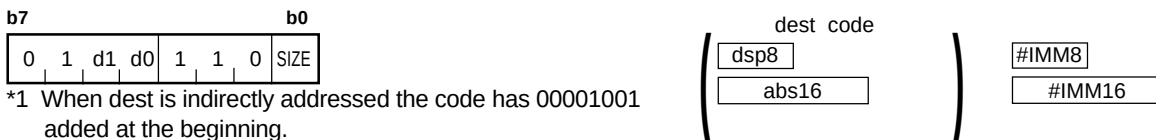
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier (.size) the number of bytes in the table is increased by 1.

AND

(2) AND.size:S #IMM, dest



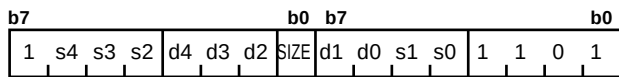
.size	SIZE	dest		d1	d0
.B	0	Rn	R0L/R0	0	0
.W	1		dsp:8[SB]	1	0
		dsp:8[SB/FB]	dsp:8[FB]	1	1
			abs16	0	1

[Number of Bytes/Number of Cycles]

dest	Rn	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/1	3/3	4/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier (.size) the number of bytes in the table is increased by 1.

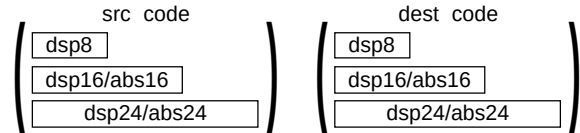
AND**(3) AND.size:G src, dest**

*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



.size	SIZE	src/dest						src/dest					
.B	0	s4	s3	s2	s1	s0		s4	s3	s2	s1	s0	
.W	1	d4	d3	d2	d1	d0		d4	d3	d2	d1	d0	
Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
		1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
		1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

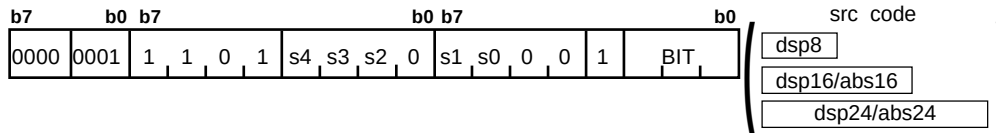
[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
An	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
[An]	2/3	2/3	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4
dsp:8[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[SB/FB]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:16[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:24[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs16	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
abs24	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

BAND

(1) BAND src



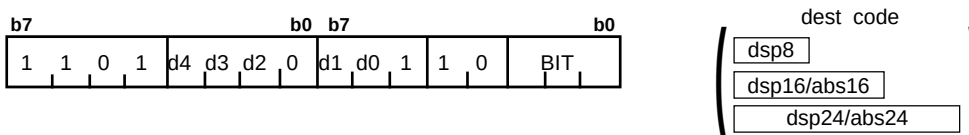
src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1	bit,base:19[An]	bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src	bit,Rn	bit,An	bit,[An]	bit,base:11[An]	bit,base:11[SB/FB]	bit,base:19[An]	bit,base:19[SB/FB]	bit,base:27[An]	bit,base:19	bit,base:27
Bytes/Cycles	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4

BCLR

(1) BCLR dest



dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1	bit,base:27[An]	bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	bit,Rn	bit,An	bit,[An]	bit,base:11[An]	bit,base:11[SB/FB]	bit,base:19[An]	bit,base:19[SB/FB]	bit,base:27[An]	bit,base:19	bit,base:27
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

BITINDEX

(1) BITINDEX.size src

b7	b6	b5	b4	b3	b2	b1	b0
1	1	0	0	d4	d3	d2	SIZE
1	1	0	0	d1	d0	1	0

src code
dsp8
dsp16/abs16
dsp24/abs24

.size	SIZE	src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

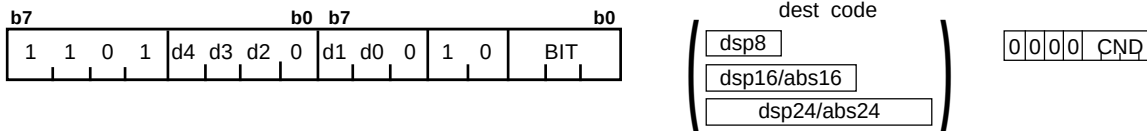
[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/4	2/4	2/6	3/3	3/6	4/6	4/6	5/6	4/6	5/6

*1 The cycles of next instruction to be executed is increased by 1.

BMcnd

(1) BMcnd dest



dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	bit,R0L	1 0 0 1 0	bit,base:11[SB/FB]	bit,base:11[SB]	0 0 1 1 0
	bit,R0H	1 0 0 0 0		bit,base:11[FB]	0 0 1 1 1
	bit,R1L	1 0 0 1 1	bit,base:19[An]	bit,base:19[A0]	0 1 0 0 0
	bit,R1H	1 0 0 0 1		bit,base:19[A1]	0 1 0 0 1
An	bit,A0	0 0 0 1 0	bit,base:19[SB/FB]	bit,base:19[SB]	0 1 0 1 0
	bit,A1	0 0 0 1 1		bit,base:19[FB]	0 1 0 1 1
[An]	bit,[A0]	0 0 0 0 0	bit,base:27[An]	bit,base:27[A0]	0 1 1 0 0
	bit,[A1]	0 0 0 0 1		bit,base:27[A1]	0 1 1 0 1
bit,base:11[An]	bit,base:11[A0]	0 0 1 0 0	bit,base:19	bit,base:19	0 1 1 1 1
	bit,base:11[A1]	0 0 1 0 1	bit,base:27	bit,base:27	0 1 1 1 0

Cnd	CND	Cnd	CND
LTU/NC	0 0 0 0	GEU/C	1 0 0 0
LEU	0 0 0 1	GTU	1 0 0 1
NE/NZ	0 0 1 0	EQ/Z	1 0 1 0
PZ	0 0 1 1	N	1 0 1 1
NO	0 1 0 0	O	1 1 0 0
GT	0 1 0 1	LE	1 1 0 1
GE	0 1 1 0	LT	1 1 1 0

[Number of Bytes/Number of Cycles]

dest	bit,Rn	bit,An	bit,[An]	bit,base:11 [An]	bit,base:11 [SB/FB]	bit,base:19 [An]	bit,base:19 [SB/FB]	bit,base:27 [An]	bit,base:19	bit,base:27
Bytes/Cycles	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4

BM*cnd***(2) BM*cnd* C**

b7				b0				b7				b0			
1	1	0	1	1	0	0	1	0	C	1	0	1	CND		

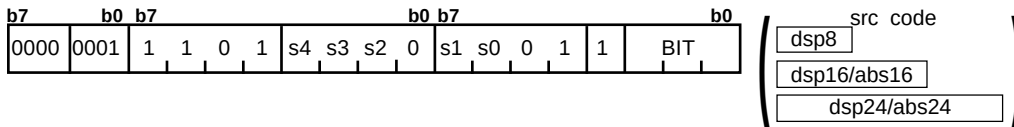
<i>Cnd</i>	C	CND	<i>Cnd</i>	C	CND
LTU/NC	0	0 0 0	GEU/C	1	0 0 0
LEU	0	0 0 1	GTU	1	0 0 1
NE/NZ	0	0 1 0	EQ/Z	1	0 1 0
PZ	0	0 1 1	N	1	0 1 1
NO	0	1 0 0	O	1	1 0 0
GT	0	1 0 1	LE	1	1 0 1
GE	0	1 1 0	LT	1	1 1 0

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/2
--------------	-----

BNAND

(1) BNAND src



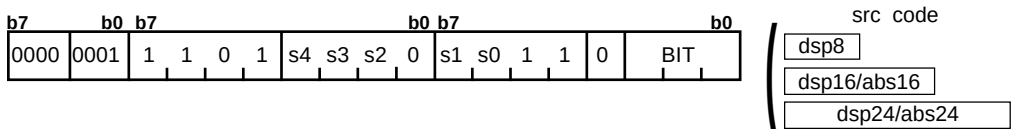
src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1	bit,base:19[An]	bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src	bit,Rn	bit,An	bit,[An]	bit,base:11 [An]	bit,base:11 [SB/FB]	bit,base:19 [An]	bit,base:19 [SB/FB]	bit,base:27 [An]	bit,base:19	bit,base:27
Bytes/Cycles	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4

BNOR

(1) BNOR src



src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1	bit,base:19[An]	bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src	bit,Rn	bit,An	bit,[An]	bit,base:11 [An]	bit,base:11 [SB/FB]	bit,base:19 [An]	bit,base:19 [SB/FB]	bit,base:27 [An]	bit,base:19	bit,base:27
Bytes/Cycles	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4

BNOT

(1) BNOT dest

b7	b0	b7	b0
1 1 0 1	d4 d3 d2 0	d1 d0 0	1 1
			BIT

dest code
dsp8
dsp16/abs16
dsp24/abs24

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	bit,R0L	1 0 0 1 0	bit,base:11[SB/FB]	bit,base:11[SB]	0 0 1 1 0
	bit,R0H	1 0 0 0 0		bit,base:11[FB]	0 0 1 1 1
	bit,R1L	1 0 0 1 1	bit,base:19[An]	bit,base:19[A0]	0 1 0 0 0
	bit,R1H	1 0 0 0 1		bit,base:19[A1]	0 1 0 0 1
An	bit,A0	0 0 0 1 0	bit,base:19[SB/FB]	bit,base:19[SB]	0 1 0 1 0
	bit,A1	0 0 0 1 1		bit,base:19[FB]	0 1 0 1 1
[An]	bit,[A0]	0 0 0 0 0	bit,base:27[An]	bit,base:27[A0]	0 1 1 0 0
	bit,[A1]	0 0 0 0 1		bit,base:27[A1]	0 1 1 0 1
bit,base:11[An]	bit,base:11[A0]	0 0 1 0 0	bit,base:19	bit,base:19	0 1 1 1 1
	bit,base:11[A1]	0 0 1 0 1	bit,base:27	bit,base:27	0 1 1 1 0

[Number of Bytes/Number of Cycles]

dest	bit,Rn	bit,An	bit,[An]	bit,base:11 [An]	bit,base:11 [SB/FB]	bit,base:19 [An]	bit,base:19 [SB/FB]	bit,base:27 [An]	bit,base:19	bit,base:27
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

BNTST

(1) BNTST src

b7	b0	b7	b0	b7	b0
0000	0001	1 1 0 1	s4 s3 s2 0	s1 s0 0 0	0
					BIT

src code
dsp8
dsp16/abs16
dsp24/abs24

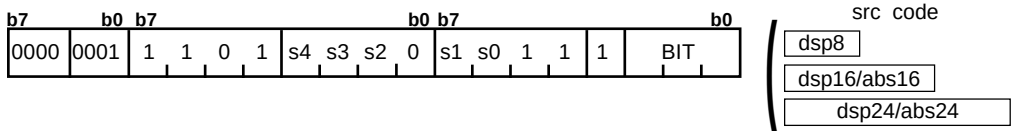
src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
Rn	bit,R0L	1 0 0 1 0	bit,base:11[SB/FB]	bit,base:11[SB]	0 0 1 1 0
	bit,R0H	1 0 0 0 0		bit,base:11[FB]	0 0 1 1 1
	bit,R1L	1 0 0 1 1	bit,base:19[An]	bit,base:19[A0]	0 1 0 0 0
	bit,R1H	1 0 0 0 1		bit,base:19[A1]	0 1 0 0 1
An	bit,A0	0 0 0 1 0	bit,base:19[SB/FB]	bit,base:19[SB]	0 1 0 1 0
	bit,A1	0 0 0 1 1		bit,base:19[FB]	0 1 0 1 1
[An]	bit,[A0]	0 0 0 0 0	bit,base:27[An]	bit,base:27[A0]	0 1 1 0 0
	bit,[A1]	0 0 0 0 1		bit,base:27[A1]	0 1 1 0 1
bit,base:11[An]	bit,base:11[A0]	0 0 1 0 0	bit,base:19	bit,base:19	0 1 1 1 1
	bit,base:11[A1]	0 0 1 0 1	bit,base:27	bit,base:27	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src	bit,Rn	bit,An	bit,[An]	bit,base:11 [An]	bit,base:11 [SB/FB]	bit,base:19 [An]	bit,base:19 [SB/FB]	bit,base:27 [An]	bit,base:19	bit,base:27
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

BNXOR

(1) BNXOR src



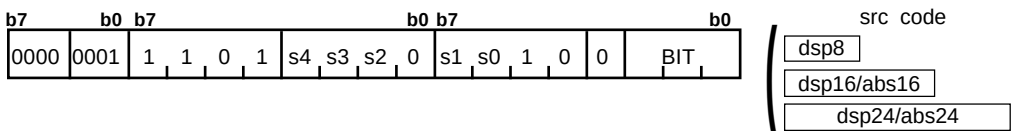
src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1	bit,base:19[An]	bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src	bit,Rn	bit,An	bit,[An]	bit,base:11 [An]	bit,base:11 [SB/FB]	bit,base:19 [An]	bit,base:19 [SB/FB]	bit,base:27 [An]	bit,base:19	bit,base:27
Bytes/Cycles	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4

BOR

(1) BOR src



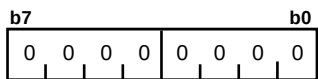
src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1	bit,base:19[An]	bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src	bit,Rn	bit,An	bit,[An]	bit,base:11 [An]	bit,base:11 [SB/FB]	bit,base:19 [An]	bit,base:19 [SB/FB]	bit,base:27 [An]	bit,base:19	bit,base:27
Bytes/Cycles	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4

BRK

(1) BRK



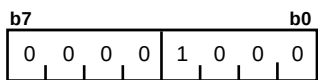
[Number of Bytes/Number of Cycles]

Bytes/Cycles	1/17
--------------	------

*1 When you specify the target address of the BRK interrupt by use of the interrupt table register (INTB) the number of cycles shown in the table increases by 2. At this time, set FF16 in address FFFFE416 through FFFFE716.

BRK2

(1) BRK2

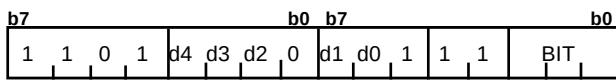


[Number of Bytes/Number of Cycles]

Bytes/Cycles	1/19
--------------	------

BSET

(1) BSET dest



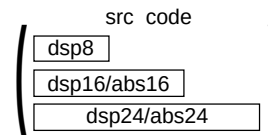
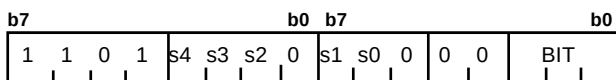
dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1	bit,base:19[An]	bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	bit,Rn	bit,An	bit,[An]	bit,base:11 [An]	bit,base:11 [SB/FB]	bit,base:19 [An]	bit,base:19 [SB/FB]	bit,base:27 [An]	bit,base:19	bit,base:27
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

BTST

(1) BTST:G src



src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1	bit,base:19[An]	bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src	bit,Rn	bit,An	bit,[An]	bit,base:11 [An]	bit,base:11 [SB/FB]	bit,base:19 [An]	bit,base:19 [SB/FB]	bit,base:27 [An]	bit,base:19	bit,base:27
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

BTST**(2) BTST:S src**

b7								b0
0	0	b2	b1	1	0	1		b0

src code

abs16

src	bit,base:19
-----	-------------

[Number of Bytes/Number of Cycles]

Bytes/Cycles	3/3
--------------	-----

BTSTC**(1) BTSTC dest**

b7								b0	b7								b0
1	1	0	1	d4	d3	d2	0	d1	d0	1	0	0					BIT

dest code

dsp8
dsp16/abs16
dsp24/abs24

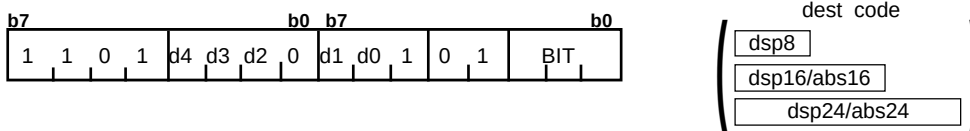
dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1	bit,base:19[An]	bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	bit,Rn	bit,An	bit,[An]	bit,base:11[An]	bit,base:11[SB/FB]	bit,base:19[An]	bit,base:19[SB/FB]	bit,base:27[An]	bit,base:19	bit,base:27
Bytes/Cycles	2/2	2/2	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4

BTSTS

(1) BTSTS dest



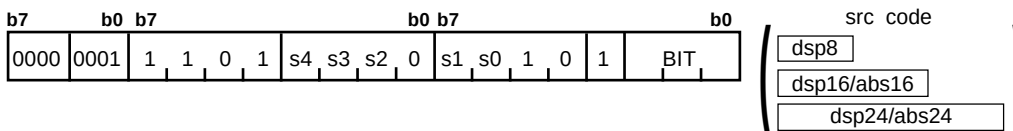
dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1		bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	bit,Rn	bit,An	bit,[An]	bit,base:11[An]	bit,base:11[SB/FB]	bit,base:19[An]	bit,base:19[SB/FB]	bit,base:27[An]	bit,base:19	bit,base:27
Bytes/Cycles	2/2	2/2	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4

BXOR

(1) BXOR src



src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
Rn	bit,R0L	1	0	0	1	0	bit,base:11[SB/FB]	bit,base:11[SB]	0	0	1	1	0
	bit,R0H	1	0	0	0	0		bit,base:11[FB]	0	0	1	1	1
	bit,R1L	1	0	0	1	1		bit,base:19[A0]	0	1	0	0	0
	bit,R1H	1	0	0	0	1		bit,base:19[A1]	0	1	0	0	1
An	bit,A0	0	0	0	1	0	bit,base:19[SB/FB]	bit,base:19[SB]	0	1	0	1	0
	bit,A1	0	0	0	1	1		bit,base:19[FB]	0	1	0	1	1
[An]	bit,[A0]	0	0	0	0	0	bit,base:27[An]	bit,base:27[A0]	0	1	1	0	0
	bit,[A1]	0	0	0	0	1		bit,base:27[A1]	0	1	1	0	1
bit,base:11[An]	bit,base:11[A0]	0	0	1	0	0	bit,base:19	bit,base:19	0	1	1	1	1
	bit,base:11[A1]	0	0	1	0	1	bit,base:27	bit,base:27	0	1	1	1	0

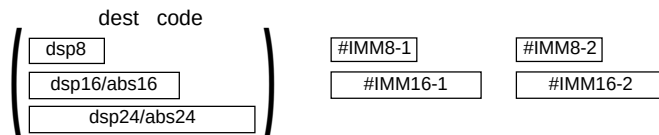
[Number of Bytes/Number of Cycles]

src	bit,Rn	bit,An	bit,[An]	bit,base:11[An]	bit,base:11[SB/FB]	bit,base:19[An]	bit,base:19[SB/FB]	bit,base:27[An]	bit,base:19	bit,base:27
Bytes/Cycles	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4

CLIP

(1) CLIP.size #IMM1, #IMM2, dest

b7	b0	b7	b0	b7	b0
0000	0001	1 0 0 0	d4 d3 d2	SIZE	d1 d0 1 1 1 1 1 0



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

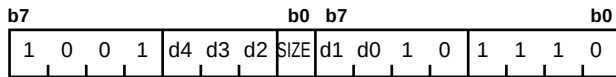
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	5/6	5/6	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8

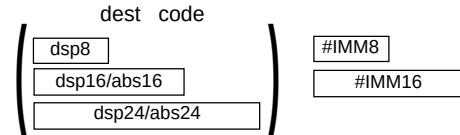
*1 When (.W) is specified for the size specifier (.size) the number of bytes in the table is increased by 2.

CMP

(1) CMP.size:G #IMM, dest



*1 When dest is indirectly addressed, the code has 00001001 added at the beginning.



.size	SIZE
.B	0
.W	1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

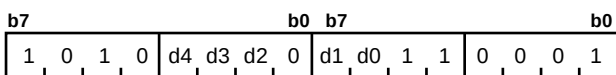
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

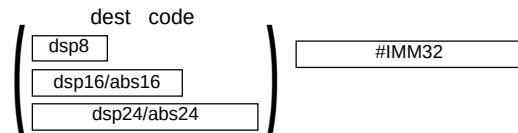
*3 When (.W) is specified for the size specifier (.size), the number of bytes in the table is increased by 1.

CMP

(2) CMP.L:G #IMM32, dest



*1 When dest is indirectly addressed, the code has 00001001 added at the beginning.

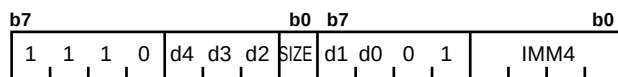


dest	d4 d3 d2 d1 d0	dest	d4 d3 d2 d1 d0
Rn	---/---/R2R0	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	dsp:8[FB]	0 0 1 1 1
	---/---/-	dsp:16[A0]	0 1 0 0 0
	---/---/-	dsp:16[A1]	0 1 0 0 1
An	A0	dsp:16[SB]	0 1 0 1 0
	A1	dsp:16[FB]	0 1 0 1 1
[An]	[A0]	dsp:24[A0]	0 1 1 0 0
	[A1]	dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	abs16	0 1 1 1 1
	dsp:8[A1]	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	6/2	6/2	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

CMP**(3) CMP.size:Q #IMM, dest**

*1 When dest is indirectly addressed, the code has 00001001 added at the beginning.



.size	SIZE	#IMM	IMM4	#IMM	IMM4
.B	0	0	0 0 0 0	-8	1 0 0 0
.W	1	+1	0 0 0 1	-7	1 0 0 1
		+2	0 0 1 0	-6	1 0 1 0
		+3	0 0 1 1	-5	1 0 1 1
		+4	0 1 0 0	-4	1 1 0 0
		+5	0 1 0 1	-3	1 1 0 1
		+6	0 1 1 0	-2	1 1 1 0
		+7	0 1 1 1	-1	1 1 1 1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

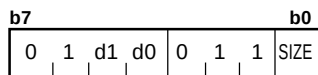
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

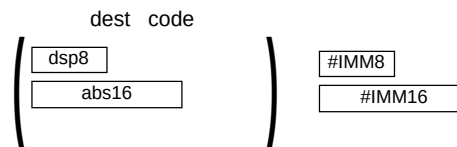
CMP

(4) CMP.size:S #IMM, dest



*1 When dest is indirectly addressed, the code has 00001001 added at the beginning.

.size	SIZE	dest		d1	d0
.B	0	Rn	R0L/R0	0	0
.W	1	dsp:8[SB/FB]	dsp:8[SB]	1	0
			dsp:8[FB]	1	1
		abs16	abs16	0	1



[Number of Bytes/Number of Cycles]

dest	Rn	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/1	3/3	4/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier (.size), the number of bytes in the table is increased by 1.

CMP**(5) CMP.size:G src, dest**

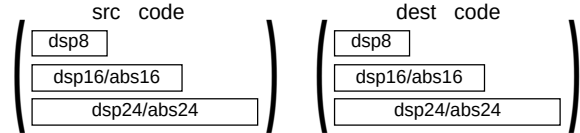
b7	b6	b5	b4	b3	b2	b1	b0
1	s4	s3	s2	d4	d3	d2	SIZE
				d1	d0	s1	s0
						0	1
						1	0

*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



.size	SIZE	src/dest		s4 s3 s2 s1 s0	src/dest		s4 s3 s2 s1 s0
.B	0			d4 d3 d2 d1 d0			d4 d3 d2 d1 d0
.W	1	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
			R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An		A0	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]		[A0]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]		dsp:8[A0]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

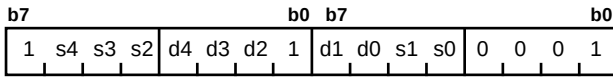
[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
An	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
[An]	2/3	2/3	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4
dsp:8[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[SB/FB]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:16[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:24[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs16	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
abs24	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

CMP

(6) CMP.L:G src, dest

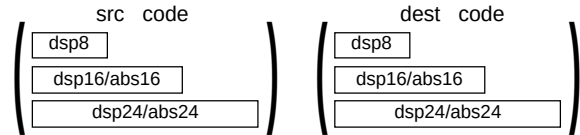


*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

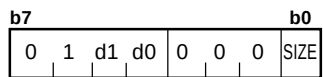
[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
An	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
[An]	2/5	2/5	2/8	3/8	3/8	4/8	4/8	5/8	4/8	5/8
dsp:8[An]	3/5	3/5	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8
dsp:8[SB/FB]	3/5	3/5	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8
dsp:16[An]	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
dsp:16[SB/FB]	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
dsp:24[An]	5/5	5/5	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8
abs16	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
abs24	5/5	5/5	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

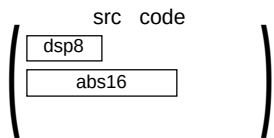
CMP

(7) CMP.size:S src, R0/R0L



*1 When src is indirectly addressed, the code has 00001001 added at the beginning.

.size	SIZE	src		d1	d0
.B	0	dsp:8[SB/FB]	dsp:8[SB]	1	0
.W	1		dsp:8[FB]	1	1
		abs16	abs16	0	1



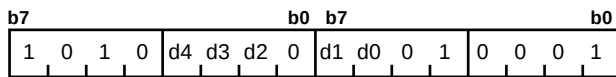
[Number of Bytes/Number of Cycles]

src	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/3	3/3

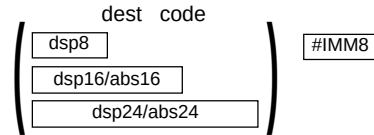
*2 When src is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

CMPX

(1) CMPX #IMM, dest



*1 When dest is indirectly addressed, the code has 00001001 added at the beginning.



dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	--- / --- /R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	--- / --- /R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	--- / --- /-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	--- / --- /-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

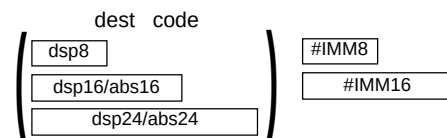
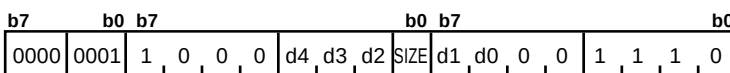
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

DADC

(1) DADC.size #IMM, dest

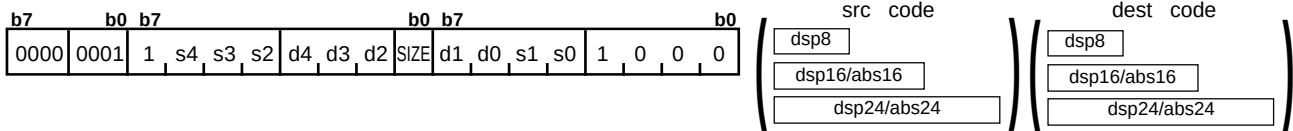


.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	4/4	4/4	4/6	5/6	5/6	6/6	6/6	7/6	6/6	7/6

*1 When (.W) is specified for the size specifier(.size), the number of bytes in the table is increased by 1.

DADC**(2) DADC.size src, dest**

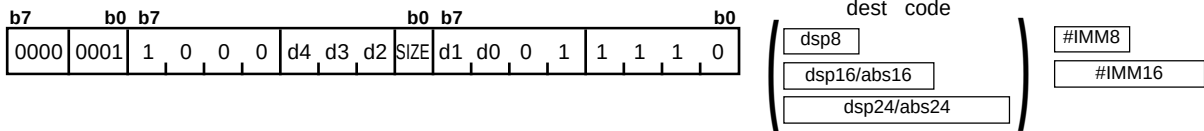
.size	SIZE	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
.B	0						
.W	1						
Rn		R0L/R0/---		1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
		R1L/R1/---		1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
		R0H/R2/-		1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
		R1H/R3/-		1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An		A0		0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
		A1		0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]		[A0]		0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
		[A1]		0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]		dsp:8[A0]		0 0 1 0 0	abs16	abs16	0 1 1 1 1
		dsp:8[A1]		0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/4	3/4	3/6	4/6	4/6	5/6	5/6	6/6	5/6	6/6
An	3/4	3/4	3/6	4/6	4/6	5/6	5/6	6/6	5/6	6/6
[An]	3/6	3/6	3/7	4/7	4/7	5/7	5/7	6/7	5/7	6/7
dsp:8[An]	4/6	4/6	4/7	5/7	5/7	6/7	6/7	7/7	6/7	7/7
dsp:8[SB/FB]	4/6	4/6	4/7	5/7	5/7	6/7	6/7	7/7	6/7	7/7
dsp:16[An]	5/6	5/6	5/7	6/7	6/7	7/7	7/7	8/7	7/7	8/7
dsp:16[SB/FB]	5/6	5/6	5/7	6/7	6/7	7/7	7/7	8/7	7/7	8/7
dsp:24[An]	6/6	6/6	6/7	7/7	7/7	8/7	8/7	9/7	8/7	9/7
abs16	5/6	5/6	5/7	6/7	6/7	7/7	7/7	8/7	7/7	8/7
abs24	6/6	6/6	6/7	7/7	7/7	8/7	8/7	9/7	8/7	9/7

DADD

(1) DADD.size #IMM, dest

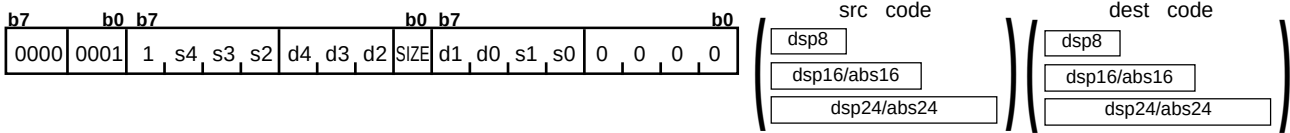


.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	4/4	4/4	4/6	5/6	5/6	6/6	6/6	7/6	6/6	7/6

*1 When (.W) is specified for the size specifier (.size), the number of bytes in the table is increased by 1.

DADD**(2) DADD.size src, dest**

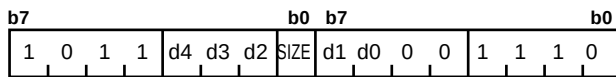
.size	SIZE	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
.B	0						
.W	1						
Rn		R0L/R0/---		1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
		R1L/R1/---		1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
		R0H/R2/-		1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
		R1H/R3/-		1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An		A0		0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
		A1		0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]		[A0]		0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
		[A1]		0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]		dsp:8[A0]		0 0 1 0 0	abs16	abs16	0 1 1 1 1
		dsp:8[A1]		0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/4	3/4	3/6	4/6	4/6	5/6	5/6	6/6	5/6	6/6
An	3/4	3/4	3/6	4/6	4/6	5/6	5/6	6/6	5/6	6/6
[An]	3/6	3/6	3/7	4/7	4/7	5/7	5/7	6/7	5/7	6/7
dsp:8[An]	4/6	4/6	4/7	5/7	5/7	6/7	6/7	7/7	6/7	7/7
dsp:8[SB/FB]	4/6	4/6	4/7	5/7	5/7	6/7	6/7	7/7	6/7	7/7
dsp:16[An]	5/6	5/6	5/7	6/7	6/7	7/7	7/7	8/7	7/7	8/7
dsp:16[SB/FB]	5/6	5/6	5/7	6/7	6/7	7/7	7/7	8/7	7/7	8/7
dsp:24[An]	6/6	6/6	6/7	7/7	7/7	8/7	8/7	9/7	8/7	9/7
abs16	5/6	5/6	5/7	6/7	6/7	7/7	7/7	8/7	7/7	8/7
abs24	6/6	6/6	6/7	7/7	7/7	8/7	8/7	9/7	8/7	9/7

DEC

(1) DEC.size dest



*1 When dest is indirectly addressed, the code has 00001001 added at the beginning.



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

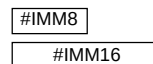
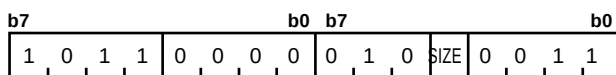
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

DIV

(1) DIV.size #IMM

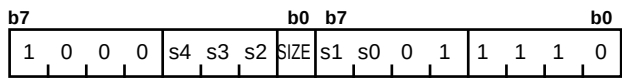


.size	SIZE
.B	0
.W	1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	3/18
--------------	------

*1 When (.W) is specified for the size specifier (.size), the number of bytes and cycles in the table are increased by 1 and 6, respectively.

DIV**(2) DIV.size src**

*1 When src is indirectly addressed, the code has 00001001 added at the beginning.



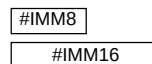
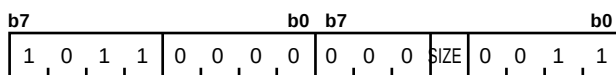
.size	SIZE	src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/18	2/18	2/20	3/20	3/20	4/20	4/20	5/20	4/20	5/20

*2 When src is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier (.size), the number of bytes in the table is increased by 6.

DIVU**(1) DIVU.size #IMM**

.size	SIZE
.B	0
.W	1

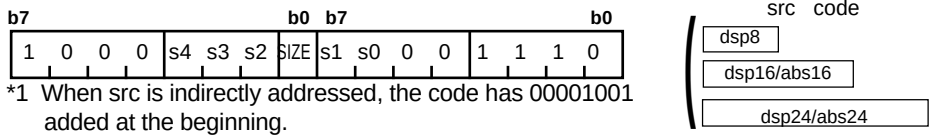
[Number of Bytes/Number of Cycles]

Bytes/Cycles	3/18
--------------	------

*1 When (.W) is specified for the size specifier (.size), the number of bytes and cycles in the table are increased by 1 and 5, respectively.

DIVU

(2) DIV.size src



.size	SIZE	src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

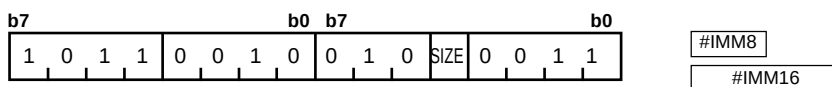
src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/18	2/18	2/20	3/20	3/20	4/20	4/20	5/20	4/20	5/20

*2 When src is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier (.size), the number of cycles in the table is increased by 5.

DIVX

(1) DIVX.size #IMM



.size	SIZE
.B	0
.W	1

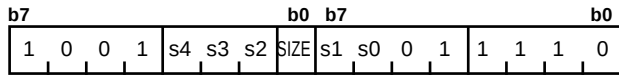
[Number of Bytes/Number of Cycles]

Bytes/Cycles	3/18
--------------	------

*1 When (.W) is specified for the size specifier (.size), the number of bytes and cycles in the table are increased by 1 and 6, respectively.

DIVX

(2) DIVX.size src



*1 When src is indirectly addressed, the code has 00001001 added at the beginning.



.size	SIZE	src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

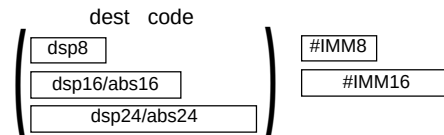
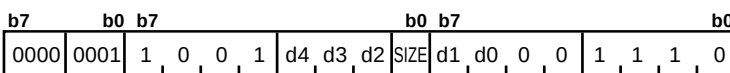
src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/18	2/18	2/20	3/20	3/20	4/20	4/20	5/20	4/20	5/20

*2 When src is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier (.size), the number of cycles in the table is increased by 6.

DSBB

(1) DSBB.size #IMM, dest



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

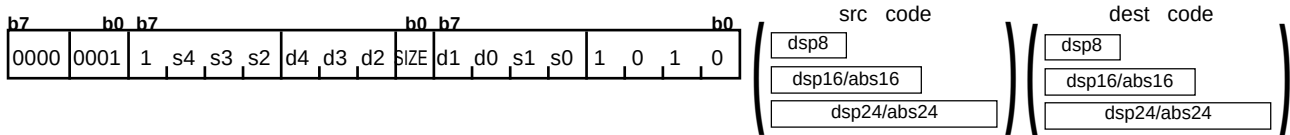
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Btyes/Cycles	4/2	4/2	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4

*1 When (.W) is specified for the size specifier (.size), the number of bytes in the table is increased by 1.

DSBB

(2) DSBB.size src, dest



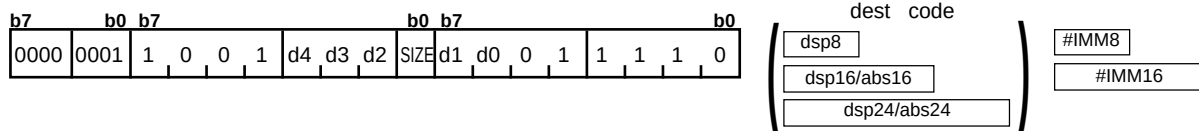
.size	SIZE																
.B	0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0						src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0					
.W	1	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0		
	R1L/R1/---		1	0	0	1	1	dsp:8[FB]		0	0	1	1	1			
	R0H/R2/-		1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0			
	R1H/R3/-		1	0	0	0	1		dsp:16[A1]	0	1	0	0	1			
	An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0			
		A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1			
	[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0			
		[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1			
	dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1			
		dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0			

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
An	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
[An]	3/4	3/4	3/5	4/5	4/5	5/5	5/5	6/5	5/5	6/5
dsp:8[An]	4/4	4/4	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5
dsp:8[SB/FB]	4/4	4/4	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5
dsp:16[An]	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
dsp:16[SB/FB]	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
dsp:24[An]	6/4	6/4	6/5	7/5	7/5	8/5	8/5	9/5	8/5	9/5
abs16	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
abs24	6/4	6/4	6/5	7/5	7/5	8/5	8/5	9/5	8/5	9/5

DSUB

(1) DSUB.size #IMM, dest



.size	SIZE	dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

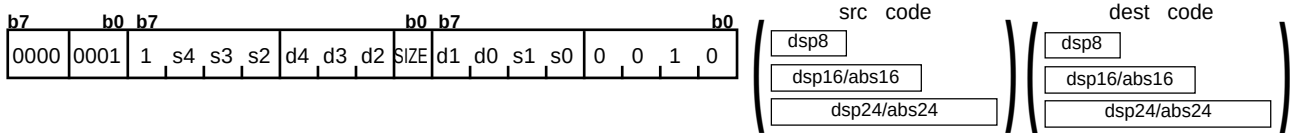
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	4/2	4/2	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4

*1 When (.W) is specified for the size specifier (.size), the number of bytes in the table is increased by 1.

DSUB

(2) DSUB.size src, dest



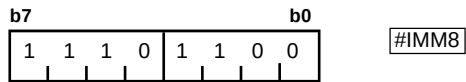
.size	SIZE																
.B	0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0						src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0					
.W	1	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0		
	R1L/R1/---		1	0	0	1	1	dsp:8[FB]		0	0	1	1	1			
	R0H/R2/-		1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0			
	R1H/R3/-		1	0	0	0	1		dsp:16[A1]	0	1	0	0	1			
	An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0			
		A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1			
	[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0			
		[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1			
	dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1			
		dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0			

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
An	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
[An]	3/4	3/4	3/5	4/5	4/5	5/5	5/5	6/5	5/5	6/5
dsp:8[An]	4/4	4/4	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5
dsp:8[SB/FB]	4/4	4/4	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5
dsp:16[An]	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
dsp:16[SB/FB]	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
dsp:24[An]	6/4	6/4	6/5	7/5	7/5	8/5	8/5	9/5	8/5	9/5
abs16	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
abs24	6/4	6/4	6/5	7/5	7/5	8/5	8/5	9/5	8/5	9/5

ENTER

(1) ENTER #IMM

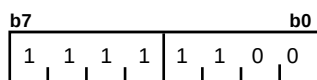


[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/4
--------------	-----

EXITD

(1) EXITD

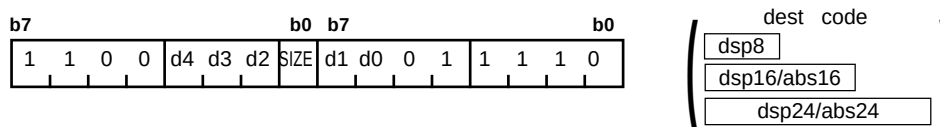


[Number of Bytes/Number of Cycles]

Bytes/Cycles	1/8
--------------	-----

EXTS

(1) EXTS.size dest



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

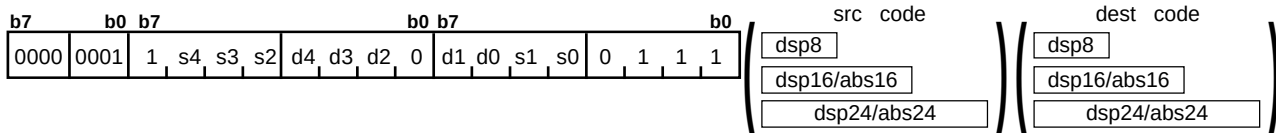
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	5/3	5/3

*1 When (.W) is specified for the size specifier(.size) the number of cycles in the table is increased by 1.

EXTS

(2) EXTS.B src,dest



src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
Rn	R0L/---/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	R1L/---/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/---/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/---/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	---	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	---	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

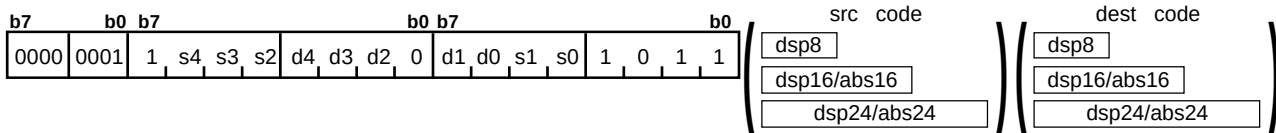
dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
Rn	---/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	---/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	---/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	---/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3
[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:8[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
dsp:16[SB/FB]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
dsp:24[An]	6/3	6/3	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4
abs16	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs24	6/3	6/3	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4

EXTZ

(1) EXTZ src,dest



src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
Rn	R0L/---/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/---/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	---	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	---	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

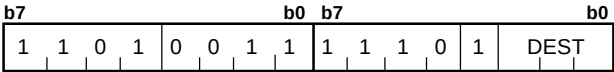
dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	---/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3
[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:8[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
dsp:16[SB/FB]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
dsp:24[An]	6/3	6/3	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4
abs16	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs24	6/3	6/3	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4

FCLR

(1) FCLR dest



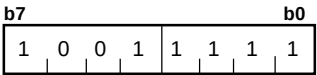
dest	DEST
C	0 0 0
D	0 0 1
Z	0 1 0
S	0 1 1
B	1 0 0
O	1 0 1
I	1 1 0
U	1 1 1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1
--------------	-----

FREIT

(1) FREIT

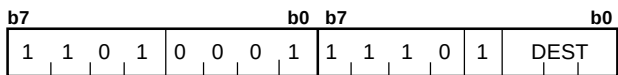


[Number of Bytes/Number of Cycles]

Bytes/Cycles	1/3
--------------	-----

FSET

(1) FSET dest



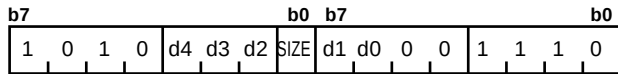
dest	DEST
C	0 0 0
D	0 0 1
Z	0 1 0
S	0 1 1
B	1 0 0
O	1 0 1
I	1 1 0
U	1 1 1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1
--------------	-----

INC

(1) INC.size dest



*1 When dest is indirectly addressed, the code has 00001001 added at the beginning.



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

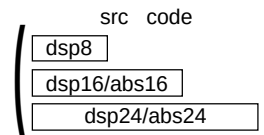
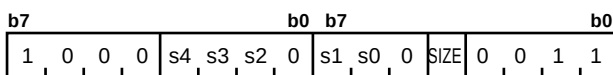
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

INDEXB

(1) INDEXB.size src



.size	SIZE	src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

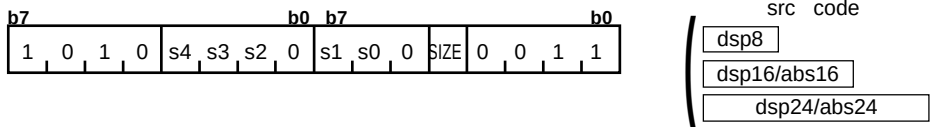
[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/2	2/2	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4

*1 When (.W) is specified for the size specifier(.size) the number of cycles in the table is increased by 2.

INDEXBD

(1) INDEXBD.size src



.size	SIZE	src	s4	s3	s2	s1	s0	src	s4	s3	s2	s1	s0
.B	0	R0L/R0/---	1	0	0	1	0	dsp:8[SB]	0	0	1	1	0
.W	1	R1L/R1/---	1	0	0	1	1	dsp:8[FB]	0	0	1	1	1
		R0H/R2/-	1	0	0	0	0	dsp:16[A0]	0	1	0	0	0
		R1H/R3/-	1	0	0	0	1	dsp:16[A1]	0	1	0	0	1
		A0	0	0	0	1	0	dsp:16[SB]	0	1	0	1	0
		A1	0	0	0	1	1	dsp:16[FB]	0	1	0	1	1
		[A0]	0	0	0	0	0	dsp:24[A0]	0	1	1	0	0
		[A1]	0	0	0	0	1	dsp:24[A1]	0	1	1	0	1
		dsp:8[A0]	0	0	1	0	0	abs16	0	1	1	1	1
		dsp:8[A1]	0	0	1	0	1	abs24	0	1	1	1	0

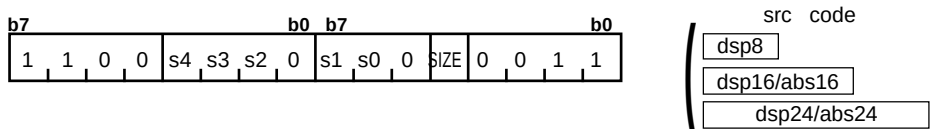
[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*1 When (.W) is specified for the size specifier(.size) the number of cycles in the table is increased by 1.

INDEXBS

(1) INDEXBS.size src



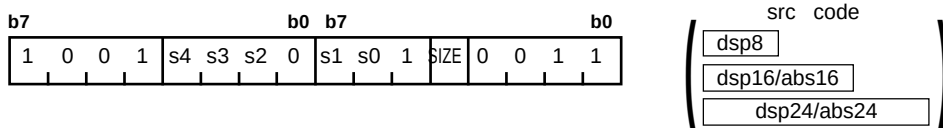
.size	SIZE	src	s4	s3	s2	s1	s0	src	s4	s3	s2	s1	s0
.B	0	R0L/R0/---	1	0	0	1	0	dsp:8[SB]	0	0	1	1	0
.W	1	R1L/R1/---	1	0	0	1	1	dsp:8[FB]	0	0	1	1	1
		R0H/R2/-	1	0	0	0	0	dsp:16[A0]	0	1	0	0	0
		R1H/R3/-	1	0	0	0	1	dsp:16[A1]	0	1	0	0	1
		A0	0	0	0	1	0	dsp:16[SB]	0	1	0	1	0
		A1	0	0	0	1	1	dsp:16[FB]	0	1	0	1	1
		[A0]	0	0	0	0	0	dsp:24[A0]	0	1	1	0	0
		[A1]	0	0	0	0	1	dsp:24[A1]	0	1	1	0	1
		dsp:8[A0]	0	0	1	0	0	abs16	0	1	1	1	1
		dsp:8[A1]	0	0	1	0	1	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*1 When (.W) is specified for the size specifier(.size) the number of cycles in the table is increased by 1.

(1) INDEXL.size **src**

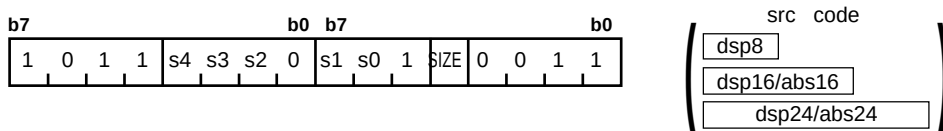


.size	SIZE	src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/4	2/4	2/6	3/6	3/6	4/6	4/6	5/6	4/6	5/6

*1 When (.W) is specified for the size specifier(.size) the number of cycles in the table is increased by 2.

(1) INDEXLD.size **src**



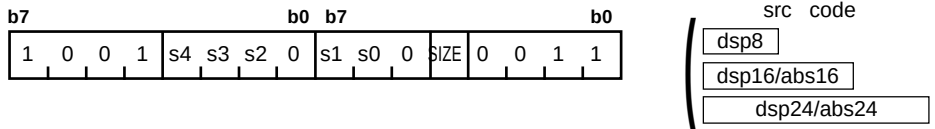
.size	SIZE	src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
.B	0		R0L/R0/---	1 0 0 1 0		dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1	dsp:8[SB/FB]	dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0		dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1	dsp:16[An]	dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0		dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1	dsp:16[SB/FB]	dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0		dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1	dsp:24[An]	dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/2	2/2	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4

*1 When (.W) is specified for the size specifier(.size) the number of cycles in the table is increased by 1.

INDEXLS

(1) INDEXLS.size src



.size	SIZE	src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

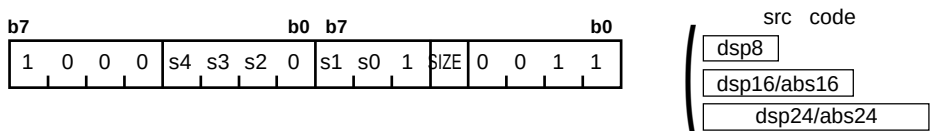
[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/2	2/2	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4

*1 When (.W) is specified for the size specifier(.size) the number of cycles in the table is increased by 1.

INDEXW

(1) INDEXW.size src



.size	SIZE	src						s4 s3 s2 s1 s0		src						s4 s3 s2 s1 s0	
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0		
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1		
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0		
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1		
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0		
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1		
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0		
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1		
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1		
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0		

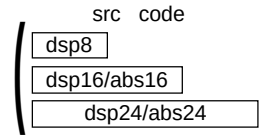
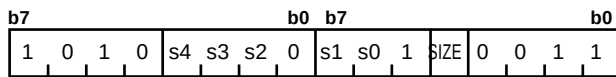
[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/2	2/2	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4

*1 When (.W) is specified for the size specifier(.size) the number of cycles in the table is increased by 2.

INDEXWD

(1) INDEXWD.size src



.size	SIZE	src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

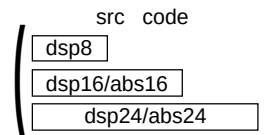
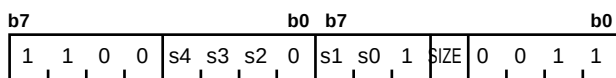
[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*1 When (.W) is specified for the size specifier(.size) the number of cycles in the table is increased by 1.

INDEXWS

(1) INDEXWS.size src



.size	SIZE	src						s4 s3 s2 s1 s0		src						s4 s3 s2 s1 s0	
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0		
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1		
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0		
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1		
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0		
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1		
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0		
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1		
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1		
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0		

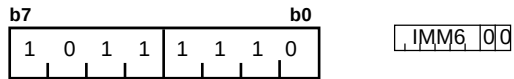
[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*1 When (.W) is specified for the size specifier(.size) the number of cycles in the table is increased by 1.

INT

(1) INT #IMM

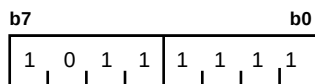


[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/ 12
--------------	-------

INTO

(1) INTO



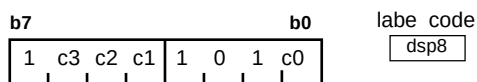
[Number of Bytes/Number of Cycles]

Bytes/Cycles	1/ 1
--------------	------

*1 When O flag is 1, the number of cycles in the table is increased by 13.

Jcnd

(1) Jcnd label



dsp8 = address indicated by label - (start address of instruction +1)

<i>Cnd</i>	c3 c2 c1 c0	<i>Cnd</i>	c3 c2 c1 c0
LTU/NC	0 0 0 0	GEU/C	1 0 0 0
LEU	0 0 0 1	GTU	1 0 0 1
NE/NZ	0 0 1 0	EQ/Z	1 0 1 0
PZ	0 0 1 1	N	1 0 1 1
NO	0 1 0 0	O	1 1 0 0
GT	0 1 0 1	LE	1 1 0 1
GE	0 1 1 0	LT	1 1 1 0

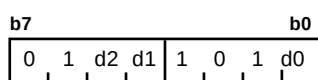
[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1
--------------	-----

*1 When branched to label the number of cycles in the table is increased by 2.

JMP

(1) JMP.S label



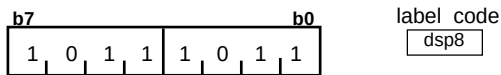
label	d2 d1 d0	label	d2 d1 d0
PC + 2	0 0 0	PC + 6	1 0 0
PC + 3	0 0 1	PC + 7	1 0 1
PC + 4	0 1 0	PC + 8	1 1 0
PC + 5	0 1 1	PC + 9	1 1 1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	1/3
--------------	-----

JMP

(2) JMP.B label



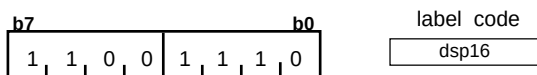
dsp8 = address indicated by label - (start address of instruction +1)

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/3
--------------	-----

JMP

(3) JMP.W label



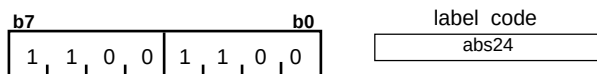
dsp16 = address indicated by label - (start address of instruction +1)

[Number of Bytes/Number of Cycles]

Bytes/Cycles	3/3
--------------	-----

JMP

(4) JMP.A label

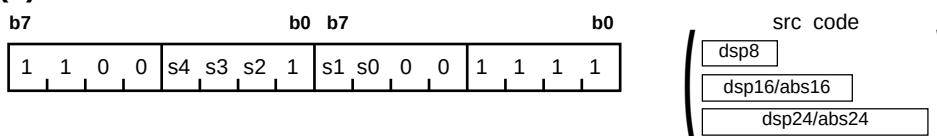


[Number of Bytes/Number of Cycles]

Bytes/Cycles	4/3
--------------	-----

JMPI

(1) JMPI.W src



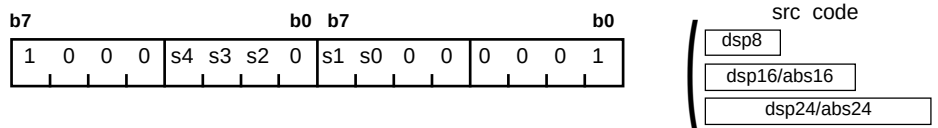
src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
Rn	---/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/7	2/7	2/8	3/8	3/8	4/8	4/8	5/8	4/8	5/8

JMPI

(2) JMP.L.A src



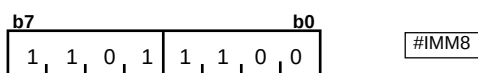
src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycle	2/5	2/5	2/7	3/7	3/7	4/7	4/7	5/7	4/7	5/7

JMPS

(1) JMPS #IMM8

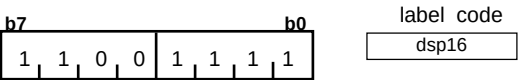


[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/8
--------------	-----

JSR

(1) JSR.W label



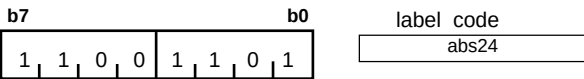
dsp16 = address indicated by label - (start address of instruction +1)

[Number of Bytes/Number of Cycles]

Bytes/Cycles	3/3
--------------	-----

JSR

(2) JSR.A label

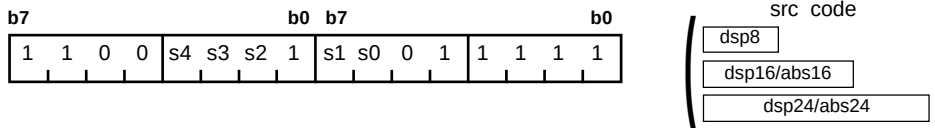


[Number of Bytes/Number of Cycles]

Bytes/Cycles	4/3
--------------	-----

JSRI

(1) JSRI.W src



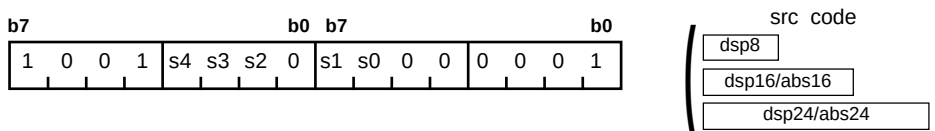
src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
Rn	---/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/7	2/7	2/8	3/8	3/8	4/8	4/8	5/8	4/8	5/8

JSRI

(2) JSRI.A src



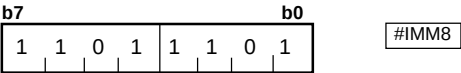
src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/5	2/5	2/7	3/7	3/7	4/7	4/7	5/7	4/7	5/7

JSRS

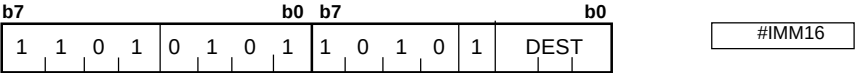
(1) JSRS #IMM8



[Number of Bytes/Number of Cycles]	
Bytes/Cycles	2/8

LDC

(1) LDC #IMM16, dest

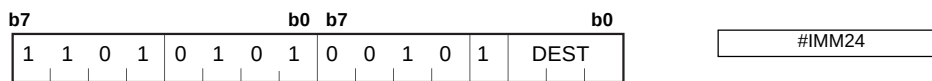


dest	DEST
DCT0	0 0 0
DCT1	0 0 1
FLG	0 1 0
SVF	0 1 1
DRC0	1 0 0
DRC1	1 0 1
DMD0	1 1 0
DMD1	1 1 1

[Number of Bytes/Number of Cycles]	
Bytes/Cycles	4/1

LDC

(2) LDC #IMM24, dest



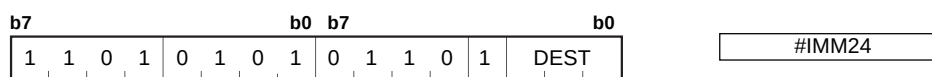
dest	DEST
INTB	0 0 0
SP	0 0 1
SB	0 1 0
FB	0 1 1
SVP	1 0 0
VCT	1 0 1
---	1 1 0
ISP	1 1 1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	5/2
--------------	-----

LDC

(3) LDC #IMM24, dest



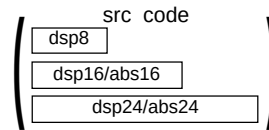
dest	DEST
---	0 0 0
---	0 0 1
DMA0	0 1 0
DMA1	0 1 1
DRA0	1 0 0
DRA1	1 0 1
DSA0	1 1 0
DSA1	1 1 1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	5/2
--------------	-----

LDC**(4) LDC src, dest**

b7	b0	b7	b0	b7	b0
0000	0001	1 1 0 1	s4 s3 s2 1	s1 s0 0 0	1 DEST



src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0	dest	DEST
Rn	---/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0	DCT0	0 0 0
	---/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1	DCT1	0 0 1
	---/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0	FLG	0 1 0
	---/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1	SVF	0 1 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0	DRC0	1 0 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1	DRC1	1 0 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0	DMD0	1 1 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1	DMD1	1 1 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1		
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0		

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycle	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

LDC**(5) LDC src, dest**

b7	b0	b7	b0
1 1 0 1	s4 s3 s2 1	s1 s0 0 0	0 DEST



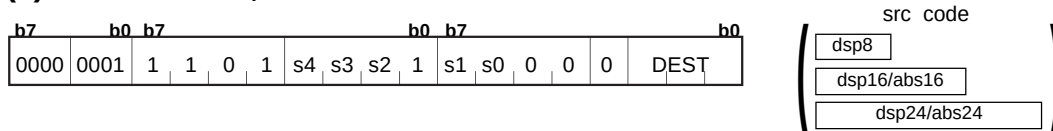
src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0	dest	DEST
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0	INTB	0 0 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1	SP	0 0 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0	SB	0 1 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1	FB	0 1 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0	SVP	1 0 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1	VCT	1 0 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0	---	1 1 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1	ISP	1 1 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1		
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0		

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/2	2/2	2/6	3/6	3/6	4/6	4/6	5/6	4/6	5/6

LDC

(6) LDC src, dest



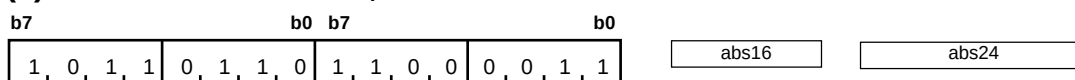
src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0	dest	DEST
Rn	---/---/R2R0	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0	---	0 0 0
	---/---/R3R1	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1	---	0 0 1
	---/---/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0	DMA0	0 1 0
	---/---/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1	DMA1	0 1 1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0	DRA0	1 0 0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1	DRA1	1 0 1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0	DSA0	1 1 0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1	DSA1	1 1 1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1		
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0		

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycle	3/3	3/3	3/6	4/6	4/6	5/6	5/6	6/6	5/6	6/6

LDCTX

(1) LDCTX abs16,abs24



[Number of Bytes/Number of Cycles]

Bytes/Cycles	7/10 + m
--------------	----------

*1 m denotes the number of transfers performed.

$$m = (\text{Number of R0,R1,R2,R3}) + 2 \times (\text{Number of A0,A1,FB,SB})$$

LDIPL

(1) LDIPL #IMM

b7	b0	b7	b0
1 1 0 1	0 1 0 1	1 1 1 0	1 IMM3

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/2
--------------	-----

MAX

(1) MAX.size #IMM,dest

b7	b0	b7	b0	b7	b0
0000	0001	1 0 0 0	d4 d3 d2	SIZE	d1 d0 1 1 1 1 1 1

dest code	
dsp8	#IMM8
dsp16/abs16	#IMM16
dsp24/abs24	

.size	SIZE
.B	0
.W	1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

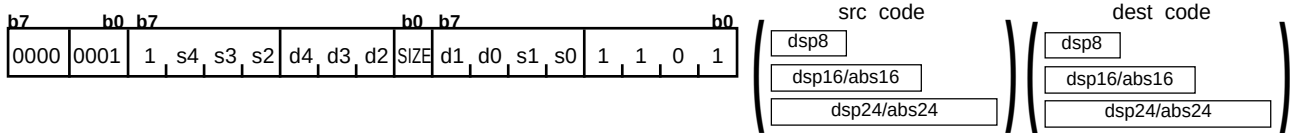
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	4/3	4/3	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5

*1 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

MAX

(2) MAX.size src, dest



.size	SIZE
.B	0
.W	1

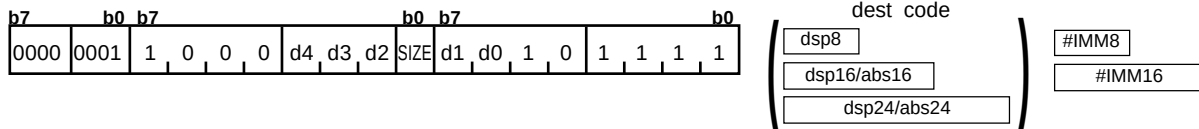
src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
An	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
[An]	3/4	3/4	3/5	4/5	4/5	5/5	5/5	6/5	5/5	6/5
dsp:8[An]	4/4	4/4	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5
dsp:8[SB/FB]	4/4	4/4	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5
dsp:16[An]	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
dsp:16[SB/FB]	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
dsp:24[An]	6/4	6/4	6/5	7/5	7/5	8/5	8/5	9/5	8/5	9/5
abs16	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
abs24	6/4	6/4	6/5	7/5	7/5	8/5	8/5	9/5	8/5	9/5

MIN

(1) MIN.size #IMM,dest

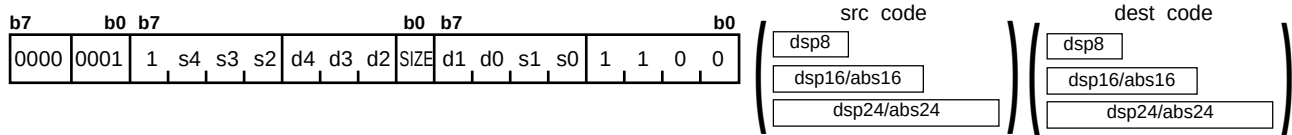


.size	SIZE	dest	d4 d3 d2 d1 d0	dest	d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	dsp:8[SB]	dsp:8[SB]
.W	1		R1L/R1/---	dsp:8[FB]	dsp:8[FB]
			R0H/R2/-	dsp:16[A0]	dsp:16[A0]
			R1H/R3/-	dsp:16[A1]	dsp:16[A1]
		An	A0	dsp:16[SB]	dsp:16[SB]
			A1	dsp:16[FB]	dsp:16[FB]
		[An]	[A0]	dsp:24[A0]	dsp:24[A0]
			[A1]	dsp:24[A1]	dsp:24[A1]
		dsp:8[An]	dsp:8[A0]	abs16	abs16
			dsp:8[A1]	abs24	abs24

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	4/3	4/3	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5

*1 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

MIN**(2) MIN.size src, dest**

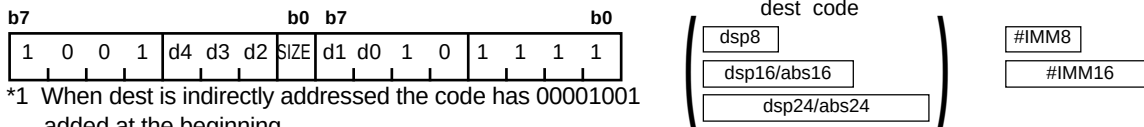
.size	SIZE	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
An	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
[An]	3/4	3/4	3/5	4/5	4/5	5/5	5/5	6/5	5/5	6/5
dsp:8[An]	4/4	4/4	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5
dsp:8[SB/FB]	4/4	4/4	4/5	5/5	5/5	6/5	6/5	7/5	6/5	7/5
dsp:16[An]	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
dsp:16[SB/FB]	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
dsp:24[An]	6/4	6/4	6/5	7/5	7/5	8/5	8/5	9/5	8/5	9/5
abs16	5/4	5/4	5/5	6/5	6/5	7/5	7/5	8/5	7/5	8/5
abs24	6/4	6/4	6/5	7/5	7/5	8/5	8/5	9/5	8/5	9/5

MOV

(1) MOV.size:G #IMM,dest



.size	SIZE
.B	0
.W	1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

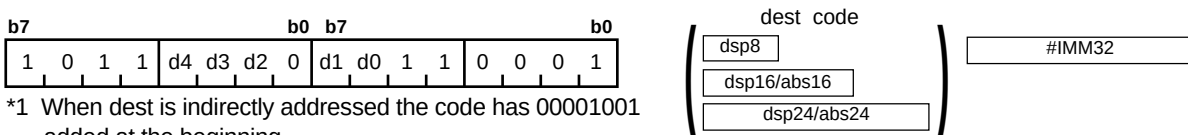
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

MOV

(2) MOV.L:G #IMM,dest



dest	d4 d3 d2 d1 d0	dest	d4 d3 d2 d1 d0
Rn	---/---/R2R0	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	dsp:8[FB]	0 0 1 1 1
	---/---/-	dsp:16[A0]	0 1 0 0 0
	---/---/-	dsp:16[A1]	0 1 0 0 1
An	A0	dsp:16[SB]	0 1 0 1 0
	A1	dsp:16[FB]	0 1 0 1 1
[An]	[A0]	dsp:24[A0]	0 1 1 0 0
	[A1]	dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	abs16	0 1 1 1 1
	dsp:8[A1]	abs24	0 1 1 1 0

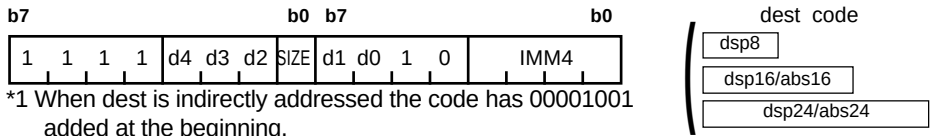
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	6/2	6/2	6/2	7/2	7/2	8/2	8/2	9/2	8/2	9/2

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

MOV

(3) MOV.size:Q #IMM4, dest



.size	SIZE	#IMM	IMM4	#IMM	IMM4
.B	0	0	0 0 0 0	-8	1 0 0 0
.W	1	+1	0 0 0 1	-7	1 0 0 1
		+2	0 0 1 0	-6	1 0 1 0
		+3	0 0 1 1	-5	1 0 1 1
		+4	0 1 0 0	-4	1 1 0 0
		+5	0 1 0 1	-3	1 1 0 1
		+6	0 1 1 0	-2	1 1 1 0
		+7	0 1 1 1	-1	1 1 1 1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

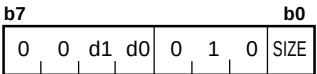
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/1	3/1	3/1	4/1	4/1	5/1	4/1	5/1

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

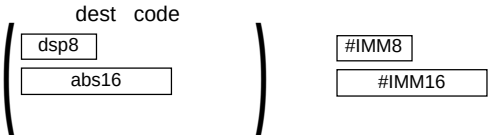
MOV

(4) MOV.size:S #IMM, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.

.size	SIZE	dest	d1	d0
.B	0	Rn	0	0
.W	1	dsp:8[SB]	1	0
		dsp:8[FB]	1	1
		abs16	0	1



[Number of Bytes/Number of Cycles]

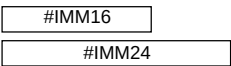
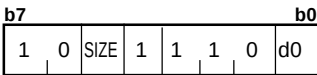
dest	Rn	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/1	3/2	4/2

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

MOV

(5) MOV.size:S #IMM,A0/A1



.size	SIZE	A0/A1	d0
.W	0	A0	0
.L	1	A1	1

[Number of Bytes/Number of Cycles]

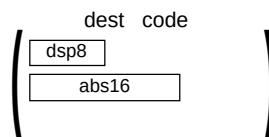
#IMM	An
#IMM16	3/1
#IMM24	4/2

MOV

(6) MOV.size:Z #0, dest

b7				b0			
0	0	d1	d0	0	0	1	SIZE

*1 When dest is indirectly addressed the code has 00001001 added at the beginning.

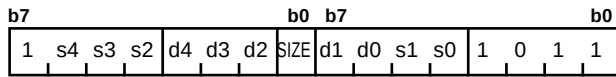


.size	SIZE	dest		d1	d0
.B	0	Rn	R0L/R0	0	0
.W	1	dsp:8[SB/FB]	dsp:8[SB]	1	0
			dsp:8[FB]	1	1
		abs16	abs16	0	1

[Number of Bytes/Number of Cycles]

dest	Rn	dsp:8[SB/FB]	abs16
Bytes/Cycles	1/1	2/1	3/1

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

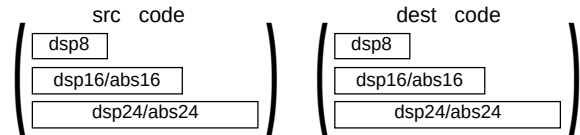
MOV**(7) MOV.size:G src, dest**

*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



.size	SIZE	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
.B	0						
.W	1						
Rn		R0L/R0/---		1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
		R1L/R1/---		1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
		R0H/R2/-		1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
		R1H/R3/-		1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An		A0		0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
		A1		0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]		[A0]		0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
		[A1]		0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]		dsp:8[A0]		0 0 1 0 0	abs16	abs16	0 1 1 1 1
		dsp:8[A1]		0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/1	2/1	2/1	3/1	3/1	4/1	4/1	5/1	4/1	5/1
An	2/1	2/1	2/1	3/1	3/1	4/1	4/1	5/1	4/1	5/1
[An]	2/3	2/3	2/3	3/2	3/2	4/2	4/2	5/2	4/2	5/2
dsp:8[An]	3/3	3/3	3/3	4/2	4/2	5/2	5/2	6/2	5/2	6/2
dsp:8[SB/FB]	3/3	3/3	3/3	4/2	4/2	5/2	5/2	6/2	5/2	6/2
dsp:16[An]	4/3	4/3	4/3	5/2	5/2	6/2	6/2	7/2	6/2	7/2
dsp:16[SB/FB]	4/3	4/3	4/3	5/2	5/2	6/2	6/2	7/2	6/2	7/2
dsp:24[An]	5/3	5/3	5/3	6/2	6/2	7/2	7/2	8/2	7/2	8/2
abs16	4/3	4/3	4/3	5/2	5/2	6/2	6/2	7/2	6/2	7/2
abs24	5/3	5/3	5/3	6/2	6/2	7/2	7/2	8/2	7/2	8/2

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3, respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

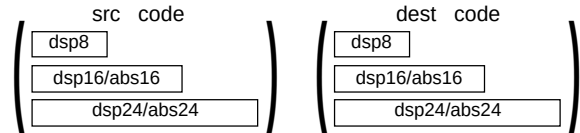
MOV

(8) MOV.L:G src, dest

b7	b0	b7	b0
1 s4 s3 s2 d4 d3 d2 1	d1 d0 s1 s0 0 0 1 1		

*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed
 00001001 when dest is indirectly addressed
 01001001 when src and dest are indirectly addressed



src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/2	2/2	2/2	3/2	3/2	4/2	4/2	5/2	4/2	5/2
An	2/2	2/2	2/2	3/2	3/2	4/2	4/2	5/2	4/2	5/2
[An]	2/4	2/4	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4
dsp:8[An]	3/4	3/4	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[SB/FB]	3/4	3/4	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:16[An]	4/4	4/4	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[SB/FB]	4/4	4/4	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:24[An]	5/4	5/4	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs16	4/4	4/4	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
abs24	5/4	5/4	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4

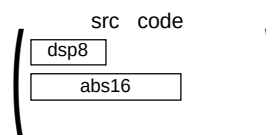
*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

MOV**(9) MOV.size:S src, R0L/R0**

b7																b0
0	0	s1	s0	1	0	0	SIZE									

*1 When src is indirectly addressed the code has 00001001 added at the beginning.

.size	SIZE	src	s1	s0
.B	0			
.W	1			
		dsp:8[SB]	1	0
		dsp:8[SB/FB]		
		dsp:8[FB]	1	1
		abs16	0	1

**[Number of Bytes/Number of Cycles]**

src	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/2	3/2

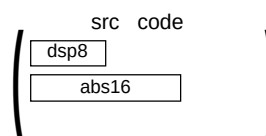
*2 When src is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

MOV**(10) MOV.size:S src, R1L/R1**

b7																b0
0	1	s1	s0	1	1	1	SIZE									

*1 When src is indirectly addressed the code has 00001001 added at the beginning.

.size	SIZE	src	s1	s0
.B	0	Rn		
.W	1	R0L/R0	0	0
		dsp:8[SB]	1	0
		dsp:8[SB/FB]		
		dsp:8[FB]	1	1
		abs16	0	1

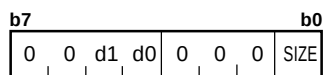
**[Number of Bytes/Number of Cycles]**

src	Rn	dsp:8[SB/FB]	abs16
Bytes/Cycles	1/3	2/3	3/3

*2 When src is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3, respectively.

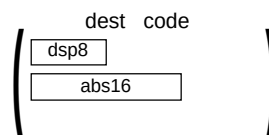
MOV

(11) MOV.size:S R0L/R0, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.

.size	SIZE	dest	d1	d0
.B	0	dsp:8[SB]	1	0
.W	1	dsp:8[SB/FB]	1	1
		abs16	0	1



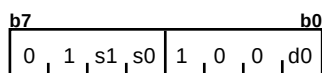
[Number of Bytes/Number of Cycles]

dest	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/1	3/1

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

MOV

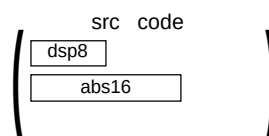
(12) MOV.L:S src, A0/A1



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.

src	s1	s0
dsp:8[SB]	1	0
dsp:8[SB/FB]	1	1
abs16	0	1

A0/A1	d0
A0	0
A1	1



[Number of Bytes/Number of Cycles]

src	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/3	3/3

*2 When src is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

MOV**(13) MOV.size:G dsp:8[SP], dest**

b7	b6	b5	b4	b3	b2	b1	b0
1	0	1	1	d4	d3	d2	SIZE
1	0	1	1	d1	d0	0	0
1	1	1	1	1	1	1	1

src code

dsp8

dest code

dsp8
dsp16/abs16
dsp24/abs24

.size	SIZE
.B	0
.W	1

	dest	d4	d3	d2	d1	d0		dest	d4	d3	d2	d1	d0
Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/3	3/3	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

MOV**(14) MOV.size:G src, dsp:8[SP]**

b7	b6	b5	b4	b3	b2	b1	b0
1	0	1	0	s4	s3	s2	SIZE
1	0	1	0	s1	s0	0	0
1	1	1	1	1	1	1	1

src code

dsp8
dsp16/abs16
dsp24/abs24

dest code

dsp8

.size	SIZE
.B	0
.W	1

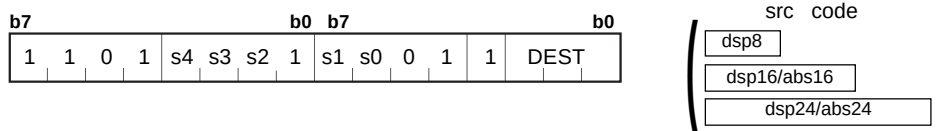
	src	s4	s3	s2	s1	s0		src	s4	s3	s2	s1	s0
Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/3	3/3	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

MOVA

(1) MOVA src, dest



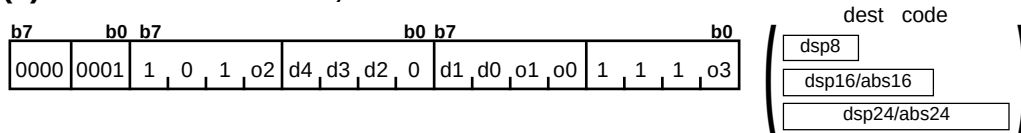
dest	DEST	src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
R2R0	0 0 0	dsp:8[An]	dsp:8[A0]	0	0	1	0	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
R3R1	0 0 1		dsp:8[A1]	0	0	1	0	1		dsp:16[FB]	0	1	0	1	1
A0	0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
A1	0 1 1		dsp:8[FB]	0	0	1	1	1		dsp:24[A1]	0	1	1	0	1
		dsp:16[An]	dsp:16[A0]	0	1	0	0	0	abs16	abs16	0	1	1	1	1
			dsp:16[A1]	0	1	0	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/2	3/2	4/2	4/2	5/2	4/2	5/2

MOVDir

(1) MOVDir R0L, dest

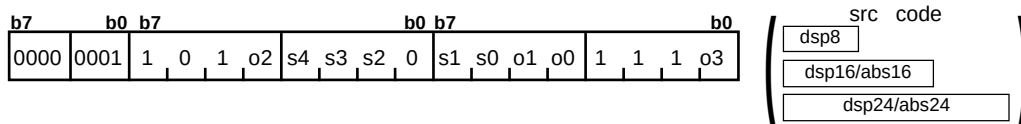


Dir	o3	o2	o1	o0
LL	0	1	0	0
HL	0	1	0	1
LH	0	1	1	0
HH	0	1	1	1

dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
Rn	R0L/---/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	R1L/---/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/---/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/---/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	---	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	---	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
MOVHH, MOVLH	3/3	3/3	3/5	4/5	4/5	5/5	5/5	6/5	5/5	6/5
MOVHL, MOVLH	3/6	3/6	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8

MOVD_{Dir}**(2) MOVD_{Dir} src, R0L**

<i>Dir</i>	o3	o2	o1	o0
LL	0	0	0	0
HL	0	0	0	1
LH	0	0	1	0
HH	0	0	1	1

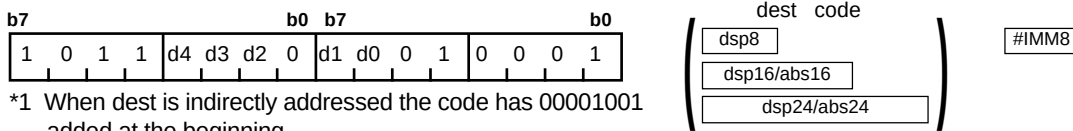
src		s4	s3	s2	s1	s0	src		s4	s3	s2	s1	s0
Rn	R0L/---/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	R1L/---/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/---/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/---/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	---	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	---	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
MOVHH, MOVLL	3/3	3/3	3/5	4/5	4/5	5/5	5/5	6/5	5/5	6/5
MOVHL, MOVLH	3/6	3/6	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8

MOVX

(1) MOVX #IMM, dest



dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
Rn	---/---/R2R0	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	---/---/R3R1	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	---/---/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	---/---/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

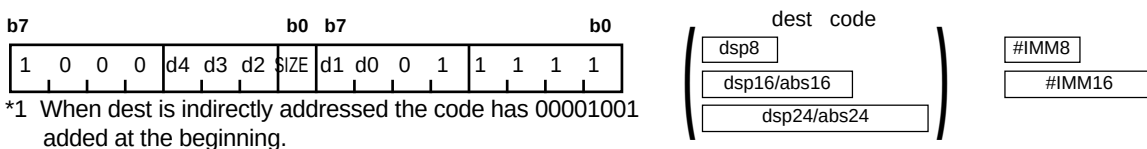
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/2	3/2	3/2	4/2	4/2	5/2	5/2	6/2	5/2	6/2

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

MUL

(1) MUL.size #IMM, dest



.size	SIZE	dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

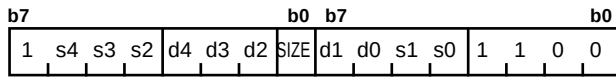
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/3	3/3	3/5	4/5	4/5	5/5	5/5	6/5	5/5	6/5

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3, respectively.

*3 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

MUL

(2) MUL.size src, dest

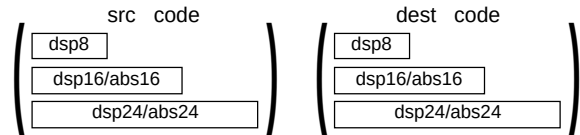


*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



.size	SIZE	src/dest		s4 s3 s2 s1 s0	src/dest		s4 s3 s2 s1 s0
.B	0			d4 d3 d2 d1 d0			d4 d3 d2 d1 d0
.W	1	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
			R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An		A0	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]		[A0]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]		dsp:8[A0]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

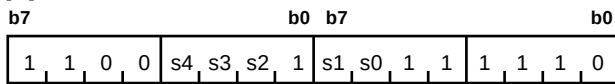
src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/3	2/3	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
An	2/3	2/3	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
[An]	2/5	2/5	2/6	3/6	3/6	4/6	4/6	5/6	4/6	5/6
dsp:8[An]	3/5	3/5	3/6	4/6	4/6	5/6	5/6	6/6	5/6	6/6
dsp:8[SB/FB]	3/5	3/5	3/6	4/6	4/6	5/6	5/6	6/6	5/6	6/6
dsp:16[An]	4/5	4/5	4/6	5/6	5/6	6/6	6/6	7/6	6/6	7/6
dsp:16[SB/FB]	4/5	4/5	4/6	5/6	5/6	6/6	6/6	7/6	6/6	7/6
dsp:24[An]	5/5	5/5	5/6	6/6	6/6	7/6	7/6	8/6	7/6	8/6
abs16	4/5	4/5	4/6	5/6	5/6	6/6	6/6	7/6	6/6	7/6
abs24	5/5	5/5	5/6	6/6	6/6	7/6	7/6	8/6	7/6	8/6

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

MULEX

(1) MULEX

src



*1 When src is indirectly addressed the code has 00001001 added at the beginning.



src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
Rn	---/---/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

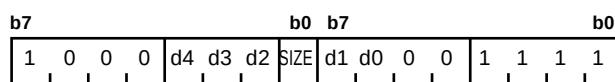
src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/8	2/8	2/10	3/10	3/10	4/10	4/10	5/10	4/10	5/10

*2 When src is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

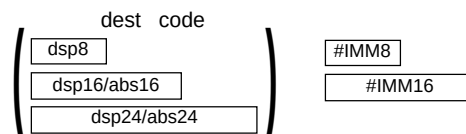
MULU

(1) MULU.size

#IMM, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

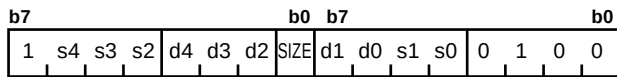
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/3	3/3	3/5	4/5	4/5	5/5	5/5	6/5	5/5	6/5

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3, respectively.

*3 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

MULU

(2) MULU.size src, dest

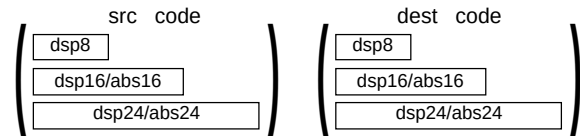


*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



.size	SIZE	src/dest						src/dest					
.B	0	s4	s3	s2	s1	s0		s4	s3	s2	s1	s0	
.W	1	d4	d3	d2	d1	d0		d4	d3	d2	d1	d0	
Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
		1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

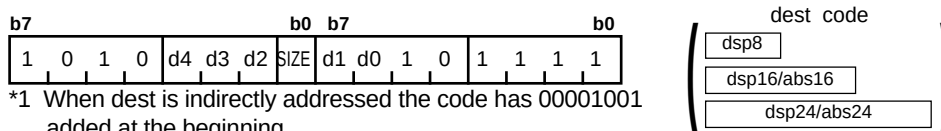
[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/3	2/3	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
An	2/3	2/3	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
[An]	2/5	2/5	2/6	3/6	3/6	4/6	4/6	5/6	4/6	5/6
dsp:8[An]	3/5	3/5	3/6	4/6	4/6	5/6	5/6	6/6	5/6	6/6
dsp:8[SB/FB]	3/5	3/5	3/6	4/6	4/6	5/6	5/6	6/6	5/6	6/6
dsp:16[An]	4/5	4/5	4/6	5/6	5/6	6/6	6/6	7/6	6/6	7/6
dsp:16[SB/FB]	4/5	4/5	4/6	5/6	5/6	6/6	6/6	7/6	6/6	7/6
dsp:24[An]	5/5	5/5	5/6	6/6	6/6	7/6	7/6	8/6	7/6	8/6
abs16	4/5	4/5	4/6	5/6	5/6	6/6	6/6	7/6	6/6	7/6
abs24	5/5	5/5	5/6	6/6	6/6	7/6	7/6	8/6	7/6	8/6

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

NEG

(1) NEG.size dest



.size	SIZE	dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

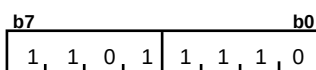
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

NOP

(1) NOP

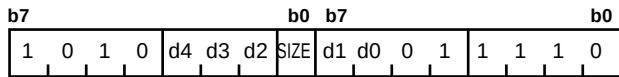


[Number of Bytes/Number of Cycles]

Bytes/Cycles	1/1
--------------	-----

NOT

(1) NOT .size dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

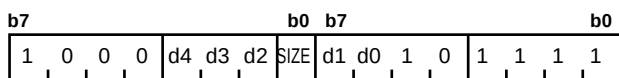
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

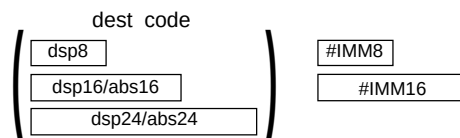
*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

OR

(1) OR.size:G #IMM, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

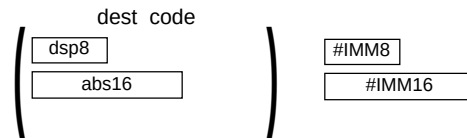
*3 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

OR**(2) OR.size:S****#IMM, dest**

b7	0	1	d1	d0	0	1	0	b0	SIZE
----	---	---	----	----	---	---	---	----	------

*1 When dest is indirectly addressed the code has 00001001 added at the beginning.

.size	SIZE	dest		d1	d0
.B	0	Rn	R0L/R0	0	0
.W	1	dsp:8[SB/FB]	dsp:8[SB]	1	0
			dsp:8[FB]	1	1
		abs16	abs16	0	1

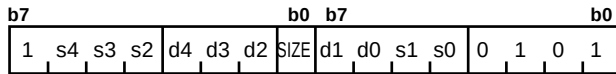
**[Number of Bytes/Number of Cycles]**

dest	Rn	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/1	3/3	4/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

OR

(3) OR.size:G src, dest

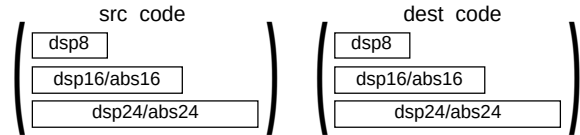


*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



.size	SIZE	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
.B	0						
.W	1						
Rn		R0L/R0/---		1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
		R1L/R1/---		1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
		R0H/R2/-		1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
		R1H/R3/-		1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An		A0		0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
		A1		0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]		[A0]		0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
		[A1]		0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]		dsp:8[A0]		0 0 1 0 0	abs16	abs16	0 1 1 1 1
		dsp:8[A1]		0 0 1 0 1	abs24	abs24	0 1 1 1 0

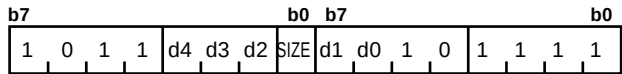
[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
An	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
[An]	2/3	2/3	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4
dsp:8[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[SB/FB]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:16[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:24[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs16	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
abs24	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

POP

(1) POP.size dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

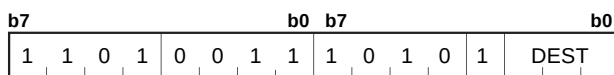
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/3	2/3	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

POPC

(1) POPC dest



dest	DEST	dest	DEST
DCT0	0 0 0	DRC0	1 0 0
DCT1	0 0 1	DRC1	1 0 1
FLG	0 1 0	DMD0	1 1 0
SVF	0 1 1	DMD1	1 1 1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/3
--------------	-----

POPC

(2) POPC dest

b7				b0				b7				b0				
1	1	0	1	0	0	1	1	0	0	1	0	1	DEST			

dest	DEST	dest	DEST
INTB	0 0 0	---	1 0 0
SP	0 0 1	---	1 0 1
SB	0 1 0	---	1 1 0
FB	0 1 1	ISP	1 1 1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/4
--------------	-----

POPM

(1) POPM dest

b7				b0				
1	0	0	0	1	1	1	0	DEST

dest							
FB	SB	A1	A0	R3	R2	R1	R0
DEST*1							

*1 The bit for a selected register is 1.
The bit for a non-selected register is 0.

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1+m
--------------	-------

*2 m denotes the number of register to be restored.
m = (number of R0, R1,R2,R3)+ 2 x (number of A0,A1,FB,SB)

PUSH

(1) PUSH.size #IMM

b7	b6	b5	b4	b3	b2	b1	b0
1	0	1	0	1	1	1	SIZE

#IMM8
#IMM16

.size	SIZE
.B	0
.W	1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1
--------------	-----

*1 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

PUSH

(2) PUSH.size src

b7	b6	b5	b4	b3	b2	b1	b0
1	1	0	0	s4	s3	s2	SIZE
				s1	s0	0	0
				1	1	1	0

*1 When src is indirectly addressed the code has 00001001 added at the beginning.

src code
dsp8
dsp16/abs16
dsp24/abs24

.size	SIZE
.B	0
.W	1

	src	s4	s3	s2	s1	s0		src	s4	s3	s2	s1	s0
Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

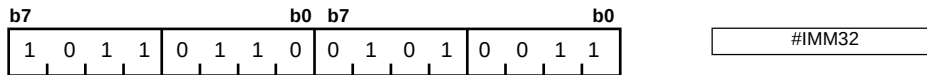
[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When src is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

PUSH

(3) PUSH.L #IMM32

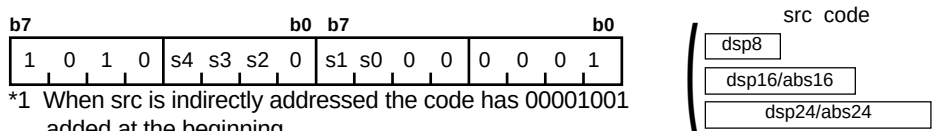


[Number of Bytes/Number of Cycles]

Bytes/Cycles	6/3
--------------	-----

PUSH

(4) PUSH.L src



*1 When src is indirectly addressed the code has 00001001 added at the beginning.

src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5

*2 When src is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

PUSHA

(1) PUSHA src

b7	b0	b7	b0
1 0 1 1	s4 s3 s2 0	s1 s0 0 0	0 0 0 1

src code
dsp8
dsp16/abs16
dsp24/abs24

src	s4 s3 s2 s1 s0	src	s4 s3 s2 s1 s0
Rn	---/---/---	dsp:8[SB/FB]	dsp:8[SB] 0 0 1 1 0
	---/---/---		dsp:8[FB] 0 0 1 1 1
	---/---/-	dsp:16[An]	dsp:16[A0] 0 1 0 0 0
	---/---/-		dsp:16[A1] 0 1 0 0 1
An	---	dsp:16[SB/FB]	dsp:16[SB] 0 1 0 1 0
	---		dsp:16[FB] 0 1 0 1 1
[An]	---	dsp:24[An]	dsp:24[A0] 0 1 1 0 0
	---		dsp:24[A1] 0 1 1 0 1
dsp:8[An]	dsp:8[A0]	abs16	abs16 0 1 1 1 1
	dsp:8[A1]	abs24	abs24 0 1 1 1 0

[Number of Bytes/Number of Cycles]

src	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/3	3/3	4/3	4/3	5/3	4/3	5/3

PUSHC

(1) PUSHC src

b7	b0	b7	b0
1 1 0 1	0 0 0 1	1 0 1 0	1 SRC

src	SRC	src	SRC
DCT0	0 0 0	DRC0	1 0 0
DCT1	0 0 1	DRC1	1 0 1
FLG	0 1 0	DMD0	1 1 0
SVF	0 1 1	DMD1	1 1 1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1
--------------	-----

PUSHC

(2) PUSHC **src**

b7	b6	b5	b4	b3	b2	b1	b0	src
1	1	0	1	0	0	0	1	0 0 1 0 1 SRC

src	SRC	src	SRC
INTB	0 0 0	---	1 0 0
SP	0 0 1	---	1 0 1
SB	0 1 0	---	1 1 0
FB	0 1 1	ISP	1 1 1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/4
--------------	-----

PUSHM

(1) PUSHM **src**

b7	b6	b5	b4	b3	b2	b1	b0	src
1	0	0	0	1	1	1	1	SRC

src							
R0	R1	R2	R3	A0	A1	SB	FB
SRC* ¹							

*1 The bit for a selected register is 1.
The bit for a non-selected register is 0.

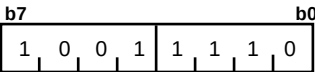
[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/m
--------------	-----

*2 m denotes the number of registers to be saved.
m = (number of R0,R1,R2,R3)+2x(number of A0,A1,FB,SB)

REIT

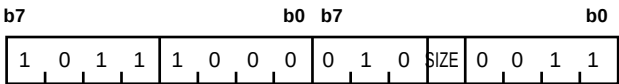
(1) REIT



[Number of Bytes/Number of Cycles]	
Bytes/Cycles	1/6

RMPA

(1) RMPA.size



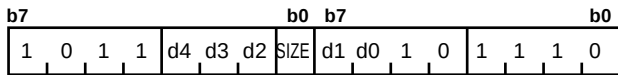
.size	SIZE
.B	0
.W	1

[Number of Bytes/Number of Cycles]	
Bytes/Cycles	2/7+2m

*1 m denotes the number of operations performed.

ROLC

(1) ROLC.size dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE	dest	d4	d3	d2	d1	d0	dest	d4	d3	d2	d1	d0
.B	0												
.W	1												
		Rn						dsp:8[SB/FB]					
								dsp:8[SB]					
								dsp:8[FB]					
								dsp:16[An]					
								dsp:16[A0]					
								dsp:16[A1]					
		An						dsp:16[SB/FB]					
								dsp:16[SB]					
								dsp:16[FB]					
		[An]						dsp:24[An]					
								dsp:24[A0]					
								dsp:24[A1]					
		dsp:8[An]						abs16					
								abs24					

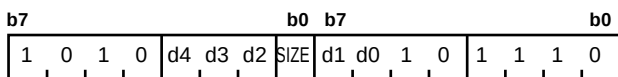
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

RORC

(1) RORC.size dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE	dest	d4	d3	d2	d1	d0	dest	d4	d3	d2	d1	d0
.B	0												
.W	1												
		Rn						dsp:8[SB/FB]					
								dsp:8[SB]					
								dsp:8[FB]					
								dsp:16[An]					
								dsp:16[A0]					
								dsp:16[A1]					
		An						dsp:16[SB/FB]					
								dsp:16[SB]					
								dsp:16[FB]					
		[An]						dsp:24[An]					
								dsp:24[A0]					
								dsp:24[A1]					
		dsp:8[An]						abs16					
								abs24					

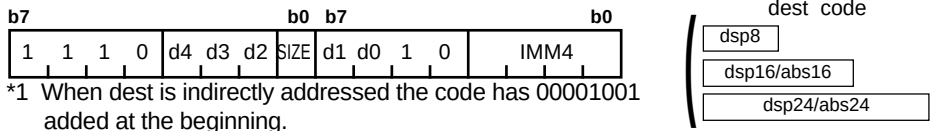
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

ROT

(1) ROT.size #IMM, dest



.size	SIZE	#IMM	IMM4	dest	IMM4
.B	0	+1	0 0 0 0	-1	1 0 0 0
.W	1	+2	0 0 0 1	-2	1 0 0 1
		+3	0 0 1 0	-3	1 0 1 0
		+4	0 0 1 1	-4	1 0 1 1
		+5	0 1 0 0	-5	1 1 0 0
		+6	0 1 0 1	-6	1 1 0 1
		+7	0 1 1 0	-7	1 1 1 0
		+8	0 1 1 1	-8	1 1 1 1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

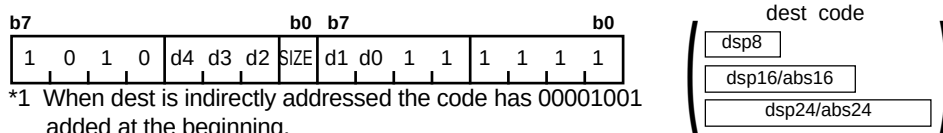
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/m	2/m	2/2+m	3/2+m	3/2+m	4/2+m	4/2+m	5/2+m	4/2+m	5/2+m

*2 m denotes the number of rotates performed.

*3 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

ROT

(2) ROT.size R1H, dest



.size	SIZE	dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/---/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			---/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

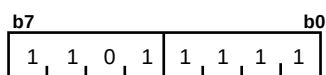
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/2+m	2/2+m	2/3+m	3/3+m	3/3+m	4/3+m	4/3+m	5/3+m	4/3+m	5/3+m

*2 m denotes the number of rotates performed.

*3 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

RTS

(1) RTS

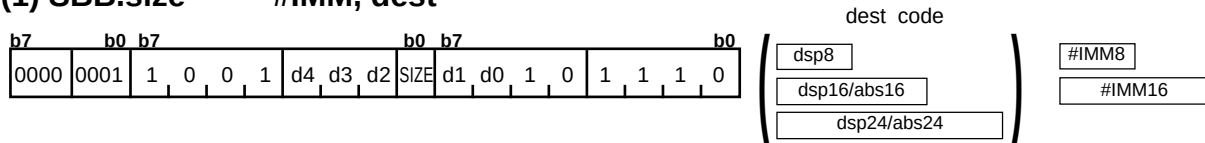


[Number of Bytes/Number of Cycles]

Bytes/Cycles	1/6
--------------	-----

SBB

(1) SBB.size #IMM, dest



.size	SIZE	dest	d4 d3 d2 d1 d0	dest	d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	dsp:8[SB]	0 0 1 1 0
			R1L/R1/---	dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	dsp:16[A1]	0 1 0 0 1
		An	A0	dsp:16[SB]	0 1 0 1 0
			A1	dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	dsp:24[A0]	0 1 1 0 0
			[A1]	dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	abs16	0 1 1 1 1
			dsp:8[A1]	abs24	0 1 1 1 0

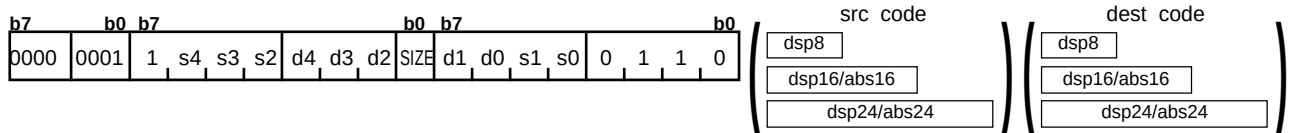
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	4/1	4/1	4/3	5/3	5/3	6/3	6/3	7/3	6/3	7/3

*1 When (.W) is specified for the size specifier(.size),the number of bytes in the table is increased by 1.

SBB

(2) SBB.size src, dest



.size	SIZE
.B	0
.W	1

src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3
An	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3
[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:8[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
dsp:16[SB/FB]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
dsp:24[An]	6/3	6/3	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4
abs16	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs24	6/3	6/3	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4

SBJNZ

(1) SBJNZ.size #IMM, dest, label



dsp8 (label code) = address indicated by label - (start address of instruction +2)

.size	SIZE	#IMM	IMM4	#IMM	IMM4
.B	0	0	0 0 0 0	+8	1 0 0 0
.W	1	-1	0 0 0 1	+7	1 0 0 1
		-2	0 0 1 0	+6	1 0 1 0
		-3	0 0 1 1	+5	1 0 1 1
		-4	0 1 0 0	+4	1 1 0 0
		-5	0 1 0 1	+3	1 1 0 1
		-6	0 1 1 0	+2	1 1 1 0
		-7	0 1 1 1	+1	1 1 1 1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

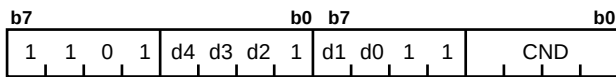
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/2	3/2	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4

*1 When branched to label the number of cycles in the table is increased by 2.

SCCnd

(1) SCCnd dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



Cnd	CND	Cnd	CND
LTU/NC	0 0 0 0	GEU/C	1 0 0 0
LEU	0 0 0 1	GTU	1 0 0 1
NE/NZ	0 0 1 0	EQ/Z	1 0 1 0
PZ	0 0 1 1	N	1 0 1 1
NO	0 1 0 0	O	1 1 0 0
GT	0 1 0 1	LE	1 1 0 1
GE	0 1 1 0	LT	1 1 1 0

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0/---/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1/---/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R2/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R3/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	---/A0/---	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	---/A1/---	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/1	2/1	2/1	3/1	3/1	4/1	4/1	5/1	4/1	5/1

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

SCMPU

(1) SCMPU.size

b7								b0	b7								b0
1	0	1	1	1	0	0	0	1	1	0	SIZE	0	0	1	1		

.size	SIZE
.B	0
.W	1

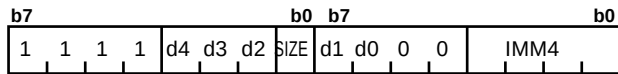
[Number of Bytes/Number of Cycles]

Size specifier	Bytes/Cycles		Remark
	Contents match and the instruction is terminated	Contents do not match and the instruction is terminated	
.B	2/6+3m	2/6+3m	The last 0 (null) is the 8 high-order bits of word
.W	2/6+1.5m	2/9+1.5m	
.W	2/8+1.5m	2/10+1.5m	The last 0(null) is the 8 low-order bits of word

*1 m denotes the number of transfers performed.

SHA

(1) SHA.size #IMM, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE
.B	0
.W	1

#IMM	IMM4	#IMM	IMM4
+1	0 0 0 0	-1	1 0 0 0
+2	0 0 0 1	-2	1 0 0 1
+3	0 0 1 0	-3	1 0 1 0
+4	0 0 1 1	-4	1 0 1 1
+5	0 1 0 0	-5	1 1 0 0
+6	0 1 0 1	-6	1 1 0 1
+7	0 1 1 0	-7	1 1 1 0
+8	0 1 1 1	-8	1 1 1 1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

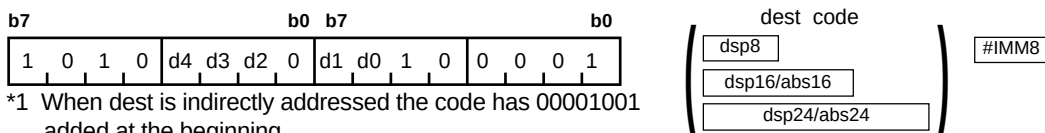
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/m	2/m	2/2+m	3/2+m	3/2+m	4/2+m	4/2+m	5/2+m	4/2+m	5/2+m

*2 m denotes the number of shifts performed.

*3 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

SHA

(2) SHA.L #IMM, dest



dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

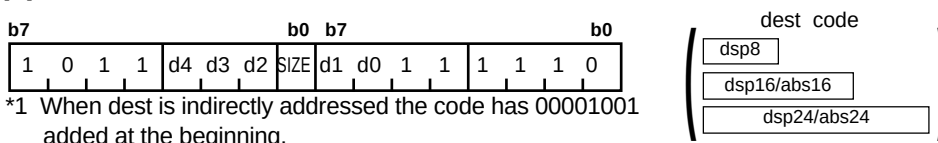
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/3+m	3/3+m	3/2+m	4/3+m	4/3+m	5/3+m	5/3+m	6/3+m	5/3+m	6/3+m

*2 m denotes the number of shifts performed.

*3 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

SHA

(3) SHA.size R1H, dest



.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/---/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			---/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

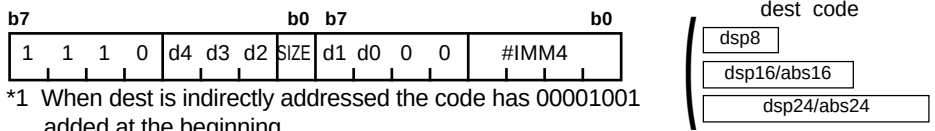
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/2+m	2/2+m	2/3+m	3/3+m	3/3+m	4/3+m	4/3+m	5/3+m	4/3+m	5/3+m

*2 m denotes the number of shifts performed.

*3 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

SHL

(1) SHL.size #IMM, dest



.size	SIZE	#IMM	IMM4	dest	IMM4
.B	0	+1	0 0 0 0	-1	1 0 0 0
.W	1	+2	0 0 0 1	-2	1 0 0 1
		+3	0 0 1 0	-3	1 0 1 0
		+4	0 0 1 1	-4	1 0 1 1
		+5	0 1 0 0	-5	1 1 0 0
		+6	0 1 0 1	-6	1 1 0 1
		+7	0 1 1 0	-7	1 1 1 0
		+8	0 1 1 1	-8	1 1 1 1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

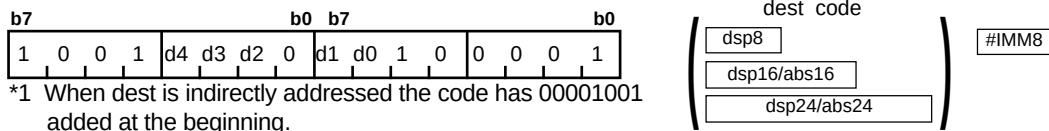
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/m	2/m	2/2+m	3/2+m	3/2+m	4/2+m	4/2+m	5/2+m	4/2+m	5/2+m

*2 m denotes the number of shifts performed.

*3 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

SHL

(2) SHL.L #IMM, dest



dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

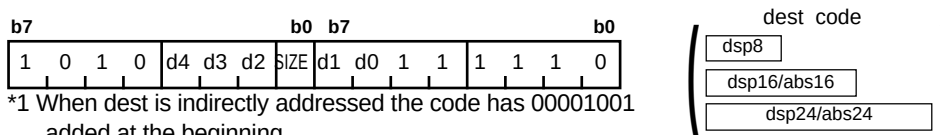
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/3+m	3/3+m	3/3+m	4/3+m	4/3+m	5/3+m	5/3+m	6/3+m	5/3+m	6/3+m

*2 m denotes the number of shifts performed.

*3 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

SHL

(3) SHL.size R1H, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.

.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			---/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

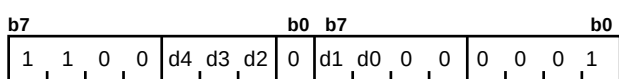
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/2+m	2/2+m	3/3+m	3/3+m	3/3+m	4/3+m	4/3+m	5/3+m	4/3+m	5/3+m

*2 m denotes the number of shifts performed.

*3 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

SHL

(4) SHL.L R1H, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

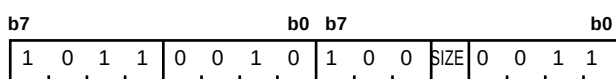
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/4+m	2/4+m	2/4+m	3/4+m	3/4+m	4/4+m	4/4+m	5/4+m	4/4+m	5/4+m

*2 m denotes the number of shifts performed.

*3 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

SIN

(1) SIN.size



.size	SIZE
.B	0
.W	1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1+2m
--------------	--------

*1 m denotes the number of transfers performed.

SMOVB

(1) SMOVB.size

b7	b6	b5	b4	b3	b2	b1	b0
1	0	1	1	0	1	1	0
1	0	0	0	SIZE	0	0	1
1	1	1	1	1	1	1	1

.size	SIZE
.B	0
.W	1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1+2m
--------------	--------

*1 m denotes the number of transfers performed.

SMOVF

(1) SMOVF.size

b7	b6	b5	b4	b3	b2	b1	b0
1	0	1	1	0	0	0	0
1	0	0	0	SIZE	0	0	1
1	1	1	1	1	1	1	1

.size	SIZE
.B	0
.W	1

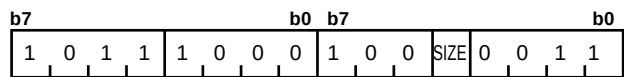
[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1+2m
--------------	--------

*1 m denotes the number of transfers performed.

SMOVU

(1) SMOVU.size



.size	SIZE
.B	0
.W	1

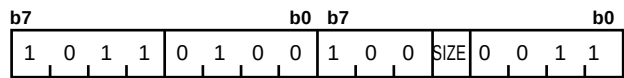
[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1+2m
--------------	--------

*1 m denotes the number of transfers performed.

SOUT

(1) SOUT.size



.size	SIZE
.B	0
.W	1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/1+2m
--------------	--------

*1 m denotes the number of transfers performed.

SSTR

(1) SSTR.size

b7	b0	b7	b0
1 0 1 1	1 0 0 0	0 0 0 0	0 0 1 1
		SIZE	

.size	SIZE
.B	0
.W	1

[Number of Bytes/Number of Cycles]

Bytes/Cycles	2/2+m
--------------	-------

*1 m denotes the number of transfers performed.

STC

(1) STC src, dest

b7	b0	b7	b0	b0
0000	0001	1 1 0 1	d4 d3 d2 1	d1 d0 0 1 0 SRC

dest code

dsp8

dsp16/abs16

dsp24/abs24

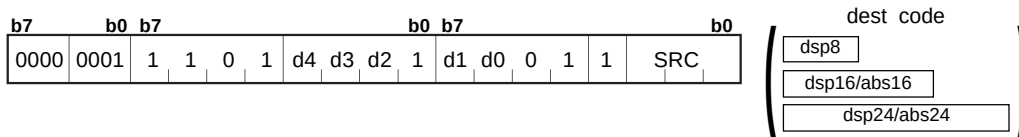
src	SRC	dest		d4 d3 d2 d1 d0					dest		d4 d3 d2 d1 d0				
-	000	Rn	---/---/R2R0	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
-	001		---/---/R3R1	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
DMA0	010		---/---/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
DMA1	011		---/---/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
DRA0	100	An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
DRA1	101		A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
DSA0	110	[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
DSA1	111		[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/3	3/3	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

STC

(2) STC src, dest



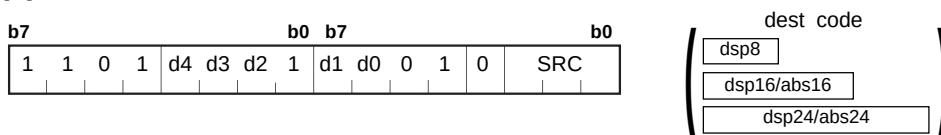
src	SRC	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
DCT0	000	Rn	---/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
DCT1	001		---/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
FLG	010		---/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
SVF	011		---/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
DRC0	100	An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
DRC1	101		A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
DMD0	110	[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
DMD1	111		[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/2	3/2	3/2	4/2	4/2	5/2	5/2	6/2	5/2	6/2

STC

(3) STC src, dest



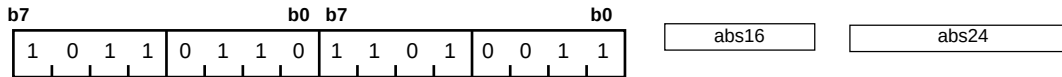
src	SRC	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
INTB	000	Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
SP	001		---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
SB	010		---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
FB	011		---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
SVP	100	An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
VCT	101		A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
-	110	[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
ISP	111		[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	2/3	2/3	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3

STCTX

(1) STCTX abs16, abs24



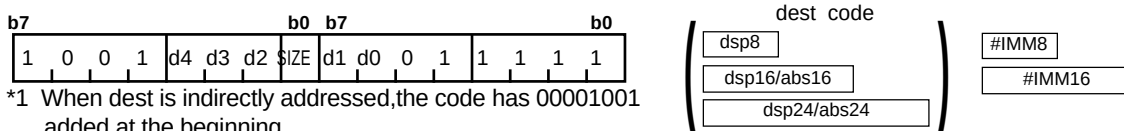
[Number of Bytes/Number of Cycles]

Bytes/Cycles	7/10+2m
--------------	---------

*1 m denotes the number of transfers performed.

STNZ

(1) STNZ.size #IMM, dest



*1 When dest is indirectly addressed, the code has 00001001 added at the beginning.

.size	SIZE	dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1		R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
			R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

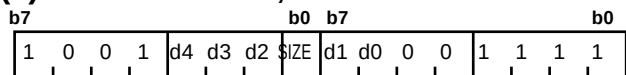
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/2	3/2	3/2	4/2	4/2	5/2	5/2	6/2	5/2	6/2

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

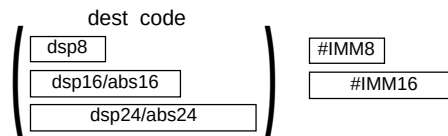
*3 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

STZ

(1) STZ.size #IMM, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE
.B	0
.W	1

dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/2	3/2	3/2	4/2	4/2	5/2	5/2	6/2	5/2	6/2

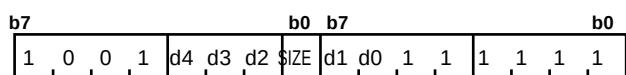
*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When Z flag is 0, the number of cycles in the table is increased by 1.

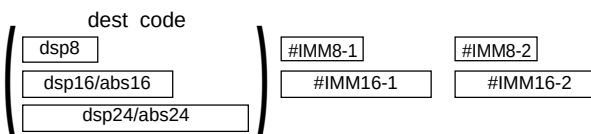
*4 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

STZX

(1) STZX.size #IMM1, #IMM2, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE
.B	0
.W	1

dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

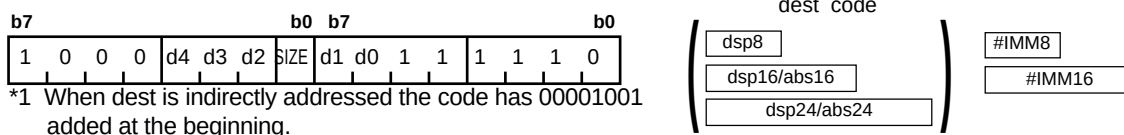
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	4/3	4/3	4/3	5/3	5/3	6/3	6/3	7/3	6/3	7/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 2.

SUB

(1) SUB.size:G #IMM, dest



.size	SIZE	dest	d4 d3 d2 d1 d0	dest	d4 d3 d2 d1 d0
.B	0	Rn	R0L/R0/---	dsp:8[SB]	0 0 1 1 0
			R1L/R1/---	dsp:8[FB]	0 0 1 1 1
			R0H/R2/-	dsp:16[An]	0 1 0 0 0
			R1H/R3/-	dsp:16[A1]	0 1 0 0 1
		An	A0	dsp:16[SB]	0 1 0 1 0
			A1	dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	dsp:24[An]	0 1 1 0 0
			[A1]	dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	abs16	0 1 1 1 1
			dsp:8[A1]	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

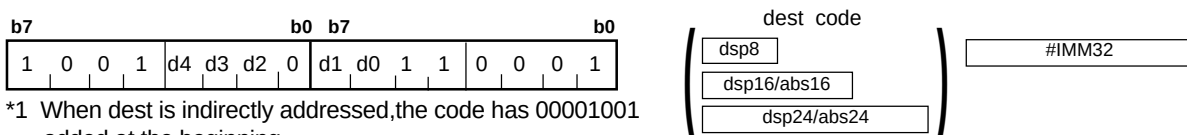
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

SUB

(2) SUB.L:G #IMM, dest



dest	d4 d3 d2 d1 d0	dest	d4 d3 d2 d1 d0
Rn	---/---/R2R0	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	dsp:8[FB]	0 0 1 1 1
	---/---/-	dsp:16[An]	0 1 0 0 0
	---/---/-	dsp:16[A1]	0 1 0 0 1
An	A0	dsp:16[SB]	0 1 0 1 0
	A1	dsp:16[FB]	0 1 0 1 1
[An]	[A0]	dsp:24[An]	0 1 1 0 0
	[A1]	dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	abs16	0 1 1 1 1
	dsp:8[A1]	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

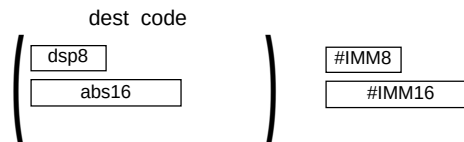
dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	6/2	6/2	6/5	7/5	7/5	8/5	8/5	9/5	8/5	9/5

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

SUB**(3) SUB.size:S #IMM, dest**

b7				b0			
0	0	d1	d0	1	1	1	SIZE

*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE	dest		d1	d0
.B	0	Rn	R0L/R0	0	0
.W	1	dsp:8[SB/FB]	dsp:8[SB]	1	0
			dsp:8[FB]	1	1
		abs16	abs16	0	1

[Number of Bytes/Number of Cycles]

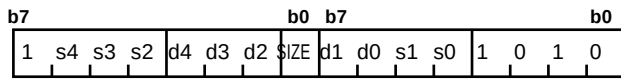
dest	Rn	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/1	3/3	4/3

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

*3 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

SUB

(4) SUB.size:G src, dest

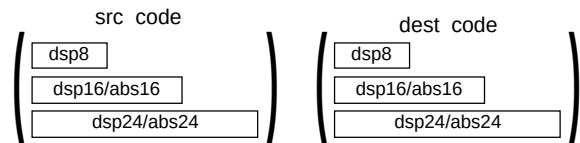


*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed

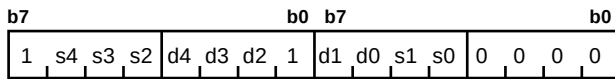


.size	SIZE	src/dest					s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0			
.B	0												
.W	1												
Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
	R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
	R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
	R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
	A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
	[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
	dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
An	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
[An]	2/3	2/3	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4
dsp:8[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[SB/FB]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:16[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:24[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs16	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
abs24	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

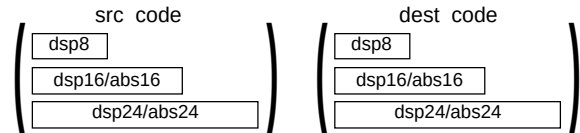
SUB**(5) SUB.L:G src, dest**

*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

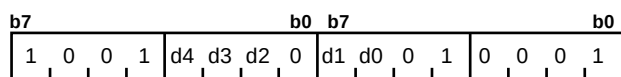
[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
An	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
[An]	2/5	2/5	2/8	3/8	3/8	4/8	4/8	5/8	4/8	5/8
dsp:8[An]	3/5	3/5	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8
dsp:8[SB/FB]	3/5	3/5	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8
dsp:16[An]	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
dsp:16[SB/FB]	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
dsp:24[An]	5/5	5/5	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8
abs16	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
abs24	5/5	5/5	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8

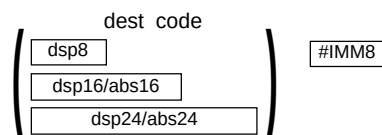
*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

SUBX

(1) SUBX #IMM, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.

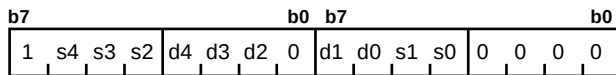


dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/2	3/2	3/5	4/5	4/5	5/5	5/5	6/5	5/5	6/5

*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

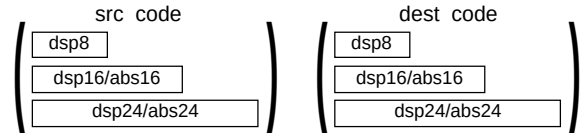
SUBX**(2) SUBX src, dest**

*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



src		s4 s3 s2 s1 s0	src		s4 s3 s2 s1 s0
Rn	R0L/---/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/---/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	---/---/R2R0	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	---/---/R3R1	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	---/---/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	---/---/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

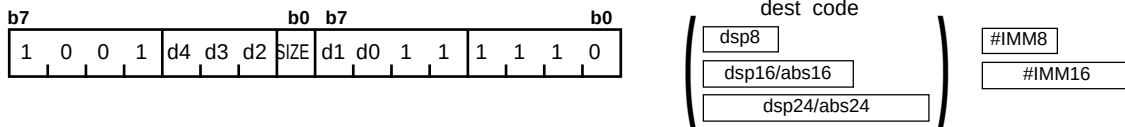
[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
An	2/2	2/2	2/5	3/5	3/5	4/5	4/5	5/5	4/5	5/5
[An]	2/5	2/5	2/8	3/8	3/8	4/8	4/8	5/8	4/8	5/8
dsp:8[An]	3/5	3/5	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8
dsp:8[SB/FB]	3/5	3/5	3/8	4/8	4/8	5/8	5/8	6/8	5/8	6/8
dsp:16[An]	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
dsp:16[SB/FB]	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
dsp:24[An]	5/5	5/5	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8
abs16	4/5	4/5	4/8	5/8	5/8	6/8	6/8	7/8	6/8	7/8
abs24	5/5	5/5	5/8	6/8	6/8	7/8	7/8	8/8	7/8	8/8

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

TST

(1) TST.size:G #IMM, dest



.size	SIZE	dest		d4	d3	d2	d1	d0	dest		d4	d3	d2	d1	d0
.B	0	Rn	R0L/R0/---	1	0	0	1	0	dsp:8[SB/FB]	dsp:8[SB]	0	0	1	1	0
.W	1		R1L/R1/---	1	0	0	1	1		dsp:8[FB]	0	0	1	1	1
			R0H/R2/-	1	0	0	0	0	dsp:16[An]	dsp:16[A0]	0	1	0	0	0
			R1H/R3/-	1	0	0	0	1		dsp:16[A1]	0	1	0	0	1
		An	A0	0	0	0	1	0	dsp:16[SB/FB]	dsp:16[SB]	0	1	0	1	0
			A1	0	0	0	1	1		dsp:16[FB]	0	1	0	1	1
		[An]	[A0]	0	0	0	0	0	dsp:24[An]	dsp:24[A0]	0	1	1	0	0
			[A1]	0	0	0	0	1		dsp:24[A1]	0	1	1	0	1
		dsp:8[An]	dsp:8[A0]	0	0	1	0	0	abs16	abs16	0	1	1	1	1
			dsp:8[A1]	0	0	1	0	1	abs24	abs24	0	1	1	1	0

[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

*1 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

TST

(2) TST.size:S #IMM, dest



.size	SIZE	dest		d1	d0
.B	0	Rn	R0L/R0	0	0
.W	1		dsp:8[SB]	1	0
		dsp:8[SB/FB]	dsp:8[FB]	1	1
			abs16	0	1

[Number of Bytes/Number of Cycles]

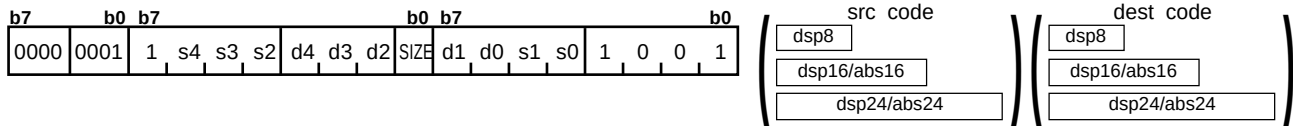
dest	Rn	dsp:8[SB/FB]	abs16
Bytes/Cycles	2/1	3/3	4/3

*1 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

TST

(3) TST.size:G

src, dest



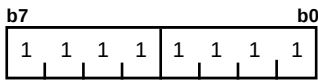
.size	SIZE	src/dest						src/dest					
.B	0	s4 s3 s2 s1 s0 d4 d3 d2 d1 d0						s4 s3 s2 s1 s0 d4 d3 d2 d1 d0					
.W	1	Rn						Rn					
		R0L/R0/---						R0L/R0/---					
		R1L/R1/---						R1L/R1/---					
		R0H/R2/-						R0H/R2/-					
		R1H/R3/-						R1H/R3/-					
		An						An					
		A0						A0					
		A1						A1					
		[An]						[An]					
		[A0]						[A0]					
		[A1]						[A1]					
		dsp:8[An]						dsp:8[An]					
		dsp:8[A0]						dsp:8[A0]					
		dsp:8[A1]						dsp:8[A1]					

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3
An	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3
[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:8[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
dsp:16[SB/FB]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
dsp:24[An]	6/3	6/3	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4
abs16	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs24	6/3	6/3	6/4	7/4	7/4	8/4	8/4	9/4	8/4	9/4

UND

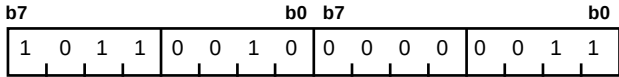
(1) UND



[Number of Bytes/Number of Cycles]	
Bytes/Cycles	1/13

WAIT

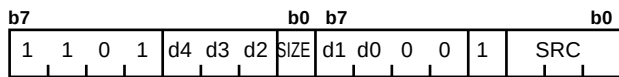
(1) WAIT



[Number of Bytes]	
Bytes	2

XCHG

(1) XCHG.size src, dest



*1 When dest is indirectly addressed the code has 00001001 added at the beginning.



.size	SIZE
.B	0
.W	1

src	SRC
R0L/R0/---	0 0 0
R1L/R1/---	0 0 1
R0H/R2/-	1 0 0
R1H/R3/-	1 0 1
A0	0 1 0
A1	0 1 1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

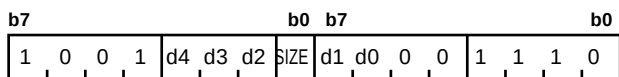
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/cycles	2/3	2/3	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4

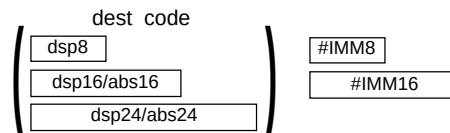
*2 When dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively.

XOR

(1) XOR.size #IMM, dest



*1 When dest is indirectly addressed, the code has 00001001 added at the beginning.



.size	SIZE
.B	0
.W	1

dest		d4 d3 d2 d1 d0	dest		d4 d3 d2 d1 d0
Rn	R0L/R0/---	1 0 0 1 0	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
	R1L/R1/---	1 0 0 1 1		dsp:8[FB]	0 0 1 1 1
	R0H/R2/-	1 0 0 0 0	dsp:16[An]	dsp:16[A0]	0 1 0 0 0
	R1H/R3/-	1 0 0 0 1		dsp:16[A1]	0 1 0 0 1
An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
	A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
	[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
	dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

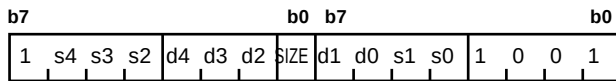
[Number of Bytes/Number of Cycles]

dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Bytes/Cycles	3/1	3/1	3/3	4/3	4/3	5/3	5/3	6/3	5/3	6/3

*2 When (.W) is specified for the size specifier(.size) the number of bytes in the table is increased by 1.

XOR

(2) XOR.size src, dest

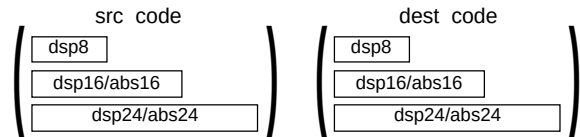


*1 For indirect addressing, the following number is added at the beginning of code:

01000001 when src is indirectly addressed

00001001 when dest is indirectly addressed

01001001 when src and dest are indirectly addressed



.size	SIZE	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0	src/dest		s4 s3 s2 s1 s0 d4 d3 d2 d1 d0
.B	0	Rn		R0L/R0/--- R1L/R1/--- R0H/R2/- R1H/R3/-	dsp:8[SB/FB]	dsp:8[SB]	0 0 1 1 0
.W	1					dsp:8[FB]	0 0 1 1 1
					dsp:16[An]	dsp:16[A0]	0 1 0 0 0
						dsp:16[A1]	0 1 0 0 1
		An	A0	0 0 0 1 0	dsp:16[SB/FB]	dsp:16[SB]	0 1 0 1 0
			A1	0 0 0 1 1		dsp:16[FB]	0 1 0 1 1
		[An]	[A0]	0 0 0 0 0	dsp:24[An]	dsp:24[A0]	0 1 1 0 0
			[A1]	0 0 0 0 1		dsp:24[A1]	0 1 1 0 1
		dsp:8[An]	dsp:8[A0]	0 0 1 0 0	abs16	abs16	0 1 1 1 1
			dsp:8[A1]	0 0 1 0 1	abs24	abs24	0 1 1 1 0

[Number of Bytes/Number of Cycles]

src \ dest	Rn	An	[An]	dsp:8[An]	dsp:8[SB/FB]	dsp:16[An]	dsp:16[SB/FB]	dsp:24[An]	abs16	abs24
Rn	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
An	2/1	2/1	2/3	3/3	3/3	4/3	4/3	5/3	4/3	5/3
[An]	2/3	2/3	2/4	3/4	3/4	4/4	4/4	5/4	4/4	5/4
dsp:8[An]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:8[SB/FB]	3/3	3/3	3/4	4/4	4/4	5/4	5/4	6/4	5/4	6/4
dsp:16[An]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:16[SB/FB]	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
dsp:24[An]	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4
abs16	4/3	4/3	4/4	5/4	5/4	6/4	6/4	7/4	6/4	7/4
abs24	5/3	5/3	5/4	6/4	6/4	7/4	7/4	8/4	7/4	8/4

*2 When src or dest is indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 3 respectively. Also, when src and dest both are indirectly addressed, the number of bytes and cycles in the table are increased by 1 and 6, respectively.

Chapter 5

Interrupt

- 5.1 Outline of Interrupt**
- 5.2 Interrupt Control**
- 5.3 Interrupt Sequence**
- 5.4 Return from Interrupt Routine**
- 5.5 Interrupt Priority**
- 5.6 Multiple Interrupts**
- 5.7 Precautions for Interrupts**
- 5.8 Exit from Stop Mode and Wait Mode**

5.1 Outline of Interrupt

When an interrupt request is acknowledged, control branches to the interrupt routine that is set to an interrupt vector table. Each interrupt vector table must have had the start address of its corresponding interrupt routine set. For details about the interrupt vector table, refer to Section 1.10, “Vector Table.”

5.1.1 Types of Interrupts

Figure 5.1.1 lists the types of interrupts. Table 5.1.1 and 5.1.2 list the source of interrupts (non-maskable) and the fixed vector tables.

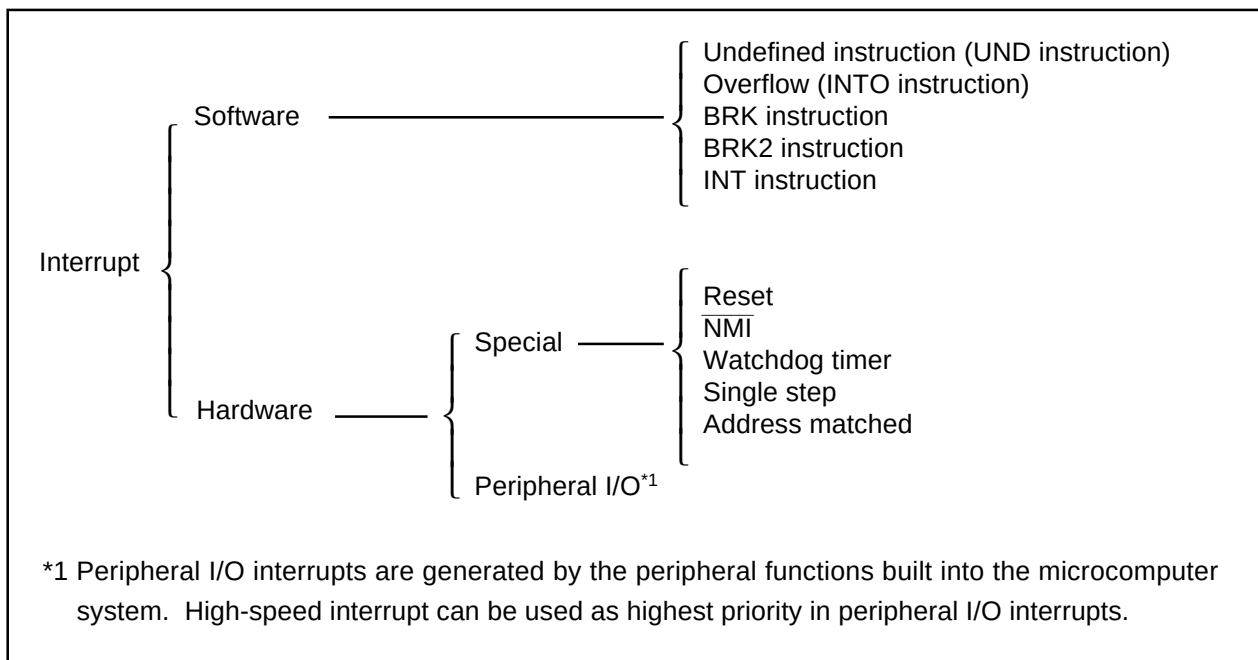


Figure 5.1.1. Classification of interrupts

Table 5.1.1 Interrupt Source (Nonmaskable) and Fixed Vector Table

Interrupt source	Vector table addresses Address (L) to address (H)	Remarks
Undefined instruction	FFFFDC ₁₆ to FFFFDF ₁₆	Interrupt generated by the UND instruction.
Overflow	FFFFE0 ₁₆ to FFFFE3 ₁₆	Interrupt generated by the INTO instruction.
BRK instruction	FFFFE4 ₁₆ to FFFFE7 ₁₆	Executed beginning from address indicated by vector in variable vector table if content of address FFFFE7 ₁₆ is FF ₁₆ .
Address match	FFFFE8 ₁₆ to FFFFEB ₁₆	Can be controlled by an interrupt enable bit.
Watchdog timer	FFFFF0 ₁₆ to FFFFF3 ₁₆	
NMI	FFFFF8 ₁₆ to FFFFFB ₁₆	External interrupt generated by driving $\overline{\text{NMI}}$ pin low.
Reset	FFFFFC ₁₆ to FFFFFF ₁₆	

Table 5.1.2 Interrupt Exclusively for Emulator (Nonmaskable) and Vector Table

Interrupt source	Vector table addresses Address (L) to address (H)	Remarks
BRK2 instruction	Interrupt vector table register exclusively for emulator 000020 ₁₆ to 000023 ₁₆	This interrupt is used exclusively for debugger purposes.
Single step		

- Maskable interrupt: This type of interrupt can be controlled by using the I flag to enable (or disable) an interrupt or by changing the interrupt priority level.
- Nonmaskable interrupt: This type of interrupt cannot be controlled by using the I flag to enable (or disable) an interrupt or by changing the interrupt priority level.

5.1.2 Software Interrupts

Software interrupts are generated by some instruction that generates an interrupt request when executed. Software interrupts are nonmaskable interrupts.

(1) Undefined-instruction interrupt

This interrupt occurs when the UND instruction is executed.

(2) Overflow interrupt

This interrupt occurs if the INTO instruction is executed when the O flag is 1.

The following lists the instructions that cause the O flag to change:

ABS, ADC, ADCF, ADD, ADDX, CMP, CMPX, DIV, DIVU, DIVX, NEG, RMPA, SBB, SCMPU, SHA, SUB, SUBX

(3) BRK interrupt

This interrupt occurs when the BRK instruction is executed.

(4) BRK2 interrupt

This interrupt occurs when the BRK2 instruction is executed. This interrupt is used exclusively for debugger purposes. You normally do not need to use this interrupt.

(5) INT instruction interrupt

This interrupt occurs when the INT instruction is executed after specifying a software interrupt number from 0 to 63. Note that software interrupt numbers 0 to 43 are assigned to peripheral I/O interrupts. This means that by executing the INT instruction, you can execute the same interrupt routine as used in peripheral I/O interrupts.

The stack pointer used in INT instruction interrupt varies depending on the software interrupt number. For software interrupt numbers 0 to 31, the U flag is saved when an interrupt occurs and the U flag is cleared to 0 to choose the interrupt stack pointer (ISP) before executing the interrupt sequence. The previous U flag before the interrupt occurred is restored when control returns from the interrupt routine. For software interrupt numbers 32 to 63, such stack pointer switchover does not occur.

However, in peripheral I/O interrupts, the U flag is saved when an interrupt occurs and the U flag is cleared to 0 to choose ISP.

Therefore movement of U flag is different by peripheral I/O interrupt or INT instruction in software interrupt number 32 to 43.

5.1.3 Hardware Interrupts

There are Two types in hardware Interrupts; special interrupts and Peripheral I/O interrupts.

(1) Special interrupts

Special interrupts are nonmaskable interrupts.

- **Reset**

A reset occurs when the $\overline{\text{RESET}}$ pin is pulled low.

- **$\overline{\text{NMI}}$ interrupt**

This interrupt occurs when the $\overline{\text{NMI}}$ pin is pulled low.

- **Watchdog timer interrupt**

This interrupt is caused by the watchdog timer.

- **Address-match interrupt**

This interrupt occurs when the program's execution address matches the content of the address match register while the address match interrupt enable bit is set (= 1).

This interrupt does not occur if any address other than the start address of an instruction is set in the address match register.

- **Single-step interrupt**

This interrupt is used exclusively for debugger purposes. You normally do not need to use this interrupt. A single-step interrupt occurs when the D flag is set (= 1); in this case, an interrupt is generated each time an instruction is executed.

(2) Peripheral I/O interrupts

These interrupts are generated by the peripheral functions built into the microcomputer system. The types of built-in peripheral functions vary with each M16C model, so do the types of interrupt causes. The interrupt vector table uses the same software interrupt numbers 0–43 that are used by the INT instruction. Peripheral I/O interrupts are maskable interrupts. For details about peripheral I/O interrupts, refer to the M16C User's Manual.

For peripheral I/O interrupts, the U flag is saved when an interrupt occurs and the U flag is cleared to 0 to choose the interrupt stack pointer (ISP) before executing the interrupt sequence. The previous U flag before the interrupt occurred is restored when control returns from the interrupt routine.

(3) High-speed interrupts

High-speed interrupts are interrupts in which the response is executed at high-speed. High-speed interrupt can be used as highest priority in peripheral I/O interrupts.

Execute a FREIT instruction to return from the high-speed interrupt routine.

For details about high-speed interrupt, refer to the M16C User's Manual.

5.2 Interrupt Control

The following explains how to enable/disable maskable interrupts and set acknowledge priority. The explanation here does not apply to non-maskable interrupts.

Maskable interrupts are enabled and disabled by using the interrupt enable flag (I flag), interrupt priority level select bit, and processor interrupt priority level (IPL). Whether there is any interrupt requested is indicated by the interrupt request bit. The interrupt request bit and interrupt priority level select bit are arranged in the interrupt control register provided for each specific interrupt. The interrupt enable flag (I flag) and processor interrupt priority level (IPL) are arranged in the flag register (FLG).

For details about the memory allocation and the configuration of interrupt control registers, refer to the M16C User's Manual.

5.2.1 Interrupt Enable Flag (I Flag)

The interrupt enable flag (I flag) is used to disable/enable maskable interrupts. When this flag is set (= 1), all maskable interrupts are enabled; when the flag is cleared to 0, they are disabled. This flag is automatically cleared to 0 after a reset is cleared.

When the I flag is changed, the altered flag status is reflected in determining whether or not to accept an interrupt request at the following timing:

- If the flag is changed by an REIT or FREIT instruction, the changed status takes effect beginning with that REIT or FREIT instruction.
- If the flag is changed by an FCLR, FSET, POPC, or LDC instruction, the changed status takes effect beginning with the next instruction.

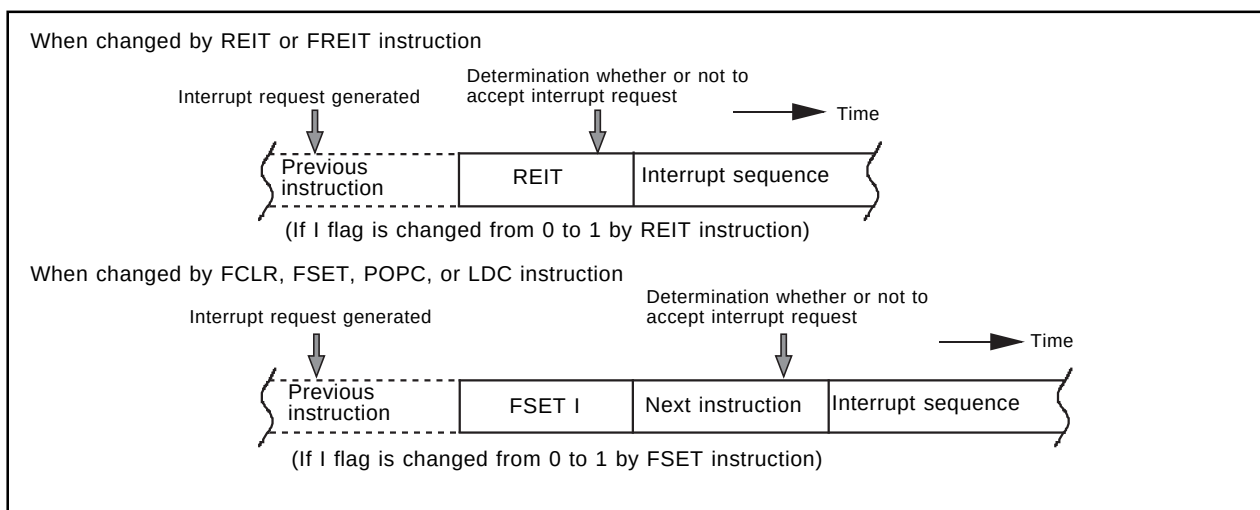


Figure 5.2.1 Timing at which changes of I flag are reflected in interrupt handling

5.2.2 Interrupt Request Bit

This bit is set (= 1) when an interrupt request is generated. This bit remains set until the interrupt request is acknowledged. The bit is cleared to 0 when the interrupt request is acknowledged.

This bit can be cleared to 0 (but cannot be set to 1) in software.

5.2.3 Interrupt Priority Level Select Bit and Processor Interrupt Priority Level (IPL)

Interrupt priority levels are set by the interrupt priority select bit in an interrupt control register. When an interrupt request is generated, the interrupt priority level of this interrupt is compared with the processor interrupt priority level (IPL). This interrupt is enabled only when its interrupt priority level is greater than the processor interrupt priority level (IPL). This means that you can disable any particular interrupt by setting its interrupt priority level to 0.

Table 5.2.1 shows how interrupt priority levels are set. Table 5.2.2 shows interrupt enable levels in relation to the processor interrupt priority level (IPL).

The following lists the conditions under which an interrupt request is acknowledged:

- Interrupt enable flag (I flag) = 1
- Interrupt request bit = 1
- Interrupt priority level > Processor interrupt priority level (IPL)

The interrupt enable flag (I flag), interrupt request bit, interrupt priority level select bit, and the processor interrupt priority level (IPL) all are independent of each other, so they do not affect any other bit.

Table 5.2.1 Interrupt Priority Levels

Interrupt priority level select bit	Interrupt priority level	Priority order
b2 0 b1 0 b0 0	Level 0 (interrupt disabled)	——
0 0 1	Level 1	Low ↓ High
0 1 0	Level 2	
0 1 1	Level 3	
1 0 0	Level 4	
1 0 1	Level 5	
1 1 0	Level 6	
1 1 1	Level 7	

Table 5.2.2 IPL and Interrupt Enable Levels

Processor interrupt priority level (IPL)	Enabled interrupt priority levels
IPL ₂ 0 IPL ₁ 0 IPL ₀ 0	Interrupt levels 1 and above are enabled.
0 0 1	Interrupt levels 2 and above are enabled.
0 1 0	Interrupt levels 3 and above are enabled.
0 1 1	Interrupt levels 4 and above are enabled.
1 0 0	Interrupt levels 5 and above are enabled.
1 0 1	Interrupt levels 6 and above are enabled.
1 1 0	Interrupt levels 7 and above are enabled.
1 1 1	All maskable interrupts are disabled.

When the processor interrupt priority level (IPL) or the interrupt priority level of some interrupt is changed, the altered level is reflected in interrupt handling at the following timing:

- If the processor interrupt priority level (IPL) is changed by an REIT or FREIT instruction, the changed level takes effect beginning with the REIT or FREIT instruction.
- If the processor interrupt priority level (IPL) is changed by a POPC, LDC, or LDIPL instruction, the changed level takes effect beginning with the next instruction.
- If the interrupt priority level of a particular interrupt is changed by an instruction such as MOV, the changed level takes effect beginning with the instruction that is executed two clock or two clock periods after the last clock of the instruction used.

5.2.4 Rewrite the interrupt control register

When an instruction to rewrite the interrupt control register is executed but the interrupt is disabled, the interrupt request bit is not set sometimes even if the interrupt request for that register has been generated. This will depend on the instruction. If this creates problems, use the below instructions to change the register.

Instructions : AND, OR, BCLR, BSET

5.3 Interrupt Sequence

An interrupt sequence — what are performed over a period from the instant an interrupt is accepted to the instant the interrupt routine is executed — is described here.

If an interrupt occurs during execution of an instruction, the processor determines its priority when the execution of the instruction is completed, and transfers control to the interrupt sequence from the next cycle. If an interrupt occurs during execution of either the SCMPU, SIN, SMOVB, SMOVF, SMOVU, SSTTR, SOUT or RMPA instruction, the processor temporarily suspends the instruction being executed, and transfers control to the interrupt sequence.

In the interrupt sequence, the processor carries out the following in sequence given:

- (1) CPU gets the interrupt information (the interrupt number and interrupt request level) by reading address 000000₁₆ (address 000002₁₆ when high-speed interrupt).
- (2) Saves the content of the flag register (FLG) as it was immediately before the start of interrupt sequence in the temporary register (Note) within the CPU.
- (3) Sets the interrupt enable flag (I flag), the debug flag (D flag), and the stack pointer select flag (U flag) to "0" (the U flag, however does not change if the INT instruction, in software interrupt numbers 32 through 63, is executed)
- (4) Saves the content of the temporary register (Note 1) within the CPU in the stack area. Saves in the flag save register (SVF) in high-speed interrupt.
- (5) Saves the content of the program counter (PC) in the stack area. Saves in the PC save register (SVP) in high-speed interrupt.
- (6) Sets the interrupt priority level of the accepted instruction in the IPL.

After the interrupt sequence is completed, the processor resumes executing instructions from the first address of the interrupt routine.

Note: This register cannot be utilized by the user.

5.3.1 Interrupt Response Time

The interrupt response time means a period of time from when an interrupt request is generated till when the first instruction of the interrupt routine is executed. This period consists of time (a) from when an interrupt request is generated to when the instruction then under way is completed and time (b) in which an interrupt sequence is executed. Figure 5.3.1 shows the interrupt response time.

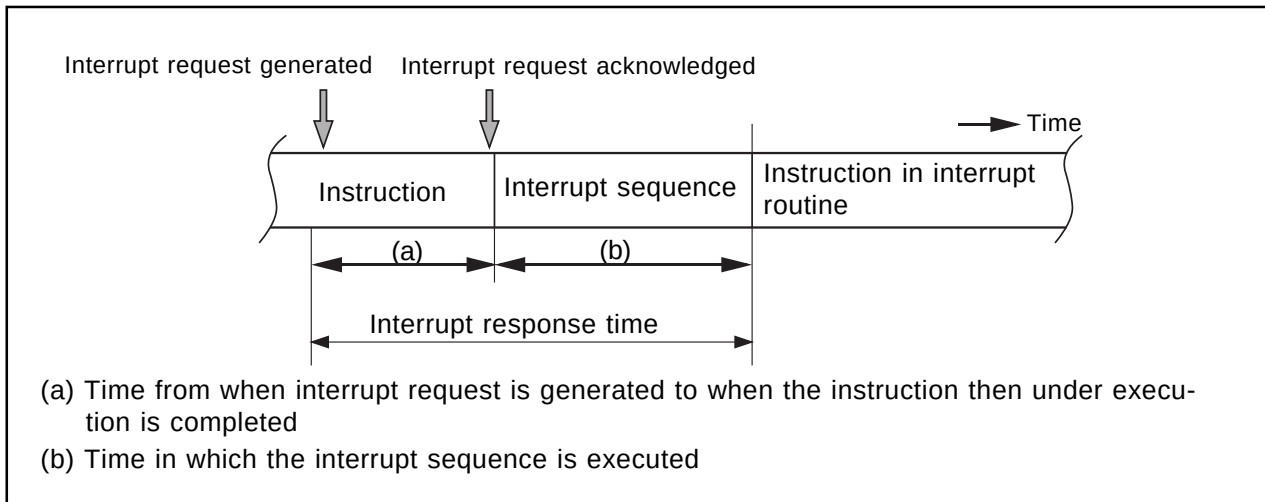


Figure 5.3.1. Interrupt response time

Time (a) varies with each instruction being executed. The DIVX instruction requires a maximum time that consists of 29* cycles.

Time (b) is shown in table 5.3.1.

* It is when the divisor is immediate or register. When the divisor is memory, the following value is added.

- Normal addressing : $2 + X$
- Index addressing : $3 + X$
- Indirect addressing : $5 + X + 2Y$
- Indirect index addressing : $5 + X + 2Y$

X is number of wait of the divisor area. Y is number of wait of the indirect address stored area. When X and Y are in odd address or in 8 bits bus area, double the value of X and Y.

Table 5.3.1 Interrupt Sequence Execution Time

Interrupt	Interrupt vector address	16 bits data bus	8 bits data bus
Peripheral I/O	Even address	14 cycles	16 cycles
	Odd address ^{*2}	16 cycles	16 cycles
INT instruction	Even address	12 cycles	14 cycles
	Odd address ^{*2}	14 cycles	14 cycles
NMI Watchdog timer Undefined instruction Address match	Even address ^{*1}	13 cycles	15 cycles
Overflow	Even address ^{*1}	14 cycles	16 cycles
BRK instruction (Variable vector table)	Even address	17 cycles	19 cycles
	Odd address ^{*2}	19 cycles	19 cycles
Single step BRK2 instruction BRK instruction (Fixed vector table)	Even address ^{*1}	19 cycles	21 cycles
High-speed interrupt ^{*3}	Vector table is internal register	5 cycles	

*1 The vector table is fixed to even address.

*2 Allocate interrupt vector addresses in even addresses as much as possible.

*3 The high-speed interrupt is independent of these conditions.

5.3.2 Changes of IPL When Interrupt Request Acknowledged

When an interrupt request is acknowledged, the interrupt priority level of the acknowledged interrupt is set to the processor interrupt priority level (IPL).

If an interrupt request is acknowledged that does not have an interrupt priority level, the value shown in Table 5.3.2 is set to the IPL.

Table 5.3.2 Relationship between Interrupts without Interrupt Priority Levels and IPL

Interrupt sources without interrupt priority levels	Value that is set to IPL
Watchdog timer, $\overline{\text{NMI}}$	7
Reset	0
Other	Not changed

5.3.3 Saving Registers

In an interrupt sequence, only the contents of the flag register (FLG) and program counter (PC) are saved to the stack area.

The order in which these contents are saved is as follows: First, the FLG register is saved to the stack area. Next, the 16 high-order bits and 16 low-order bits of the program counter expanded to 32-bit are saved. Figure 5.3.2 shows the stack status before an interrupt request is acknowledged and the stack status after an interrupt request is acknowledged.

In a high-speed interrupt sequence, the contents of the flag register (FLG) is saved to the flag save register (SVF) and program counter (PC) is saved to PC save register (SVP).

If there are any other registers you want to be saved, save them in software at the beginning of the interrupt routine. The PUSHM instruction allows you to save all registers except the stack pointer (SP) by a single instruction.

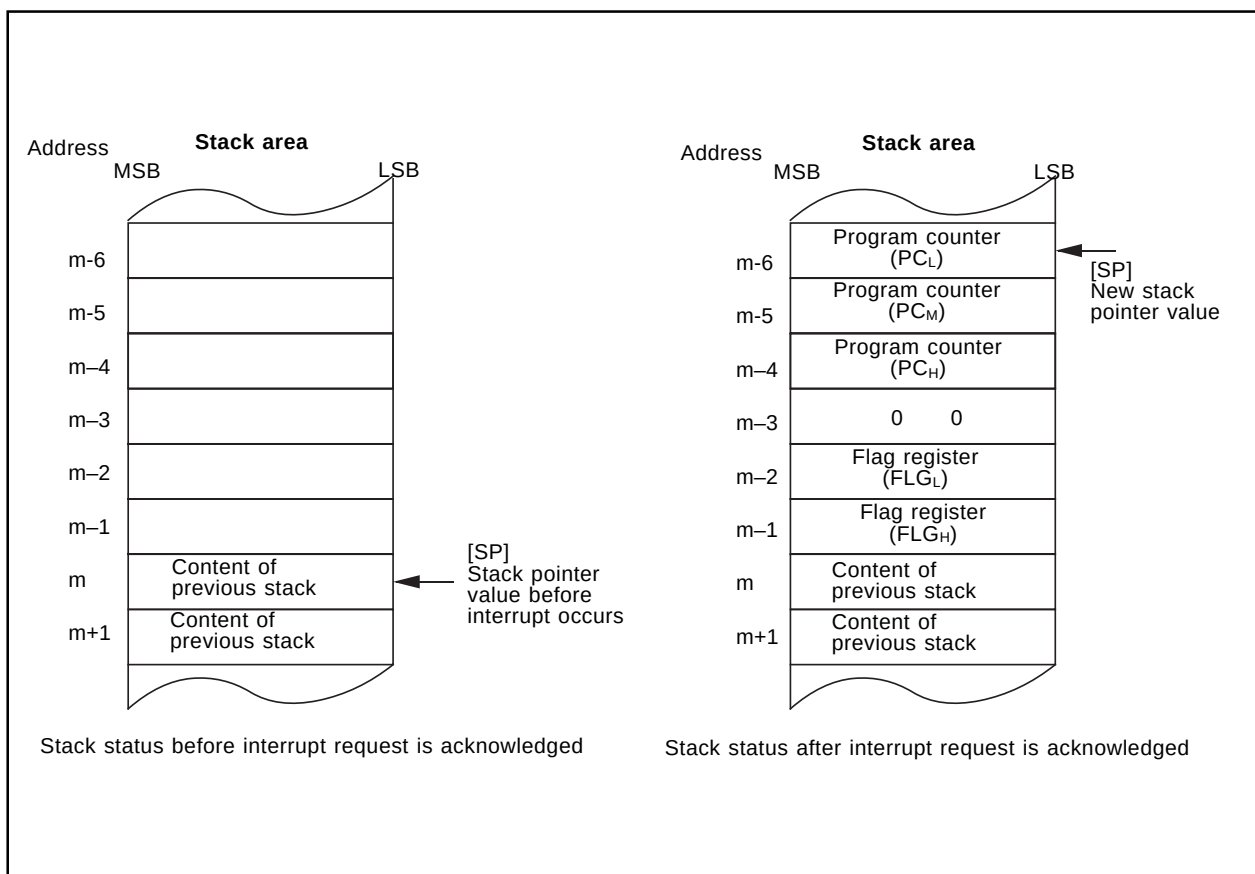


Figure 5.3.2 Stack status before and after an interrupt request is acknowledged

5.4 Return from Interrupt Routine

As you execute the REIT instruction at the end of the interrupt routine, the contents of the flag register (FLG) and program counter (PC) that have been saved to the stack area immediately preceding the interrupt sequence are automatically restored. In high-speed interrupt, as you execute the REIT instruction at the end of the interrupt routine, the contents of the flag register (FLG) and program counter (PC) that have been saved to the save registers immediately preceding the interrupt sequence are automatically restored.

Then control returns to the routine that was under execution before the interrupt request was acknowledged, and processing is resumed from where control left off.

If there are any registers you saved via software in the interrupt routine, be sure to restore them using an instruction (e.g., POPM instruction) before executing the REIT or FREIT instruction.

5.5 Interrupt Priority

If two or more interrupt requests are sampled active at the same time, whichever interrupt request is acknowledged that has the highest priority.

Maskable interrupts (Peripheral I/O interrupts) can be assigned any desired priority by setting the interrupt priority level select bit accordingly. If some maskable interrupts are assigned the same priority level, the interrupt that a request came to most in the first place is accepted at first, and then, the priority between these interrupts is resolved by the priority that is set in hardware^{*1}.

Certain nonmaskable interrupts such as a reset (reset is given the highest priority) and watchdog timer interrupt have their priority levels set in hardware. Figure 5.5.1 lists the hardware priority levels of these interrupts.

Software interrupts are not subjected to interrupt priority. They always cause control to branch to an interrupt routine whenever the relevant instruction is executed.

*1 Hardware priority varies with each M16C model. Please refer to your M16C User's Manual.

Reset > $\overline{\text{NMI}}$ > Watchdog > Peripheral I/O > Single step > Address match

Figure 5.5.1. Interrupt priority that is set in hardware

5.6 Multiple Interrupts

The following shows the internal bit states when control has branched to an interrupt routine:

- The interrupt enable flag (I flag) is cleared to 0 (interrupts disabled).
- The interrupt request bit for the acknowledged interrupt is cleared to 0.
- The processor interrupt priority level (IPL) equals the interrupt priority level of the acknowledged interrupt.

By setting the interrupt enable flag (I flag) (= 1) in the interrupt routine, you can reenables interrupts so that an interrupt request can be acknowledged that has higher priority than the processor interrupt priority level (IPL). Figure 5.6.1 shows how multiple interrupts are handled.

The interrupt requests that have not been acknowledged for their low interrupt priority level are kept pending. When the IPL is restored by an REIT and FREIT instruction and interrupt priority is resolved against it, the pending interrupt request is acknowledged if the following condition is met:

$$\begin{array}{ccc} \text{Interrupt priority level of} & & \text{Restored processor interrupt} \\ \text{pending interrupt request} & > & \text{priority level (IPL)} \end{array}$$

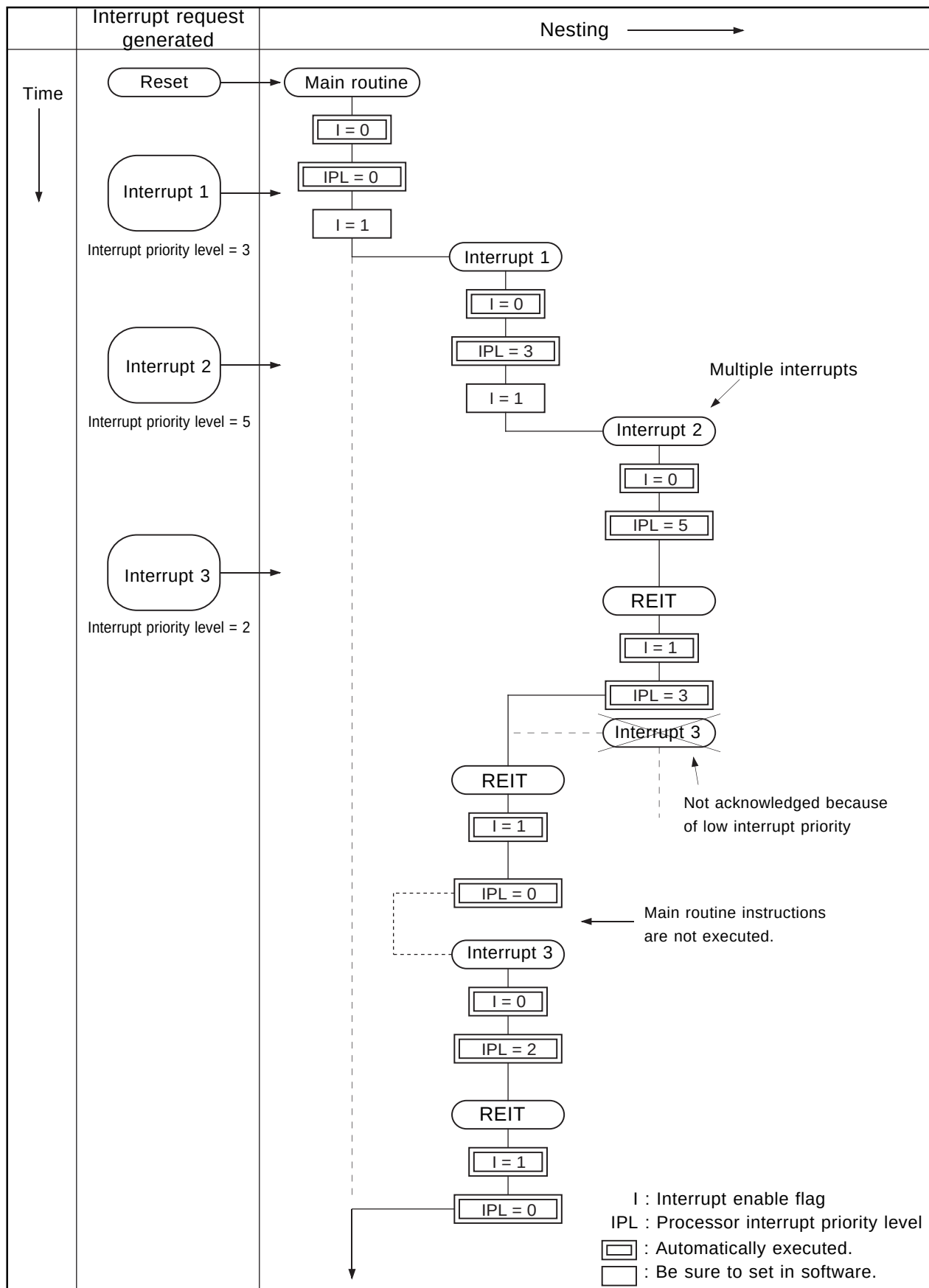


Figure 5.6.1. Multiple interrupts

5.7 Precautions for Interrupts

(1) Reading addresses 000000₁₆ and 000002₁₆

- When maskable interrupt is occurred, CPU read the interrupt information (the interrupt number and interrupt request level) in the interrupt sequence from address 000000₁₆. When high-speed interrupt is occurred, CPU read from address 000002₁₆.

The interrupt request bit of the certain interrupt will then be set to "0".

However, reading addresses 000000₁₆ and 000002₁₆ by software does not set request bit to "0".

(2) Setting the stack pointer

- The value of the stack pointer immediately after reset is initialized to 000000₁₆. Accepting an interrupt before setting a value in the stack pointer may become a factor of runaway. Be sure to set a value in the stack pointer before accepting an interrupt. When using the $\overline{\text{NMI}}$ interrupt, initialize the stack pointer at the beginning of a program. Any interrupt including the $\overline{\text{NMI}}$ interrupt is generated immediately after executing the first instruction after reset. Set an even number to the stack pointer. When an even number is set, execution efficiency is increased.

(3) Rewrite the interrupt control register

- When a instruction to rewrite the interrupt control register is executed but the interrupt is disabled, the interrupt request bit is not set sometimes even if the interrupt request for that register has been generated. This will depend on the instruction. If this creates problems, use the below instructions to change the register.

Instructions : AND, OR, BCLR, BSET

5.8 Exit from Stop Mode and Wait Mode

When using an peripheral I/O interrupt to exit stop mode or wait mode, the relevant interrupt must have been enabled and set to a priority level above the level set by the interrupt priority set bits for exiting a stop/wait state. Set the interrupt priority set bits for exiting a stop/wait state to the same level as the processor interrupt level (IPL) of flag register (FLG).

RESET and $\overline{\text{NMI}}$ interrupt are independent of the interrupt priority set bits for exiting a stop/wait state, and stop/wait state is exited.

Chapter 6

Calculation Number of Cycles

6.1 Instruction queue buffer

6.1 Instruction queue buffer

The M16C/80 series have 8-stage (8-byte) instruction queue buffers. If the instruction queue buffer has a free space when the CPU can use the bus, instruction codes are taken into the instruction queue buffer. This is referred to as “prefetch”. The CPU reads (fetches) these instruction codes from the instruction queue buffer as it executes a program.

Explanation about the number of cycles in Chapter 4 assumes that all the necessary instruction codes are placed in the instruction queue buffer, and that data is read or written to the memory connected via a 16-bit bus (including the internal memory) beginning with even addresses without software wait or $\overline{\text{RDY}}$ or other wait states. In the following cases, more cycles may be needed than the number of cycles shown in this manual:

- When not all of the instruction codes needed by the CPU are placed in the instruction queue buffer...
Instruction codes are read in until all of the instruction codes required for program execution are available. Furthermore, the number of read cycles increases in the following cases:
 - (1) The number of read cycles increases as many as the number of wait cycles incurred when reading instruction codes from an area in which software wait or $\overline{\text{RDY}}$ or other wait states exist.
 - (2) When reading instruction codes from memory chips connected to an 8-bit bus, more read cycles are required than for 16-bit bus.
- When reading or writing data to an area in which software wait or $\overline{\text{RDY}}$ or other wait states exist...
The number of read or write cycles increases as many as the number of wait cycles incurred.
- When reading or writing 16-bit data to memory chips connected to an 8-bit bus...
The memory is accessed twice to read or write one 16-bit data. Therefore, the number of read or write cycles increases by one for each 16-bit data read or written.
- When reading or writing 16-bit data to memory chips connected to a 16-bit bus beginning with an odd address...
The memory is accessed twice to read or write one 16-bit data. Therefore, the number of read or write cycles increases by one for each 16-bit data read or written.

Note that if prefetch and data access occur in the same timing, data access has priority. Also, if more than seven bytes of instruction codes exist in the instruction queue buffer, the CPU assumes there is no free space in the instruction queue buffer and, therefore, does not prefetch instruction code.

Figures 6.1.1 to 6.1.8 show examples of instruction queue buffer operation and CPU execution cycles.

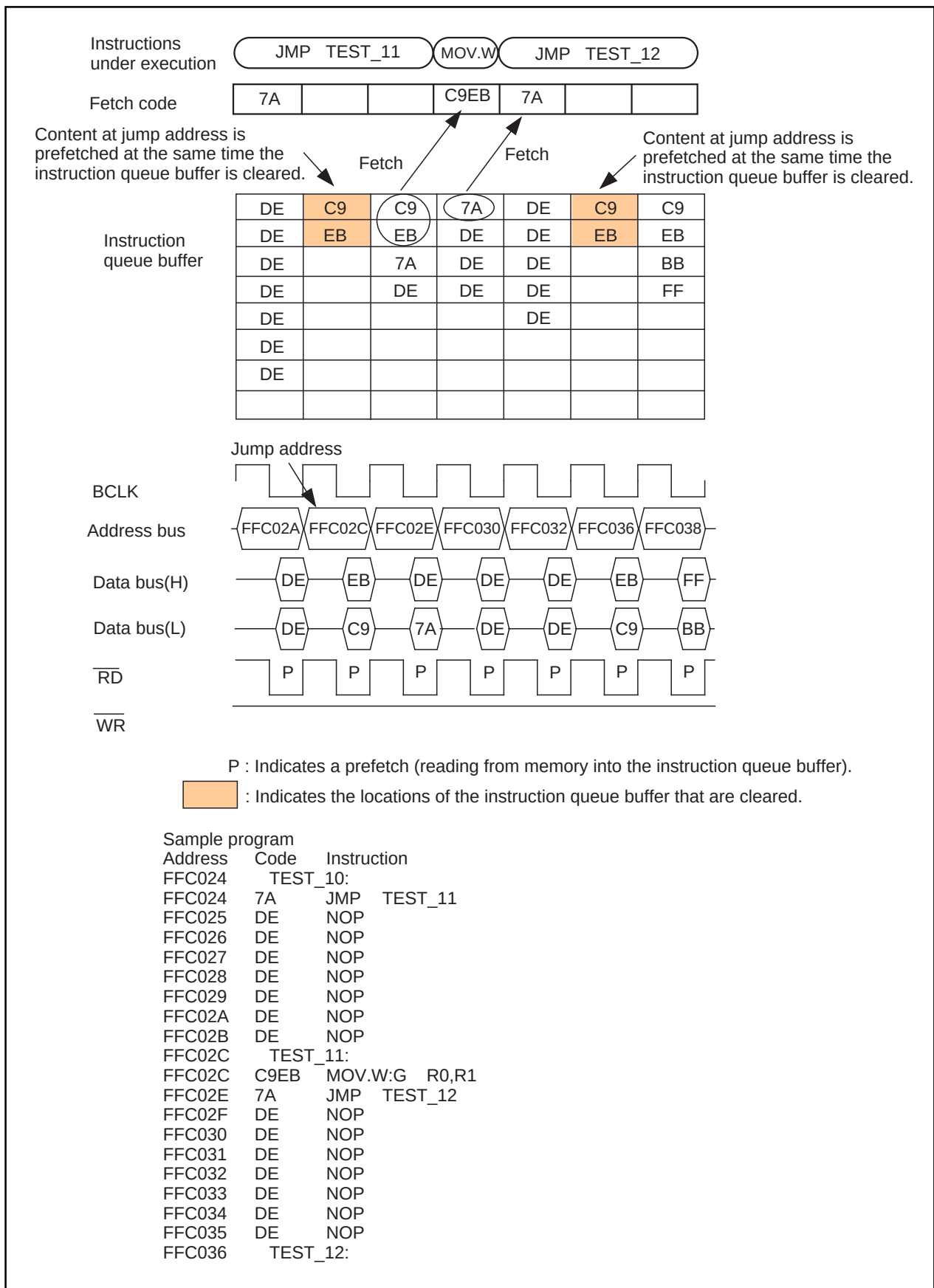


Figure 6.1.1. When executing a register transfer instruction starting from an even address (Program area: 16-bit bus without wait state; Data area: 16-bit bus without wait state)

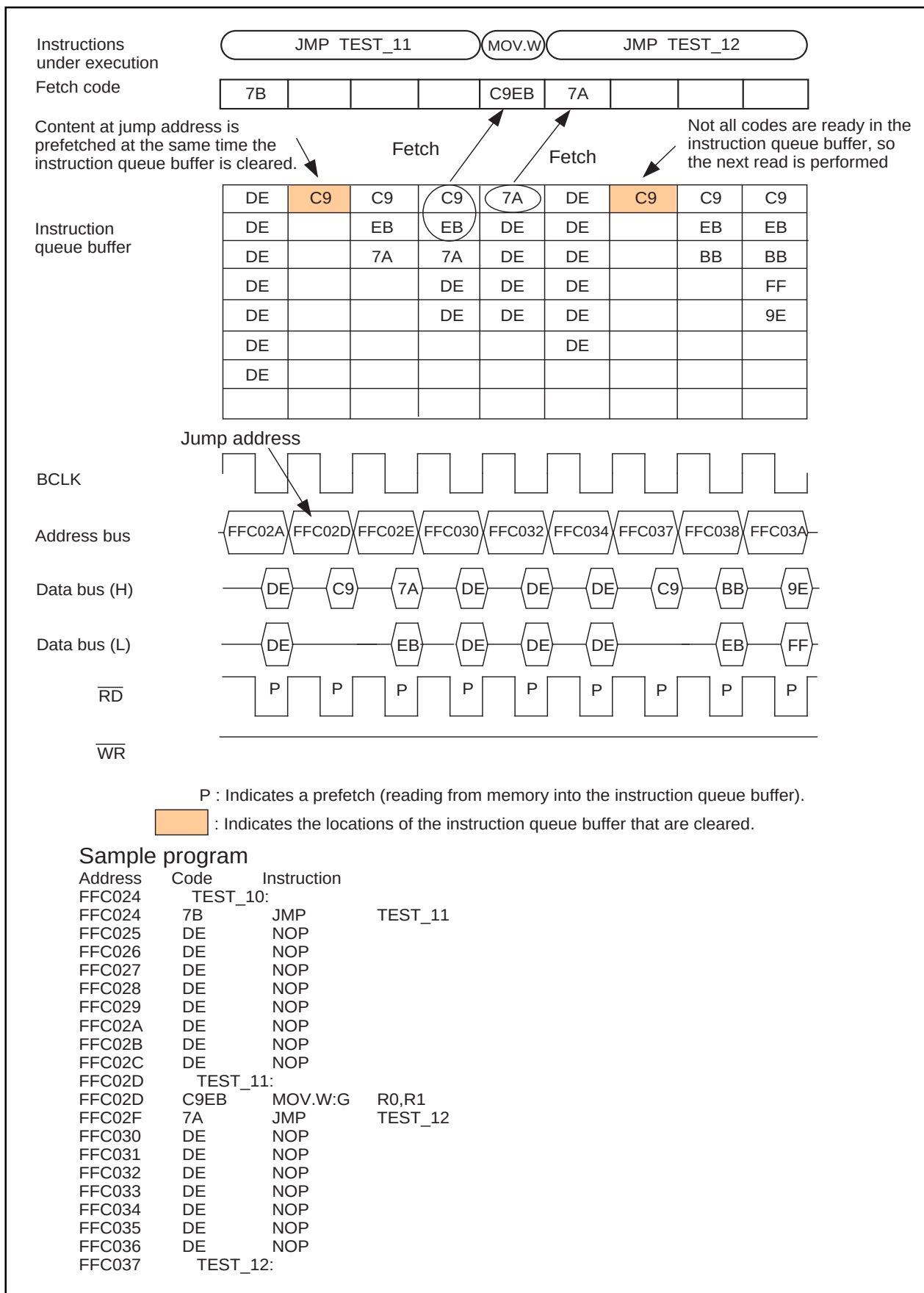


Figure 6.1.2. When executing a register transfer instruction starting from an odd address (Program area: 16-bit bus without wait state; Data area: 16-bit bus without wait state)

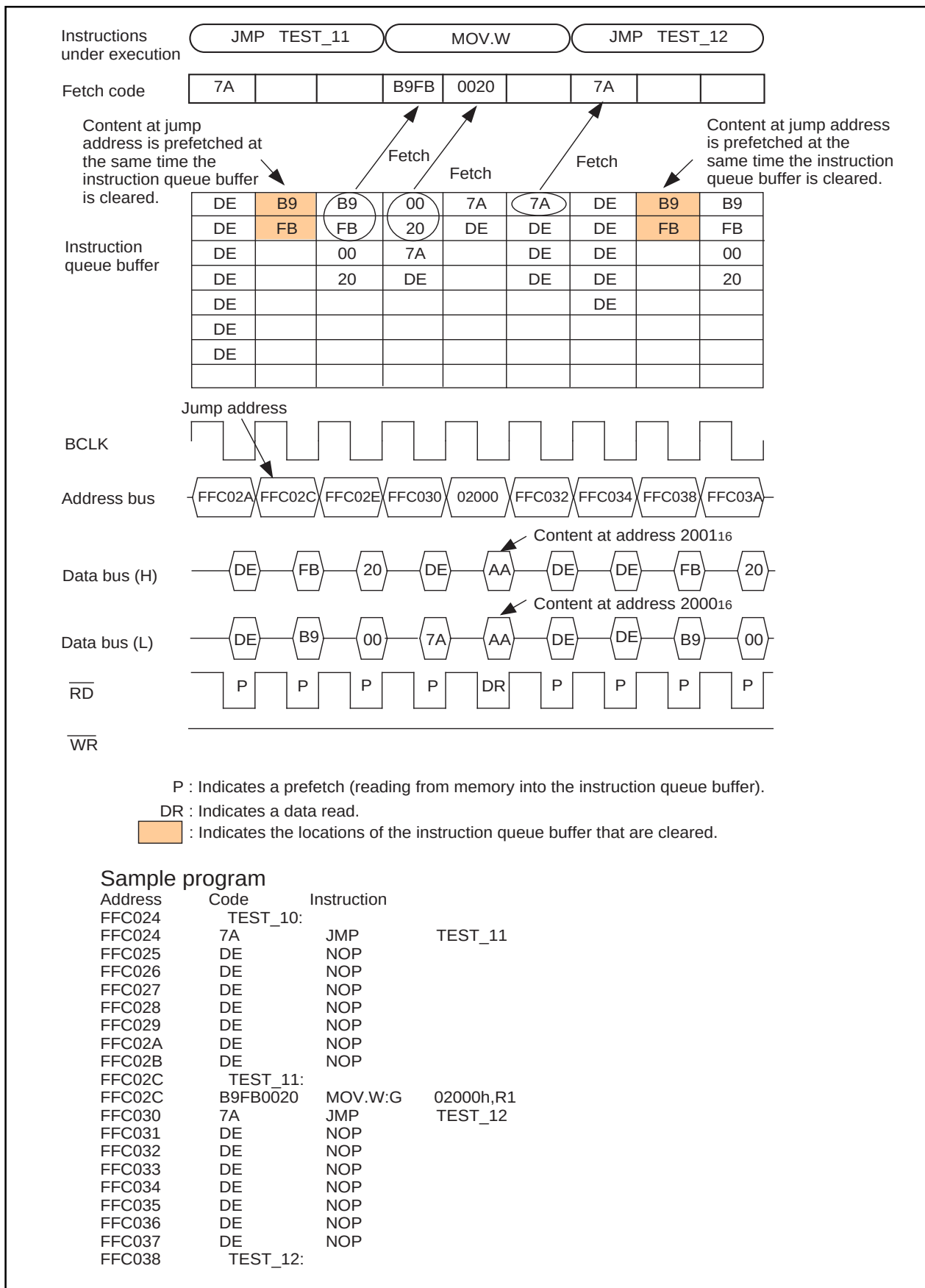


Figure 6.1.3. When executing an instruction to read from even addresses starting from an even address (Program area: 16-bit bus without wait state; Data area: 16-bit bus without wait state)

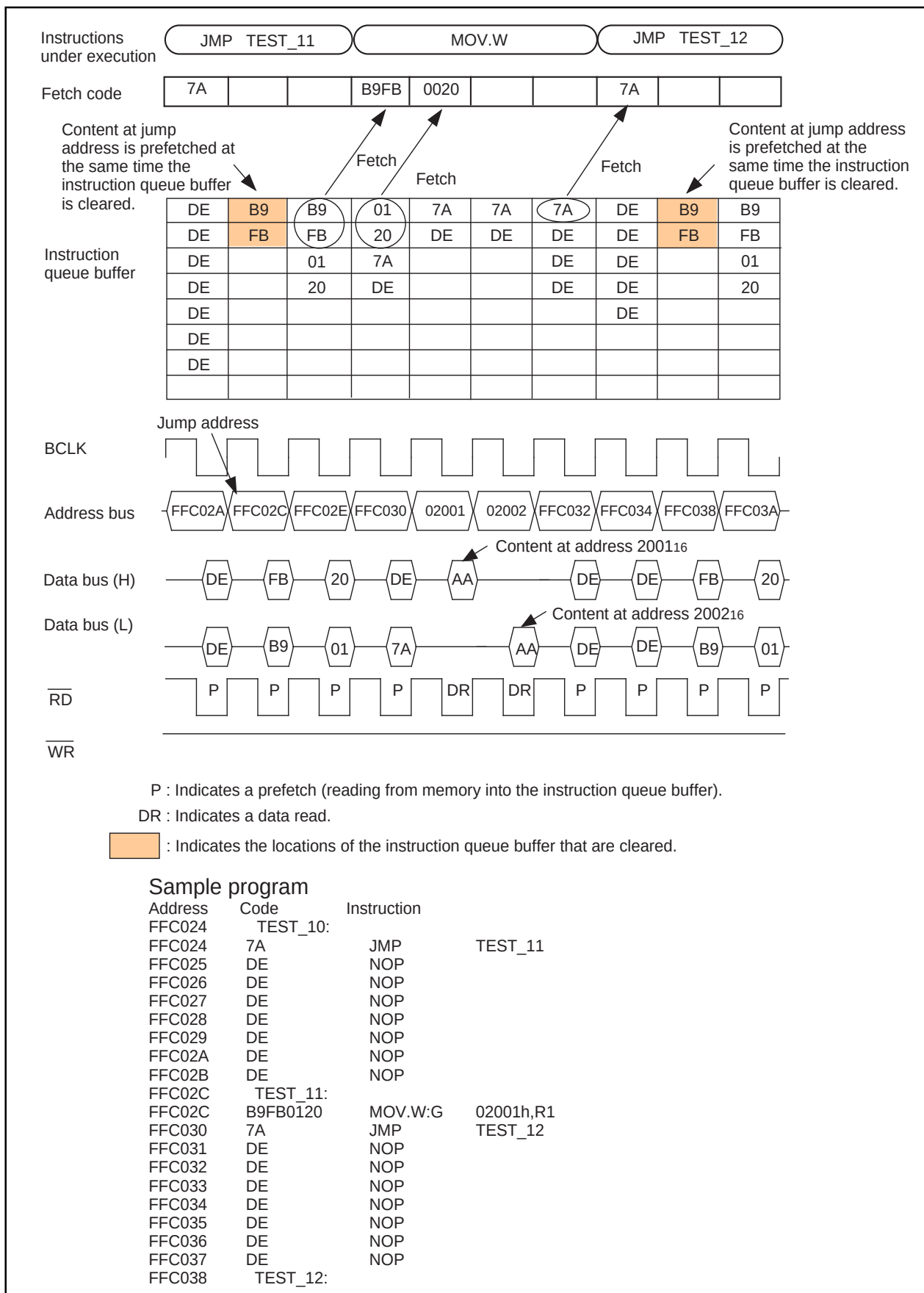


Figure 6.1.4. When executing an instruction to read from odd addresses starting from an even address (Program area: 16-bit bus without wait state; Data area: 16-bit bus without wait state)

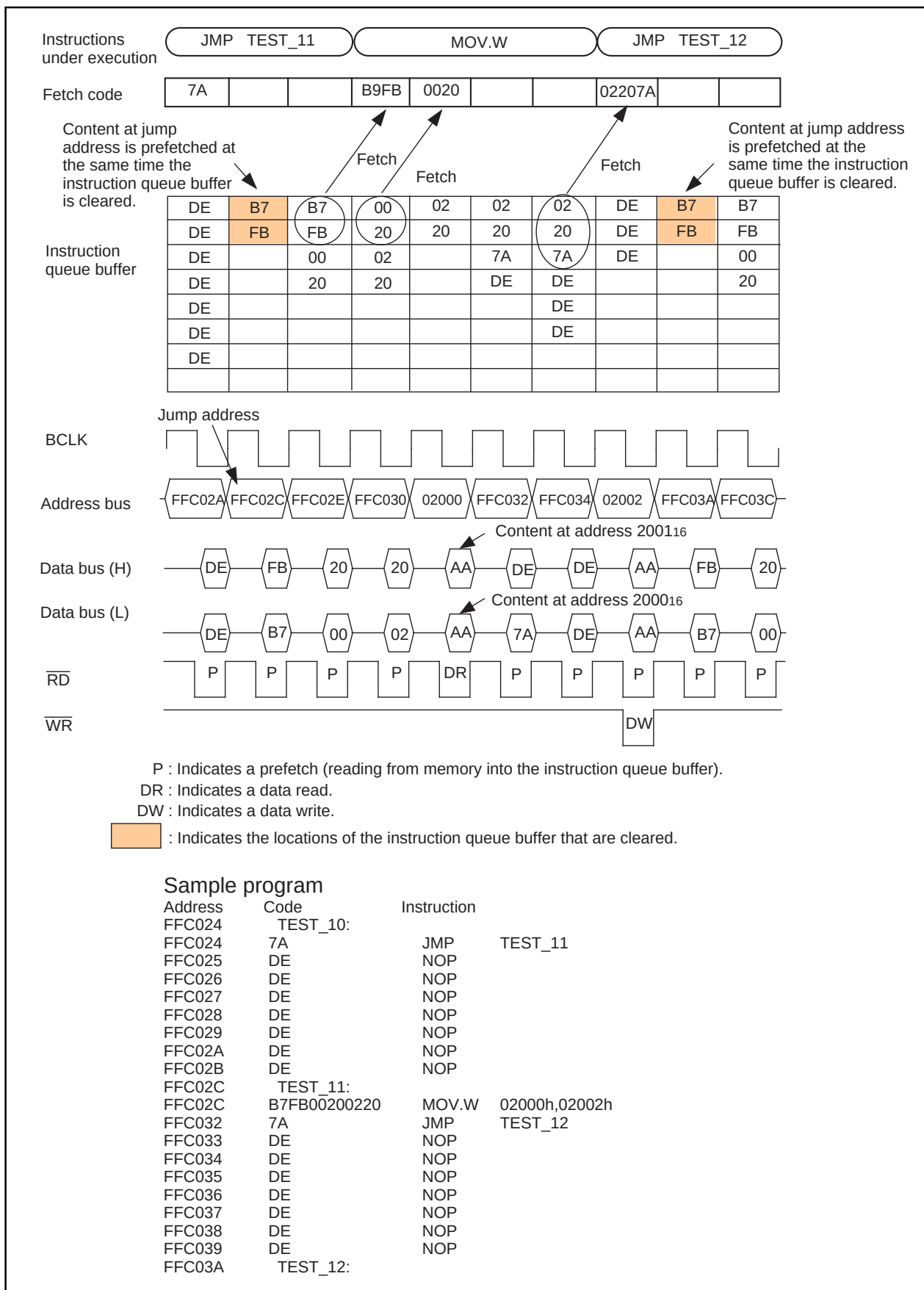


Figure 6.1.5. When executing an instruction to transfer data between even addresses starting from an even address (Program area: 16-bit bus without wait state; Data area: 16-bit bus without wait state)

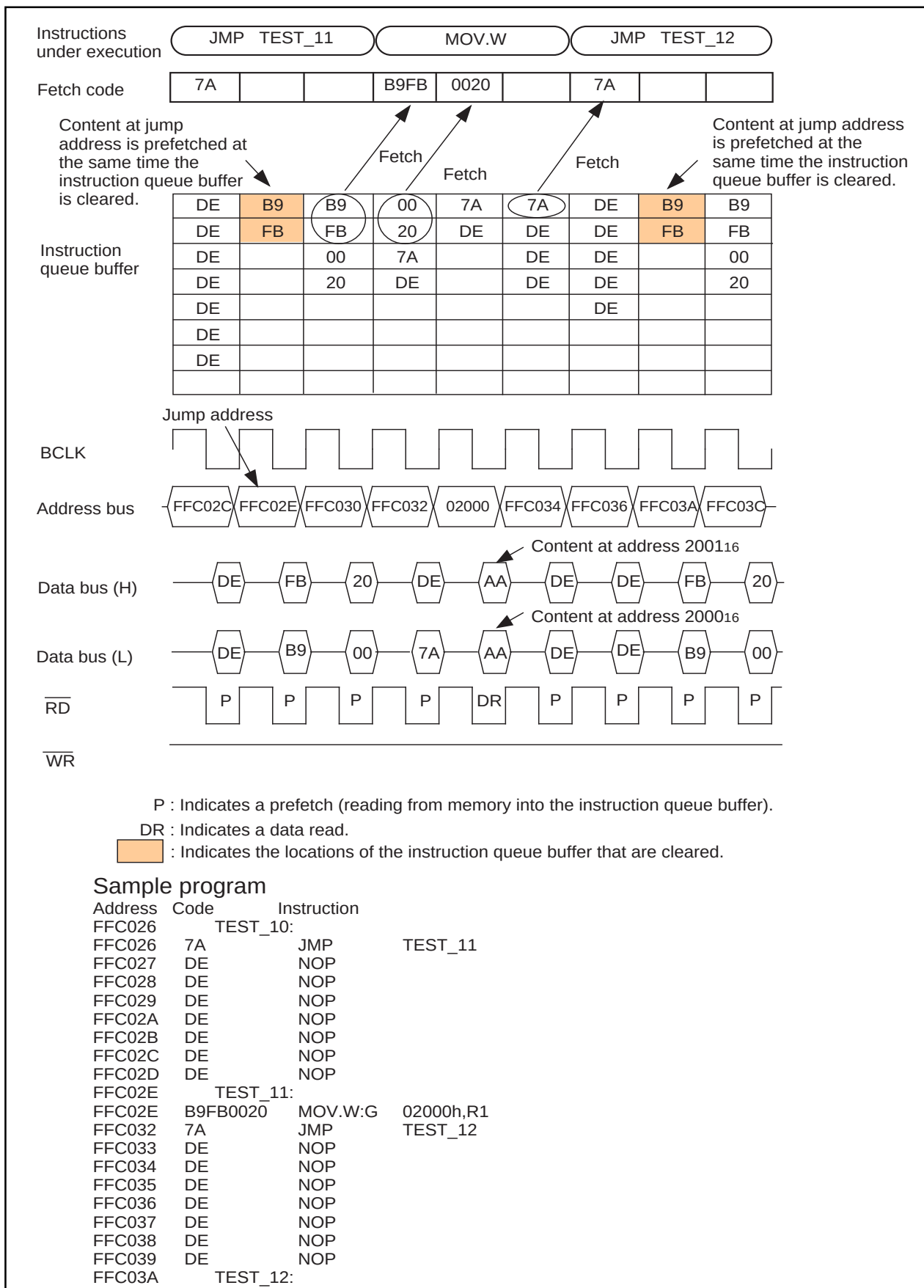


Figure 6.1.6. When executing an instruction to read from even addresses starting from an even address (Program area: 16-bit bus without wait state; Data area: 16-bit bus with wait state)

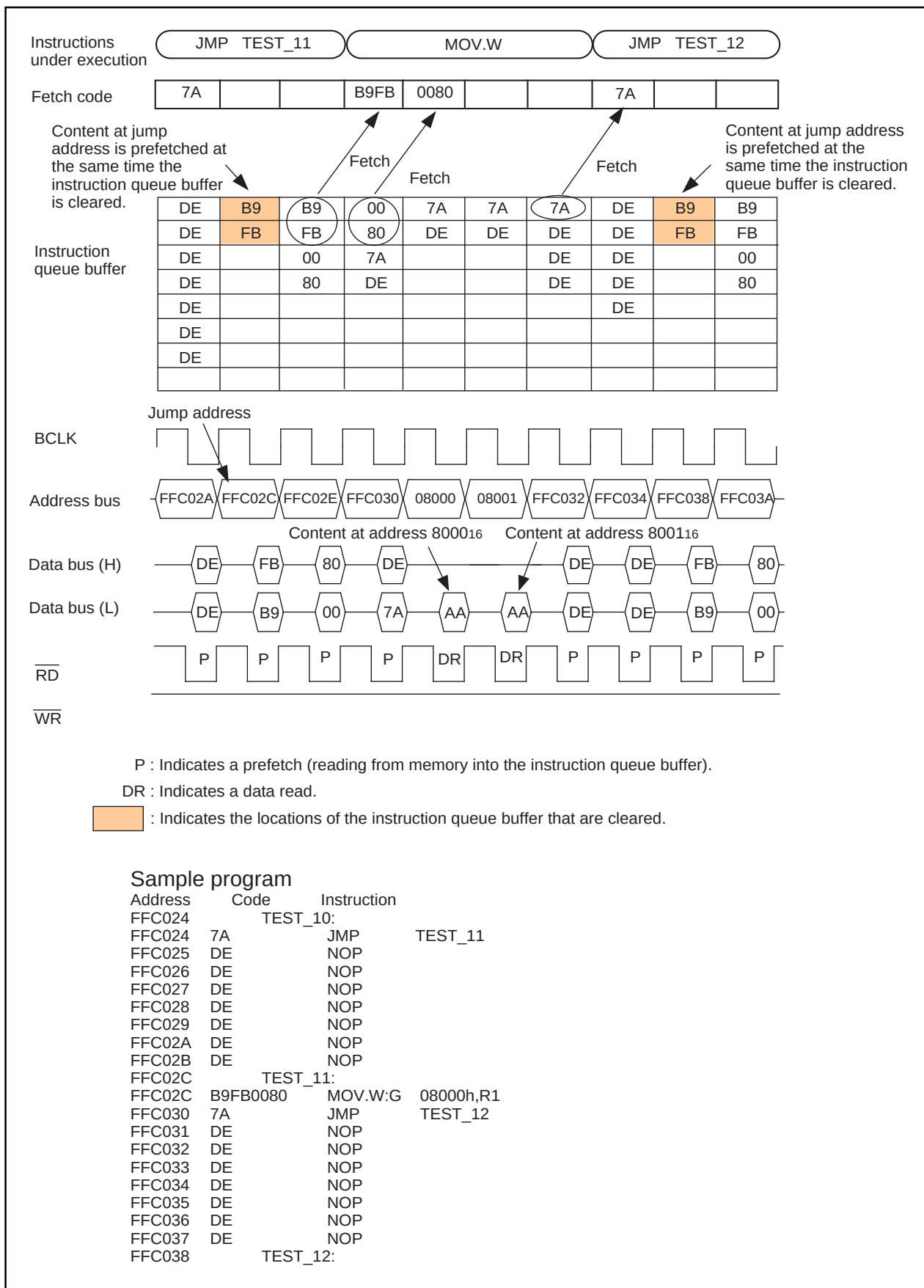


Figure 6.1.7. When executing a read instruction for memory connected to 8-bit bus
 (Program area: 16-bit bus without wait state; Data area: 8-bit bus without wait state)

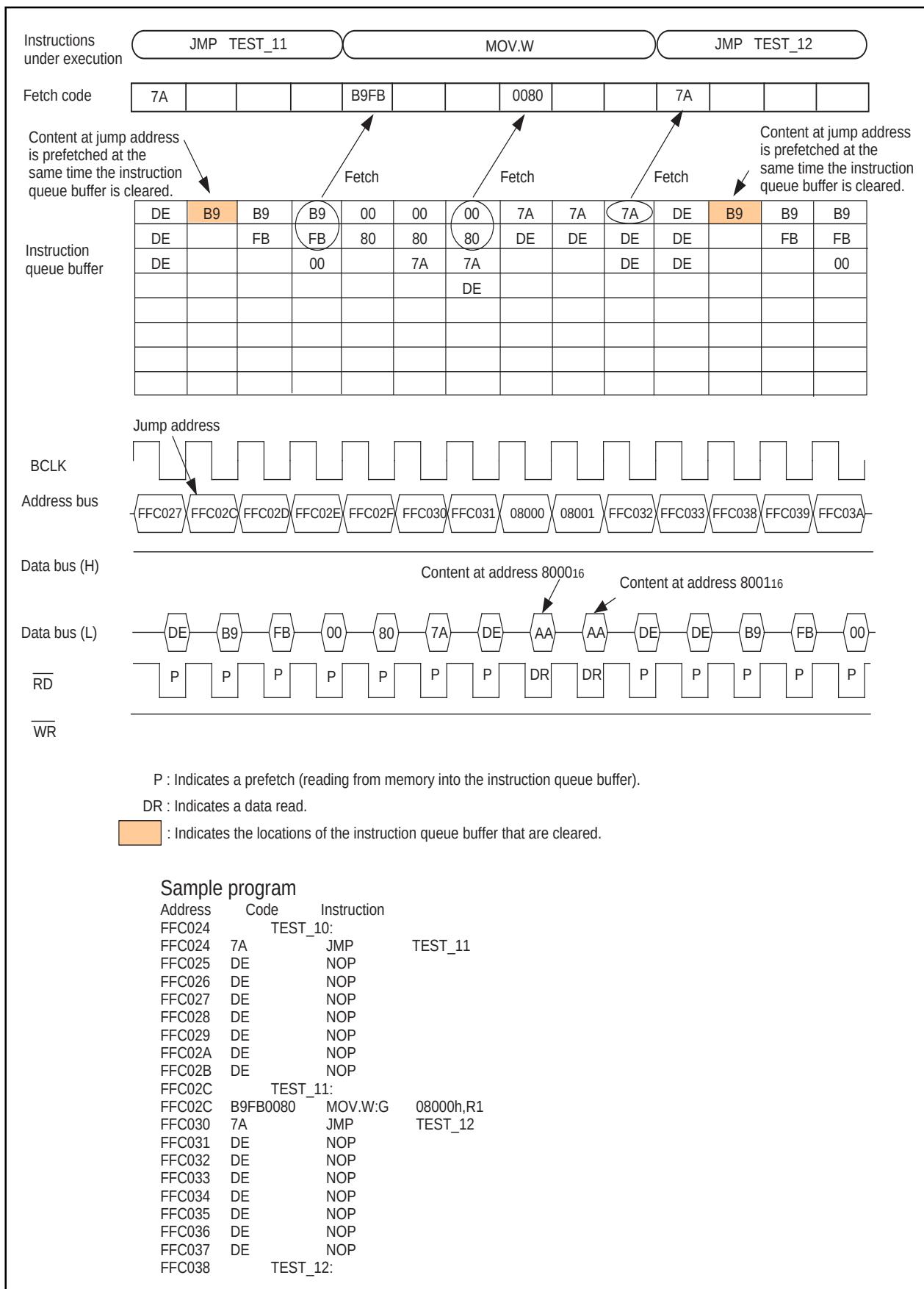


Figure 6.1.8. When executing a read instruction for memory connected to 8-bit bus
 (Program area: 8-bit bus without wait state; Data area: 8-bit bus without wait state)

Q & A

Information in a Q&A form to be used to make the most of the M16C family is given below.

Usually, one question and the answer to it are given on one page; the upper section is for the question, and the lower section is for the answer (if a pair of question and answer extends over two or more pages, a page number is given at the lower-right corner).

Functions closely connected with the contents of a page are shown at its upper-right corner.

Q

How do I distinguish between the static base register (SB) and the frame base register (FB)?

A

Only positive displacement is allowed in SB Relative Addressing, while FB Relative Addressing can be with positive or negative displacement.

If you write a program in C, Mitsubishi C compiler uses FB as a stack frame base register.

You can use SB and FB as intended in programming in the assembly language.

Q

What is the difference between the user stack pointer (USP) and the interrupt stack pointer (ISP)?, What are their roles?

A

You use USP when using the OS. When several tasks run, the OS secures stack areas to save registers of individual tasks. Also, stack areas have to be secured, task by task, to be used for handling interrupts that occur while tasks are being executed. If you use USP and ISP in such an instance, the stack for interrupts can be shared by these tasks; this allows you to efficiently use stack areas.

Q

What is the difference between the DIV instruction and the DIVX instruction?

A

Either of the DIV instruction and the DIVX instruction is an instruction for signed division, the sign of the remainder is different.

The sign of the remainder left after the DIV instruction is the same as that of the dividend, on the contrary, the sign of the remainder of the DIVX instruction is the same as that of the divisor.

In general, the following relation among quotient, divisor, dividend, and remainder holds.
$$\text{dividend} = \text{divisor} \times \text{quotient} + \text{remainder}$$

Since the sign of the remainder is different between these instructions, the quotient obtained either by dividing a positive integer by a negative integer or by dividing a negative integer by a positive integer using the DIV instruction is different from that obtained using the DIVX instruction.

For example, dividing 10 by -3 using the DIV instruction yields -3 and leaves +1, while doing the same using the DIVX instruction yields -4 and leaves -2.

Dividing -10 by +3 using the DIV instruction yields -3 and leaves -1, while doing the same using the DIVX instruction yields -4 and leaves +2.

Q

Is it possible to change the value of the interrupt table register (INTB) while a program is being executed?

A

Yes. But there can be a chance that the microcomputer runs away out of control if an interrupt request occurs in changing the value of INTB. So it is not recommended to frequently change the value of INTB while a program is being executed.

Table of symbols

Symbols used in this software manual are explained below. They are good in this manual only.

Symbol	Meaning
$\leftarrow$	Transposition from the right side to the left side
$\leftrightarrow$	Interchange between the right side and the left side
+	Addition
—	Subtraction
$\times$	Multiplication
$\div$	Division
$\wedge$	Logical conjunction
$\vee$	Logical disjunction
∇	Exclusive disjunction
—	Logical negation
dsp24	24-bit displacement
dsp16	16-bit displacement
dsp8	8-bit displacement
EVA()	An effective address indicated by what is enclosed in ()
EXTS()	Sign extension indicated by what is enclosed in ()
EXTZ()	Zero extension indicated by what is enclosed in ()
(HH)	Higher-order byte of higher-order word of a register or memory (highest byte)
H4:	Four higher-order bits of an 8-bit register or 8-bit memory
(HL)	Lower-order byte of higher-order word of a register or memory
	Absolute value
(LH)	Higher-order byte of lower-order word of a register or memory
(LL)	Lower-order byte of lower-order word of a register or memory (lowest byte)
L4:	Four lower-order bits of an 8-bit register or 8-bit memory
LSB	Least Significant Bit
M()	Content of memory indicated by what is enclosed in ()
MSB	Most Significant Bit
PC _H	Higher-order byte of the program counter
PC _{ML}	Middle-order byte and lower-order byte of the program counter
FLGH	Four higher-order bits of the flag register
FLGL	Eight lower-order bits of the flag register
[]	Indirect addressing

Glossary

Technical terms used in this software manual are explained below. They are good in this manual only.

Term	Meaning	Related word
borrow	To move a digit to the next lower position.	carry
carry	To move a digit to the next higher position.	borrow
context	Registers that a program uses.	
decimal addition	An addition in terms of decimal system.	
displacement	The difference between the initial position and later position.	
effective address	An after-modification address to be actually used.	
LSB	Abbreviation for Least Significant Bit The bit occupying the lowest-order position of a data item.	MSB

Term	Meaning	Related word
MSB	Abbreviation for Most Significant Bit The bit occupying the highest-order position of a data item.	
operand	A part of instruction code that indicates the object on which an operation is performed.	LSB
operation	A generic term for move, comparison, bit processing, shift, rotation, arithmetic, logic, and branch.	operation code
operation code	A part of instruction code that indicates what sort of operation the instruction performs.	
overflow	To exceed the maximum expressible value as a result of an operation.	operand
pack	To join data items. Used to mean to form two 4-bit data items into one 8-bit data item, to form two 8-bit data items into one 16-bit data item, etc.	
SFR area	Abbreviation for Special Function Area. An area in which control bits of peripheral circuits embodied in a microcomputer and control registers are located.	unpack

Term	Meaning	Related word
shift out	To move the content of a register either to the right or left until fully overflowed.	
sign bit	A bit that indicates either a positive or a negative (the highest-order bit).	
sign extension	To extend a data length in which the higher-order to be extended are made to have the same sign of the sign bit. For example, sign-extending FF ₁₆ results in FFFF ₁₆ , and sign-extending 0F ₁₆ results in 000F ₁₆ .	
stack frame	An area for automatic variables the functions of the C language use.	
string	A sequence of characters.	
unpack	To restore combined items or packed information to the original form. Used to mean to separate 8-bit information into two parts — 4 lower-order bits and four higher-order bits, to separate 16-bit information into two parts — 8 lower-order bits and 8 higher-order bits, or the like.	pack
zero extension	To extend a data length by turning higher-order bits to 0's. For example, zero-extending FF ₁₆ to 16 bits results in 00FF ₁₆ .	

Index

A

A0 and A1 ... 5
A1A0 ... 5
Address register ... 5
Address space ... 3
Addressing mode ... 22

B

B flag ... 6
Byte (8-bit) data ... 16

C

C flag ... 6
Carry flag ... 6
Cycles ... 139

D

D flag ... 6
Data arrangement in memory ... 17
Data arrangement in Register ... 16
Data register ... 4
Data type ... 10
Debug flag ... 6
Description example ... 37
dest ... 18

F

FB ... 5
Fixed vector table ... 19
Flag change ... 37
Flag register ... 5
FLG ... 5

Frame base register ... 5

Function ... 37

I

Interrupt table register ... 5
I flag ... 6
Instruction code ... 139
Instruction Format ... 18
Instruction format specifier ... 35
INTB ... 5
Integer ... 10
Interrupt enable flag ... 6
Interrupt stack pointer ... 5
Interrupt vector table ... 19
IPL ... 7
ISP ... 5

L

Long word (32-bit) data ... 16

M

Maskable interrupt ... 248
Memory bit ... 12
Mnemonic ... 35, 38

N

Nibble (4-bit) data ... 16
Nonmaskable interrupt ... 248

O

O flag ... 6
Operand ... 35, 38

Operation ... 37

Overflow flag ... 6

P

PC ... 5

Processor interrupt priority level ... 7

Program counter ... 5

R

R0, R1, R2, and R3 ... 4

R0H, R1H ... 4

R0L, R1L ... 4

R2R0 ... 4

R3R1 ... 4

Register bank ... 8

Register bank select flag ... 6

Register bit ... 12

Related instruction ... 37

Reset ... 9

S

S flag ... 6

SB ... 5

Selectable src / dest (label) ... 37

Sign flag ... 6

Size specifier ... 35

Software interrupt number ... 20

Special page number ... 19

Special page vector table ... 19

src ... 18

Stack pointer ... 5

Stack pointer select flag ... 6

Static base register ... 5

String ... 15

Syntax ... 35, 38

U

U flag ... 6

User stack pointer ... 5

USP ... 5

V

Variable vector table ... 20

W

Word (16-bit) data ... 16

Z

Z flag ... 6

Zero flag ... 6

Revision History

Revision History

Version	Contents for change	Revision date
REV.C	Chapter 5 addition • Page 20 line 20 (IPL) --> (ISP) • Page 32 Absolute 000FFF₁₆ --> 000FFFF₁₆ • Page 95 JMPS Operation FFFFFF₁₆ --> FFFE₁₆ • Page 96 JSR Operation SP - 1 --> SP - 2 • Page 98 JMRS Operation FFFFFF₁₆ --> FFFE₁₆ • Page 133 SCMPU Operation temp --> tmp • Page 276 SCCnd dest An --- --> ---/A0/--- --- --> ---/A1/---	'99.1.26
	• Page 4 line 2 13 registers --> 28 registers • Page 89 INDEXType [Description Example] INDEXB R0 --> INDEXB.W R0 INDEXLS [A0] --> INDEXLS.B [A0] • Page 138-143 SIN, SMOVB, SMOVF, SOUT, SSTR [Operation] Delete 'Repeat' and 'Until ...'	'99.1.26
	• Page 62- BRK, BRK2, ENTER, EXITD, INT, INTO, POPC, POPM, REIT, RTS, UND Note for PCH, FBH and M(SP) is added. • Page 120 PUSH *2 The 8 high-order bits are 0 --> indeterminate • Page 133 SCMPU • When the size specifier (.size) is (.W) If M(A)=M(A1) then M(A0+1)–M(A1+1) --> If M(A)=M(A1) and M(A0)≠0 then M(A0+1)–M(A1+1) • Page 135 SHA [Flag change] O • Page 173 (4) Table of cycles • Page 268 PUSHM [Byte number/ cycle number] 1/m --> 2/m	'99.3.12
	• Page 5 (9) Save flag register 24 bits --> 16 bits	'99.7.12
Revision history		M16C/80 Series Software Manual

Revision History

Version	Contents for change	Revision date
REV.D	Chapter 6 addition • Page 5 (9) Save flag register (SVF) 24 bit --> 16 bit • Page 10 1.6 Internal State after Reset is Cleared • Save flag register (SVF) : indeterminate --> addition • Save PC register (SVP) : indeterminate --> addition • Vector register (VCT) : indeterminate --> addition • Page 69 CLIP [Function] • Src1 and src2 are set "src1<src2". --> addition • Page 99 LDC [Function] *3 SP and ISP --> SP, ISP and INTB • Page 118 POPC [Operation] *3 --> addition • Page 120 PUSH [Operation] *2 ..., the 8 high-order bits become indeterminate. --> become 0 • Page 120 PUSHC [Operation] *3 --> addition • Page 149 SUB [Function] Line 10 When <i>src</i> is the address register, <i>src</i> is zero-extended to perform operation in 32 bits. --> addition • Page 193 BNTST [Number of Bytes/Number of Cycles] dest --> src • Page 196 BSET [Number of Bytes/Number of Cycles] dest --> src • Page 229 JMP dsp = address indicated by label - (start address of instruction +2) --> Delete [Number of Bytes/Number of Cycles] 1/4 --> 1/3 • Page 231 JMPI [Number of Bytes/Number of Cycles] dest --> src • Page 232 JMPI [Number of Bytes/Number of Cycles] dest --> src • Page 234 JSRI (1) and (2) [Number of Bytes/Number of Cycles] dest --> src • Page 231 JMPI (2) d4 d3 d2 d1 d0 --> s4 s3 s2 s1 s0 • Page 257 MULEX [Number of Bytes/Number of Cycles] dest --> src	99.10.25
	• Page 120 PUSH [Operation] *2 When <i>src</i> is address register(A0, A1), the 8 high-order bits become indeterminate. --> ... become 0. • Page 234 (2)JSRI.A d4 d3 d2 d1 d0 --> s4 s3 s2 s1 s0	99.10.28
REV.D1	• Page 303(2) Overflow interrupt CMPX addition	00.03.02
Revision history		M16C/80 Series Software Manual

—Keep safety first in your circuit designs!—

- Mitsubishi Electric Corporation puts the maximum effort into making semiconductor products better and more reliable, but there is always the possibility that trouble may occur with them. Trouble with semiconductors may lead to personal injury, fire or property damage. Remember to give due consideration to safety when making your circuit designs, with appropriate measures such as (i) placement of substitutive, auxiliary circuits, (ii) use of non-flammable material or (iii) prevention against any malfunction or mishap.

—Notes regarding these materials—

- These materials are intended as a reference to assist our customers in the selection of the Mitsubishi semiconductor product best suited to the customer's application; they do not convey any license under any intellectual property rights, or any other rights, belonging to Mitsubishi Electric Corporation or a third party.
- Mitsubishi Electric Corporation assumes no responsibility for any damage, or infringement of any third-party's rights, originating in the use of any product data, diagrams, charts, programs, algorithms, or circuit application examples contained in these materials.
- All information contained in these materials, including product data, diagrams, charts, programs and algorithms represents information on products at the time of publication of these materials, and are subject to change by Mitsubishi Electric Corporation without notice due to product improvements or other reasons. It is therefore recommended that customers contact Mitsubishi Electric Corporation or an authorized Mitsubishi Semiconductor product distributor for the latest product information before purchasing a product listed herein.
The information described here may contain technical inaccuracies or typographical errors. Mitsubishi Electric Corporation assumes no responsibility for any damage, liability, or other loss rising from these inaccuracies or errors.
Please also pay attention to information published by Mitsubishi Electric Corporation by various means, including the Mitsubishi Semiconductor home page (<http://www.mitsubishichips.com>).
- When using any or all of the information contained in these materials, including product data, diagrams, charts, programs, and algorithms, please be sure to evaluate all information as a total system before making a final decision on the applicability of the information and products. Mitsubishi Electric Corporation assumes no responsibility for any damage, liability or other loss resulting from the information contained herein.
- Mitsubishi Electric Corporation semiconductors are not designed or manufactured for use in a device or system that is used under circumstances in which human life is potentially at stake. Please contact Mitsubishi Electric Corporation or an authorized Mitsubishi Semiconductor product distributor when considering the use of a product contained herein for any specific purposes, such as apparatus or systems for transportation, vehicular, medical, aerospace, nuclear, or undersea repeater use.
- The prior written approval of Mitsubishi Electric Corporation is necessary to reprint or reproduce in whole or in part these materials.
- If these products or technologies are subject to the Japanese export control restrictions, they must be exported under a license from the Japanese government and cannot be imported into a country other than the approved destination.
Any diversion or reexport contrary to the export control laws and regulations of Japan and/or the country of destination is prohibited.
- Please contact Mitsubishi Electric Corporation or an authorized Mitsubishi Semiconductor product distributor for further details on these materials or the products contained therein.

M16C/80 Series Software Manual



Renesas Electronics Corporation

1753, Shimonumabe, Nakahara-ku, Kawasaki-shi, Kanagawa 211-8668 Japan